

目录

系统分析师-按照知识点导出历年真题（案例题） 2

系统分析与设计 2

数据库系统 46

企业信息化战略与实施 77

Web开发 81

软件工程 121

系统规划 135

嵌入式系统设计 147

软件需求工程 190

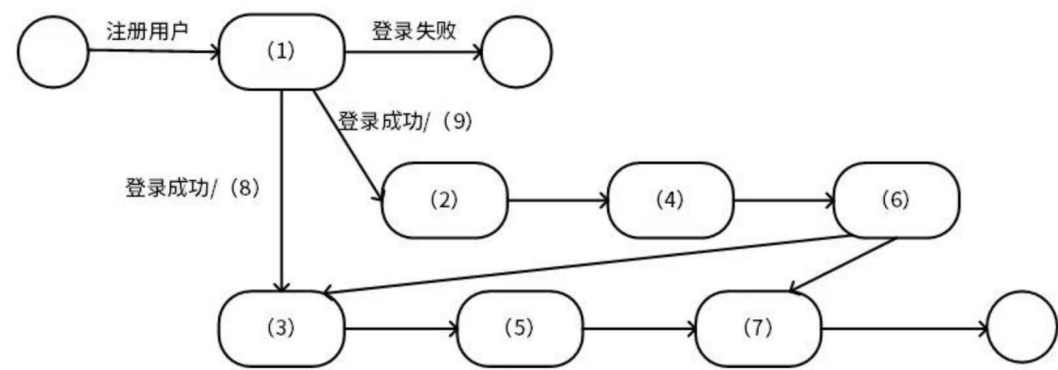
注：版权提醒：本资料已通过国家版权局登记（登记号：渝作登字-2025-A-00624260），（一旦发现资料恶意流出，直接封号不退款，后果严重的话会请求平台披露你的个人信息，并于互联网法院起诉，请珍惜账号权益和个人名誉，谢谢）。

系统分析师-按照知识点导出历年真题（案例题）

第1章——系统分析与设计

(2023年5月系统分析师真题-第5题)

阅读以下关于软件系统分析与建模的叙述，在纸上回答问题1至3。
某软件公司拟开发一套汽车租赁系统，科学安全和方便的管理租赁公司的各项业务，提高公司效率，提升利率。注册用户在使用系统镜像车辆预约时需执行以下操作
(a) 用户登录系统 (b) 查询车辆信息 (c) 预订车辆 (d) 提取车辆 (e) 归还车辆 (f) 计算结余 (g) 查询还车时间



问题（1）

在面向对象软件分析过程中，状态机图和活动图用于建立软件的动态模型，主要描述系统随时间变化的行为，请分别阐述这两种图的概念和特点。

思路解析

状态机图和活动图都是面向对象建模中用于描述系统动态行为的重要工具。状态机图着重展示对象如何随着时间推移在不同的状态之间转换，关注的是事件的触发及状态的反应。而活动图主要用于描述系统中不同活动的流程和执行顺序，强调操作的先后顺序和控制流。两者可以结合使用，帮助更好地理解 and 实现系统的动态行为。

问题（2）

图1-1给出了采用状态机图描写的注册用户的状态交换过程，请使用题干给出的（a）～（g）补充完善图1-1中的（1）～（7），并给出（8）和（9）空白处的状态转移执行动作。将正确的答案填写在答题纸上。

思路解析

状态机图中描述的是用户与系统交互的不同状态。根据题干给出的用户操作流程，补充了状态机图中的具体步骤：

- 用户首先登录系统（a），接着可以查询车辆信息（b）并选择预定（c）。
- 提车操作（d）后，用户会归还车辆（e），并计算结余（f）。
- 查询还车时间（g）有助于了解还车的时效性。

状态转移的执行动作“查询到有预定车辆”和“查询到无预定车辆”用于描述系统对预定车辆的检测，确保用户在没有预定的情况下不能提车。这里题干很简洁，所有的信息都在这7个操作里。它没

有明确描述具体业务流程只能凭感觉来做。一条路径是用户租车路径、登陆、查询车辆信息、预定车辆、提车。一条路径是用户还车路径、登陆、查询还车时间、还车。然后最后是结算。当然为什么租车要结算可能是定金之类的，还车的时候二次结算归还定金，并扣除费用。

问题（3）

RUP 中有9个核心工作流，分6个核心过程和3个核心支持工作流。9个核心工作流在项目中迭代被使用在每次迭代中以不同的重点和强度重复。结合题干案例，用300字以内的文字简要说明 RUP 中9个核心工作流的主要作用。

思路解析

RUP中的9个核心工作流涵盖了从需求分析、设计、开发、测试到部署的全过程，确保项目的每个方面都得到有效管理。这些工作流不仅包括直接影响开发的核心过程，还涉及项目管理和变更控制等支持性工作流，为项目的成功交付提供全方位的保障。

答案-问题（1）

（1）状态图用来描述对象状态和事件之间的关系，强调一个实体基于事件反应的动态行为。状态图适合用于表述在不同用例之间的对象行为。

（2）活动图描述的是对象活动的顺序关系所遵循的规则，它着重表现系统的行为，强调了行为活动的顺序和条件控制。

（3）活动图是由状态图变化而来的，他们各自用于不同的目的。活动图依据对象状态的变化来捕获动作（将要执行的工作或活动）与动作的结果。活动图中一个活动结束后立即进入下一个活动（在状态图中，状态的变迁可能需要事件的触发）。

答案-问题（2）

- （1）a
- （2）b
- （3）g
- （4）c
- （5）e
- （6）d
- （7）f
- （8）查询到有预定车辆
- （9）查询到无预定车辆

答案-问题（3）

RUP中有9个核心工作流，分为6个核心过程和3个核心支持工作流。这些工作流是：

（1）业务建模：定义新组织的构想，并在商业用例模型和商业对象模型中描述组织的过程、角色和责任。

（2）需求：描述系统应该做什么，并确保开发人员和用户对需求达成共识。

（3）分析和设计：将需求转化为未来系统的设计，创建设计模型和可选的分析模型。

（4）实现：以层次化的子系统形式组织代码，实现类和对象，并进行单元测试和集成测试。

(5) 测试：验证对象间的交互作用，验证软件组件的正确集成，检验需求的正确实现，并发现和处理缺陷。

(6) 部署：成功生成版本并将软件分发给最终用户，包括打包、安装、提供帮助等活动。

(7) 配置和变更管理：控制多个项目成员和大量产物，管理软件的版本和变更。

(8) 项目管理：平衡各种目标，管理风险，成功交付满足用户需求的产品。

(9) 环境：提供软件开发环境，包括过程和工具，支持项目配置和规范，以及为组织实施过程提供指导。

(2023年5月系统分析师真题-第1题)

阅读以下关于企业信息系统结构化分析的叙述，回答问题1和2。

某软件公司为企业开发一套员工在线教育系统支持员工利用业余时间开展专业技术培训，提升员工技能。在项目开展初期，采用结构化分析进行开发，并对系统中培训部员工和培训教师的相关功能进行分析，具体需求如下：

(1) 培训部根据企业技术发展需求，负责策划培训课程，并形成课程计划，针对不同的员工设置不同的课程；

(2) 员工首先在系统进行注册，填写自己的编号，学历，专业，岗位等信息，生成员工注册信息，然后将自己的培训需求录入系统，系统自动评估并进行课程推荐，员工确认后形成课程需求；

(3) 培训教师也通过系统进行注册，填写自己的编号、学历、专业等信息，形成培训教师注册信息

(4) 系统根据课程计划、员工注册信息，课程需求和培训教师注册信息，为员工和培训教师生成对应的课程表。

工时系统分析师对上述流程进行了审核，并指出需补充数据字典，从而更完整地対系统建模。

问题(1)

数据流图(DFD)是结构分析方法的重要工具。请用300字以内的文字描述DFD的定义。

思路解析

数据流图是结构化方法中的核心工具之一，其作用是简洁明了地展示系统中的数据如何流动，以及系统如何处理这些数据。它不关注物理细节，而是强调数据的流动和处理，从而为后续的系统设计提供清晰的逻辑框架。通过使用DFD，开发人员可以更清晰地识别系统的主要功能模块和数据处理方式，进一步为系统设计和实现提供依据。

问题(2)

项目组针对题干描述的业务需求，初步绘制了系统流图(2-1)，请分析途中的三类错误并对每类错误进行简单解释。

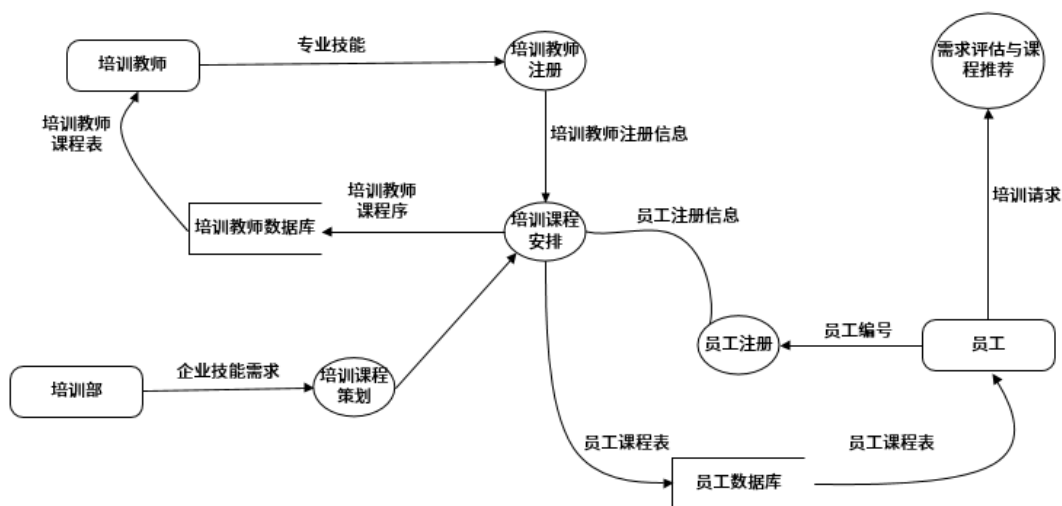


图 2-1 员工在线教育系统数据流图

思路解析

在绘制和审查DFD时，需要确保图中的每个元素都符合结构化分析的标准。DFD的核心目标是清晰地表达系统的数据流动和处理过程，因此在设计DFD时必须遵循一定的规则：

每个加工都必须有输入和输出。

数据流要有明确的方向，数据的流动路径必须清晰。

每个加工和数据流都应有清晰的命名，以确保图形的可读性。

编号系统帮助在不同的层次之间建立清晰的关系，避免混淆。

通过检查和修正这些错误，可以使DFD更准确地反映系统的数据处理逻辑，进一步推动系统设计和实现的工作。

答案-问题（1）

数据流图或称数据流程图，是一种便于用户理解、分析系统数据流程的图形工具。它摆脱了系统的物理内容，精确地在逻辑上描述系统的功能、输入、输出和数据存储等，是系统逻辑模型的重要组成部分。数据流图是结构化分析方法中使用的工具，它以图形的方式描绘数据在系统中流动和处理的过程，由于它只反映系统必须完成的逻辑功能，所以它是一种功能模型。在结构化开发方法中，数据流图是需求分析阶段产生的结果。

答案-问题（2）

第1类错误：加工只有输入没有输出。"需求评估与课程推荐"这个加工只有输入数据流，但是没有任何输出数据流。在DFD中，每个加工过程必须至少有一个输入流和一个输出流。如果一个加工没有输出流，就说明这个过程没有任何可见的效果，违反了数据流图的基本要求。

第2类错误：数据流没有方向箭头。"员工注册信息"这一数据流没有标明方向箭头。数据流必须明确指示数据从哪里流向哪里，没有方向箭头会使得数据流的起点和终点不明确，从而影响图形的理解和准确性。数据流必须具备方向性，指明数据传递的路径。

第3类错误：所有的加工没有编号。"需求评估与课程推荐"和"培训教师注册"等加工没有编号。在DFD中，每个加工都应有一个编号，编号可以帮助分析人员和开发人员跟踪和引用这些过程，特别是当DFD分层时，编号能够帮助关联不同层次之间的流程和数据。

第4类错误：数据流没有名称。"培训课程规划"到"培训课程安排"的这条数据流没有命名。在DFD中，每条数据流都必须有明确的名称，以便准确表示数据的含义和作用。没有名称的数据流使得图形不够清晰，影响其可读性和准确性。

第5类错误：不能从一个存储直接到另一个存储。"员工课程库"到"员工实体"之间的直接数据流是不允许的，同样“培训教师数据库”到“培训教师实体”也不能直接连接。数据存储和数据实体之间的流动应该通过加工来进行转换，而不能直接连接。因为数据存储是静态的，它们本身不进行处理，数据流动应通过相应的处理过程进行管理。

(2022年5月系统分析师真题-第2题)

阅读以下关于软件系统分析与建模的叙述，在答题纸上回答问题 1 至问题 3。

某软件公司拟开发一套博客系统，要求能够向用户提供一个便捷发布自己心得，及时有效的与他人进行交流的平台。新用户发布个人博客之前，需要创建一个新的博客账户，以下为新用户注册的操作行为：

- (a) 向系统请求创建一个新的博客账户；
- (b) 输入个人详细信息；
- (c) 使用证件数据库验证个人详细信息；
- (d) 选择账户类型；
- (e) 身份验证成功，创建新的博客账户；
- (f) 用户身份信息验证不成功；
- (g) 以电子邮件的方式将账户详细信息发送给用户；
- (h) 博客账户申请被拒绝。

问题（1）

在结构化和面向对象的软件分析过程中，通常会使用到数据流图、活动图和流程图，请分别描述这三种模型的特点和适用场景。

思路解析

这三种图形模型分别代表了不同的分析和设计需求：

数据流图适合功能建模，特别是在系统数据处理和流动的描述上。

活动图适合展示系统内的控制流和操作流程，尤其在面向对象设计时发挥作用。

流程图则专注于反映业务处理的各个环节，是理解和管理业务流程的重要工具。根据不同的建模需求选择合适的图形工具，可以帮助更清晰地描述系统。

问题（2）

采用用例图和用例描述建模系统需求，请使用题干给出的（a）～（h），完善"博客账户创建用例描述"中的（1）～（6），如表 1-1 所示。将正确答案填在答题纸上。

用例名称	创建一个新的博客账户	
相关的需求	新用户的个人资料通过证件致数据库验证成功后，CMS系统将允许用户创建一个新的博客账户	
上下文目标	新用户请求创建一个新的博客账户	
前置条件	新用户具有真实有效的身份证明	
成功的结束条件	为新用户创建博客账户	
失败的结束条件	创建博客账户的申请被拒绝	
主要参与者	新用户	
次要参与者	作者证件数据库	
触发器	新用户向 CMS请求建立一个新的博客账户	
基本流程	步骤	
	1	(1)
	2	(2)
	3	(3)
	4	(4)
	5	身份验证成功，创建新的博客账户
	6	(5)
扩展流程	步骤	分支动作
	1	用户身份信息验证不成功
	2	(6)

思路解析

在用例图和用例描述中，我们通常会根据业务流程来进行步骤的描述。这里，（a）代表用户请求创建新账户，（d）代表选择账户类型，接下来的步骤顺序是符合该业务逻辑的。当验证失败时，通过（h）进行账户拒绝的描述，而成功的操作则通过（g）发送邮件通知用户。

问题（3）

需求评审是通过将需求规格说明书递交给相关人员检查，以发现其中存在缺陷的过程。在需求工程中，需求评审是一个非常重要的过程。结合题干案例，请用 300 字以内的文字简要说明需求评审的内容及作用。

思路解析

需求评审是需求工程中的核心活动之一，它不仅确保了需求的准确性和完整性，还帮助团队发现潜在问题并采取措施进行修正。通过需求评审，项目团队能够验证需求文档是否满足各方要求，确保需求的一致性、准确性和完整性。这一过程能够提高系统设计的质量，并为后续的开发、测试和维护奠定坚实的基础。

答案-问题（1）

数据流图（DFD）：特点：数据流图是用来表达系统中数据流动的模型，描述数据如何在系统中流动以及被各个处理单元（加工）如何处理。数据流图主要由数据流、外部实体、加工和数据存储组成。它强调的是数据在系统内的流动，通常用于功能建模。适用场景：数据流图是结构化分析方法中的重要工具，适用于进行功能建模，特别是在描述系统如何处理和传递数据时。

活动图：特点：活动图用来表示系统中的动态过程，描述系统操作的执行顺序及其相互之间的控制流和数据流。活动图专注于系统的动态视图，尤其强调对象间的控制流程。它是面向对象分析和设计中的重要建模工具。适用场景：活动图广泛用于面向对象分析与设计建模中，适用于业务流程建模和系统功能的流程建模，帮助团队理解系统内部的操作流程。

流程图：特点：流程图是一种反映现有系统业务处理过程的工具，描绘了各部门的业务分工与联系，以及业务流程中信息流、物料流的传递。它适合用于展示简单的业务过程和工作流。适用场景：流程图主要用于结构化需求分析与设计，适用于梳理和描述系统中的业务流程、工作流等

，帮助开发团队理清系统的业务逻辑。

答案-问题（2）

(1) (a) (2) (d) (3) (b) (4) (c) (5) (g) (6) (h)

答案-问题（3）

需求评审的内容：

正确性：需求规格说明书是否正确地描述了系统的行为和特征，确保符合项目干系人的需求。

推导过程：软件需求是否从系统需求、业务规格以及其他来源正确推导而来。

完整性和高质量：需求是否完整，且没有遗漏，且保证了系统设计和后续开发的可行性。

一致性：需求在所有地方是否一致，避免了冲突或矛盾。

后续支持：需求是否为系统的设计、实现和测试提供了充分的基础。

需求评审的作用：

发现遗漏的需求：评审可以帮助团队发现未被捕捉的需求，避免遗漏关键功能。

解决需求冲突：需求评审帮助揭示不同干系人之间未达成共识的部分，促使各方沟通解决。

消除二义性：通过审查需求，发现并修正模糊或不明确的描述，确保需求的准确性。

降低风险：通过评审过程，能够有效发现并消除潜在的问题，减少因需求不一致带来的风险，提高软件质量。

(2022年5月系统分析师真题-第1题)

阅读以下关于系统设计的叙述，在答题纸上回答问题 1 至问题 3。

某软件公司为某企业开发一套在线办公自动化系统，以提升企业管理服务的质量和效率。公司决定采用面向对象方法开发该系统，张工负责对员工请假审批业务进行了分析，具体描述如下：

企业员工通过请假申请页面提出请假申请后，首先由部门经理进行审核，若请假理由不合理，部门经理拒绝请假申请，将申请退回给员工，员工对请假申请表进行修改再次提交，或放弃请假申请。

若请假理由合理，且请假天数小于等于 3 天，则部门经理直接批准请假申请，生成请假批准单，申请结束。

若请假天数大于 3 天，则在部门经理批准请假申请后，需要提交给副总经理审核批准。副总经理若认为该申请合理，则审核通过，生成请假批准单，流程结束；若副总经理认为申请不合理，则拒绝请假申请，将请假申请退回给员工。

问题（1）

识别设计类是面向对象设计过程中的重要环节之一，设计类表达了类的职责，即该类所承担的任务。请用 300 字以内的文字说明设计类通常包含哪 3 种类型，以及每种类型的主要职责，并从题干描述案例涉及的具体类为每种类型的设计类举出两个实例。

思路解析

设计类是面向对象分析和设计中的核心要素，帮助将系统分解为易于管理和维护的部分。实体类代表系统的核心数据，边界类则是用户或外部系统与系统的接口，控制类负责管理和实现系统的业务逻辑。通过清晰地划分这三类设计类，能够使系统更加模块化和松耦合，便于扩展和维护。

问题（2）

识别类之间的关系是面向对象分析过程中的重要环节之一，常见的类之间关系包括泛化关系、关联关系、聚合关系、组合关系等。请用 300 字以内的文字对泛化关系和关联关系的内涵进行描述，并根据题干具体类的描述为这两种关系各举出一种实例。

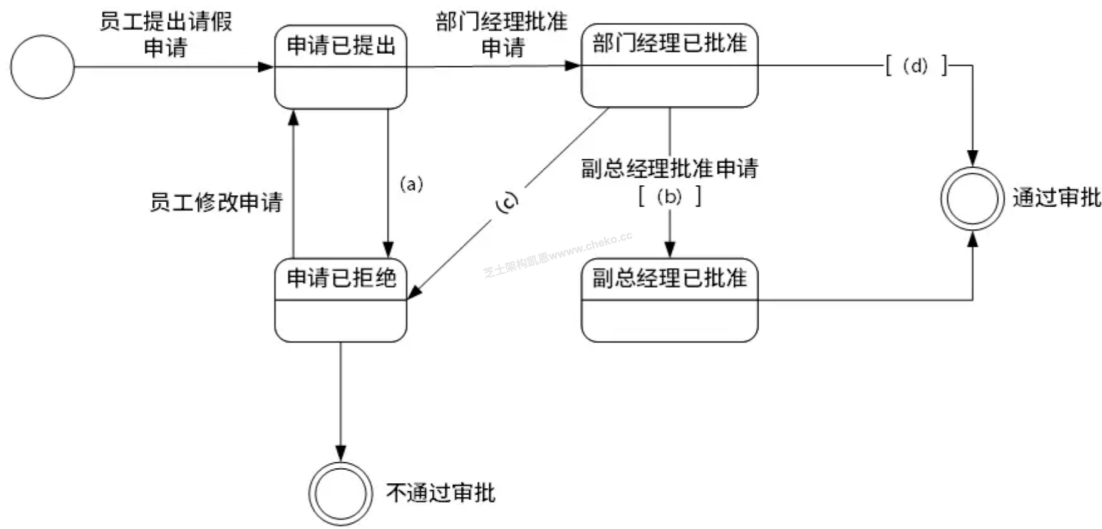
思路解析

在面向对象分析中，识别类与类之间的关系有助于形成类图。泛化关系强调继承性，属于“is-a”关系，父类定义通用属性与方法，子类在此基础上扩展。题中“员工”是通用角色，而“部门经理”是具有审批权限的特殊员工，因此属于泛化关系。

关联关系则强调对象之间的协作，组合和聚合属于关联关系，属于强“has-a”关系。例如“员工”必须拥有“请假申请单”才能发起请假流程；同时，一个员工可能会提交多个申请，一个申请单也只能属于一个员工，这体现了多对一或一对多的关联特性。这里两者是组合关系，“请假申请单”必须依附于“员工”，没有员工就没有请假申请单，申请单的生命周期完全依赖于员工。

问题（3）

在面向对象的设计过程中，状态图描述了一个实体基于事件反应的动态行为。请根据题干描述，填写图 2-1 中的（a）~（d）空白，完成请假审批的状态图。



思路解析

在状态图中，状态表示实体（如请假申请）在生命周期中的各个状态，而事件则是引起状态转换的触发因素。根据题干的描述：

- 当部门经理拒绝请假申请时，系统的状态会转换到“请假申请被拒绝”。
- 如果请假天数大于3天，则流程将进入副总经理审核阶段。
- 副总经理可能会拒绝申请，导致申请返回给员工。
- 如果请假天数小于或等于3天，则部门经理可以直接批准申请，生成请假批准单。

答案-问题（1）

实体类。实体对象的抽象，通常来自域模型（现实世界），用来描述具体的实体，通常映射到数据库表格与文件中。案例中的员工是实体类，请假申请单，请假批准单。

边界类。边界类是指在系统中负责与外部交互的类，主要用于处理系统与用户或其他系统之间的接

口。边界类充当系统与外部实体（如用户、外部系统）的桥梁，负责接收输入、显示输出以及传递信息给控制类或实体类。例如：From表单、对话框、菜单、接口等。案例中的请假表单，请假批准单是边界类（这里指的是表单类，和实体类中的请假申请单，请假批准单还不太一样）。

控制类。控制对象的抽象，主要用来体现应用程序的执行逻辑，将其抽象出来，可以使变化不影响用户界面和数据库中的表。案例中的提交请假、审核批准是控制类。

答案-问题（2）

泛化描述了一般元素和特殊元素之间的关系，是父类和子类之间的关系。员工，部门经理的关系就是泛化关系。关联关系描述了给定类的单独对象之间语义上的连接。关联提供了不同类之间的对象可以相互作用的连接。员工与请假记录之间的关系就是关联关系。

答案-问题（3）

- (a) 部门经理拒绝请假申请 (b) 请假天数大于3天
(c) 副总经理拒绝请假申请 (d) 请假天数小于等于3天

(2021年5月系统分析师真题-第5题)

阅读以下关于系统分析与设计的叙述，在答题纸上回答问题1至问题3。

某高校拟开发一套图书馆管理系统，在系统分析阶段，系统分析师整理的核心业务流程与需求如下：

系统为每个读者建立一个账户，并给读者发放读者证（包含读者证号、读者姓名），账户中存储读者的个人信息、借阅信息以及预订信息等，持有读者证可以借阅图书、返还图书、查询图书信息、预订图书、取消预订等。

在借阅图书时，需要输入读者所借阅的图书名、ISBN 号，然后输入读者的读者证号，完成后提交系统，以进行读者验证。如果读者有效，借阅请求被接受，系统查询读者所借阅的图书是否存在，若存在，则读者可借出图书，系统记录借阅记录；如果读者所借阅的图书已被借出，读者还可预订该图书。读者如期还书后，系统清除借阅记录，否则需缴纳罚金，读者还可以选择续借图书。

同时，以上部分操作还需要系统管理员和图书管理员参与。

问题（1）

采用面向对象方法进行软件系统分析与设计时，一项重要的工作是进行类的分析与设计。请用200字以内的文字说明分析类图与设计类图的差异。

思路解析

分析类图和设计类图的区别在于其产生的阶段、关注点和详细程度。分析类图着眼于系统的需求和业务逻辑，而设计类图则是在此基础上进行详细的系统设计，关注实现的可操作性和技术细节。分析类图更侧重于领域建模，而设计类图则具体到如何通过编程实现这些需求。

问题（2）

设计类图的首要工作是进行类的识别与分类，该工作可分为两个阶段：首先，采用识别与筛选法，对需求分析文档进行分析，保留系统的重要概念与属性，删除不正确或冗余的内容；其次，将识别出来的类按照边界类、实体类和控制类等三种类型进行分类。请用200字以内的文字对边界类、实

体类和控制类的作用进行简要解释，并对下面给出的候选项进行识别与筛选，将合适的候选项编号填入表2-1中的（1）~（3）空白处，完成类的识别与分类工作。

表 2-1 图书管理系统类识别与分类表格

类型	实例
边界类	(1)
实体类	(2)
控制类	(3)

候选项:

- a) 系统管理员 b) 图书管理员 c) 读者 d) 读者证 e) 账户
f) 图书 g) 借阅 h) 归还 i) 预订 j) 罚金
k) 续借 l) 借阅记录

思路解析

控制类前面说过一般是动宾结构或者动词，所以这里的g（借阅）、h（归还）、i（预订）、k（续借）都是控制类。边界类这里不好判断，按照定义，边界类主要负责系统与外部参与者（如用户、其他系统）之间的交互，包括数据的输入和输出界面等。它处于系统边界，起到隔离和交互的作用。之前凯恩说过边界类窗口、通信协议、打印机接口、报表等，好像看起来都不像。那只能从定义判断，就是可以沟通外界和控制类的东西。这里读者证是最合适的。其余的都可以看做实体类。

除非：j. 罚金：涉及系统与用户在费用结算等交互场景下的输出信息展示等交互功能。l. 借阅记录：可用于向用户展示其借阅情况等信息，属于系统和用户交互。它们都是界面的定义范畴。

问题（3）

根据类之间的相关性特点，可以将类之间的关系分为组合（composition）、继承（inheritance）、关联（association）、聚合（aggregation）和依赖（dependency）等5种，请用300字以内的文字分别对这5种关系的内涵进行叙述，并从封装性、动态组合和创建对象的方便性三个方面对组合和继承关系的优缺点进行比较。

思路解析

组合和继承是面向对象设计中的两种重要关系，组合通常在封装性和灵活性上优于继承。组合使得系统设计更加灵活，部分对象的创建和生命周期由整体对象控制，可以支持运行时的动态组合；而继承虽然在代码复用上有优势，但由于紧密耦合，破坏了封装性，限制了灵活性。在设计时，通常推荐优先使用组合关系，特别是在需要高灵活性和松耦合的场景下。答案中如果非要挑毛病，就是继承里虽然“不需要单独创建父类对象”，但实际上系统依然会帮你自动创建并初始化父类部分，只是对开发者透明而已。

答案-问题（1）

分析类图（Analysis Class Diagram）：分析类图通常在需求分析阶段产生。分析类图主要关注系统的概念模型，反映系统的业务需求和业务对象。它更多使用了业务领域的语言和词汇，描述

系统中对象的关系，但不涉及系统的实现细节。分析类图中的类通常不包含具体的属性和方法，仅描述对象之间的关系及交互。

设计类图 (Design Class Diagram)：设计类图通常在系统设计阶段产生。设计类图关注系统的实现结构，描述系统如何实现业务需求。它会细化类的属性、方法，并考虑类的可见性、数据类型等细节。设计类图更为详细，包括了类名、属性、方法、方法参数、返回值等实现细节。

答案-问题 (2)

边界类主要用于描述外部参与者与系统之间的交互。边界类是一种用于对系统外部环境与其内部运作之间的交互进行建模的类。这种交互包括转换事件，并记录系统表示方式 (例如接口) 中的变更。

实体类主要是作为数据管理和业务逻辑处理层面上存在的类。实体类的主要职责是存储和管理系统内部的信息，它也可以有行为，甚至很复杂的行为，但这些行为必须与它所代表的实体对象密切相关。

控制类用于描述一个用例所具有的事件流控制行为，控制一个用例中的事件顺序。控制类是控制其他类工作的类。每个用例通常有一个控制类，控制用例中的事件顺序，控制类也可以在多个用例间共用。其他类通常并不向控制类发送消息，而是由控制类发出消息。

第二问官方的答案如下 (说不通)

(1) j、l (2) a、 b、 c、 f (c可替换成 d e) (3) g、 h、 i、 k

凯恩给出的答案是

(1) d (2) a、 b、 c、 e、 f 、 j、 l (3) g、 h、 i、 k

答案-问题 (3)

组合 (composition)：是整体与部分的关系，但部分不能离开整体而单独存在。

继承 (inheritance)：表示一般与特殊的关系，它指定了子类如何特化父类的所有特征和行为。

关联 (association)：是一种拥有的关系，它使一个类知道另一个类的属性和方法。

聚合 (aggregation)：是整体与部分的关系，且部分可以离开整体而单独存在。

依赖 (dependency)：是一种使用的关系，即一个类的实现需要另一个类的协助。

(1) 从封装性方面看，组合关系不破坏封装性，整体类与局部类之间松耦合，彼此互相独立；继承关系破坏封装性，子类与父类之间紧密耦合，子类依赖于父类的实现，子类缺乏独立性。

(2) 从动态组合方面看，组合关系支持动态组合，在运行时整体对象可以选择不同的局部对象；继承关系不支持动态继承，在运行时，子类无法选择不同的父类。

(3) 从创建对象的方便性方面看，组合关系在创建整体类的对象时，需要创建所有局部类对象；继承关系在创建子类对象时，无须单独创建父类的对象。

(2021年5月系统分析师真题-第4题)

阅读以下关于Web系统架构设计的叙述，在答题纸上回答问题1至问题3。

某公司拟开发一个基于Web的远程康复系统。该系统的主要功能需求如下：

(1) 康复设备可将患者的康复训练数据实时传入云数据库；

(2) 医生可随时随地通过浏览器获取患者康复训练的数据，并进行康复训练的结果评估和康复处方的更新；

(3) 患者可通过此系统查看自己的康复训练记录和医生下达的康复训练处方，并可随时与医生进行在线沟通交流；

（4）平台管理员可借助此系统实现用户的管理和康复设备的监控和管理，及时获悉设备的数据信息，便于设备的维护和更新。

该公司针对上述需求组建了项目组，并召开了项目开发讨论会。会上，张工建议云数据库采用关系型数据库来实现数据存储；李工提出采用三层架构实现该远程康复系统。

问题（1）

请用200字以内的文字简要说明什么是云数据库以及云数据库的特点。

思路解析

云数据库不仅具备传统数据库的基本功能，而且充分利用了云计算的优势，如高弹性、高可用性和高性能，使得用户可以在没有硬件投资的情况下使用强大的数据库服务。云数据库适合数据量大、需要高可用性和灵活扩展的应用场景。

问题（2）

根据该系统的功能需求，请列举出该系统中存在的实体，以辅助张工进行关系数据库设计。

思路解析

在关系数据库设计中，实体的识别和抽象是一个关键步骤。实体通常代表了业务数据中具体的事物或对象。例如，一个订单处理系统中的实体可能包括订单、客户、产品、配送地址等。此题中的实体有用户、医生、患者、平台管理员、康复设备、设备数据、训练数据、康复处方、训练记录等。

问题（3）

根据李工的建议，该系统将采用三层架构。请用300字以内的文字分析层次型架构的优势，并从下面给出的（a）~（i）候选项中进行选择，补充完善图5-1中（1）~（6）处空白的内容，完成该系统的架构设计方案。

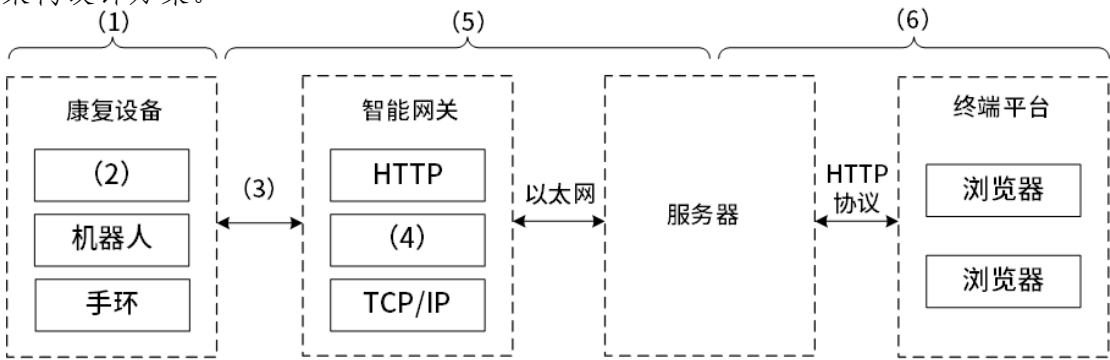


图 5-1 基于 Web 的远程康复系统

候选项：

- (a) 治疗仪
- (b) 接入层
- (c) Socket
- (d) Spring
- (e) 应用层
- (f) MySQL
- (g) MVC

(h) 无线通信

(i) 网络层

思路解析

基于层次型架构的层次系统组成一个层次结构，每一层为上层服务，并作为下层客户。在一些层次系统中，除了一些精心挑选的输出函数外，内部的层接口只对相邻的层可见，这样的系统中构件在层上实现了虚拟机。连接件通过决定层间如何交互的协议来定义，拓扑约束包括对相邻层间交互的约束。由于每一层最多只影响两层，同时只要给相邻层提供相同的接口，允许每层用不同的方法实现，同样为软件重用提供了强大的支持。

因此，层次型架构具有的优势为：开发人员进行专业分工，专注理解某一层：系统可修改性高，只要前后提供的服务(接口)相同，即可用新的实现来替换原有层次的实现：每一层中的组件保持内聚性，层之间保持松散耦合，降低了系统间的依赖：有利于复用。结合该系统需求、层次型体系结构风格特点和题干给出的相关技术，可完成该系统的架构设计。

答案-问题(1)

云数据库是指被优化或部署到一个虚拟计算环境中的数据库，具有按需付费、按需扩展、高可用性以及存储整合等能力。云数据库的特性有实例创建快速、支持只读实例、读写分离、故障自动切换、数据备份、Binlog备份、SQL审计、访问白名单、监控与消息通知等。

答案-问题(2)

用户、医生、患者、平台管理员、康复设备、设备数据、训练数据、康复处方、训练记录等

答案-问题(3)

层次型架构的优势如下：

(1) 在层次型架构中，各层次的开发人员可以专注于自己的领域，例如，前端开发人员主要关注用户界面和交互，后端开发人员则关注数据处理和业务逻辑。这种分工方式使得开发人员能够更加深入地理解某一层的技术细节，提高开发效率。

(2) 层次型架构通过定义清晰的接口，使得各层之间的依赖关系变得相对简单。只要保证前后端提供的服务(接口)一致，就可以轻松地替换某一层的实现。这种灵活性使得系统更加易于维护和升级。

(3) 在层次型架构中，每一层的组件都紧密相关，且专注于完成某一特定的功能或任务。这种内聚性使得组件之间的耦合度降低，提高了系统的可扩展性和可维护性。

(4) 由于层次型架构的清晰分层，某一层的组件可以很容易地被其他系统复用。例如，一个通用的数据访问层可以应用于多个不同的业务系统。这种复用性不仅提高了开发效率，还降低了系统的复杂性。

(5) 层次型架构的另一个优势是结构清晰，层次分明。这种结构使得系统更容易被理解和管理，降低了系统的复杂性和出错率。

(1) (b) 接入层

(2) (a) 治疗仪

(3) (h) 无线通信

(4) (c) Socket

(5) (i) 网络层

(2020年5月系统分析师真题-第5题)

阅读以下关于软件系统分析与设计的叙述，在答题纸上回答问题1至问题3。

某企业拟开发一套数据处理系统，在系统分析阶段，系统分析师整理的核心业务流程与需求如下：

(a) 系统分为管理员和用户两类角色，其中管理员主要进行用户注册与权限设置，用户主要完成业务功能；

(b) 系统支持用户上传多种类型的数据，主要包括图像、文本和二维曲线等；

(c) 数据上传完成后，用户需要对数据进行预处理操作，预处理操作包括图像增强、文本摘要，曲线平滑等；

(d) 预处理操作完成后，需要进一步对数据进行智能分析，智能分析操作包括图像分类、文本情感分析、曲线未来走势预测等；

(e) 上述预处理和智能分析操作的中间结果均需要进行保存；

(f) 用户可以将数据分析结果以图片、文本、二维图表等多种方式进行展示，并支持结果汇总，最终导出为符合某种格式的报告。

问题(1)

数据流图(Data Flow Diagram, DFD) 是一种重要的结构化系统分析方法，重点表达系统内数据的传递关系，并通过数据流描述系统功能。请用300字以内的文字说明DFD在进行系统需求分析过程中的主要作用。

思路解析

数据流图(DFD)是结构化分析方法中的一个重要工具，它通过图形化的方式描述系统的数据流动过程，是系统分析阶段的核心工具之一。DFD不仅帮助系统分析人员理解和表达系统的需求，还能够有效地与客户沟通需求，避免在开发过程中的需求误解。通过将系统的功能和数据流直观地展示出来，DFD帮助团队把复杂的需求抽象成易于理解的图示，特别是对非技术人员，DFD的简洁明了使其成为交流的重要桥梁。

DFD通过逐层细分的方式揭示系统各部分之间的关系，是系统设计的起点，后续的系统设计可以基于DFD的内容进行详细设计和开发。DFD作为一个文档，还可以在后期的开发和维护过程中作为存档材料，帮助开发团队进一步优化系统。

问题(2)

顶层图(也称作上下文数据流图)是描述系统最高层结构的DFD，它的特点是将整个待开发的系统表示为一个加工，将所有的外部实体和进出系统的数据流都画在一张图中。请参考题干描述，将合适的内容填入图2---1中(1) — (5)空白处，完成该系统的顶层图。

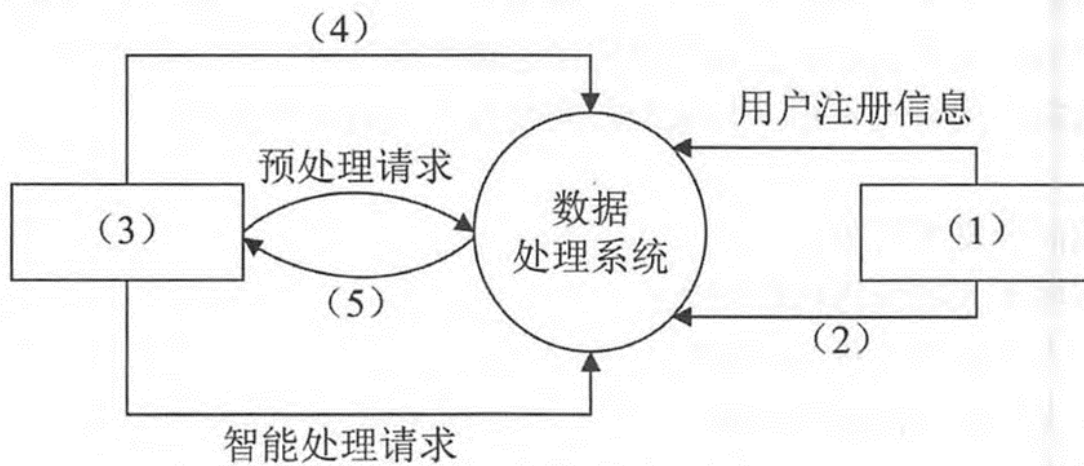


图 2-1 数据处理系统顶层图

思路解析

在DFD中，通常会出现4种基本符号，分别是数据流、加工、数据存储和外部实体（数据源及数据终点）。数据流是具有名字和流向的数据，在DFD中用标有名字的箭头表示。加工是对数据流的变换，一般用圆圈表示。数据存储是可访问的存储信息，一般用直线段表示。外部实体是位于被建模的系统之外的信息生产者或消费者，是不能由计算机处理的成分，它们分别表明数据处理过程的数据来源及数据去向，用标有名字的方框表示。

所以（1）（3）应填入外部实体名，（2）（3）（4）应填入加工名，根据（a）系统分为管理员和用户两类角色，其中管理员主要进行用户注册与权限设置，用户主要完成业务功能，可以得出

（1）为管理员（2）为用户权限信息。根据（b）系统支持用户上传多种类型的数据，主要包括图像、文本和二维曲线等，可以得出（4）为多种类型数据。根据（c）数据上传完成后，用户需要对数据进行预处理操作，预处理操作包括图像增强、文本摘要，曲线平滑等，可以得出（3）为用户。根据（f）用户可以将数据分析结果以图片、文本、二维图表等多种方式进行展示，并支持结果汇总，最终导出为符合某种格式的报告，可以得出（5）为导出报告/展示结果。

问题（3）

在结构化设计方法中，通常采用流程图表示某一处理过程，这种过程既可以是生产线上的工艺流程，也可以是完成一项任务必需的管理过程。而在面向对象的设计方法中，则主要采用活动图表示某个用例的工作流程。请用300字以内的文字说明流程图和活动图在表达业务流程时的三个主要不同点。

思路解析

流程图和活动图在表达业务流程时的不同，首先体现在关注点上。流程图着重描述业务操作的顺序和时间关系，主要表现为流程中的各个步骤如何衔接。而活动图更侧重表现系统内对象之间的交互，描述的是业务流程中的行为和活动的转换。其次，活动图具备并发执行的能力，能够表达多个并行进行的操作，而流程图则无法直接表达这种并发的概念。最后，活动图的多终止状态特点使得它能应对复杂的流程逻辑，允许多个不同的终点，而流程图则通常依赖于单一的流程结束点。

这使得活动图更适合面向对象的设计方法，特别是当系统涉及到多个并行处理和复杂交互时，活动图能够提供更清晰、更灵活的建模能力。

答案-问题（1）

DFD的主要作用如下：

- （1） DFD是理解和表达用户需求的工具，是需求分析的手段。
- （2） DFD概括地描述了系统的内部逻辑过程，是需求分析结果的表达工具，也是系统设计的重要参考资料，是系统设计的起点。
- （3） DFD作为一个存档的文字材料，是进一步修改和充实开发计划的依据。

答案-问题（2）

- （1） 管理员
- （2） 用户权限信息
- （3） 用户
- （4） 多种类型数据
- （5） 导出报告/展示结果

答案-问题（3）

流程图和活动图有如下三个主要区别：

- （1） 流程图着重描述处理过程，它的主要控制结构是顺序、分支和循环，各个处理过程之间有严格的顺序和时间关系。而活动图描述的是对象活动的顺序关系所遵循的规则，它着重表现的是系统的行为，而非系统的处理过程。
- （2） 流程图只能表达顺序执行过程，活动图则可以表达并发执行过程。
- （3） 活动图可以有多个结束状态，而流程图只能有一个结束状态。

（2020年5月系统分析师真题-第1题）

阅读下列说明，回答问题1至问题3，将解答填入答题纸的对应栏内。

某软件企业拟采用面向对象方法开发一套体育用品在线销售系统，在系统分析阶段，"提交订单"用例详细描述如表1-1所示。

表 1-1 提交订单用例表

用例名称	提交订单	
用例编号	SGS-RS01	
优先级	高	
主要参与者	注册会员	
其他参与者	商家、仓库、支付系统、快递公司	
前置条件	会员已成功登录系统	
触发器	会员选择商品加入购物车	
执行步骤	1. 会员选择商品并加入购物车； 3. 会员确认购物车商品类型及商品数量，并提交结算； 5. 用户提交订单； 7. 用户选择支付方式； 9. 用户提交支付申请；	2. 系统显示购物车已选商品列表； 4. 系统显示订单配送信息、订单商品列表及价格、订单总价； 6. 系统显示支付信息； 8. 用户输入支付密码； 10. 系统显示成功支付页面。
可选步骤	5A. 用户取消订单，用例结束。 9A. 用户放弃支付，转（3）。 10A. 系统显示未成功支付，转（6）。	
后置条件	商家通知仓库打包订单商品，并按照配送地址交付快递公司发货。	

问题（1）

面向对象系统开发中，实体对象、控制对象和接口对象的含义是什么？

思路解析

在面向对象开发中，根据对象的职责不同，我们将对象分为三类：

实体对象：主要用于表示系统中的实际数据，通常是业务领域中的核心数据。它们需要持久化存储，并且是系统运行时的基础。例如，在在线销售系统中，商品、订单、支付信息等都属于实体对象。

控制对象：负责处理系统中的业务逻辑和规则，通常用于调度实体对象的操作，并通过执行应用逻辑来完成业务需求。例如，订单处理过程中的状态转换、优惠计算等逻辑，通常由控制对象负责。

接口对象：用于表示用户与系统之间的交互，它负责接收用户的输入并反馈相应的输出，通常与前端交互有关，如用户界面、显示

问题（2）

面向对象系统分析与建模中，从潜在候选对象中筛选系统业务对象的原则有哪些？

思路解析

在面向对象的分析和建模中，正确识别和筛选出系统的业务对象是至关重要的。通过以下几个原则来筛选系统中的潜在候选对象：

去除具有相同含义的对象：多个候选对象可能有重复或相似的含义，应该去掉这些冗余对象，保留具有唯一意义的对象。

去除不属于本系统范围内的对象：有些候选对象可能与系统的实际功能无关，应该排除系统外部的对象，只保留与系统本身密切相关的对象。

去除没有特定独立行为的对象：如果某些对象只是数据容器或没有独立行为，无法独立执行操作，它们不应作为业务对象。

去除含义解释不清楚的对象：如果对象的定义不清晰，难以理解其功能和行为，它不应被纳入系

统的设计中。

去除属于另一个对象属性或行为的对象：有些对象实际上是属于其他对象的属性或行为，应该避免重复定义，确保每个对象在系统中有清晰且独立的角色。

问题（3）

根据题目所示"提交订单"用例详细描述，可以识别出哪些业务对象？

思路解析

在面向对象分析中，业务对象是系统中用来处理数据和实现功能的核心实体。根据"提交订单"用例的描述，以下是识别出的业务对象：

会员：指代注册的用户或消费者，会员拥有订单提交、购物等行为，并且是系统的重要组成部分。

商品：指代在线销售平台中的销售品，用户通过购物车选择商品并进行订单提交。

购物车：是用户选购商品的容器，用户将商品添加到购物车后，最终提交为订单。

订单：是用户提交的请求，包含订单的详细信息，如商品、数量、总价等，属于系统的核心对象。

配送信息：指代与订单相关的配送地址和配送方式等信息。

支付记录：记录了用户支付订单时的支付方式、支付状态等信息。

答案-问题（1）

- （1）实体对象：用来表示业务域的事实数据并需要持久化存储的对象类型；
- （2）控制对象：用来表示业务系统中应用逻辑和业务规则的对象类型；
- （3）接口对象：用来表示用户与系统之间交互方式的对象类型。

答案-问题（2）

- （1）去除具有相同含义的对象；
- （2）去除不属于本系统范围内的对象；
- （3）去除没有特定独立行为的对象；
- （4）去除含义解释不清楚的对象；
- （5）去除属于另一个对象属性或行为的对象。

答案-问题（3）

会员、商品、购物车、订单、配送信息、支付记录

（2018年5月系统分析师真题-第1题）

阅读以下关于系统分析任务的叙述，在答题纸上回答问题1至问题3。

某公司是一家以运动健身器材销售为主营业务的企业，为了扩展销售渠道，解决原销售系统存在的许多问题，公司委托某软件企业开发一套运动健身器材在线销售系统。目前，新系统开发处于问题分析阶段，所分析各项内容如下所述：

- （a）用户需要用键盘输入复杂且存在重复的商品信息；

- (b) 订单信息页面自动获取商品信息并填充;
- (c) 商品订单需要远程访问库存数据并打印提货单;
- (d) 自动生成电子提货单并发送给仓库系统;
- (e) 商品编码应与原系统商品编码保持一致;
- (f) 商品订单处理速度太慢;
- (g) 订单处理的平均时间减少30%;
- (h) 数据编辑服务器CPU性能较低;
- (i) 系统运维人员数量不能增加。

问题（1）

问题分析阶段主要完成对项目开发的问题、机会和/或指示的更全面的理解。请说明系统分析师在问题分析阶段通常需要完成哪四项主要任务。

思路解析

在系统分析阶段，系统分析师的目标是深入了解当前系统的现状，识别出存在的问题以及改进的机会，从而为系统的设计和 implement 阶段打下坚实的基础。具体任务可以分为以下几个主要部分：

- 研究问题领域：系统分析师需要全面了解并定义问题领域，这意味着要分析与系统相关的所有数据、过程和接口。数据部分包括当前系统使用的所有数据存储，如数据库、文件等。过程部分需要识别出当前系统中的业务响应和处理过程，尤其是关键的业务事件。接口部分则要明确系统中各个地点和用户之间的交互。
- 分析问题和机会：通过对系统的现状进行深入分析，找出当前系统中存在的问题，例如性能瓶颈、功能缺失、用户体验差等。同时，还需要挖掘出可能的改进机会，比如优化现有功能、引入新技术或流程等。这一环节对于后续设计方案至关重要，只有深入了解问题和机会，才能找到更有效的解决方法。
- 分析业务过程：业务过程分析通常是可选的，但对于复杂系统来说，它帮助系统分析师识别出哪些业务流程可能会影响系统功能。通过分析业务流程，分析师能够更好地了解用户需求，并确保系统设计能有效支持这些流程。
- 制定系统改进目标：在问题分析的基础上，系统分析师需要明确系统改进的目标。这些目标应该是针对已识别问题和机会的解决方案，并应具备可衡量性，例如提升系统响应速度、降低操作复杂度、增加用户满意度等。

问题（2）

因果分析是问题分析阶段一项重要技术，可以得出对系统问题的真正理解，并且有助于得到更具有创造性和价值的方案。请将题目中所列（a） - （i）各项内容填入表中（1） - （4）对应位置。

因果分析		系统改进目标	
问题或机会	原因和结果	系统目标	系统约束条件
(1)	(2)	(3)	(4)

思路解析

因果分析是一种技术，通过分析问题、原因、目标和约束的关系，可以帮助分析师更清晰地识别出系统的根本问题，并设计出更合适的解决方案。在此题中，我们需要将问题、原因、改进目标和约束分别填入表格中。

系统真正的问题（列（1））：通过分析系统中的问题，最核心的问题是** “商品订单处理速

度太慢” **。这个问题会直接影响到用户体验和系统效率，因此需要在改进过程中解决。

问题产生的原因（列（2））：从题目中描述的情况来看，导致订单处理速度慢的原因包括：用户输入复杂且重复的商品信息（a）。商品订单需要远程访问库存数据并打印提货单（c）。数据编辑服务器CPU性能较低（h）。这些因素共同导致了系统的低效运行。

用户输入复杂且重复的商品信息（a）。

商品订单需要远程访问库存数据并打印提货单（c）。

数据编辑服务器CPU性能较低（h）。这些因素共同导致了系统的低效运行。

系统改进目标（列（3））：为了改善系统性能，目标应该是：自动化订单信息获取（b）。自动生成电子提货单并发送给仓库系统（d）。提高订单处理速度，减少处理时间（g）。

自动化订单信息获取（b）。

自动生成电子提货单并发送给仓库系统（d）。

提高订单处理速度，减少处理时间（g）。

设计约束（列（4））：设计约束通常是系统在设计 and 实施时必须遵循的条件，且不能轻易改变。此系统的约束包括：商品编码应与原系统商品编码保持一致（e）。系统运维人员数量不能增加（i）。

商品编码应与原系统商品编码保持一致（e）。

系统运维人员数量不能增加（i）。

问题（3）

系统约束条件可以分为四类，请将类别名称填入表中（1） - （4）对应的位置。

约束条件	类别
新系统必须在五月底上线运行	(1)
新系统开发费用不超过 20 万元	(2)
新系统必须能够实现在线实时处理	(3)
新系统必须满足 GB31524-2005 电商平台技术规范	(4)

思路解析

系统约束是指在系统开发过程中必须遵守的一些条件或限制，这些条件通常无法改变或非常困难改变。约束条件可能来源于不同方面，常见的约束条件分为四类：进度、成本、技术、政策/标准。理解这些约束可以帮助项目团队在设计和开发过程中做出合理的决策。

进度约束（（1））：进度约束通常指的是项目必须在规定的时间内完成。进度约束往往影响项目的各个方面，例如资源分配、工作分配和任务优先级。进度是项目管理中非常关键的一部分，特别是在大规模软件开发过程中，进度的控制关系到项目是否能够按时交付。

成本约束（（2））：成本约束涉及到项目预算的限制。每个项目都有预设的预算，必须在预算范围内完成开发工作。成本约束要求开发团队在有限的资金下，尽可能高效地开发系统，并控制开销。

技术约束（（3））：技术约束涉及到开发中使用的技术栈、工具、平台等。某些项目可能需要在特定的技术框架或硬件平台上进行开发，或者受限于现有的技术能力，无法使用最先进的技术。技术约束通常会影响系统的设计和开发方向。

政策/标准约束（（4））：政策和标准约束包括行业标准、公司政策、法规要求等。这些约束通常是不可避免的，必须遵守相应的政策和标准才能确保系统的合规性。例如，涉及到数据隐私保护的项目可能需要遵守GDPR等法律要求。

答案-问题（1）

问题分析阶段的四项主要任务包括：

- 1) 研究问题领域
- 2) 分析问题和机会
- 3) 分析业务过程
- 4) 制定系统改进目标

答案-问题（2）

- (1) (f)
- (2) (a) (c) (h)
- (3) (b) (d) (g)
- (4) (e) (i)

答案-问题（3）

- (1) 进度
- (2) 成本
- (3) 技术
- (4) 政策/标准

(2018年5月系统分析师真题-第2题)

阅读以下关于系统分析设计的叙述，在答题纸上回答问题1至问题3。

某软件公司为共享单车租赁公司开发一套单车租赁服务系统，公司项目组对此待开发项目进行了分析，具体描述如下：

- 1) 用户（非注册用户）通过手机向租赁服务系统进行注册，成为可租赁共享单车的合法用户，其中包括提供身份、手机号等信息，并支付约定押金；
- 2) 将采购的共享单车注册到租赁服务系统后方可投入使用。即将单车的标识信息（车辆编号、二维码等）录入到系统；
- 3) 用户（注册或非注册用户）通过手机查询可获得单车的地理位置信息以便就近取用；
- 4) 用户（注册用户）通过手机登录到租赁服务系统中，通过扫描二维码或输入车辆编号以进行系统确认，系统后台对指定车辆状态（可用或不可用），以及用户资格进行确认，通过确认后对车辆下达解锁指令；
- 5) 用户在使用完车辆后关闭车锁，车辆自身将闭锁状态上报到租赁服务系统中，完成车辆状态的更新和用户租赁费用结算；
- 6) 系统应具备一定的扩容能力，以满足未来市场规模扩张的需要。

项目组李工认为该系统功能相对独立，系统可分解为不同的独立功能模块，适合采用结构化分析与设计方法对系统进行分析与设计。但王工认为，系统可管理的对象明确，而且项目团队具有较强的面向对象系统开发经验，建议采用面向对象分析与设计方法。经项目组讨论，决定采用王工的建议，采用面向对象分析与设计方法开发系统。

问题（1）

在系统分析阶段，结构化分析和面向对象分析方法主要分析过程和分析模型均有所区别，请将

(a)~(g) 各项内容填入表2-1 (1)~(4) 处对应位置。

系统分析方法	主要分析内容	分析结果呈现形式
结构化分析方法	(1)	(2)
面向对象分析方法	(3)	(4)

- (a) 确定目标系统概念类；
- (b) 实体关系图（ERD）；
- (c) 用例图；
- (d) 通过功能分解方式把系统功能分解到各个模块中；
- (e) 交互图；
- (f) 数据流图（DFD）；
- (g) 建立类间交互关系。

思路解析

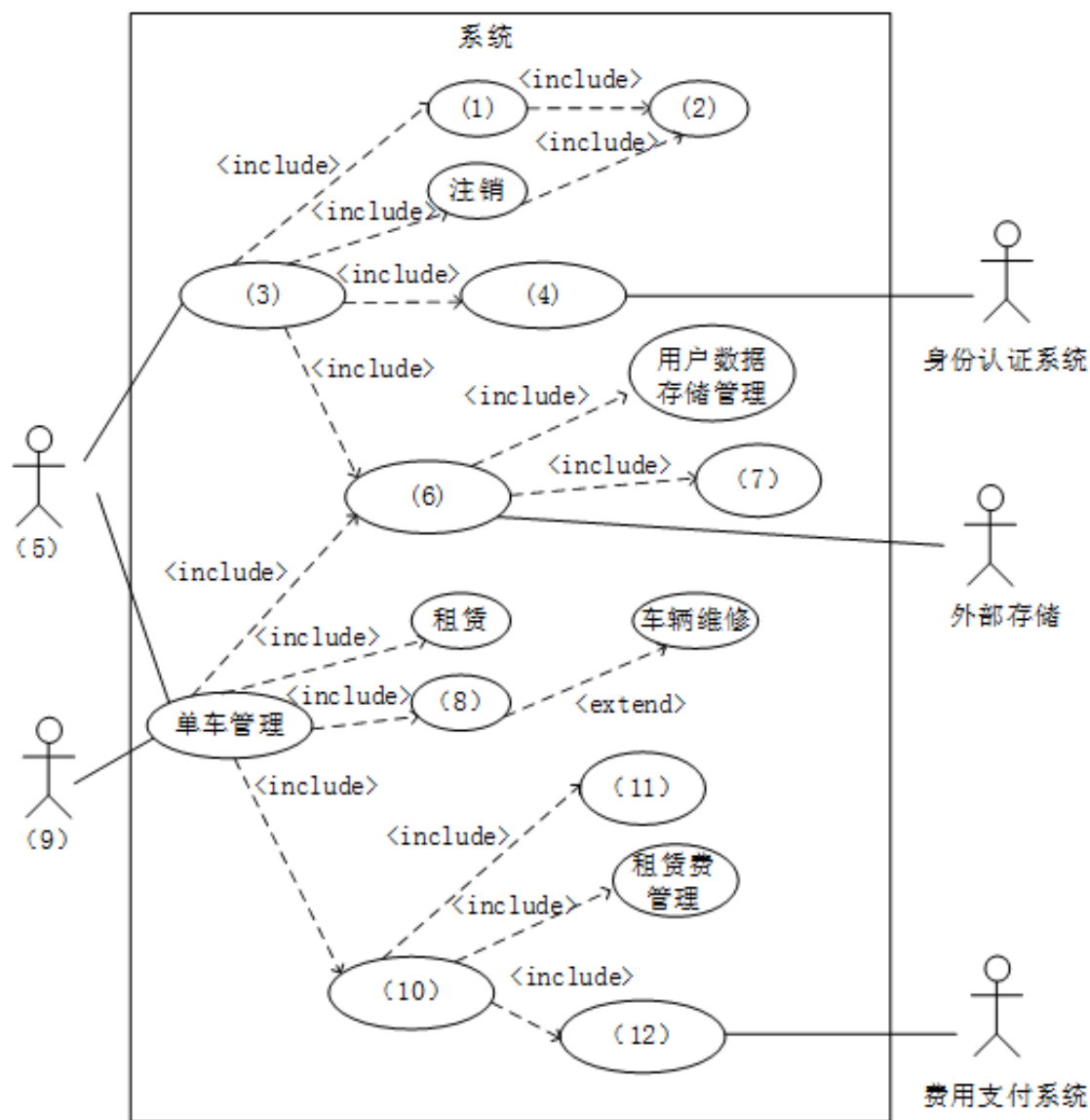
结构化分析（SA）主要分析内容是对系统进行模块划分，并把识别出来的功能分配到各模块中，通过细分的模块功能来达到系统整体功能的目的。结构化分析中以数据流图DFD表示模块间数据的交互关系，以实体关系图ERD表示数据模型，以及状态转换图STD表示行为模型。

面向对象分析（OOA）主要分析内容是对系统进行概念类定义，确定类与类之间的关系，以及确定类的职责，建立交互图等，从而对系统功能进行完整描述。在面向对象分析中，以用例图来表示概念类之间的关系，以交互图来表示相关对象之间的行为。

问题（2）

请分析下面A~Q所列出的共享单车租赁服务系统中的概念类及其方法，在图2-1所示用例图（1）~（12）处补充所缺失信息。

A. 用户，B. 共享单车，C. 用户管理，D. 注册，E. 注销，F. 用户查询，G. 单车管理，H. 租赁，I. 归还，J. 单车查询，K. 费用管理，L. 保证金管理，M. 租赁费管理，N. 数据存储管理，O. 用户数据存储管理，P. 单车数据存储管理，Q. 费用结算，R. 身份认证



思路解析

这道题目要求根据共享单车租赁服务系统的用例图补充完整缺失的信息。首先，需要理解系统中的各个功能模块及其对应的概念类。这些模块可以是对系统中不同功能的封装，如用户管理、单车管理、费用管理等，每个功能模块对应着系统的一部分关键行为。

注册（D）：用户通过注册成为系统的一员，注册是租赁流程的起始点，所有用户都必须进行注册。

用户查询（F）：提供用户查看信息的功能，帮助用户查询自己的租赁信息或单车状况等。

用户管理（C）：涉及对用户信息的管理，包括注册、修改、查询用户数据等操作。

身份认证（R）：在用户注册和登录时，系统需要对用户身份进行验证。

用户（A）：代表共享单车系统中的用户，用户是系统的重要组成部分，系统的核心对象之一。

数据存储管理（N）：负责管理系统中的数据存储，包括用户信息、单车信息、租赁记录等。

单车数据存储管理（P）：负责管理单车的注册信息、状态信息等。

归还（I）：用户使用完单车后将其归还，系统根据归还信息更新车辆状态和租赁记录。

共享单车（B）：代表单车的基本实体，所有共享单车的基本信息和状态都存储在系统中。

费用管理（K）：负责管理用户的租赁费用，包括租赁价格、计费方式等。

保证金管理（L）：用户在注册时支付的保证金的管理。

费用结算（Q）：在用户归还单车后，系统进行租赁费用结算的操作。

问题（3）

随着共享单车投放量以及用户量的增加会存在系统性能或容量下降问题，请用200字以内的文字说明，在系统设计之初，如何考虑此类问题？

思路解析

随着共享单车系统的规模扩展，系统的性能和容量是重要考虑因素。解决这些问题的关键在于通过扩展硬件和优化架构设计。数据存储、服务器性能、以及网络通信能力是核心瓶颈，需要通过灵活的架构设计，如分布式存储、负载均衡和可扩展性设计来满足不断增长的需求。这样可以保证系统能够稳定处理更多用户和设备的请求，并支持未来的扩展。

答案-问题（1）

- （1）（d）
- （2）（b）（f）
- （3）（a）（g）
- （4）（c）（e）

答案-问题（2）

- （1）D：注册
- （2）F：用户查询
- （3）C：用户管理
- （4）R：身份认证
- （5）A：用户
- （6）N：数据存储管理
- （7）P：单车数据存储管理
- （8）I：归还
- （9）B：共享单车
- （10）K：费用管理
- （11）L：保证金管理
- （12）Q：费用结算

答案-问题（3）

1. 数据存储容量。采用独立存储系统如阵列，对于数据存储可灵活扩展。
2. 服务器处理性能
考虑可扩展性问题，利用集群，扩展时采用水平扩展方式。
利用分布式存储方式，将各个城市的数据分散存储，减少压力，提升处理性能。
利用负载均衡技术，解决高并发问题。
3. 通信性能。服务器网口提速，对服务器接口进行灵活扩展如采用端口聚合等。

阅读以下关于系统设计的叙述，在答题纸上回答问题1至问题3。

某软件公司为电子商务企业开发一套网上交易订单管理系统，以提升服务的质量和效率。在项目之初，项目组决定采用面向对象的开发方法进行系统开发，并对系统的核心业务功能进行了分析，具体描述如下：

注册用户通过商品信息页面在线浏览商品，将需要购买的商品添加进购物车内，点击"结算"按钮后开始录入订单信息。

用户在订单信息录入页面上选择支付方式，填写并确认收货人、收货地址和联系方式等信息。点击"提交订单"按钮后产生订单，并开始进行订单结算。

订单需要在30分钟内进行支付，否则会自动取消，用户也可以手工取消订单。

用户支付完成，经确认后，系统开始备货，扣除该商品可接单数量，并移除用户购物车中的所有商品资料。

生成订单表单，出货完毕，订单生效。为用户快递商品，等待用户接收。

用户签收商品，交易完成。

问题（1）

识别设计类是面向对象设计过程中的重要工作，设计类表达了类的职责，即该类所担任的任务。请用300字以内的文字说明设计类通常分为哪三种类型，每种类型的主要职责，并针对题干描述案例涉及的具体类为每种类型的设计类举出2个实例。

思路解析

面向对象分析设计中，实体/边界/控制（EBC）三分法有助于将领域模型与用例行为解耦，实现高内聚、低耦合。

实体类映射稳定的业务名词，负责数据持久化与不变式（如订单必须含至少一条订单项、库存数量不得为负），常与仓储/ORM协作；它们变化频率相对较低例如，用户、商品、购物车等。

。边界类面向系统边界，负责交互协议与数据适配，如 Web 界面、REST 网关、支付回调接口等；边界变化快，但不应包含核心业务规则，仅做输入校验与数据组装。例如订单表单等。

控制类聚合用例步骤，负责事务边界、流程编排、领域服务调用与异常补偿，例如“结算→创建订单→调用支付→根据回调更新状态→触发备货”。

问题（2）

在面向对象的设计过程中，活动图（activity diagram）阐明了业务用例实现的工作流程。请用300字以内的文字给出活动图与流程图（flow chart）的三个主要区别。

思路解析

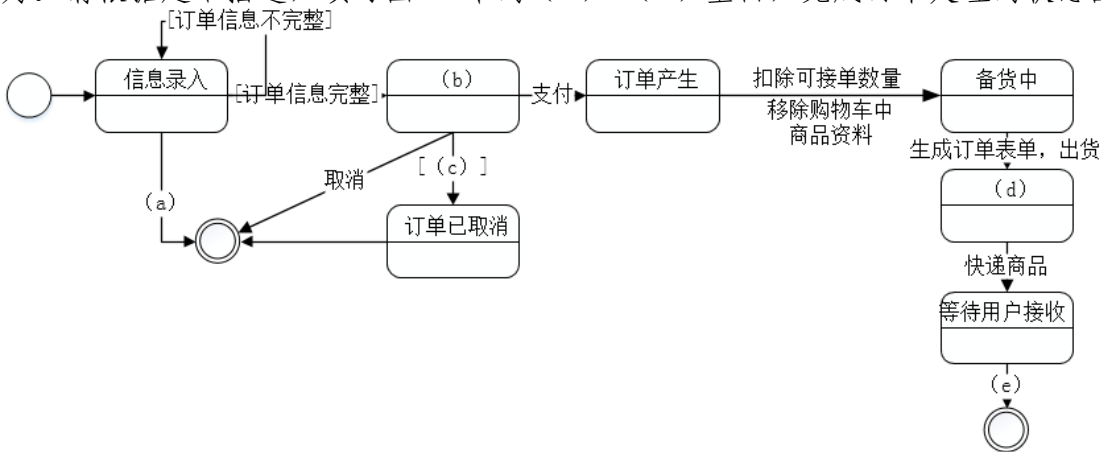
两者均用于刻画流程，但语义与建模场景不同。

活动图继承自 UML，具备令牌流语义，可使用决定/合并、分叉/汇合、泳道、对象节点、输入输出引脚、信号发送/接收等元素描述跨对象、跨角色的活动协作，并可标注守卫条件与并发，适合表达用例实现、业务流程与服务编排。其语义与状态机/时序图可互补，用于从行为角度校验需求。

流程图则侧重算法或程序控制结构的线性/分支/循环表示，符号体系更简洁，缺少对对象责任划分、对象流与并发同步的原生支持，多用于局部过程或教学场景。在电商订单示例中，“结算→创建订单→支付→备货/出库并发→汇合→配送”更适合用活动图表达，其泳道可区分“用户、订单服务、库存服务、物流服务”，而传统流程图虽能画出步骤，但难以准确体现并发与对象间协作。

问题（3）

在面向对象的设计过程中，状态图（statechart diagram）描述了一个实体基于事件反应的动态行为。请根据题干描述，填写图2-1中的（a）-（e）空白，完成订单处理的状态图。



思路解析

本题考查将业务规则映射为订单状态机的能力。依据题干：购物车进入结算后，订单尚未创建时可标注为**（b）待结算**；提交后创建待支付订单。若用户主动触发或系统定时任务在**（c）大于30分钟未支付，则转入（a）取消**；取消需要触发库存释放与购物车恢复（如适用）等补偿。支付完成并经第三方回调确认，系统进入备货/出库流程，完成后订单达到**（d）订单生效**，此时应保证订单与库存一致并生成配送单。物流妥投后，用户完成**（e）用户签收**，状态转为已完成。该状态机需具备外部事件驱动（支付回调、签收确认）、计时器/超时事件（30分钟）与异常路径（取消/失败重试）等机制，才能在实现上支持一致性、可补偿与可观测性；同时应限制从“生效/已完成”回退的非法转换，确保状态闭环。

答案-问题（1）

- （1）实体类。实体类映射需求中的每个实体，保存需要存储在永久存储体中的信息，例如，用户、商品、购物车等。
- （2）控制类。控制类是用于控制用例工作的类，用于对一个或几个用例所特有的控制行为进行建模。例如，结算、备货等。
- （3）边界类。边界类用于封装在用例内、外流动的信息或数据流。例如订单表单等。

答案-问题（2）

- （1）活动图描述的是对象活动的顺序关系所遵循的规则，它着重表现系统的行为，而非处理过程；而流程图着重描述处理过程。
- （2）流程图一般都限于顺序进程，而活动图则可以支持并发进程。
- （3）活动图是面向对象的，而流程图是面向过程的。

答案-问题（3）

- （a）取消
- （b）待结算
- （c）大于30分钟
- （d）订单生效

(e) 用户签收

(2015年5月系统分析师真题-第2题)

阅读以下关于系统业务架构分析的叙述，在答题纸上回答问题1至问题3。

某集团下属煤矿企业委托软件公司开发一套煤炭运销管理系统，该系统属于整个集团企业信息化架构中的业务层，系统针对煤矿企业开发，包括合同管理、磅房管理、质检化验、运费结算等功能。部分业务详细描述如下：

(1) 合同管理：合同签订、合同查询、合同跟踪等。

(2) 磅房管理：系统可以从所有类型的电子磅自动读数；可以自动从电子磅上读取车辆皮重、毛重，计算出净重；可根据合同内容自动减少相应提货单剩余数量，如果实际发货量超过合同额则拒绝发货。

(3) 质检化验：根据过磅单、车号，生成化验分析委托单，生成化验分析报告。

(4) 运费结算：依据过磅单上的净重、化验单、合同规定，自动计算出原料结算单、运费结算单。

煤矿企业根据集团的工作计划制订本企业的业务计划，煤矿企业根据集团划拨指标和提供的原料生产煤炭，所生产的煤炭交由集团统一管理和销售给客户。软件公司采用Zachman框架对企业业务架构和业务过程进行分析，结果如表2-1所示。

	(a)	(b)	(c)	(d)	时间	(e)
目标范围	A11	A12	A13	计划部、财务部、运销部	A15	A16
企业模型	A21	A22	A23	A24	A25	企业业务计划
系统模型	A31	A32	A33	合同界面、过磅界面、质检界面……	企业计划处理结构	A36
技术模型	A41	系统层、数据层、功能层、决策层	系统架构、软硬件配置	A44	A45	A46
详细展现	数字定义 Car、User……	A52	A53	A54	A55	程序逻辑规格说明
功能系统	A61	A62	A63	A64	A65	A66

问题（1）

Zachman框架是什么？请在表2-1中（a）-（e）位置补充企业业务架构中的信息类别。

思路解析

Zachman 框架为企业架构提供了一个结构化的思维工具。它将复杂的企业信息系统分解为“六大关注点”和“六种视角”。六大关注点包括：

数据（What）：系统要处理的核心数据与信息对象；

功能（How）：系统如何实现业务目标；

网络（Where）：涉及的分布、节点与通信关系；

角色（Who）：系统中承担任务的人员或组织；

时间（When）：业务事件的时间约束与流程计划；

动机（Why）：业务目标、策略与规则。

六种视角则从规划者、所有者、设计者、构建者、分包商、运行系统等不同立场出发，形成一个 6×6 的矩阵。该矩阵既是知识分类法，又是文档索引工具，能够保证企业架构建模时不遗漏关键要素。

在本案例中，（a）（e）恰好对应六大关注点中的五个，说明该系统业务架构分析基于 Zachman

的标准框架展开。

问题（2）

项目组在该煤炭企业业务架构分析中完成了四项主要工作：数据流图、实体联系图、网络拓扑结构和计划时间表，这四项工作在表2-1中处于什么位置，请用表2-1中的位置编号表示。

思路解析

在 Zachman 框架中，每个单元格代表了某一视角下的某一关注点描述。根据题干给出的四项工作：

数据流图（DFD）：属于 功能视角（How），反映逻辑流程与控制流，且位于 设计者层次 → 对应 A32。

实体联系图（ER 图）：关注数据对象及关系，属于 数据视角（What），位于 设计者层次 → 对应 A31。

网络拓扑结构：关注分布节点与网络连接，属于 网络视角（Where），位于 构建者层次 → 对应 A53。

计划时间表：强调时间约束与任务安排，属于 时间视角（When），位于 所有者层次 → 对应 A25。

问题（3）

根据题目所述业务描述，请分别给出表2-1中A11和A23位置应该填入的内容。（物流关系用"→"表示）

思路解析

在 Zachman 框架中：

A11（规划者 × 数据）：要求列出对企业最重要的数据元素。根据题干描述，煤炭运销的关键文档与单据包括：合同（业务起点）、提货单（控制发货量）、化验委托单（质检流程）、原料结算单和运费结算单（结算结果）。这些构成了系统的核心数据对象，直接支撑合同履约与财务结算。

A23（所有者 × 网络）：要求描述企业业务层面的网络分布。在煤炭运销中，物流是关键环节，应抽象为 节点（如煤矿、磅房、质检中心、客户交付点）与链接（运输路径），从而体现煤炭运输与交付的物理网络关系。

答案-问题（1）

Zachman框架是由约翰扎科曼（John Zachman ）在1987年创立的全球第一个企业架构理论。Zachman框架是一种组织构架工具（用来设计文档、需求说明和模型的工具）的一种分类学。包括工具的目标（例如，商业拥有者、创建者）是谁，哪些特殊的问题（例如，数据、功能）需要阐明。

（a）数据 （b）功能 （c）网络 （d）角色 （e）动机

答案-问题（2）

数据流图： A32，实体联系图： A31，网络拓扑结构： A53，计划时间表： A25

答案-问题（3）

A11: 列出对业务至关重要的元素: 合同、提货单、委托单、原料结算单、运费结算单

A23: 物流网络(节点和链接)

(2013年5月系统分析师真题-第1题)

某软件公司拟为物流企业开发一套库存管理系统, 该系统的部分需求陈述如下:

(1) 库存管理系统主要包括货物入库管理、货物出库管理、仓库管理、统计报表和系统管理等功能;

(2) 库存管理系统的用户包括仓库管理员、仓库经理和系统管理员, 用户必须在注册后才能使用系统功能; 用户可以选择使用邮件注册或电话注册。

(3) 仓库管理员在进行出入库操作前必须先登录; 仓库经理可以通过系统查看统计报表, 如果前一个月的报表未生成, 则系统自动生成统计报表, 否则直接显示。

(4) 系统管理员可以在系统中设置仓库温度范围, 当仓库内温度超过最高值或者低于最低值时, 系统自动调用温控管理操作, 连接温度调节系统进行制冷或加热。

(5) 仓库管理功能要求每个月1日零点对前一个月货物入库和出库记录进行数据汇总操作。项目组决定构造用例模型以描述系统需求。

问题(1)

用例建模的首要任务是识别系统中的参与者。请根据题目中所描述的需求, 识别出系统中有哪些参与者?

思路解析

在用例建模中, 参与者(Actor)指与系统发生交互、对系统行为产生影响或从系统获得服务的外部实体, 既可以是人, 也可以是外部系统或装置, 甚至可以是时间事件。依据题干: 第一, 仓库管理员、仓库经理、系统管理员与系统直接交互, 发起登录、查看报表、配置阈值等, 属于主要人类参与者; 第二, 系统需在每月 1 日 00:00 自动汇总上月数据, 这一触发源并非某个用户主动点击, 而是时间事件/计时服务对系统提出的触发, 建模时常被抽象为参与者“时间”, 用以清晰表达系统边界与外部驱动; 第三, 仓库温度由温度监测系统/传感器产生并上报, 当温度越界时系统需联动设备进行制冷/加热, 因此温度监测系统(或以“温度”抽象表示)与温度调节系统均应辨识为外部系统型参与者。在考试场景中, 是否采用“温度”这一名词直接作为参与者名称, 并非关键, 关键在于体现**“温度读数来自系统边界之外的外部信号/系统”这一事实。因此, 将其规范命名为“温度监测系统(传感器)”更专业; 同理, 执行制冷/加热的设备群体宜命名为“温度调节系统”。通过上述识别, 能够完整覆盖题干涉及的人工角色、时间触发、设备联动三类交互主体, 体现用例模型对系统边界与外部交互的清晰刻画。这种做法不仅有助于后续用例关系与时序场景的描述, 也为与硬件/第三方系统确定接口契约与通信协议**奠定基础。

问题(2)

用例建模的主要工作是书写用例规约。用例规约通常包括哪几部分内容?

思路解析

完善的用例规约应围绕意图—边界—流程—结果四个层面展开。首先, 用例名要具体、面向业务价值, 如“生成上月统计报表”; 参与者明确谁发起、谁协作; 用例目标强调业务价值与成功判据

，使开发与测试对“完成”的判定一致。其次，前置条件限定进入用例的状态前提（如用户已登录、阈值已配置）；触发条件刻画何时/何因启动（如时间到达每月 1 日 00:00，或经理点击“查看报表”），与前置条件共同定义用例入口门槛。再次，基本事件流以**“正常成功路径”为核心，逐步陈述参与者与系统的交互步骤；扩展/备选事件流用于表达分支、异常与回退，如“上月报表不存在则自动生成”“温度采集失败则重试/告警”等，确保覆盖主干+变体**。最后，后置条件定义用例完成后的可观测状态变化（数据持久化、报表已生成、设备指令已下发），便于测试断言。为便于验收，还可记录业务规则与约束（如报表粒度、温度阈值计算规则）、非功能需求（性能、可用性、可审计性）、优先级与频次（有助于迭代排期与容量评估）、异常处理与日志/审计要求。

问题（3）

建立了用例模型后，可以利用用例之间的关系调整用例模型，用例之间的关系包括哪几种？对于每种关系，请根据题目中所描述的需求分别给出一组用例。

思路解析

包含（include）用于表达被多个用例复用且不可或缺的子功能，体现强复用与必经路径。在本题中，“登录”显然是“入库”“出库”等一线操作的共同前提，适合被建模为被包含用例，从而避免在多个用例的基本流中重复描述认证步骤，同时也便于今后将登录扩展为多因子认证或单点登录时实现“一处变更，多处受益”。

扩展（extend）强调在特定条件下对基础用例的可选增强，更贴近条件分支与可插拔能力。题干要求“若上月报表未生成则自动生成”，此处“查看统计报表”是经理的主意图；是否需要“生成”取决于运行时条件（报表存在性），因此将“生成统计报表”建模为扩展用例最贴切：在主用例定义扩展点（上月报表不存在），当条件满足时切入执行，完成后回到主用例继续展示。

泛化（generalization）反映行为的继承与特化关系：父用例描述通用流程与约束，子用例在此基础上补充或替换差异步骤。题中“用户注册”是通用过程（采集身份信息、校验唯一性、创建账号、发送激活信息等），而“电话注册”“邮件注册”在“验证码发送与校验”“联系方式介质”“反垃圾策略”等步骤存在差异，适合作为并列的子用例从父用例泛化而来。

若在考试中需要画用例图，可将“用户注册”为父用例，连出“邮件注册”“电话注册”的泛化箭头；将“查看统计报表”指向“生成统计报表”的扩展关系（带条件注释“上月报表不存在”）；将“入库”“出库”指向“登录”的包含关系；并将时间（计时服务）、温度监测系统、温度调节系统作为外部参与者放在系统边界之外。

答案-问题（1）

用例模型的参与者：仓库管理员、仓库经理、系统管理员、时间、温度、温度调节系统。

答案-问题（2）

用例建模的主要工作是书写用例规约（use case specification），而不是画图。用例模板为一个给定项目的所有人员定义了用例规约的结果，其内容至少包括用例名、参与者、目标、前置条件、事件流（基本事件流和扩展事件流）和后置条件等，其他的还可以包括非功能需求和用例优先级等。

答案-问题（3）

(1) 用例之间的关系包括：包含关系、扩展关系、泛化关系。

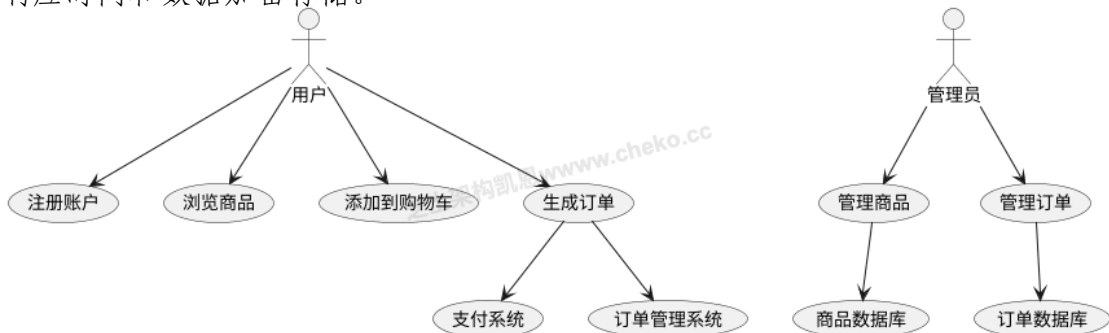
(2) "出入库操作"与"登录"属于包含关系；

"查看统计报表"与"生成统计报表"属于扩展关系；"用户注册"与"电话注册"、"邮件注册"与"电话注册"属于典型的泛化关系。

(2024年5月系统分析师真题-第1题)

某电商平台公司正在进行一项新项目的系统设计，该平台将为用户提供在线购物、订单管理、支付处理以及用户管理等核心功能。开发团队已通过与产品经理的讨论确定了系统需求，并进行了初步的需求分析。为了确保系统能够满足业务需求，同时具备较高的可用性、性能和可扩展性，团队正在进行详细的需求获取与系统设计工作。

该系统的主要目标包括支持多种操作系统（如Windows和Linux）、保证用户数据的安全性、确保高并发请求处理以及支持动态扩展等。系统还将根据用户角色提供不同的访问权限，同时保证较短的响应时间和数据加密存储。



问题（1）

请写出 6 种需求获取技术。

思路解析

需求获取技术包括了多种方法，每种方法适用于不同的场景和需求类型。用户访谈通过与用户的互动获取需求信息，但需要良好的沟通技巧。问卷调查则适合大规模收集意见，但有时可能存在样本偏差问题。采样是通过选择代表性用户群体进行调查，可以提高效率。情节串联板有助于通过视觉手段促进对需求的理解。联合需求计划强调开发者与客户之间的紧密合作，特别是对于存在歧义的需求。最后，其他方法包括从现有文档或实践中提取信息，能够帮助团队了解已有系统或用户的实际操作。

问题（2）

根据用例图和顺序图（暂缺），举例说明用例描述参与者前置条件和后置条件。

思路解析

用例描述通常包括多个部分，其中最重要的部分之一就是前置条件和后置条件。前置条件确保在执行用例之前，系统已经满足了某些关键状态和条件。如果前置条件没有满足，系统可能无法执行该用例。后置条件则说明用例执行后的结果，确保系统的状态符合预期。例如，用户需要先登录才能下订单，因此“用户已登录”是一个前置条件；订单生成后，系统会发送邮件通知用户，这就是该用例的后置条件。

完整用例描述如下所示：

- (1) 用例名称。用例名称应该与用例图相符，并写上其相应的编号。
 - (2) 简要说明。对用例为参与者所传递的价值结果进行描述，应注意语言简要，使用用户能够阅读的自然语言。
 - (3) 事件流。事件流是指当参与者和系统试图达到一个目标时所发生的一系列活动，
 - (4) 非功能需求。因为用例所涉及的非功能需求通常很难在事件流中进行表达，因此单列为一小节进行描述。在非功能需求的描述方面，一定要注意使其可度量和可验证。否则，就容易流于形式，形同摆设。
 - (5) 前置条件和后置条件。前置条件是执行用例之前必须存在的系统状态，如果前置条件不满足，则用例无法启动。后置条件是用例执行完毕系统可能处于的一组状态。
 - (6) 扩展点。如果包括扩展（或包含）用例，则写出扩展（或包含）用例名，并说明在什么情况下使用。
 - (7) 优先级。说明用户对该用例的期望值，为以后的开发工作确定先后顺序。
- 此题让你填写前置条件和成功条件。前置条件比较好理解，做这个事情要符合哪些基本要求。后置条件要看流程结束之后会带来哪些状态。

问题（3）

请简述需求获取用例的三类对象分别是什么。

思路解析

在需求获取的用例分析中，参与者是系统外部的角色，他们通过与系统的交互来触发用例的执行。用例是系统为参与者提供的具体服务或功能，是系统操作的核心部分。系统则是执行用例的环境，负责对参与者的请求做出响应，并完成指定任务。在需求分析时，理解这三类对象有助于清晰地定义系统与外部世界的交互关系，明确系统需要提供的功能和服务。

答案-问题（1）

用户访谈：通过与用户的面对面或远程访谈，获取用户的需求。这种方法可以采取结构化或非结构化访谈，旨在从用户的视角了解他们对系统的需求和期望。

问卷调查：通过设计问卷收集大量用户的意见和反馈。问卷调查适合于对大量用户进行调查，但需要精心设计，避免出现偏差。

采样：通过选择代表性样本进行数据采集。采样技术确保了收集到的需求数据具有代表性，并能在一定程度上推断总体需求。

情节串联板：通过让参与者通过图片或图表的形式讲述需求，这种方法帮助理解用户的需求和背景情境，适用于需求较为复杂的情况。

联合需求计划（JRP）：这是一种通过集体讨论的方式获取需求的方法，主要适用于开发者与客户之间的沟通，能够帮助澄清不清晰或存在歧义的需求。

其他需求获取方法：如收集相关资料、获取已有文档、参加用户的业务实践等，这些方式有助于获取现有系统的运作方式与用户的需求背景。

答案-问题（2）

前置条件：是指在执行用例之前，系统需要满足的条件。如果这些条件不满足，用例将无法执行。例如：

用户需要先登录才能进行订单操作。

系统必须连接到支付网关，才能进行支付处理。

后置条件：是指用例执行后的状态或效果，描述系统经过该用例处理后的期望状态。例如：用户下单成功后，系统将生成订单，并发送订单确认邮件。

支付完成后，系统将更新订单状态并生成支付凭证。

答案-问题（3）

需求获取用例中通常涉及参与者(Actor)、用例(Use Case)和系统(System)三类对象。参与者是与系统交互的人员或外部系统，用例描述了参与者如何通过系统完成特定任务，而系统是执行用例所涉及的软件或硬件环境。

(2024年5月系统分析师真题-第2题)

某电商平台开发团队正在进行系统设计，以下是该平台的10个需求描述：

用户可以在系统中创建账户。

系统可以处理每秒1000个请求。

系统需要保证数据加密存储。

系统必须支持Windows和Linux操作系统。

用户可以浏览商品并添加到购物车。

系统的响应时间不超过2秒。

系统必须能够记录用户登录历史。

系统能够根据不同用户角色提供不同的权限。

系统需要支持动态扩展。

系统能够生成订单并发送确认邮件。

请根据上述需求，作答以下子问题。

问题（1）

分别将10种需求描述通过代码编号填写至功能需求，非功能需求，和设计约束

思路解析

在需求分类中，通常将需求分为三大类：功能需求、非功能需求和设计约束。

功能需求：指系统应实现的具体功能。这些需求通常涉及到系统的核心操作与用户交互。例如，需求 1、5、7、8 和 10 都是明确系统需要实现的功能。

非功能需求：指系统的质量特性，如性能、安全性、可靠性等。非功能需求通常不涉及具体的功能实现，而是描述系统的“做得怎么样”。例如，需求 2（系统性能）、需求 3（数据加密）、需求 6（响应时间）和需求 9（系统可扩展性）都属于非功能需求。

设计约束：是对系统设计的限制条件。这些约束影响系统的架构、技术选型等，例如，需求 4（操作系统支持）就是一个设计约束，它限定了平台必须兼容的操作系统。

问题（2）

解释一下判定表，然后找到题干里哪个需求会用到判定表。

思路解析

判定表主要应用于多条件决策的场景，能够系统化地描述不同条件组合下的系统行为。在电商平台中，尤其是在用户角色权限控制方面，判定表能够帮助开发人员明确不同角色之间的权限关系，确保系统的安全性和灵活性。

问题（3）

说明设计数据流图需要遵循的基本原则有哪些。

思路解析

设计数据流图时，必须遵循的原则主要集中在平衡性上。首先，父图和子图之间的数据流应该在数量和方向上保持一致。其次，子图内部的平衡是非常重要的，如果一个加工节点没有输出数据或者输入数据不足，系统的业务流程将无法正常运行，这可能导致数据流的“黑洞”或“灰洞”现象。数据存储的设计也非常关键，每个数据存储必须能够同时支持数据的读写操作，否则可能会导致系统的功能不全或性能瓶颈。

答案-问题（1）

功能需求，需求描述：用户可以在系统中创建账户。

非功能需求，需求描述：系统可以处理每秒 1000 个请求。

非功能需求，需求描述：系统需要保证数据加密存储。

设计约束，需求描述：系统必须支持 Windows 和 Linux 操作系统。

功能需求，需求描述：用户可以浏览商品并添加到购物车。

非功能需求，需求描述：系统的响应时间不超过 2 秒。

功能需求，需求描述：系统必须能够记录用户登录历史。

功能需求，需求描述：系统能够根据不同用户角色提供不同的权限。

非功能需求，需求描述：系统需要支持动态扩展。

功能需求，需求描述：系统能够生成订单并发送确认邮件。

答案-问题（2）

判定表用于描述具有多个条件和多个结果的情况，它可以有效地帮助设计决策流程，尤其是在业务规则复杂、条件多样的场景中。在电商系统中，判定表可以用于描述不同用户角色权限的控制。

- 左上部分：列出判断条件，例如是否为管理员、是否为注册用户等。
- 左下部分：列出对应的行动，例如允许访问商品管理、允许修改订单状态等。
- 右上部分：列出条件组合的不同组合状态（如真或假）。
- 右下部分：列出相应的动作（如执行或不执行）。

在该电商系统中，根据用户角色（如管理员、普通用户、游客）来控制访问权限时，判定表可以明确不同角色的权限控制。

应用需求：在需求 8 "系统能够根据不同用户角色提供不同的权限" 中，将用到判定表来定义不同角色（如管理员、注册用户等）在系统中可以执行的操作。

答案-问题（3）

（1）父图（上层数据流图）与子图（下层数据流图）平衡。个数一致：两层数据流图中的数

据流个数一致。方向一致：两层数据流图中的数据流方向一致。

(2) 子图内部的平衡。黑洞：加工只有输入没有输出。奇迹：加工只有输出没有输入。灰洞：加工中输入不足以产生输出。每个数据存储必须既有读的数据流，又有写的数据流。但是在某张子图中，可能只有读没有写，或者只有写没有读。

(2024年11月系统分析师真题-第2题)

阅读以下关于系统分析与设计的叙述，在答题纸上回答问题1至问题3。

【说明】

某高校需开发一套在线论文管理系统，功能包括：
学生用户通过界面登录、查看可选课题列表、提交选题
教师用户可发布课题、批阅论文
管理员管理用户权限
系统需支持论文上传下载、审核状态跟踪等功能

问题（1）

请用 150 字简要说明什么是面向对象设计的开闭原则。

思路解析

此题考察的是面向对象设计的核心原则之一。开闭原则的关键思想是“扩展开放，修改关闭”，这有助于提高系统的灵活性、可维护性和稳定性。它鼓励通过增加新代码（如子类化、接口等方式）来扩展功能，而不是直接修改已有的实现，从而降低了因修改现有代码导致的风险和错误。

问题（2）

请简要说明边界类、实体类、控制类的含义和作用，并将以下候选类填入到下表对应的位置

(1) 论文、(2) 课题、(3) 学生、(4) 选题窗口、(5) 课题列表、(6) 论文上传、(7) 论文批阅、(8) 登录界面、(9) 用户管理

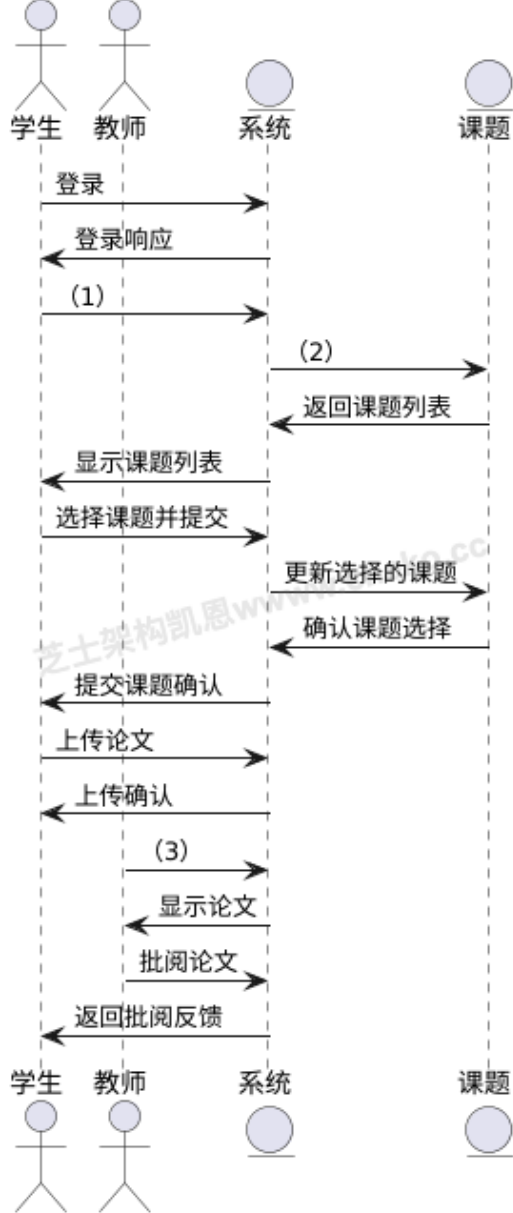
类别	含义	候选类
控制类		
实体类		
边界类		

思路解析

在面向对象的系统设计中，边界类主要是与外部环境进行交互的类，通常是用户界面或外部接口，它们负责接受用户输入并将数据传递到系统内部的类。实体类则是表示系统中的核心数据结构，通常直接映射到数据库或文件中，负责存储和处理数据。控制类的作用是协调和控制数据的流动，处理系统中的业务逻辑，通常作为边界类和实体类之间的中介。在本题中，通过对类的作用和命名规则进行分析，可以将候选类合理地分类。

问题（3）

请根据题目描述完善下方时序图



思路解析

- 学生登录系统：首先，学生通过界面发送登录请求，系统进行验证并返回结果。
- 选择课题并提交：学生选择感兴趣的课题并提交，系统更新选题状态并返回结果。
- 上传论文：学生提交论文文件后，系统确认上传是否成功，并反馈给学生。
- 教师批阅论文：教师查看学生提交的论文，提交批阅结果后，系统将批阅结果反馈给学生。

答案-问题（1）

面向对象设计的开闭原则（Open Closed Principle, OCP）指的是一个软件实体（如类、模块、函数等）应该对扩展开放，对修改关闭。也就是说，在设计一个模块时，应当使该模块可以在不被修改源代码的前提下被扩展，即能够在不必修改原有代码的情况下改变这个模块的行为。

答案-问题（2）

控制类：负责协调和控制其他类之间的交互，处理业务逻辑，起到一个 "桥梁" 的作用。它接收来

自边界类的用户请求，调用实体类的方法进行处理，并将处理结果返回给边界类。

实体类：用于模拟现实世界中的实体，代表系统中需要持久化存储和处理的信息，如业务对象、数据对象等。实体类通常包含属性和操作，属性用于描述实体的特征，操作则用于定义实体的行为。

边界类：位于系统与外界的交界处，负责与用户或外部系统进行交互。它为用户提供操作界面，或者与外部系统进行数据交换，将用户的输入转换为系统能够理解的形式，同时将系统的输出呈现给用户。

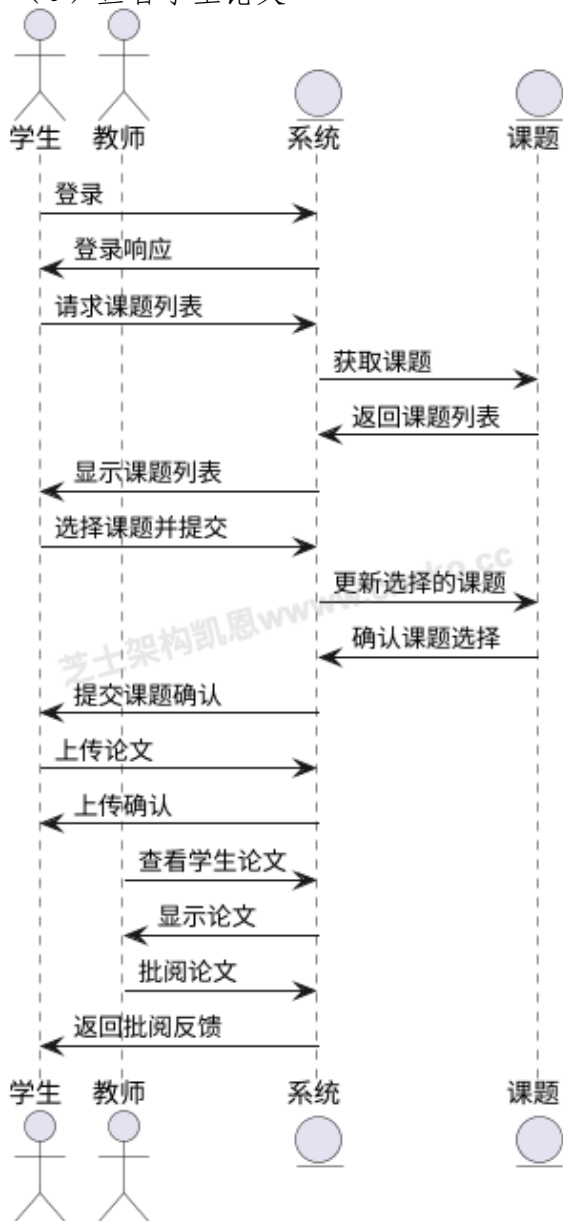
控制类 (6) 论文上传、(7) 论文批阅、(9) 用户管理

实体类 (1) 论文、(2) 课题、(3) 学生

边界类 (4) 选题窗口、(5) 课题列表、(8) 登录界面

答案-问题（3）

- (1) 请求课题列表
- (2) 获取课题
- (3) 查看学生论文



(2025年5月系统分析师真题-第1题)

某客户关系管理系统（CRM）正在进行面向对象分析。系统涉及多个用户角色，其主要职责如下：

系统管理员：管理系统基础数据，包括系统参数配置、操作日志查看、业务字典维护等。

客户经理：管理客户数据，包括客户信息、联系人信息、客户交易记录维护和客户历史记录打印。处理客户服务，包括服务创建、服务分配、服务处理和服务反馈。

客户总监：负责客户统计报表管理，包括客户构成分析报表、客户贡献分析报表和客户满意度分析报表等。

管理员：负责用户及权限管理，包括用户注册（电话注册、邮件注册）、用户登录、权限分配等。系统要求用户必须登录后才能进行操作。

问题（1）

用例构建通常包括哪四个阶段，请用文字简单说明。

思路解析

用例构建是面向对象分析中至关重要的环节，通常包括四个阶段。

用例构建是一个从明确交互对象到形成完整功能模型的结构化过程：首先通过识别参与者（包括用户、外部系统等与系统交互的外部实体），确定“谁会与系统打交道”；接着从零散需求中合并提炼出用例，将参与者的目标转化为系统可提供的独立功能单元；再细化用例描述，明确其前置条件、执行流程、后置状态等细节，让功能逻辑清晰可执行；最后调整用例模型，通过梳理用例间的包含、扩展等关系，优化整体结构，确保模型能准确、完整地反映系统功能边界与交互逻辑，为后续开发提供清晰的需求蓝图。

问题（2）

根据上述描述，系统的参与者有哪些？分别可以归纳出哪些用例？

思路解析

这里要对提供的参与者和用例列表进行分类整理，辨别出哪些是角色（参与者），哪些是功能行为（用例），并将它们按照角色一一对应起来。确保每一个参与者所承担的用例都被准确归类，避免遗漏和混乱。

问题（3）

上述用例之间存在哪些常见的关系？请举出一个扩展（extend）、包含（include）和泛化（generalization）关系的示例。

思路解析

第一种关系：包含关系（include）

包含关系表示某一用例在其执行过程中必须调用另一个用例，后者是前者逻辑的一部分，且可以被多个用例共享使用。其本质是“公共行为提取”，用于提升建模的复用性和结构清晰度。

例如，在“创建客户服务”过程中，系统一定需要用户填写相关的服务信息，包括客户名称、服务类型、服务说明等。这一行为不只属于“创建”，也可能被“修改服务”所使用，因此我们将“填写服务信息”提取为独立用例，通过 include 被多个用例调用。这体现了对业务逻辑的抽象与复用，减少冗余。

第二种关系：扩展关系（extend）

扩展关系表示某用例在特定条件下可能扩展另一个用例的功能，核心在于条件触发与非强依赖性。被扩展用例可以独立存在，扩展用例是可选的、附加的功能逻辑。

以“查询客户交易记录”为例，这是一个基本的查询操作，但并非每次查询都要打印结果。只有当用户选择打印时，才执行“打印客户历史记录”这一扩展用例。因此，扩展关系适合用于描述那些上下文相关的行为变化，如错误处理、额外操作、UI交互等，是系统灵活性与用户体验支持的关键建模手段。

第三种关系：泛化关系（generalization）

泛化关系表示两个用例或两个参与者之间存在继承结构，即子用例或子参与者继承了父用例/父参与者的结构与行为，同时具备自身特化内容。泛化强调共性提取与差异表达。

以“电话注册”和“邮件注册”为例，它们都属于“用户注册”的范畴，拥有共同的注册流程，如填写账户信息、提交验证，但在验证方式上略有不同（电话 vs 邮件）。因此我们将“电话注册”和“邮件注册”作为“用户注册”的子用例建模，这种方式有助于系统更灵活地支持多种业务实现方式，同时保持建模的一致性与扩展性。

答案-问题（1）

用例构建通常包括是识别参与者、合并需求获得用例、细化用例描述和调整用例模型

答案-问题（2）

该系统的参与者包括系统管理员、客户经理、客户总监、管理员（或用户管理员）以及注册用户（或系统用户）。这些参与者分别代表了不同角色在系统中的职责和使用权限，涵盖了系统管理、业务处理和信息查询等方面。

系统管理员主要负责系统层面的配置与维护，相关用例包括管理系统基础数据、查看操作日志和维护业务字典。

客户经理是系统中的核心业务人员，负责客户信息的日常管理和客户服务处理，其用例如管理客户信息、管理联系人信息、维护客户交易记录、打印客户历史记录、创建客户服务、分配客户服务、处理客户服务以及客户服务反馈。

客户总监则关注客户数据的分析和决策支持，其典型用例有生成客户构成分析报表、客户贡献分析报表和客户满意度报表。

管理员负责用户相关功能，包括用户注册（支持电话注册和邮件注册）、用户登录和用户权限管理。注册用户是系统的最终使用者，必须登录后才能访问相关服务。

答案-问题（3）

在用例建模中，不同用例之间可能存在多种关系，其中最常见的是包含关系。包含（include）关系表示一个用例的执行必须包含另一个用例的行为，是一种“被重用”的机制。例如，“创建客户服务”用例在执行过程中必须“填写服务信息”，因此“填写服务信息”是“创建客户服务”的一个包含用例。

扩展（extend）关系用于描述某个用例在特定条件下对另一个用例的功能进行扩展，通常表示一种可选的、条件触发的行为。例如，“打印客户历史记录”只有在执行了“查询客户交易记录”之后才会出现，因此“打印客户历史记录”是“查询客户交易记录”的扩展用例。

泛化（generalization）关系描述的是用例或参与者之间的继承关系，用于抽象和复用。例如，“电话注册”和“邮件注册”是“用户注册”用例的两种具体实现方式，它们继承了“用户注册

”的基本逻辑，但实现方式不同，因此形成泛化关系。这种关系有助于简化模型，提升结构的清晰度与可维护性。

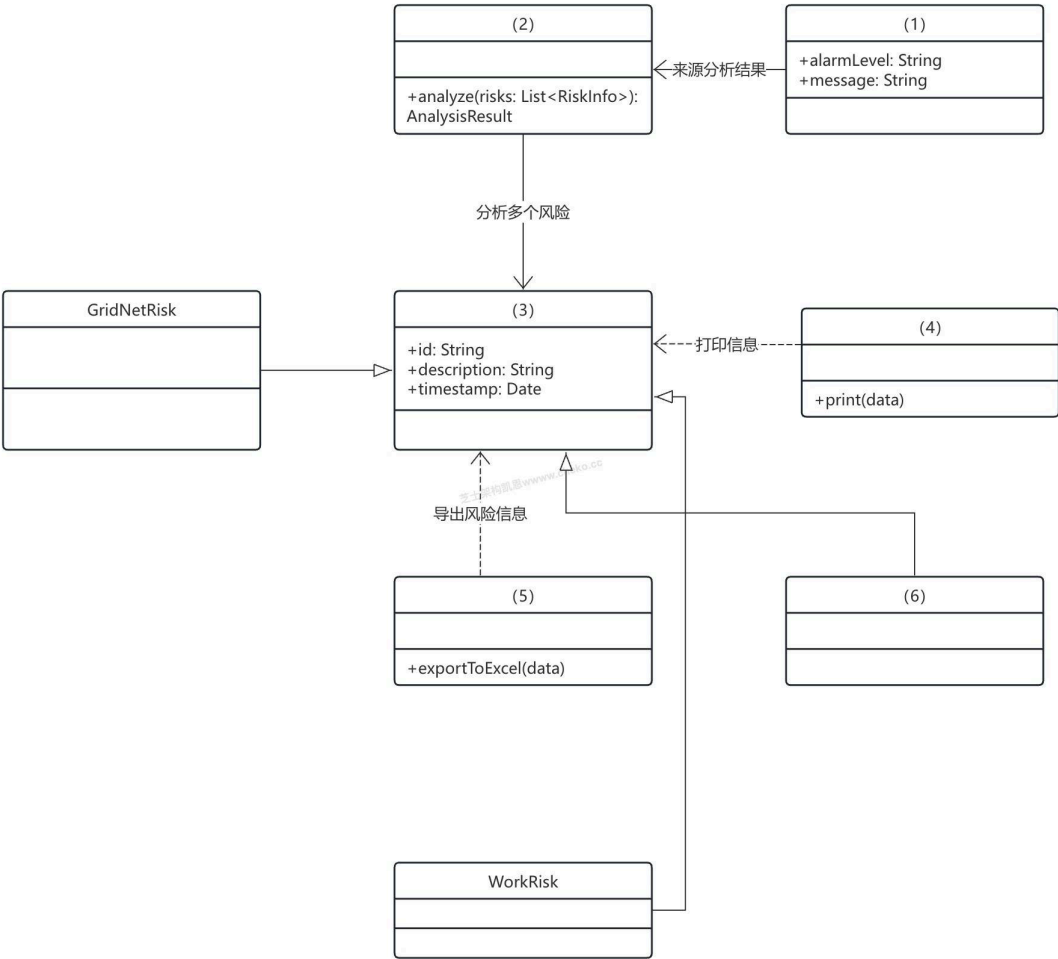
(2025年5月系统分析师真题-第2题)

某电力风险管理系统用于对电网运行中的各类风险信息进行统一管理。系统功能包括录入和维护各类风险信息，如工作风险、基建风险、电网运行风险等；支持风险信息的建档分类管理；对已登记的风险信息进行统计分析，并提供打印输出、导出 Excel 报表等功能。此外，系统具备风险报警机制，可根据分析结果生成相应的告警信息。

系统分析人员根据业务需求绘制了一张初步的风险管理类图，其中部分类名、类关系及模型视图尚不完整，请根据以下问题进行补全与分析。

问题（1）

在类图中，待填空处编号为 1~6，请从候选项 a~h 中选择适当的类名进行填充。候选项包括：
a. RiskInfo（风险信息）、b. ProjectRisk（基建风险）、c. RiskAnalysis（风险分析）、d. AlarmInfo（告警信息）、e. ExcelExporter（导出功能）、f. Printer（打印功能）



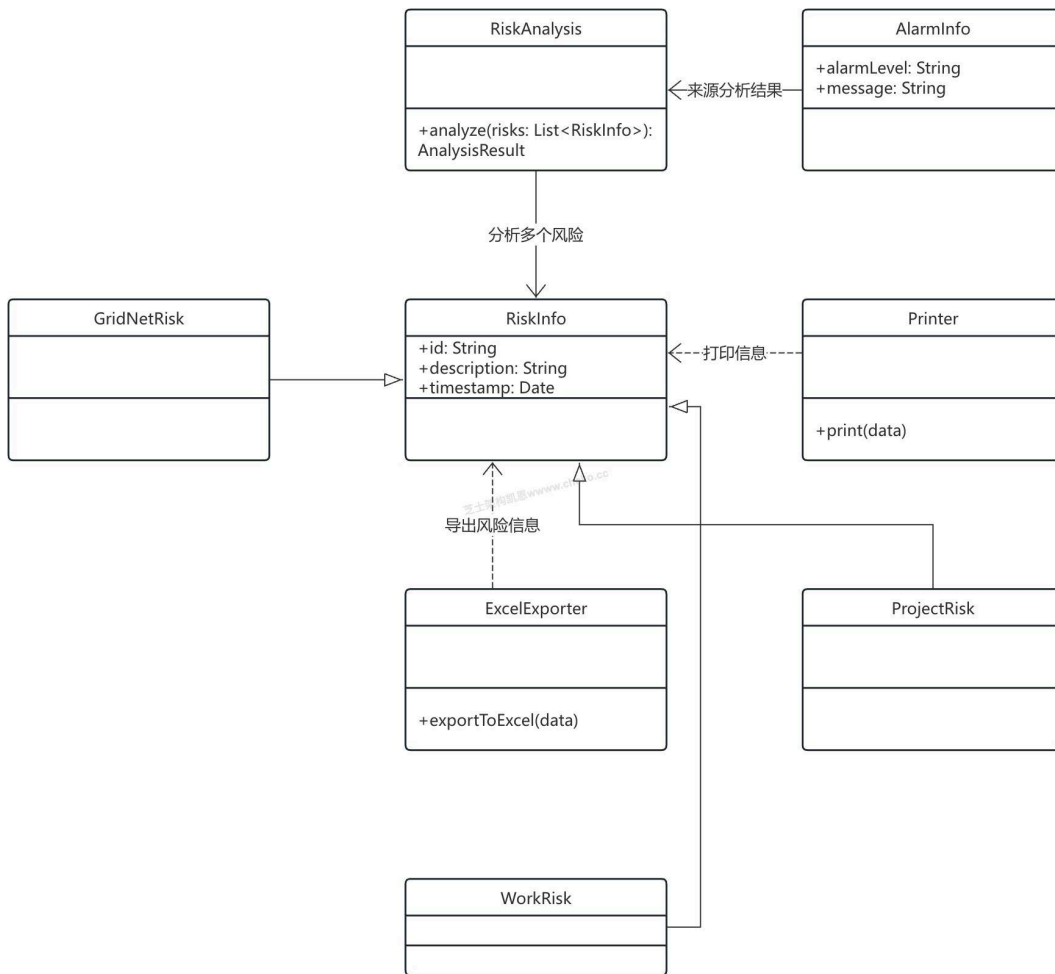
思路解析

- (1) 包含 alarmLevel 和 message 字段，用于描述告警，显然为 AlarmInfo。
- (2) 含有 analyze(RiskInfo) 方法，表示分析功能，故为 RiskAnalysis。
- (3) 是三类风险的父类，具有基本属性，是整个系统风险对象的核心类，应为 RiskInfo。
- (4) 提供 print(data) 方法，用于打印风险信息，应为 Printer。

(5) 提供 exportToExcel 方法，并且与风险信息有导出依赖，显然为 ExcelExporter。

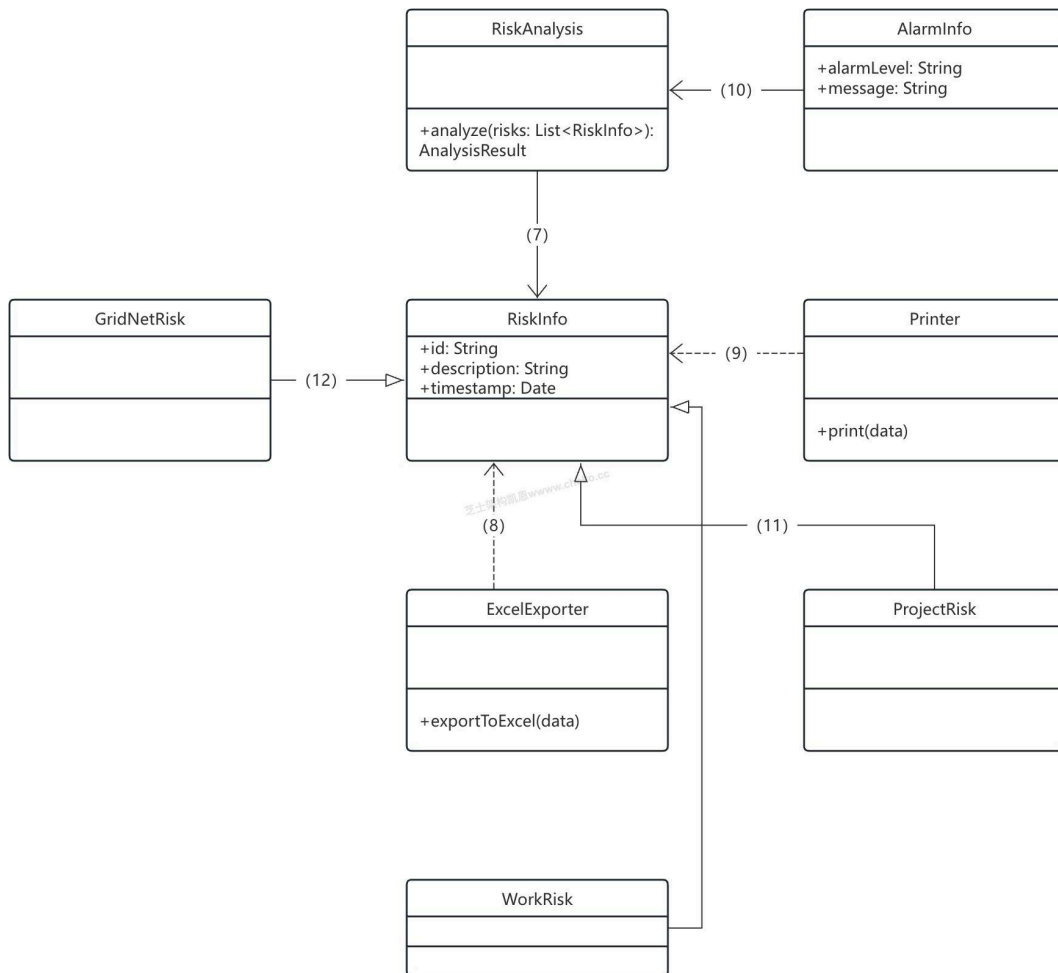
(6) 图中与 RiskInfo 存在继承关系，是 WorkRisk 与 GridNetRisk 并列的另一个风险子类，应为 ProjectRisk。

询问 ChatGPT



问题（2）

类图中的编号 7~12 表示类与类之间的关系。请判断这些关系是否符合 UML 规范，并指出其中不正确的关系，说明错误原因及其应修改为的关系类型。



思路解析

编号 7: RiskAnalysis → RiskInfo, 关系类型: 依赖: RiskAnalysis 通过 analyze(risks: List<RiskInfo>) 方法, 接受多个 RiskInfo 实例进行分析。分析行为与数据之间存在明确的结构联系, 属于典型的依赖关系 (虚线箭头)。

编号 8: ExcelExporter → RiskInfo, 关系类型: 依赖: ExcelExporter 提供 exportToExcel(data) 方法, 方法参数中使用了 RiskInfo, 但不长期持有 RiskInfo 的实例, 仅在方法执行中短暂使用。属于典型的依赖关系 (虚线箭头), 反映临时使用、非结构持有的建模场景。非常符合 UML 的依赖定义。

编号 9: Printer → RiskInfo, 关系类型: 依赖: Printer 类中的 print(data) 方法同样只是以 RiskInfo 作为临时参数进行调用, 不创建、不保存 RiskInfo 对象, 也不会生命周期中依赖它的存在, 属于非结构性使用关系。因此建模为依赖关系是准确的。

编号 10: AlarmInfo → RiskAnalysis, 关系类型: 关联: AlarmInfo 的数据源是 RiskAnalysis 的分析结果, 两者之间存在结果数据与来源之间的语义关联。使用单向关联关系表示 AlarmInfo 可溯源到 RiskAnalysis 的结果, 符合“数据结果与生成来源”之间的逻辑建模。该表达在面向对象分析中是合规的建模方式。

编号 11: ProjectRisk → RiskInfo, 关系类型: 继承: ProjectRisk 是 RiskInfo 的一种具体风险类型, 属于 RiskInfo 的子类。图中使用了继承关系 (实线带三角箭头) 表示 ProjectRisk 泛化于 RiskInfo, 这是标准的泛化 (继承) 关系。符合面向对象的继承结构表达。

编号 12: GridNetRisk → RiskInfo, 关系类型: 继承: 与编号 11 相同, GridNetRisk 作为电网类风险, 是 RiskInfo 的一种类型, 使用泛化关系建模其继承自 RiskInfo 的属性与行为, 表达准

确无误。

问题（3）

UML有哪些视图，分别对应 UML 哪些建模组件？

思路解析

逻辑视图（Logical View）

逻辑视图用于描述系统的功能结构和类之间的静态关系，是开发人员最关注的视角。它主要通过类图（Class Diagram）和对象图（Object Diagram）来表示。类图展示类的属性、方法及其继承、关联、依赖等关系，是面向对象建模的核心工具；对象图则是类图在某一时刻的实例化快照，反映具体对象的关系。该视图主要用于面向对象分析、模块设计与职责划分。

实现视图（Implementation View / Development View）

实现视图描述系统的模块划分和代码结构，是构建、维护和管理软件开发的重要依据。它主要使用组件图（Component Diagram）来展示模块（如类库、包、服务组件）之间的依赖关系和组织方式。实现视图支持模块化设计、代码包管理和构建过程控制。

进程视图（Process View / 动态视图）

进程视图展示系统在运行时的动态行为，强调交互顺序、状态变化和并发处理机制。对应的 UML 图包括活动图（Activity Diagram）、状态图（State Machine Diagram）和序列图（Sequence Diagram）。其中，活动图用于描述业务流程的控制逻辑，状态图用于建模对象生命周期的变化过程，序列图则描绘对象之间的消息传递顺序。进程视图常用于行为建模、线程管理和业务流程控制。

部署视图（Deployment View / 物理视图）

部署视图反映系统在物理层面上的分布结构，用于建模软件与硬件之间的部署关系。其核心 UML 表达为部署图（Deployment Diagram），展示系统组件在不同节点、服务器、设备中的分布情况。部署视图适用于运维部署设计、网络拓扑结构建模及分布式系统设计。

用例视图（Use Case View）（+1）

用例视图从用户或外部系统的角度出发，描述系统所提供的功能以及用户与系统之间的交互关系，是 4+1 模型中最基础也最重要的视角。通过用例图（Use Case Diagram）来展示参与者（Actor）与系统之间的交互场景。用例视图是需求分析的核心表达形式，常用于功能建模和用户行为分析，并为其他视图的设计提供指导依据。

答案-问题（1）

（1）d （2）c （3）a （4）f （5）e （6）b

答案-问题（2）

编号 7：错误，题目中是关联，实际上应该是依赖。RiskAnalysis 把 RiskInfo 实例作为参数传入，不长期持有，用依赖关系表示最合理。

编号 8：正确，依赖。ExcelExporter 只是临时用到 RiskInfo 数据进行导出，不长期持有，用依赖关系表示最合理。

编号 9：正确，依赖。Printer 使用 RiskInfo 作为打印参数，仅是短暂调用，也属于依赖关系。

编号 10：正确，关联。AlarmInfo 来源于 RiskAnalysis 的结果，二者存在结果与来源的关系，用单向关联表达清楚且规范。

编号 11：正确，继承。ProjectRisk 是 RiskInfo 的一种，属于典型的子类，用继承关系表示准

确。

编号 12: 正确, 继承。GridNetRisk 同样是 RiskInfo 的子类, 继承关系体现了它对父类属性的复用。

答案-问题 (3)

逻辑视图: 类图、对象图 (对应系统结构建模)

用例视图: 用例图 (体现系统功能与用户交互)

实现视图: 组件图 (表示代码模块组织结构)

进程视图: 活动图、状态图、序列图 (建模运行时行为)

部署视图: 部署图 (反映硬件与节点部署关系)

(2025年11月系统分析师真题-第4题)

某智能监控与设备控制系统用于对生产现场设备进行集中监控、自动控制和人工干预。系统主要功能包括:

采集设备运行状态信息;

根据系统配置和规则库自动生成控制决策;

向设备发送控制执行指令;

支持管理人员通过监控系统进行手动配置和调整控制规则。

系统采用数据流图 (DFD) 进行需求分析和功能建模, 设计了顶层数据流图以及若干子层数据流图。在设计和检查过程中, 发现部分数据流设置不合理, 影响了系统逻辑的正确性。

请根据数据流图建模原理和系统业务逻辑, 回答下列问题。(此题另外小问很难还原)

问题 (1)

简要说明顶层数据流图和子层数据流图在设计时应遵循哪些原则。

思路解析

顶层数据流图与子层数据流图的设计原则是高频考点, 重点考查考生对DFD建模规范性的理解。

首先是平衡原则。所谓平衡, 是指子层数据流图中的输入和输出数据流, 必须与其父层 (上一层) 中对应加工的输入和输出保持一致。不能在子图中无中生有新的数据流, 也不能遗漏父图已有的数据流。这一原则确保了系统功能分解前后的一致性, 是数据流图正确性的核心要求。

其次是逐层分解原则。顶层数据流图用于描述系统的整体功能和外部交互关系, 而子层数据流图是在此基础上对加工进行逐步细化。每一层只关注一个抽象层次的问题, 避免在高层图中出现过多细节, 使系统结构清晰、易理解。

第三是功能一致性原则。子层图中所有加工的组合功能, 应当等价于父层中被分解的那个加工, 不能偏离原有功能目标。这保证了系统建模在逻辑上的连贯性。这个可写可不写, 书本没有提到。

最后是数据流守恒原则。加工只能对数据进行处理、转换, 不能凭空产生数据或消除数据。所有输出数据都应能在输入数据中找到来源。

答案-问题 (1)

顶层数据流图和子层数据流图设计应遵循以下原则:

平衡原则。子层数据流图中的输入和输出数据流, 必须与其父层 (上一层) 中对应加工的输入和

输出保持一致。

逐层分解原则。顶层数据流图用于描述系统的整体功能和外部交互关系，而子层数据流图是在此基础上对加工进行逐步细化。

数据流守恒原则。加工只能对数据进行处理、转换，不能凭空产生数据或消除数据。

第2章——数据库系统

(2023年5月系统分析师真题-第3题)

某国际大型企业拟向企业开发一个在线社交网络服务(SNS)系统，让企业内部的员工能够通过类似社交网络的方式进行工作和自我管理，以实现企业内部员工高效、透明、便捷的沟通与协作。其基本的工序包括：

- 1、企业内部部门、员工等基本信息的维护和管理
- 2、提供常见的SNS服务，如 Blog，WiKi，RSS，即时通信和邮件等
- 3、提供社交网络，推荐系统等基本功能提高业务协作

除了上述功能性的需求外，还需要满足系统的可拓展、灵活部署以及高性能和高可用性需求企业设计系统小组

问题（1）

李工建议采用SNS中常见的NoSQL数据库技术，原因是传统的关系数据库不能满足实际需求，请用200字内说明该系统不能采用传统关系数据库的原因，并给出NoSQL数据的基本特点。

思路解析

传统关系数据库（RDBMS）是基于表格和固定模式存储数据的，它通过ACID（原子性、一致性、隔离性、持久性）特性来保证数据的完整性和可靠性。虽然这种设计适用于一些事务处理应用，但面对社交网络平台的高并发、快速变化的非结构化数据（例如，用户活动日志、社交媒体帖子等），它的扩展性和灵活性不足。NoSQL数据库是为了解决这些问题而设计的，具有高度的灵活性和扩展性，可以应对SNS系统对性能和规模的要求。NoSQL的横向扩展性使得它能够通过增加机器来处理不断增长的数据量，而关系型数据库通常依赖于纵向扩展，即通过增强单个服务器的性能来提升系统处理能力，这在面对海量数据时显得不够灵活。

问题（2）

李工对当前主流的 NoSQL 数据库镜像了调研形成了如下表

类型	典型应用场景	数据模型	优点	缺点
Key-Value	缓存、日志等	key-value / 全键值对	(c)	数据无结构
列式存储	(a)	列簇式存储	查找速度快、可扩展性强	功能相对局限
文档型	web 应用	key-value, value 为结构化数据	(d)	查询性能不高、缺乏统一语言
图数据库	社交网络、推荐系统等	(b)	利用图结构的各类算法	需对图整计算，分布集群方案，实现困难

思路解析

（a）大数据、OLAP应用： 列数据库（Columnar Database）以列存储数据，并且特别适合批量

数据处理和实时分析，广泛应用于大数据场景以及**在线分析处理（OLAP）**系统中。列数据库通过将同一列的数据存储在一起，优化了列级别的查询性能，非常适合进行大量的统计和数据分析。

（b）图由节点和关系组成：图数据库（Graph Database）是基于图论的数据库系统，数据由节点、边和属性组成，节点代表实体，边代表实体之间的关系。图数据库的特点是能够高效地处理复杂的关系查询，特别适合社交网络、推荐系统等应用场景。它通过将数据和关系同等对待进行存储，使得复杂的查询变得更加高效。

（c）高性能读写：Key-Value数据库（例如Redis）主要优化了数据的读取和写入性能，适合需要高速读写的场景。它通常以键值对的形式存储数据，支持高并发访问，因此适合用于缓存、高频次的数据操作。

（d）以文档为存储信息的基本单位：文档数据库（Document Database）将数据存储为文档（如JSON、XML等格式），每个文档可以有不同的结构，非常适合处理结构不规则的数据。文档数据库非常适合快速变化的应用场景，并且能够灵活地存储半结构化数据。

问题（3）

针对李工的建议，王工认为单纯使用 NoSQL 数据库无法完全满足系统要求，考虑到系统功能所涉及到的数据类型建议使用混合数据库模型来实现，请用100字以内文字说明。混合数据库模式如何组织。

思路解析

混合数据库模型将传统的关系型数据库与现代的NoSQL数据库结合使用，充分利用它们的优势。关系型数据库适用于结构化数据，如需要严格事务控制和完整性保证的员工信息、财务数据等。而NoSQL数据库则适合处理高并发、大规模、非结构化的数据，如用户行为数据、社交网络信息等。通过使用混合数据库模型，可以确保系统能够在处理复杂数据时具备足够的灵活性，同时保持高效的性能和可扩展性。例如，SNS系统中的用户社交关系图可以用图数据库处理，消息和日志数据可以使用键值数据库（如Redis），而员工基本信息则可以使用关系型数据库进行存储。这种模型能够根据不同数据的特性进行优化，提高整体系统的处理能力和扩展性。

答案-问题（1）

不能采用传统关系数据库的原因：

在线社交网络服务（SNS）系统要求对海量数据进行高并发、高效率的存储与访问，并且需要高扩展性和高可用性。传统的关系数据库（如MySQL、Oracle）虽然在数据一致性和事务处理上有优势，但在处理大量并发读写、高容量数据存储、灵活扩展等方面存在性能瓶颈。关系型数据库不擅长应对SNS系统中动态生成的非结构化数据（如用户动态、评论等），并且其扩展性通常是纵向扩展，而非横向扩展，难以满足SNS系统对性能和可扩展性的需求。

NoSQL数据库基本特点：

高并发读写：NoSQL数据库能够支持高并发的数据访问，性能优越，能够高效处理大量的请求。

灵活的数据存储结构：NoSQL数据库支持各种数据类型的存储，如键值对、文档、列簇等，适应多样化的数据格式。

海量数据存储和扩展性：NoSQL数据库设计为分布式架构，支持水平扩展，能够存储海量数据。

高可用性：基于分布式数据存储，NoSQL数据库避免了单点故障，提供了较高的可用性。

答案-问题（2）

- (a) 大数据、OLAP应用
- (b) 图由节点和关系组成
- (c) 高性能读写
- (d) 以文档为存储信息的基本单位

答案-问题（3）

混合数据库模型是结合了多种类型的数据库，如关系型数据库和NoSQL数据库。

在系统中，可以使用关系型数据库（如MySQL、PostgreSQL）来处理具有结构化、关系性的数据，如员工信息、部门管理等；NoSQL数据库（如MongoDB、Redis）则用来处理高并发、非结构化数据，如即时通讯记录、社交网络关系等。通过结合不同类型数据库的优点，可以根据数据的特性优化系统的性能与功能。

（2020年5月系统分析师真题-第3题）

阅读以下关于数据管理的叙述，在答题纸上回答问题1至问题3。

某全国连锁药店企业在新冠肺炎疫情期间，紧急推出在线口罩预约业务系统。该业务系统为普通用户提供口罩商品查询、购买、订单查询等业务，为后台管理人员提供订单查询、订单地点分布汇总、物流调度等功能。该系统核心的关系模式为预约订单信息表。

推出业务系统后，几天内业务迅速增长到每日10万多笔预约订单，系统数据库服务器压力剧增，导致该业务交易响应速度迅速降低，甚至出现部分用户页面无法刷新、预约订单服务无响应的情况。为此，该企业紧急成立技术团队，由张工负责，以期尽快解决该问题。

问题（1）

经过分析，张工认为当前预约订单信息表存储了所有订单信息，记录已达到了百万级别。系统主要的核心功能均涉及对订单信息表的操作，应首先优化预约订单信息表的读写性能，建议针对系统中的SQL语句，建立相应索引，并进行适当的索引优化。

针对张工的方案，其他设计人员提出了一些异议，认为索引过多有很多副作用。请用100字以内的文字简要说明索引过多的副作用。

思路解析

索引本质上是用来加速数据检索的工具，但在数据库中建立过多的索引会带来额外的存储和维护成本。每次数据修改都会触发索引的更新，因此增加了系统的负担。同时，过多的索引可能导致查询优化器难以选择最优的查询路径，影响系统的整体性能。在数据库设计时，需要权衡索引的使用，确保其对性能的提升大于其带来的开销。

问题（2）

作为团队成员之一，李工认为增加索引并进行优化并不能解决当前问题，建议采用物理分区策略，可以根据预约订单信息表中"所在城市"属性进行表分区，并将每个分区分布到独立的物理磁盘上，以提高读写性能。常见的物理分区特征如表41所示。李工建议选择物理分区中的列表分区模式。

表 4-1 物理分区模式比较表

分区模式	分区依据	适合数据	数据管理能力	数据分布
范围分区	属性取值范围	(b)	能力强	不均匀
哈希分区	属性的哈希值	静态数据	能力弱	(d)
列表分区	(a)	单属性、离散值	(c)	不均匀
组合分区	属性组合分区	周期、离散等	能力强	

请填补表4-1中的空（a）—（d）处，并用100字以内的文字解释说明李工选择该方案的原因。

思路解析

李工选择列表分区的原因是基于“所在城市”这一离散属性来进行分区。列表分区模式适用于属性具有离散值的情况，如城市或地区代码，每个分区包含一组明确的城市数据。

- （a）属性的离散值：表明分区基于离散的城市代码属性进行。
- （b）周期性数据/周期数据：可能描述按时间或其他周期性属性进行数据分区的特性。
- （c）能力强：说明列表分区能够处理大量数据，提高查询和管理效率。
- （d）均匀：在多个城市的数据分区中，数据的分布尽可能均匀，避免了某些分区数据过多而影响性能。

通过按城市分区，李工确保每个城市的数据存储和查询效率得到提升，从而优化数据库性能。

问题（3）

在系统运行过程中，李工发现后台管理人员执行的订单地址信息汇总等操作，经常出现与普通用户的预约订单操作形成读写冲突，影响系统的性能。因此李工建议采用读写分离模式，采用两台数据库服务器，并采用主从复制的方式进行数据同步。请用100字以内的文字简要说明主从复制的基本步骤。

思路解析

主从复制是一种数据库同步机制，用于在多个数据库间保持数据一致性。在这种模式中，主服务器负责处理所有写操作并记录修改内容，而从服务器则通过读取主服务器的日志来同步这些修改。通过主从复制，读操作可以被分配到从服务器，从而减轻主服务器的负担，优化系统性能，避免读写冲突。在李工的方案中，读写分离能够有效缓解数据库性能瓶颈，提升系统的并发处理能力。

答案-问题（1）

索引过多的副作用有：

- （1）过多的索引会占用大量的存储空间；
- （2）更新开销，更新语句会引起相应的索引更新；
- （3）过多索引会导致查询优化器需要评估的组合增多；
- （4）每个索引都有对应的统计信息，索引越多则需要的统计信息越多；
- （5）聚集索引的变化会导致非聚集索引的同步变化。

答案-问题（2）

- （a）属性的离散值
- （b）周期性数据/周期数据
- （c）能力强

(d) 均匀

李工建议根据预约订单所在城市进行表分区，而所在城市属性为离散值，根据所在城市属性建立列表分区，也方便不同城市处理自己的数据，方便数据管理。

答案-问题（3）

主从复制的基本步骤：

- （1）主服务器将所做修改通过自己的I/O线程，保存在本地二进制日志中；
- （2）从服务器上的I/O线程读取主服务器上面的二进制日志，然后写入从服务器本地的中继日志；
- （3）从服务器上同时开启一个SQL thread，定时检查中继日志，如果发现有更新则立即把更新的内容在本机的数据库上面执行一遍。

(2019年5月系统分析师真题-第4题)

阅读以下关于数据管理的叙述，在答题纸上回答问题1至问题3。

某软件企业开发了一套新闻社交类软件，提供常见的新闻发布、用户关注、用户推荐、新闻点评、新闻推荐、热点新闻等功能，项目采用MySQL数据库来存储业务数据。系统上线后，随着用户数量的增加，数据库服务器的压力不断加大。为此，该企业设立了专门的工作组来解决此问题。

张工提出对MySQL数据库进行扩展，采用读写分离，主从复制的策略，好处是程序改动比较小，可以较快完成，后续也可以扩展到MySQL集群，其方案如图4-1所示。李工认为该系统的诸多功能，并不需要采用关系数据库，甚至关系数据库限制了功能的实现，应该采用NoSQL数据库来替代MySQL，重新构造系统的数据层。而刘工认为张工的方案过于保守，对该系统的某些功能，如关注列表、推荐列表、热搜榜单等实现困难，且性能提升不大；而李工的方案又太激进，工作量太大，短期无法完成，应尽量综合二者的优点，采用Key-Value数据库+MySQL数据库的混合方案。

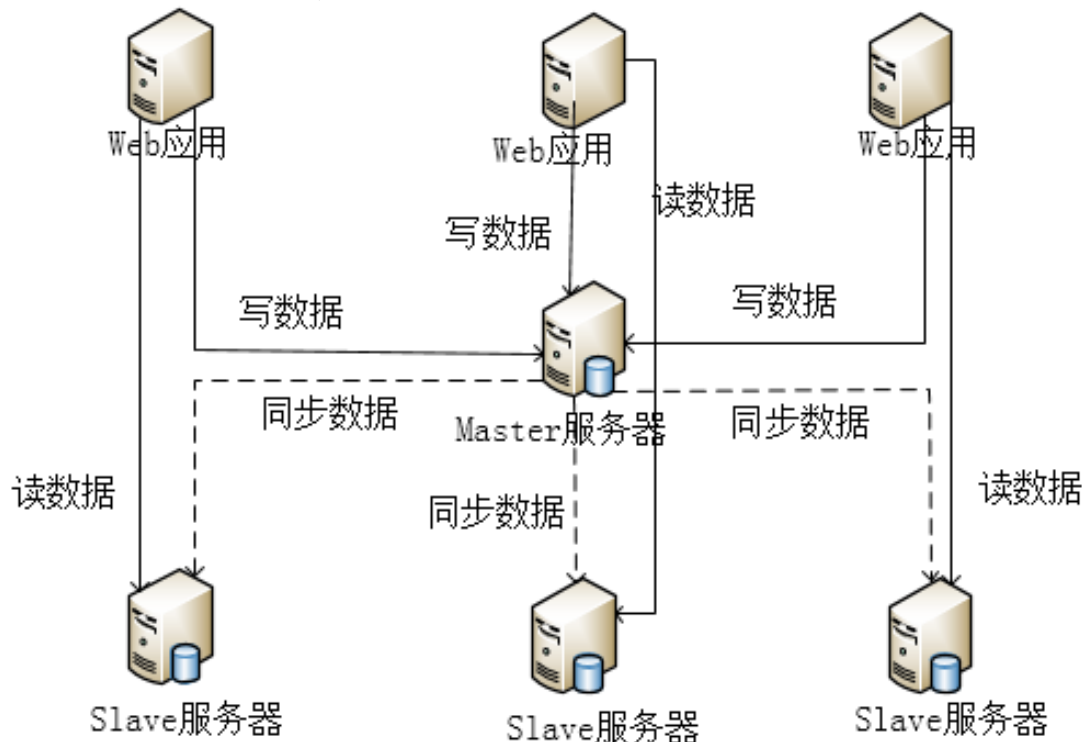


图4-1 张工方案示意图

经过组内多次讨论，该企业最终决定采用刘工提出的方案。

问题（1）

张工方案中采用了读写分离，主从复制策略。其中，读写分离设置物理上不同的主/从服务器，让主服务器负责数据的（a）操作，从服务器负责数据的（b）操作，从而有效减少数据并发操作的（c），但却带来了（d）。因此，需要采用主从复制策略保持数据的（e）。

MySQL数据库中，主从复制是通过binary log来实现主从服务器的数据同步，MySQL数据库支持的三种复制类型分别是（f）、（g）、（h）。

请将答案填入（a）~（h）处的空白，完成上述描述。

思路解析

MySQL 的主从复制机制主要依赖 binary log（二进制日志）来实现。主服务器将数据更新操作记录到 binary log 中，从服务器通过 I/O 线程拉取日志，再由 SQL 线程进行重放，从而实现数据同步。张工提出的读写分离方案是数据库性能优化的常见手段，主库专门负责写操作，从库承担读操作，这样能有效分担压力，提升并发能力，尤其适用于读多写少的应用场景。但是这种方案带来的一个问题就是数据延迟与不一致性风险。例如，当用户刚刚完成写入操作，查询从库可能还未同步最新数据，造成结果延迟。因此必须通过主从复制来保持数据一致性。MySQL 提供三种复制方式：SBR（基于 SQL 语句），存储效率高但可能存在语句副作用；RBR（基于行），能保证一致性但存储量大；MBR（混合模式）则结合二者的优势，按场景选择。选择合适的复制策略是数据库扩展的关键环节。

问题（2）

李工方案中给出了关系数据库与NoSQL数据的比较，如表4-1所示，以此来说明该新闻社交类软件更适合采用NoSQL数据库。请完成表4-1中的（a）~（d）处空白。

特征	关系数据库	NoSQL 数据库
数据一致性	实时一致性	(a)
数据类型	结构化数据	(b)
事务	高事务性	(c)
水平扩展	弱	强
数据容量	有限数据	(d)

思路解析

NoSQL数据库相较于传统的关系型数据库，在以下几个方面表现出了优势，这些特性使得它们特别适合用于需要高扩展性、处理海量数据以及快速响应的应用场景：

- 弱一致性：NoSQL数据库通常采用最终一致性，而非严格的ACID事务，这样可以提高系统的可用性和性能，尤其适合对一致性要求不高的场景。
 - 非结构化：NoSQL数据库通常支持存储非结构化或半结构化的数据，如JSON、XML等，而关系型数据库则依赖于严格的表结构和固定的字段类型。
 - 弱事务性：NoSQL数据库通常不支持传统的强事务性，这对于需要快速写入、读写分离的系统非常有用。
 - 海量数据：NoSQL数据库如键值存储、文档数据库等能更高效地处理大规模数据，尤其是在数据量不断增长时，能够支持水平扩展。
- 对于新闻社交类软件来说，NoSQL数据库非常适合处理大量的实时数据和非结构化数据（如用户评论、推荐内容等），可以提供更高的性能和灵活性。

问题（3）

刘工提出的方案采用了 Key-Value 数据库+MySQL数据库的混合方案，是根据数据的读写特点将数据分别部署到不同的数据库中。但是由于部分数据可能同时存在于两个数据库中，因此存在数据同步问题。请用 200 字以内的文字简要说明解决该数据同步问题的三种方法。

思路解析

在混合数据库架构中，数据同步 是最核心的挑战。不同数据源往往有不同的用途：关系数据库用于事务性操作和持久化，而 Key-Value 数据库常作为缓存或快速查询的支撑。如果两者数据不同步，就可能出现缓存数据与数据库数据不一致的问题，导致用户体验下降甚至业务错误。

实时同步 方案适合对一致性要求高的场景，通过“读缓存、写数据库并更新缓存”方式，保证最新数据可用。但这会增加写操作的开销。异步队列方式 则通过消息中间件解耦写操作，将更新任务异步推送，适合高并发场景，但可能出现短时间延迟。数据库插件或触发器同步 能在底层自动监听数据变更并更新缓存，减少人工逻辑编写，提升自动化程度。不同方法各有优劣，实际应用中常根据数据一致性要求与业务性能需求进行组合使用，以平衡系统的可扩展性和稳定性。

答案-问题（1）

- (a) 写
- (b) 读
- (c) 延迟
- (d) 数据不一致风险
- (e) 一致性
- (f) (g) (h) 基于SQL语句的复制 (statement-based replication, SBR)
- (g) 基于行的复制 (row-based replication, RBR)
- (h) 混合模式复制 (mixed-based replication, MBR)

答案-问题（2）

- (a) 弱一致性
- (b) 非结构化
- (c) 弱事务性
- (d) 海量数据

答案-问题（3）

实时同步方案：通过缓存机制，先查询缓存，如果缓存未命中再查询数据库，同时将数据库更新后的数据同步到缓存中。缓存更新时，先更新数据库，再同步更新缓存。

异步队列方式同步：使用消息中间件（如Kafka、RabbitMQ等）作为队列，将数据更新操作异步地推送到消息队列中，由消费端处理数据同步，适合高并发场景。

通过数据库插件完成数据同步：使用数据库插件（如MySQL的Binlog同步插件）自动将数据库更新的变更数据同步到另一个数据库中，确保数据的一致性。

利用触发器进行缓存同步：在数据库中使用触发器（Trigger），当数据发生变化时，触发同步操作，将数据同步到缓存系统中。

(2018年5月系统分析师真题-第4题)

阅读以下关于数据库设计的叙述，在答题纸上回答问题1至问题3。

某软件企业开发一套类似于淘宝网上商城业务的电子商务网站。该系统涉及多种用户角色，包括购物用户、商铺管理员，系统管理员等。

在数据库设计中，该系统数据库的核心关系包括：

产品（产品编码，产品名称，产品价格，库存数量，商铺编码）

商铺（商铺编码，商铺名称，商铺地址，商铺邮箱，服务电话）

用户（用户编码，用户名称，用户地址，联系电话）

订单（订单编码，订单日期，用户编码，商铺编码，产品编码，产品数量，订单总价）

不同用户角色有不同的数据需求，为此该软件企业在基本数据库关系模式的基础上，定制了许多视图。其中，有很多视图涉及到多表关联和聚集函数运算。

问题（1）

商铺用户需要实时统计本商铺的货物数量和销售情况，以便及时补货，或者为商铺调整销售策略。为此专门设计了可实时查看当天商铺中货物销售情况和存货情况的视图，商铺产品销售情况日报表（商铺编码，产品编码，日销售产品数量，库存数量，日期）。数据库运行测试过程中，发现针对该视图查询性能比较差，不满足用户需求。

请说明数据库视图的基本概念及其优点，并说明本视图设计导致查询性能较差的原因。

思路解析

此题主要考查视图的定义与优缺点，以及性能下降的原因。在该场景中，视图的使用没有对性能进行优化，导致在涉及到大量数据的统计时，性能变得非常差。视图的性能瓶颈通常出现在需要进行复杂计算或大量数据处理时。

问题（2）

为解决该视图查询性能比较差的问题，张工建议为该数据建立单独的商品当天货物销售、存货情况的关系表。但李工认为张工的方案造成了数据不一致的问题，必须采用一定的手段来解决。

1) 说明张工方案是否能够对该视图查询性能有所提升，并解释原因。

2) 解释说明李工指出的数据不一致问题产生的原因。

思路解析

此题主要考察了通过分表来优化查询性能的思路，并指出了分表导致的数据一致性问题。在分表的方案下，性能提升是显而易见的，因为我们将计算量集中在了一个表中，而不是每次查询时都进行计算。然而，分表引发的数据不一致问题则需要特别关注。在数据更新时，必须确保所有相关表的数据保持一致，否则会导致查询时出现错误。这个问题需要通过某些同步机制（如触发器或应用层逻辑）来解决。

问题（3）

针对李工提出的问题，常见的解决手段有应用程序实现，触发器实现和物化视图实现等，请用300字以内的文字解释说明这三种方案。

思路解析

此题考察了三种常见的数据库数据同步和性能优化方法。

应用程序实现需要手动管理数据同步，适合一些小型应用，但当数据量大时可能导致复杂度增加。

触发器实现可以在数据库层面自动保持数据一致性，适合频繁更新的场景，但也可能对性能产生影响。

物化视图实现通过存储查询结果来提高性能，是一种常见的优化方案，但也需要注意刷新机制的设计，以避免数据不一致。理解这些方案的适用场景，有助于在不同需求下选择合适的解决方案。

答案-问题（1）

视图是虚表，是从一个或几个基本表（或视图）中导出的表，在系统的数据字典中仅存放了视图的定义，不存放视图对应的数据。视图的优点如下所示：

- 1、视图能简化用户的操作
- 2、视图机制可以使用户以不同的方式查询同一数据
- 3、视图对数据库重构提供了一定程度的逻辑独立性
- 4、视图可以对机密的数据提供安全保护

查询性能较差的原因是视图中"日销售产品数量"需要针对订单表做统计分析，订单表中有数量庞大的历史销售记录，所以这种操作极为耗时。

答案-问题（2）

1) 张工方案的效果：

张工的方案能够提升查询性能。通过建立一个单独的“当天货物销售、存货情况表”，可以将当天的销售情况和存货情况事先计算好并存储。这种做法大大减少了查询时需要计算和聚合的数据量，因为统计和汇总的工作在数据插入时就已经完成了，而不是在查询时进行实时计算。这样，查询操作只需要从预先计算好的表中读取数据，而不需要每次都对历史订单数据进行聚合运算，从而大幅提升了查询性能。

2) 数据不一致问题的原因：

数据不一致问题的根本原因在于“当天货物销售、存货情况表”与“订单表”之间存在冗余数据存储。当订单表中的数据发生变化（例如修改了订单的数量或删除了某个订单），这些变化没有同步到“当天货物销售、存货情况表”中。这样，就会导致两个表中的数据不一致，造成查询结果不准确。例如，某个订单的产品数量改变了，但未同步更新“当天货物销售、存货情况表”，那么查询时就可能会返回错误的库存或销售数据。

答案-问题（3）

应用程序实现：在进行订单的添加、修改、删除操作时，从应用程序中，控制对两个数据表都进行相关操作，以保障数据的一致性。

触发器实现：在应用程序中，只对订单表进行操作。但写触发器，当订单表发生变化时，把当日订单内容同步更新到当天货物销售、存货情况表中。

物化视图实现：建立"当天货物销售、存货情况"的物化视图，物化视图会把相应的数据物理存储起来，而且在订单表发生变化时，会自动更新。 - 试题解析：

视图（View）是从一个或多个表（或视图）导出的表。视图与表（有时为与视图区别，也称表为基本表-----Base Table）不同，视图是一个虚表，即视图所对应的数据不进行实际存储，数据库中

只存储视图的定义，在对视图的数据进行操作时，系统根据视图的定义去操作与视图相关联的基本表。

(2017年5月系统分析师真题-第2题)

阅读以下关于系统数据分析与建模的叙述，在答题纸上回答问题 1至问题 3。
某软件公司受快递公司委托，拟开发一套快递业务综合管理系统，实现快递单和物流信息的综合管理。项目组在系统逻辑数据模型设计中，需要描述的快递单样式如图2-1所示，图 2-2 是项目组针对该快递单所设计的候选实体及其属性。

快递详情单

9 7 9 1 0 0 0 1

寄件人姓名		目的地	
单位名称		单位名称	
寄件人详细地址		收件人详细地址	
联系电话(非常重要)		手机	
联系电话(非常重要)		手机	
☐ 文件 ☐ 物品		付款方式 ☐ 现金 ☐ 到付 ☐ 协议结算	
重量		保价 保价金额	
体积		代收货款	
内件品名及数量		运费	
寄件人签名		加急费	
收件人签名		包装费	
证件号		保价费	
主属性: PK1<pi>		总计	

图 2-1 快递单样式图

寄件人	快递单
姓名	Variable characters (20)
始发地	Variable characters (20)
单位名称	Text
详细地址	Text
联系电话	characters (12)
证件号	characters (20)
主属性: PK1<pi>	

收件人	快递单
姓名	Variable characters (20)
目的地	Variable characters (20)
单位名称	Text
详细地址	Text
联系电话	characters (12)
证件号	characters (20)
主属性: PK3<pi>	

快递单
编号
类型
重量
体积
名称
数量
收寄员
日期
付款方式
保价金额
代收货款
运费
加急费
包装费
保价费
总计
备注
主属性: PK2<pi>

问题 (1)

数据库设计主要包括概念设计、逻辑设计和物理设计三个阶段，请用 200字以内文字说明这三个阶段的主要任务。

思路解析

数据库设计的三个阶段：概念设计、逻辑设计和物理设计是数据库设计的核心过程。每个阶段都从不同的角度对系统进行优化和调整，确保数据库能够支持系统的高效运行。

概念设计阶段主要是抽象化的思考，设计者不需要考虑实际的技术实现，而是专注于业务需求和数据结构的抽象，确保整个数据库模型符合需求。

逻辑设计则更为具体，开始考虑数据的存储形式和数据库管理系统的实现要求，此时需要考虑数据的具体关系和约束条件，使得设计能够实际运行。

物理设计是最具体的阶段，设计师必须考虑硬件和性能需求，确保数据库的运行效率，包括存储方式、索引优化和数据访问路径。

问题（2）

根据快递单样式图，请说明：

- （1）图 2-2中三个候选实体对应的主属性PK1、PK2和PK3分别是什么？
- （2）图 2-2 中应设计哪些实体之间的联系，并说明联系的类型。

思路解析

在数据库设计中，实体及其主属性的选择至关重要，因为主属性需要确保数据的唯一性和完整性。这里，寄件人和收件人实体都以证件号作为主属性，这可以确保在快递单的管理中唯一地标识每一个寄件人和收件人。而快递单实体以编号作为主属性，这个编号保证每一单快递都能被唯一标识。在设计实体间的联系时，我们考虑了业务流程中的实际需求。寄件人与快递单之间是一种1: N关系，因为一个寄件人可能会寄送多张快递单，而每张快递单只有一个寄件人。同样，收件人与快递单之间也是1: N关系，因为一个收件人可以接收多个快递单，而每张快递单只会会有一个收件人。

问题（3）

在图2-2中添加实体之间的联系后，该实体联系图是否满足第一范式、第二范式和第三范式中的要求（对于每种范式判定时，假定已满足低级别范式要求）。如果不满足，请用 200 字以内文字分别说明其原因。

思路解析

数据库的范式设计是保证数据一致性、减少冗余和提高查询效率的关键。在本题中，寄件人和收件人实体由于属性较少且简单，因此容易满足范式要求。它们避免了部分依赖和传递依赖，符合1NF、2NF和3NF。

而快递单实体则相对复杂，它包含多个依赖于主键的属性，同时也涉及到一些派生属性（如总计），这种设计容易引入传递依赖。为满足3NF，应该对快递单进行优化，避免数据的冗余和不一致问题。

答案-问题（1）

概念设计，也称为概念结构设计，其主要任务是在需求分析阶段生成的需求说明书基础上，将实际业务需求抽象成数据模型，确保模型的独立性和通用性。该阶段的设计不依赖于具体的数据库管理系统（DBMS），其输出通常是实体-关系图（ER图），通过ER图来描述实体、属性、关系等内容。

逻辑设计，也称为逻辑结构设计，其主要任务是将概念设计阶段的ER图转化为符合特定数据库管理

系统（DBMS）支持的逻辑结构。例如，在关系型数据库中，ER图中的实体将转换为数据表，关系将转换为表之间的外键约束。逻辑设计阶段的输出通常是关系模式，它以关系的形式描述了数据库中的表格、字段以及约束条件。

物理设计，也称为物理结构设计，主要任务是根据数据库的实际使用环境和性能需求，设计数据库的物理存储结构和存取方式。此阶段需要考虑数据的存储位置、索引、访问路径等问题，以提高数据库的查询和更新性能。物理设计的输出包括存储结构和存取方法，例如，选择合适的索引、分区策略、存储介质等。

答案-问题（2）

PK1: 证件号

PK2: 编号

PK3: 证件号

联系1: 寄件人与快递单之间应有联系，联系类型: 1:N。

联系2: 收件人与快递单之间应有联系，联系类型: 1:N。

答案-问题（3）

寄件人，收件人均满足第3范式，因为这两个关系均消除了部分函数依赖与传递函数依赖。（自然也就同时满足第1范式与第2范式）。快递单满足第2范式，但不满足第3范式，快递单的主键为编号，编号确定：保价金额、代收货款、运费、加急费、包装费、保价费，而这一系列费用的组合确定总计。所以存在传递函数依赖。

（2017年5月系统分析师真题-第4题）

阅读以下关于数据库分析与建模的叙述，在答题纸上回答问题1至问题 3。

某电子商务企业随着业务不断发展，销售订单不断增加，每月订单超过了50万笔，急需开发一套新的互联网电子订单系统。同时该电商希望建立相应的数据中心，能够对订单数据进行分析挖掘，以便更好地服务用户。

王工负责订单系统的数据库设计与开发，初步设计的核心订单关系模式为：

orders (order_no, customer_no , order_date, product_no, price,) ;

考虑订单数据过多，单一表的设计会对系统性能产生较大影响，仅仅采用索引不足以解决性能问题。因此，需要将订单表拆分，按月存储。

王工采用反规范化设计方法来解决，给出了相应的解决方案。李工负责数据中心的设计与开发。李工认为王工的解决方案存在问题，建议采用数据物理分区技术。在解决性能问题的同时，也为后续的数据迁移、数据挖掘和分析等工作提供支持。

问题（1）

常见的反规范化设计包括增加冗余列、增加派生列、重新组表和表分割。为解决题干所述需求，王工采用的是哪种方法?请用300字以内的文字解释说明该方法，并指出其优缺点。

思路解析

王工采用的表分割是反规范化设计的一种常见方式。表分割的目的是通过拆分表格来减轻数据库负载，提升性能。具体来说，表分割包括水平分割和垂直分割两种形式。

水平分割：水平分割是将数据按某一列的值（例如时间）分割成多个表，例如按月将订单数据存储在不同的表中。查询当月订单时，只需要访问相应月份的表，从而避免了全表扫描，提升了性能。

垂直分割：垂直分割则是将表中的列根据业务需求分割到不同的表中。例如，一个订单表中可能包含客户信息、商品信息等，垂直分割后，客户信息和商品信息可以分别存储到不同的表中。

优点是：

提升查询性能：由于每个分割后的表包含的数据量较小，查询时无需扫描整个表，减少了查询时间，尤其是在处理大数据量时效果显著。

数据管理灵活：每个表的数据量相对较小，便于进行管理，如备份、恢复、迁移等。按月分割后，每个月的数据可以独立处理，有助于提高系统的可扩展性。

缺点是：

破坏数据的完整性：表分割后，数据的完整性和一致性管理变得更为复杂。跨表查询时需要进行集合操作，这增加了查询的复杂度。

维护困难：随着时间推移，分割后的表越来越多，DBA需要手动维护分表策略，特别是在查询历史数据时，跨表查询可能会增加系统的负担。

问题（2）

物理数据分区技术一般分为水平分区和垂直分区，数据库中常见的是水平分区。水平分区分为范围分区、哈希分区、列表分区等。请阅读下表，在(1)~(8)中填写不同分区方法在数据值、数据管理能力、实施难度与可维护性、数据分布等方面的特点。

	范围分区	哈希分区	列表分区
数据值	(1)	连续离散均可	(2)
数据管理能力	强	(3)	(4)
实施难度与可维护性	(5)	好	(6)
数据分布	(7)	(8)	不均匀

思路解析

分区技术是对数据库进行优化的重要手段，它能够有效地提升大规模数据的访问性能和管理能力。常见的物理水平分区技术包括范围分区、哈希分区和列表分区。每种分区方法都有不同的优缺点，适用于不同的场景。

范围分区：

数据值：连续，范围分区是根据数据的某个范围（如时间）来进行划分。例如，可以按月、季度或年度来划分数据，这样可以有效减少查询时需要扫描的数据量。

数据管理能力：强，范围分区便于根据时间、地域等维度进行数据的管理和分析。分区的数据通常符合业务需求，便于管理。

实施难度与可维护性：差，范围分区的管理和维护较为复杂，特别是当数据分布不均匀时，需要手动调整分区策略。

数据分布：不均匀，由于数据的分布可能集中在某些特定时间段，导致某些分区的数据量大，而其他分区数据量少，从而影响性能。

哈希分区：

数据值：离散，哈希分区通过哈希算法将数据均匀分布到不同的分区上，通常适用于数据值离散且不具有明显时间顺序的场景。

数据管理能力：弱，哈希分区不依赖于数据的实际内容，因此无法根据业务需求进行灵活管理。

实施难度与可维护性：强，哈希分区不需要手动调整分区规则，管理较为简便。

数据分布：均匀，哈希分区能够均匀地将数据分布到各个分区，避免某些分区的过载问题。

列表分区：

数据值：离散，列表分区通常用于处理离散的、重复的值。例如，可以根据某个类别（如地域标签）进行分区，适用于数据类别明显且重复率较高的场景。

数据管理能力：强，这里是从找数据的角度，毕竟列表分区是根据某个指定类别划分的，通过列表分区方便处理和查找数据。

实施难度与可维护性：差，由于列表分区的分区依据是离散的数值，容易导致数据不均衡，维护起来比较麻烦。

数据分布：不均匀，列表分区可能会导致某些分区的数量不平衡，特别是在数据值不均匀的情况下，某些分区的数据量过大，导致性能下降。

问题（3）

根据需求，李工宜选择物理水平分区中的哪种分区方法？请用300字以内的文字分别解释说明该方法的优缺点。

思路解析

范围分区是通过将数据按某一列或多列的值的范围来划分数据的存储区域，常见的划分方式是按照时间（如订单的创建时间）进行分区。例如，可以将订单数据按月、季度或年度进行分区，查询某一时间段的订单时，只需访问相应时间段的分区，从而大大提高查询效率。

范围分区的优点：

数据管理能力强：范围分区特别适用于有自然范围的数据（如订单日期、交易金额等）。它可以在分区时按时间、金额等维度来划分，便于后期的数据迁移、备份、清理等管理操作。

查询效率高：当查询某个特定时间段的数据时，范围分区只需要访问相应时间段的分区，而不需要扫描整个表，减少了查询的时间。

备份和恢复：每个分区的数据相对独立，可以独立备份、恢复，这样在进行系统维护时，可以减少停机时间和提升恢复效率。

范围分区的缺点：

分区管理较复杂：范围分区的管理通常需要数据库管理员根据数据的变化来手动调整分区策略。

例如，分区规则可能需要根据实际情况调整，这对DBA提出了较高的要求。

数据不均匀：如果某些时间段的数据量过大（如某个月的订单量异常增多），可能会导致某些分区的负载过大，影响系统的整体性能。

答案-问题（1）

王工采用的是表分割的方式进行反规范化设计。

表分割包括水平分割与垂直分割两种形式：

水平分割：根据一列或多列数据的值把数据行放到两个独立的表中。

垂直分割：把主码和一些列放到一个表，然后把主码和另外的列放到另一个表中。

优点：

（1）表分割的方式将数据分布到多个逻辑与物理上均独立的不同的表。对于电子商务中的订单，最频繁的操作针对的是当月的订单表，表分割后有效地减少了操作表（即当月订单表）的记录数

，可有效提升性能。同时按月进行表分割后，也可以针对各月份数据进行管理，有利于数据迁移、备份和管理。

缺点：

(1) 表分割的方式从逻辑上破坏了关系概念的完整性，由一个关系变为多个关系。因此，进行历史数据的数据挖掘和分析时，必须执行多表集合并操作，相对于单表形式，复杂度较高，增加了数据维护的难度，应用软件设计和实现也更为复杂。

答案-问题(2)

(1) 连续 (2) 离散 (3) 弱 (4) 强 (5) 差 (6) 差 (7) 不均匀 (8) 均匀

答案-问题(3)

李工宜选择范围分区方式。范围分区优点包括：数据管理能力强、可以提高查询效果，备份管理也相对容易。但是因为范围分区和数据本身强关联，需要DBA对分区进行手动维护，相对哈希分区来说实施难度较高。

(2016年5月系统分析师真题-第4题)

阅读以下关于数据库设计的叙述，在答题纸上回答问题1至问题3。

某航空公司要开发一个订票信息处理系统，以方便各个代理商销售机票。开发小组经过设计，给出该系统的部分关系模式如下：

航班（航班编号，航空公司，起飞地，起飞时间，目的地，到达时间，剩余票数，票价）

代理商（代理商编号，代理商名称，客服电话，地址，负责人）

机票代理（代理商编号，航班编号，票价）

旅客（身份证号，姓名，性别，出生日期，电话）

购票（购票单号，身份证号，航班编号，搭乘日期，购票金额）

在提供给用户的界面上，其核心功能是当用户查询某航班时，将该航班所有的代理商信息及其优惠票价信息，返回给用户，方便用户购买价格优惠的机票。在实现过程中发现，要实现此功能，需要在代理商和机票代理两个关系模式上进行连接操作，性能很差。为此开发小组将机票代理关系模式进行了扩充，结果为：

机票代理（代理商编号，航班编号，代理商名称，客服电话，票价）

这样，用户在查找信息时只需对机票代理关系模式进行查询即可，提高了查询效率。

问题(1)

机票代理关系模式的修改，满足了用户对代理商机票价格查询的需求，提高了查询效率。但这种修改导致机票代理关系模式不满足3NF，会带来存储异常的问题。

(1) 请具体说明其问题，并举例说明。(2) 这种存储异常会造成数据不一致，请给出解决该存储异常的方案。

思路解析

本题的关键在于指出逆规范化后出现的对码的非主属性依赖问题，并据此解释为什么产生冗余与更新异常。

规范化前（或规范化的正确做法）应将代理商基本信息独立存放于“代理商”表，机票代理表只保

留用于描述“代理商×航班”关联与该组合上的业务属性（如优惠票价）。在逆规范化之后，机票代理（代理商编号，航班编号，代理商名称，客服电话，票价）以（代理商编号，航班编号）作为候选码，其中代理商名称、客服电话仅依赖于代理商编号（主码的一部分），这构成了对主码的部分函数依赖，违反2NF；因此不满足3NF。由此带来典型异常：数据冗余（相同代理商信息在多航班记录中重复）、修改异常（单点更新需波及多行；漏改则不一致）、插入异常（尚未有具体航班时无法插入代理商信息）与删除异常（删除某航班代理记录时可能意外丢失该代理商的事实）。例如：代理商X代理了100个航班，客服电话从“400-123-4567”改为“400-765-4321”，若只更新了其中98条，系统中将同时存在两种客服电话，数据不一致直接危害用户联系与对账流程。

在保持性能收益的前提下消解不一致，常见方案有三：其一，回归3NF（机票代理仅保留代理商编号、航班编号、票价；代理商名称与客服电话通过连接获取），可彻底消除异常，但会增加查询连接成本；其二，触发器同步：在“代理商”表上建立AFTER UPDATE触发器，当代理商名称或客服电话变更时，自动UPDATE 机票代理 SET 名称/电话=... WHERE 代理商编号=...，提供数据库内的强一致性与较好的实时性；其三，应用层事务同步：将更新代理商与批量更新机票代理冗余列纳入同一事务，失败则回滚，适合业务逻辑集中在应用层的系统。此外，还可考虑定时批处理（弱一致）、物化视图/索引视图（由数据库维护视图物化数据以供快速查询）或缓存（如Redis）+订阅通知（在变更时失效缓存重建），在读多写少的场景权衡一致性与性能。

问题（2）

在机票销售信息处理系统中，两个代理商的售票并发执行，可能产生的操作序列如表4-1所示。

时间点	代理商 1	代理商 2
T1	查询剩余票数	
T2		查询剩余票数
T3	剩余票数减 1	
T4		剩余票数减 1
T5	提交	
T6		提交

- 假设两个代理商执行之前，该航班仅剩1张机票。
- （1）请说明上述两个代理商操作的结果。
 - （2）并发操作会带来数据不一致的问题，请具体说明3种问题。

思路解析

将场景具体化：初始“剩余票数=1”。若系统实现为“先查库存>0则减1并出票”，且缺乏合适的并发控制（如行级锁或原子更新），两代理商事务可能同时读到“1”。随后各自执行“减1”，都将其本地计算结果写回（一般是UPDATE 航班 SET 剩余票数=剩余票数-1 WHERE 航班编号=... AND 剩余票数>0或读改写模式）。

如果是读-改-写且没有条件保护，最后提交的事务会覆盖先提交的事务写入值，最终库存可能为0（题设正确答案），而业务上却卖出了两张票（超卖）。若实现是条件更新但不加锁，也存在时间窗引发的并发成功，依旧可能出现超卖或覆盖。

此结果正体现了丢失修改（Lost Update）：两个事务不隔离地对同一行进行读改写，后写覆盖前写，使得前一事务的修改从逻辑上“消失”。

并发环境中还会出现另外两类经典异常。读脏数据（Dirty Read）：事务A更新“剩余票数=0并附带其他变更”后尚未提交，事务B就读取到了该未提交的新值并继续出票逻辑；若A因风控失败或业务校验回滚，B基于未生效值做出的业务决策即为“脏”。

不可重复读（Non-repeatable Read）：事务A在一次交易流程中两次读取余票，本以为两次结果一

致，但事务B在期间完成了减库存提交，导致A第二次读取与第一次不同，产生逻辑分歧与流程歧义。进一步，在范围查询（如统计多航班剩余票>0）中，B可能插入或删除满足条件的新行，A两次统计结果不一致则称为幻读（Phantom）。为了消除上述问题，数据库通常提供并发控制与隔离级别：如读已提交（RC）避免脏读、可重复读（RR）避免不可重复读（在MySQL InnoDB下配合间隙锁也可抑制部分幻读），以及可串行化（Serializable）从根本上以牺牲并发换取严格一致性。在票务扣减这种强一致、热点写的场景，工程实践多采用：① 行级锁+原子条件更新（UPDATE ... SET 剩余票数=剩余票数-1 WHERE 航班编号=... AND 剩余票数>0，检查受影响行数=1决定是否成功）；② 悲观锁（SELECT ... FOR UPDATE）或乐观锁（版本号/时间戳，CAS式更新）；③ 去中心化扣减（如库存预占/令牌桶/消息队列串行化）；④ 限流与幂等（防止重复扣减与重试放大）。

问题（3）

请填写表中的空白项，并用150字以内的文字说明读写锁机制的缺点。

时间点	代理商 1	代理商 2
T1	查询剩余票数，加读锁	
T2		查询剩余票数，（2）
T3	剩余票数减 1，（1）	
T4		剩余票数减 1，（3），（4）
T5	提交，释放全部锁	
T6		（5），剩余票数减 1，（6）
T7		提交，释放全部锁

思路解析

本题考核两方面：其一是对读写锁在并发过程中的行为判断（何时加读锁、写锁、等待与被唤醒）；其二是对读写锁在数据库并发控制与应用层并发编程中的局限性与代价的系统性理解。读写锁（RWLock）以“读共享、写独占”为核心规则：多个读者可以并发获取读锁，写者获取写锁时需要排他地占用资源，且通常要求在持有写锁期间无其他读者存在。结合题设时序：在T1/T2两个代理商先后执行查询时，均为只读访问，故加读锁是正确的；当任意一方进入库存扣减阶段，这是写操作，需要对同一行（或同一逻辑资源）加写锁以保证排他性，从而避免并发写的冲突。由于写锁与读锁互斥，当一方已经或即将升级/获取写锁时，另一方必须等待；在持锁事务提交（或回滚）后，锁被释放，等待方被唤醒/得到通知再继续争用，最终成功加写锁后完成修改。故填空给出的“加读锁→加写锁→等待/得到通知”的序列与规则一致。

关于缺点，需要从性能与正确性两条线展开。性能层面：读写锁的收益建立在“读远多于写”的工作负载上，读者之间共享并发可显著提升吞吐；然而当写比例上升或存在写密集热点时，写锁的排他性会阻塞大量读请求，系统吞吐急剧下降，甚至退化为串行。频繁的锁竞争还会导致线程上下文切换与锁队列调度的额外开销，降低整体效率。正确性/可用性层面：读写锁引入了死锁与饥饿风险。典型如“读者优先策略”下，持续涌入的读请求可能让写者长期饥饿；“写者优先策略”虽然缓解写饥饿，却可能在读多场景引发读方等待激增。再者，实际工程常遇到锁升级/降级问题：线程先持有读锁后尝试升级为写锁，如果实现不支持原地升级且存在其他并发读者，就必须先释放读锁再争抢写锁，这在竞争下容易丢失原子性保证或诱发活锁。此外，在数据库内部（如InnoDB）与应用层RWLock之间还可能发生优先级反转：低优先级线程持有关键写锁阻塞高优先级业务线程，影响SLA。

答案-问题（1）

机票代理（代理商编号，航班编号，代理商名称，客服电话，票价）以（代理商编号，航班编号）作为候选码，其中代理商名称、客服电话仅依赖于代理商编号（主码的一部分），这构成了对主码的部分函数依赖，违反2NF；因此不满足3NF。

不满足3NF的关系模式存在存储异常问题，包括数据冗余和修改异常。

（1）数据冗余：如果某代理商代理100个航班的售票，那么在机票代理的关系模式中就要出现100个元组，该代理商的名称、客服电话也随之重复出现100次。

（2）修改异常：由于上述冗余问题，当需要修改该代理商的客服电话时，就要修改100个元组中的客服电话值，否则就会出现客服电话值不一致的现象。

解决存储异常的典型方案是进行模式分解，但在本题中机票代理已是逆规范化的产物，故可使用触发器在修改时，检查并更新对应数据的方式来解决数据不一致的问题。

答案-问题（2）

2个代理商都成功售出1张票，剩余票数为0。

数据库的并发操作会带来一些数据不一致问题，例如，丢失修改、读脏数据和不可重复读等。

（1）丢失修改。事务A与事务B从数据库中读入同一数据并修改，事务B的提交结果破坏了事务A提交的结果，导致事务A的修改被丢失。

（2）读脏数据。事务A修改某一数据，并将其写回磁盘，事务B读取同一数据后，事务A由于某种原因被撤消，这时事务A已修改过的数据恢复原值，事务B读到的数据就与数据库中的数据不一致，是不正确的数据。

（3）不可重复读。事务A读取数据后，事务B执行了更新操作，事务A使用的仍是更新前的值，造成了数据不一致性。

答案-问题（3）

（1）加写锁

（2）加读锁

（3）加写锁

（4）等待

（5）得到通知

（6）加写锁

采用封锁的方法虽然可以有效防止数据的不一致性，但封锁本身也会产生一些麻烦，最主要就是死锁问题。也就是说，多个事务申请不同封锁，由于申请者均拥有一部分封锁权，而又需等待另外事务拥有的部分封锁而引起的永无休止的等待。

（2013年5月系统分析师真题-第4题）

阅读以下关于数据库分析与设计的叙述，在答题纸上回答问题1至问题4。

某集团公司在全省均设有分公司，现欲建立全国统一的销售管理信息系统，以便总公司及时掌握各分公司的销售情况。公司成立专门的项目组进行该系统的研发工作，其中张工负责其中的数据库设计工作。

张工和需求分析小组紧密合作，在设计出数据流图和数据字典的基础上，给出了数据库关系模式和相应的索引设计。同时考虑到未规范化关系模式可能引起的各类数据错误，对关系模式进行了全面的规范化处理，使所有关系模式均达到了3NF或BCNF。

在项目实施过程中，应用开发小组认为该设计方案未考虑应用功能的实际需求。如果严格按照设计方案实施，会对应用系统中整体性能产生较大影响。主要的原因在于进行数据查询时，会产生大量的多表连接操作，影响性能。而设计方案中的索引设计，并不能完全满足数据查询的性能要求。

应用开发小组还认为，该设计方案未考虑到信息系统中核心销售数据处理的特点：各分公司在使用该信息系统时只能操作自己分公司的销售数据，无权操作其他分公司的销售数据；只有总公司有权利操作所有销售数据，以便进行统计分析。

应用开发小组要求，在数据库设计方案中，必须针对实际应用功能的实现来考虑关系模式的规范化，必要时需要采用逆规范化或解除规范化的方法来保证性能要求。

问题（1）

系统需要管理供应商和货物等信息，具体包括供应商姓名、地址以及货物名称、价格等，供应商可以提供0~n种货物，其公司地址也可能发生变化。请以供应商关系模式

supplier (name, address, product, price) 为例，解释不规范的关系模式存在哪些问题。

思路解析

在数据库设计过程中，如果直接采用未经过规范化的关系模式，就可能导致一系列数据管理上的问题。以 supplier (name, address, product, price) 为例，该关系模式将供应商的基本信息与所提供的货物信息混合在一张表中，造成了结构上的冗余与潜在风险。

首先，数据冗余是最显著的问题。比如一个供应商A提供了5种货物，那么供应商A的地址信息将在该表中重复存储5次。这不仅占用额外存储空间，而且一旦需要更新地址，必须在多条记录中同步更新，极易导致数据不一致。其次，插入异常也很明显。当数据库需要新增一个供应商的信息时，如果此时供应商还未提供任何货物，则只能插入“货物名”和“价格”字段为空值的记录。这种记录破坏了数据的完整性与一致性，同时还可能违反某些字段的非空约束。第三，修改异常表现在地址信息的更新上。当供应商A更换地址时，系统必须在多条记录中逐一更新，否则会出现同一供应商的不同地址，造成信息冲突。最后，删除异常同样严重。当系统删除供应商某一货物时，该供应商的基本信息可能也随之丢失，从而丢失了该供应商仍然存在的事实，这对管理系统的逻辑正确性是极大的破坏。

因此，从冗余、插入、修改、删除四个方面来看，未规范化的关系模式会严重影响数据库的数据一致性与可靠性。通过对关系模式进行规范化处理（如将供应商表与货物表拆分为两个关系，再通过外键关联），可以很好地解决这些问题。

问题（2）

应用开发小组认为张工的规范化设计虽然解决了未规范化关系模式带来的问题，但实际实现功能时会造成系统性能的下降，请解释其原因。

思路解析

数据库规范化的本质在于通过分解关系模式来消除数据冗余和更新异常。经过规范化处理的关系模式，通常能够达到3NF甚至BCNF，这在理论上能最大限度保证数据一致性。然而，规范化带来的一个直接后果就是数据分布到多个表之中，而应用系统在查询时往往需要从多个表中获取数据。这种情况下，系统必须执行多表连接（JOIN）操作才能满足查询需求。

在实践中，多表连接操作的开销通常很大，尤其在面对大规模数据时，性能下降更加明显。具体来说，连接操作会涉及大量的磁盘I/O、内存缓存以及索引查找，即使索引设计合理，也很难完全抵消因表数量过多带来的性能损失。比如一个规范化后的系统中，查询某一供应商的基本信息和其货物

信息，至少需要连接“供应商表”和“货物表”，而在更复杂的场景下，可能需要三张甚至更多表的连接，造成SQL语句复杂、执行时间延长。

因此，虽然规范化能有效避免冗余、插入、修改、删除异常，但在性能需求较高的实际应用场景下，它可能并不是最优解。为了解决这种矛盾，数据库设计往往需要在规范化与性能之间进行权衡。在需要频繁查询的大数据量场景下，可以考虑在保持一定规范化的同时，适当引入逆规范化手段，以减少连接次数，提高查询效率。

问题（3）

请解释逆规范化方法，说明其优缺点。

思路解析

逆规范化（Denormalization）是指在数据库设计中，出于系统性能优化的目的，有意地放宽规范化要求，将多个已分解的关系模式合并，或者冗余部分字段，从而减少多表连接操作，提升查询与统计效率。它是数据库设计中的一种折中方法，用于解决规范化与性能之间的矛盾。

优点方面，逆规范化最大的优势是提高查询和统计性能。例如，在一个高度规范化的系统中，如果用户频繁查询供应商及其商品价格，就必须进行多表连接。而采用逆规范化，可以将“供应商名称、地址、货物名称、价格”等信息存储在同一表中，查询时只需访问一张表即可，大大降低连接操作的代价。同时，逆规范化还可能简化应用程序逻辑，减少代码复杂性，因为数据已经冗余存储在单一结构中。

缺点方面，逆规范化会带来一系列数据管理问题。首先是数据冗余，相同的信息可能存储在多个地方，增加了存储空间消耗。其次是更新代价，当某个字段发生变化时，系统需要同步更新所有相关记录，否则就会造成数据不一致。例如供应商地址修改时，如果分布在多条记录中，必须同时更新，否则会出现同一供应商多个地址的冲突。此外，逆规范化还可能重新引入插入异常、删除异常等问题，使得数据库完整性受到破坏。

问题（4）

针对该信息系统中核心销售数据处理的特点，如采用关系表水平分割的逆规范化方法，请给出具体的解决方案，并说明该方案存在的问题。

思路解析

在该销售管理信息系统中，存在一个明显的权限和数据分布需求：分公司只能访问和管理本公司数据，而总公司需要整合全国各地的销售数据。针对这种特点，可以采用水平分割（Horizontal Partitioning）的方法，即按照地域或分公司将销售数据分表存储。例如，可以在数据库中建立 sales-beijing、sales-shanghai、sales-guangdong 等表，分别存储各地分公司的销售数据。这样，分公司在操作时只需访问本地表，保证了数据访问的隔离性和安全性。

该方案的优点是明显的：

性能优化：每个分公司只需操作本地表，数据量相对较小，查询和更新效率较高。

安全控制：通过权限分配，分公司只能访问自己的表，避免了误操作或越权访问。

管理清晰：数据的物理划分符合企业的组织架构，便于管理和维护。

然而，该方案也存在一定问题：

首先，总公司统计效率低。由于数据分散在多个表甚至多个服务器中，总公司进行全国范围的数据汇总时，必须访问所有分表并进行合并操作，这一过程比单表聚合复杂且低效，尤其在跨服务器查

询时，网络开销较大。其次，应用程序复杂性增加。为了支持不同分公司访问不同的数据表，应用层需要动态判断用户所属分公司并路由到对应的表，这增加了开发和维护成本。同时，如果未来公司组织结构调整或分公司合并拆分，数据库表结构也必须随之调整，灵活性降低。

答案-问题（1）

- （1）数据冗余：关系模式中多次重复记录了同一供应商的地址。
- （2）插入异常：如果还未确定一个供应商有哪些货物，只是想添加一个供应商的地址信息，则会产生产品与价格均为空的记录。
- （3）修改异常：当修改一个供应商的地址时，需要将多条记录同时更新，若未同时更新，则数据产生不一致。
- （4）删除异常：当删除一个供应商的货物时，其地址信息被一并删除。

答案-问题（2）

数据库规范化的过程，实际是对数据表的不断拆分，以达到更高的规范程度。这样处理，带来的问题是：系统中大量查询不能通过单表完成，而需要将多表进行连接查询，所以表拆分得越多，查询性能也就越差。

答案-问题（3）

逆规范化方法优点：提高统计、查询效率。逆规范化方法缺点：增加了数据冗余，浪费存储空间，增、删、改操作的效率降低，可能导致数据不一致，可能产生添加、修改、删除异常。

答案-问题（4）

解决方案：将各省的数据存放于各省分公司。该方案主要问题：（1）在于总公司进行全国数据统计时，需要从各省服务器调取数据，效率较低。（2）执行应用功能时需要动态选择分公司的数据库表，增加了应用程序的复杂度。

(2024年5月系统分析师真题-第3题)

在某大型大学的教务管理系统中，学校需要管理大量的教学数据。包括教师信息、学生信息、课程信息、学生选课情况等。为了提高系统的查询效率并保证数据一致性，开发团队采用了数据库规范化技术，进行了一系列的范式设计，以消除数据冗余、避免更新异常、插入异常和删除异常等问题。

教师信息（Teacher） 属性：教师ID（TID）、教师姓名（TName）、教师所在院系（Department） 主键：教师ID（TID） 函数依赖：TID → TName, Department
属性：教师ID（TID）、教师姓名（TName）、教师所在院系（Department）
主键：教师ID（TID）
函数依赖：TID → TName, Department

学生信息（Student） 属性：学生ID（SID）、学生姓名（SName）、学生出生日期（DOB） 主键：学生ID（SID） 函数依赖：SID → SName, DOB
属性：学生ID（SID）、学生姓名（SName）、学生出生日期（DOB）
主键：学生ID（SID）

函数依赖: $SID \rightarrow SName, DOB$

课程信息 (Course) 属性: 课程ID (CID)、课程名称 (CName)、学分 (Credits) 主键: 课程ID (CID) 函数依赖: $CID \rightarrow CName, Credits$

属性: 课程ID (CID)、课程名称 (CName)、学分 (Credits)

主键: 课程ID (CID)

函数依赖: $CID \rightarrow CName, Credits$

学生选课情况 (Enrollment) 属性: 学生ID (SID)、课程ID (CID)、教工 ID (TID) 主键: 学生ID (SID)、课程ID (CID); 学生ID (SID)、教工 ID (TID) 函数依赖: $SID, CID \rightarrow TID, SID, TID \rightarrow CID, TID \rightarrow CID$

属性: 学生ID (SID)、课程ID (CID)、教工 ID (TID)

主键: 学生ID (SID)、课程ID (CID); 学生ID (SID)、教工 ID (TID)

函数依赖: $SID, CID \rightarrow TID, SID, TID \rightarrow CID, TID \rightarrow CID$

问题 (1)

请问这四种关系是否满足BCNF, 如果不满足用什么方式满足。

思路解析

这里就要用到凯恩提到的判断方法。若一个关系为 $R(U)$, 它是满足第一范式的, 当 R 中不存在任何属性对候选码的传递函数依赖时, 则称 R 符合BCNF, 即 $R \in BCNF$ 。这个书本写的很抽象。翻看别的定义也是如此。所以直接看例子理解。BCNF一定是3NF,

这个比较直观, 因为直接看它的定义, 包含了对3NF的定义。一个判断的简单方法是在BCNF范式中, 关系模式中任何函数依赖的左侧必须是码, 不能存在非码的属性。每名教师只教授一门课程, 每门课程由若干名教师教授, 若某一学生选定某门课程, 就确定了一名教师; 同样地, 某名学生选修了某名教师就确定了所选课程的名称。

(教工号, 学号), (学号, 课程号) 可以作为主键

但是由于存在教工号 $\rightarrow$ 课程号。

推论判断: 因为教工号不是码, 所以不是BCNF范式

(如何规范化) 可将授课分解为两个关系模式, 即(学号, 教工号)和(教工号, 课程号), 它们之中不存在任何属性对候选码的部分函数依赖和传递函数依赖, 所以符合 Boyce-Codd 范式。

问题 (2)

根据数据表, 补全 SQL 语句。再谈谈如何通过反规范化实现, 然后再说反规范化的缺点以及解决思路。

```
SELECT S.SName, C.CName, T.TName
FROM Enrollment E
JOIN Student (1)
JOIN Course C ON E.CID = C.CID
JOIN Teacher T ON E.TID = T.TID
WHERE (2);
```

思路解析

SQL 语句补全: 提供了查询学生选课情况的 SQL 语句。通过联接 (JOIN) Enrollment、Student、Course 和 Teacher 表, 按学生 ID 查询该学生的所有课程和对应教师信息。

反规范化：反规范化通常用于优化查询性能，尤其是在高频查询场景下。常见的反规范化策略包括将计算结果存储到数据库中、减少表之间的联接（通过冗余存储数据）以及对表进行水平或垂直分割等。

反规范化的缺点：虽然反规范化可以提高查询性能，但它带来了冗余数据、更新异常、插入异常和删除异常等问题，可能导致数据一致性问题。

解决方法：为了减少反规范化的缺点，可以使用以下方法：

部分反范式化：只对高频查询的表或字段进行反规范化，避免影响整个系统。

触发器或应用层同步：使用触发器确保数据一致性，或在应用层手动同步冗余数据。

3. 物化视图：通过物化视图来存储计算结果，避免查询时的冗余存储。

定期批处理作业：定期检查并修复冗余数据，保持数据一致性。

问题（3）

针对于王工采用触发器来解决关系表问题，采用哪两种触发器。以及对于触发器这种方式是否合理，其有什么缺陷。如何不采用触发器实现。

思路解析

触发器的类型：

AFTER INSERT 触发器：用于在插入数据后自动执行操作。例如，当一个学生选课时，可以使用该触发器确保相应的教师和课程信息更新。

AFTER UPDATE 触发器：用于在数据更新后自动执行操作。例如，当学生修改选课信息时，可以使用该触发器更新相关数据。

触发器的缺点：

性能问题：触发器的执行会增加数据库的开销，尤其在大量数据操作时，可能会影响数据库的整体性能。

调试困难：触发器是隐式执行的，调试时难以发现问题，错误也较难定位。

维护复杂：随着系统复杂度的增加，触发器的管理和维护变得更加困难，可能导致意外的副作用。

不采用触发器的实现方式：

应用程序级别的约束和验证：在应用程序中实现逻辑检查，确保数据一致性。例如，可以在应用层面检查插入数据的有效性，再提交到数据库。

存储过程：将业务逻辑封装在数据库的存储过程中，确保数据操作遵循一致性规则。

定期批处理作业：可以通过定期的批处理作业来检查和修复数据库中的数据不一致性，这适用于数据一致性要求较低的场景。

答案-问题（1）

教师信息（Teacher） 属性：教师ID（TID）、教师姓名（TName）、教师所在院系（Department） 主键：教师ID（TID） 函数依赖：TID → TName, Department

这个表满足BCNF，因为左侧的属性（TID）是主键，且函数依赖是基于主键的，没有任何非主键属性依赖于其他非主键属性。

属性：教师ID（TID）、教师姓名（TName）、教师所在院系（Department）

主键：教师ID（TID）

函数依赖：TID → TName, Department

这个表满足BCNF，因为左侧的属性（TID）是主键，且函数依赖是基于主键的，没有任何非主键属性依赖于其他非主键属性。

学生信息（Student） 属性：学生ID（SID）、学生姓名（SName）、学生出生日期（DOB） 主键：学生ID（SID） 函数依赖：SID → SName, DOB

同样，这个表满足BCNF，因为左侧的属性（SID）是主键。
属性：学生ID（SID）、学生姓名（SName）、学生出生日期（DOB）
主键：学生ID（SID）

函数依赖：SID → SName, DOB

同样，这个表满足BCNF，因为左侧的属性（SID）是主键。
课程信息（Course） 属性：课程ID（CID）、课程名称（CName）、学分（Credits） 主键：课程ID（CID） 函数依赖：CID → CName, Credits

这个表也满足BCNF，因为左侧的属性（CID）是主键。
属性：课程ID（CID）、课程名称（CName）、学分（Credits）
主键：课程ID（CID）

函数依赖：CID → CName, Credits

这个表也满足BCNF，因为左侧的属性（CID）是主键。
学生选课情况（Enrollment） 属性：学生ID（SID）、课程ID（CID）、教师ID（TID） 主键：学生ID（SID）、课程ID（CID）；学生ID（SID）、教师ID（TID） 函数依赖：SID, CID → TID | SID, TID → CID

问题点： 虽然（SID, CID）和（SID, TID）是候选键，但存在函数依赖 TID → CID，而TID不是候选键，这意味着它违反了BCNF的规则。 为了满足BCNF，可以将表拆分为两个：

Enrollment: （SID, TID）作为主键，属性为 SID 和 TID。 Teaching: （TID）作为主键，属性为 TID 和 CID。

属性：学生ID（SID）、课程ID（CID）、教师ID（TID）
主键：学生ID（SID）、课程ID（CID）；学生ID（SID）、教师ID（TID）
函数依赖：SID, CID → TID | SID, TID → CID

问题点：
虽然（SID, CID）和（SID, TID）是候选键，但存在函数依赖 TID → CID，而TID不是候选键，这意味着它违反了BCNF的规则。

为了满足BCNF，可以将表拆分为两个： Enrollment: （SID, TID）作为主键，属性为 SID 和 TID。 Teaching: （TID）作为主键，属性为 TID 和 CID。

Enrollment: （SID, TID）作为主键，属性为 SID 和 TID。

Teaching: （TID）作为主键，属性为 TID 和 CID。

这样，TID → CID 变成了一个基于主键的依赖，满足BCNF。

答案-问题（2）

（1）Student S ON E.SID = S.SID

（2）WHERE S.SID = '某学生ID'；

优点如下：

- （1）将常用的计算属性（例如，总计和最大值等）存储到数据库实体中。
- （2）重新定义实体，以减少外部属性数据或行数据的开支。
- （3）将关系进行水平或垂直分割，以提升并行访问度。

缺点如下:

- (1) 数据冗余增。反规范化会导致相同的数据在多个地方重复存储。
- (2) 更新异常。由于数据冗余,当一处数据需要更新时,可能会遗漏其他重复存储的地方,导致数据不一致。
- (3) 插入异常。某些情况下,为了满足反规范化的结构要求,可能需要先插入一些不必要的数据。
- (4) 删除异常。删除某些数据可能会意外删除其他有用的信息。

常用方法:

部分反范式化: 仅对高频查询的表或字段进行反范式化,避免全面反范式化带来的问题。

触发器或应用层同步: 使用数据库触发器或应用代码确保冗余数据的一致性。

物化视图: 使用视图加速查询,并避免手动进行冗余存储。

定期批处理作业: 定时检查并修复不一致数据。

答案-问题(3)

两种触发器: AFTER INSERT 和 AFTER UPDATE 触发器。

触发器的问题: 性能问题、调试困难、维护复杂。

不采用触发器的实现方式: 应用程序级别的约束和验证、存储过程、定期批处理作业。

(2024年11月系统分析师真题-第3题)

阅读以下关于数据库设计的叙述,在答题纸上回答问题1至问题3。

【说明】

某公司为了鼓励员工考取技能证书,提升公司竞争力,准备设计一套技能需求水平管理系统。王工在设计之处进行了系统分析,建立了以下数据库模型:

技能表(技能ID、技能描述),主键是技能ID;

员工表(员工ID、姓名、部门、联系方式),主键是员工ID;

项目表(项目ID、项目名称、项目描述)、主键是项目ID;

员工技能表(员工ID、技能ID、技能水平);

项目技能需求表(项目ID、技能ID、需求水平)。

备注:技能水平分为初级、中级、高级,一个员工有多个技能。一个项目需要多个技能,一个项目也可以同一个技能需要多个水平。

问题(1)

员工技能表的主键、函数依赖是?满足第几范式?项目技能需求表主键、函数依赖是?满足第几范式?

思路解析

对员工技能表,由于一个员工可对应多个技能、一个技能也可由多名员工掌握,用单列(员工ID或技能ID)均无法唯一标识记录,故候选码为(员工ID,技能ID),取其为主码;在该表中唯一的非主属性是“技能水平”,它对整个主码完全函数依赖:给定某员工与某技能的组合即可唯一确定其水平,不存在对主码某一真子集的依赖,因而无部分函数依赖;同时表内也不存在“非主属性→非主属性”的传递依赖。据此该表满足3NF。更进一步地,表内唯一的非平凡决定因素就是主码本身,任何决定因素都是超码,因而亦满足BCNF。这一判断与业务语义一致:员工的技能水平不依赖于姓名

、部门等外部属性，仅由(员工ID, 技能ID)决定。

对项目技能需求表，为满足“同一项目对同一技能可要求多个水平”的前提，必须将需求水平纳入主码，否则将无法同时保存“中级”“高级”两条需求，或被迫以重复列等非规范化方式建模。采用主键(项目ID, 技能ID, 需求水平)后，表中不存在非主属性（或仅有主属性），其非平凡函数依赖只有主码对自身（恒等依赖），因此自然满足3NF。同时，任一决定因素都是超码，亦满足BCNF。若业务改为“至少达到某一最低水平”，则可将需求水平改为“最低水平”并取(项目ID, 技能ID)为主键，但那是另一种业务假设，本题不采纳以避免与原题设矛盾。

问题（2）

为查询各个项目中满足技能的员工姓名和技能水平，王工创建一个视图，视图伪代码是：

```
select 项目名称, (1), 技能ID, 技能水平
from 员工表t1, 项目表t2, (2) t3, 项目技能需求表 t4
where t3.员工ID=t1.员工ID and t4.项目 id=t2.项目ID and (3) and t3.技能ID=t4.技能ID.
请根据题意完善上述伪代码。
```

上面的语句连接有性能问题，如何优化，优化的好处和缺点是什么？

思路解析

原伪代码最大的问题是把t4.员工ID = t1.员工ID写在了条件中，但项目技能需求表本身并不包含员工ID，这是错误的连接条件，会导致语义不成立或被数据库拒绝。正确的做法应当先用员工技能表(t3)把“具备某技能且达到某水平的员工”筛选出来，再与“项目技能需求表”(t4)的(技能ID, 需求水平)做等值连接，进而再关联项目表与员工表以取出展示字段，因此(2)必须是“员工技能表”，(3)必须包含两条关键连接：t1.员工ID = t3.员工ID与t3.技能水平 = t4.需求水平。

在性能层面，主连接谓词集中在等值连接与低基数列（如技能水平仅三种），若索引设计不当会出现扫描放大。为提升选择性，应在“连接共同出现的列”上构建复合索引：例如在员工技能表建立(技能ID, 技能水平, 员工ID)顺序，使优化器可先据技能ID缩小范围，再据技能水平过滤，最后回表或直接覆盖输出员工ID；在项目技能需求表建立(技能ID, 需求水平, 项目ID)，与之“对称匹配”。此外，确保员工表、项目表以主键索引支撑最终回表。仅选必需列减少网络与I/O；对于高复用报表型查询，可考虑物化视图或结果缓存。

这些优化的优点是显著降低I/O、提升QPS与稳定性；缺点在于写入与变更时需要维护更多索引、占用额外空间，物化视图还需制定刷新策略并处理与源表的一致性问题。

问题（3）

王工设计师觉得还有数据冗余指出更新会异常，请问如何优化？请给出新表。

思路解析

观察现有数据库模型，在员工技能表中，每个员工的技能水平重复记录；项目技能需求表中，项目对技能的需求水平也可能多次出现。比如有多个员工具备“中级编程”技能，“中级”这个水平信息在员工技能表中会多次存储；多个项目都要求“中级编程”技能，同样“中级”在项目技能需求表中也会重复。

这种重复存储就导致数据冗余。当要修改“中级”技能水平的定义或描述时，若在员工技能表中改了，而项目技能需求表中没改，就出现更新异常。

答案-问题（1）

员工技能表主键：（员工 ID，技能 ID）。函数依赖：员工 ID，技能 ID → 技能水平。该表满足第三范式（3NF）。

项目技能需求表：主键是（项目 ID，技能 ID，需求水平）。函数依赖：（（项目 ID，技能 ID，需求水平）→ （项目 ID，技能 ID，需求水平）。该表满足第三范式（3NF）。

答案-问题（2）

员工姓名、员工技能表、t3. 技能水平=t4. 需求水平。

添加索引：在员工表(员工ID)、项目表(项目ID)、员工技能表(员工ID, 技能ID)创建复合索引
优化内容：

改写为ANSI JOIN，配合复合索引：员工技能表(技能ID，技能水平，员工ID)、项目技能需求表(技能ID，需求水平，项目ID)、员工表(员工ID)、项目表(项目ID)。

仅取所需列，避免SELECT *；用EXISTS替代不必要的笛卡尔扩张；考虑物化视图/结果缓存（视业务而定）。

优点：明显减少随机I/O与回表次数、提升连接选择性与执行计划稳定性。缺点：写入时索引维护成本上升、占用存储、物化视图需刷新与一致性管理。

答案-问题（3）

当前数据库模型中，员工技能表和项目技能需求表可能存在数据冗余。比如多个项目对同一技能有相同的需求水平，或者多个员工有相同的技能水平，这些重复的技能水平信息在表中多次存储。在更新技能水平相关信息时，可能会出现部分数据更新，而其他相同数据未更新的更新异常情况。可以通过引入一个技能水平表，将技能水平相关信息独立出来，减少数据冗余和更新异常。

技能水平表：（技能水平 ID，技能水平描述），主键为技能水平 ID。这样，技能水平的具体描述信息只存储一次。

员工技能表（优化后）：（员工 ID，技能 ID，技能水平 ID），主键为（员工 ID，技能 ID），通过技能水平 ID 关联技能水平表，记录员工所具备技能对应的水平。

项目技能需求表（优化后）：（项目 ID，技能 ID，技能水平 ID），主键为（项目 ID，技能 ID，技能水平 ID），通过技能水平 ID 关联技能水平表，记录项目对技能的需求水平。

通过这种方式，将技能水平信息集中管理，当技能水平的描述等信息需要更新时，只需在技能水平表中进行一次更新操作，减少了数据冗余和更新异常的风险。同时，原有的员工表、技能表、项目表结构可以保持不变。

（2024年11月系统分析师真题-第4题）

阅读以下关于物联网（IoT）的叙述，在答题纸上回答问题1至问题 3。

【说明】

在现代农业中，为了提高生产效率和作物质量，越来越多的农场开始利用物联网（IoT）技术。某农业企业计划建设一个农业物联网项目，涉及多种农业传感器的数据采集，支持多种协议，并搭建一个高效的数据处理平台。该项目使用了 IoTDB 时序数据库，以及Netty和Kafka Cluster技术。

问题（1）

请简述时序数据库的特点，结合本项目说明为什么使用时序数据库。

思路解析

时序数据库 (Time Series Database, TSDB) 是近年来在大数据和物联网场景中广泛使用的一类数据库, 它与传统关系型数据库或NoSQL数据库最大的不同点在于其设计目标专注于处理带时间戳的大量数据。在物联网项目中, 传感器数据天然符合时间序列的定义, 因此采用时序数据库往往能够在性能、存储和分析能力方面体现出明显优势。

首先, 时序数据库具备高写入性能。农业物联网项目中可能部署成百上千个传感器, 这些传感器会以秒级甚至毫秒级的频率采集数据。如果采用传统关系型数据库, 大量高并发写入操作将导致锁冲突和性能瓶颈, 系统可能无法实时处理。而时序数据库针对这一特性做了专门优化, 例如批量写入、内存缓冲、数据分区等机制, 能够确保系统在高并发情况下依然保持稳定写入能力, 避免数据堆积。

其次, 存储效率的提升也是重要原因。传感器数据往往在时间上连续且规律性较强, 例如温度在短时间内变化幅度有限。时序数据库通常采用差分压缩、基于时间间隔的存储优化等技术, 大幅降低存储开销。对于长期运行的农业物联网系统, 这意味着能够在保证长期历史数据可用的前提下, 大幅减少存储成本。

第三, 查询与分析功能的优化。农业场景中不仅需要实时监控数据, 还需要进行长期趋势分析。例如, 农场管理者可能会关注过去三个月内的平均气温对作物生长的影响, 或者比较不同灌溉周期下湿度的变化趋势。时序数据库内置的窗口函数和降采样功能使得这些查询能够高效完成, 避免了传统数据库需要复杂SQL和大量数据扫描的问题。

问题 (2)

简述为什么使用 Kafka Cluster 模式

思路解析

Kafka 是一种分布式流处理平台, 其 Cluster (集群) 模式尤其适合大规模物联网场景。在农业物联网项目中, 大量传感器会同时上传数据, 如果单机处理, 很快会出现性能瓶颈甚至数据丢失。因此, 使用 Kafka Cluster 模式具有以下重要意义。

首先, 是高吞吐量。Kafka 采用分布式架构, 通过分区 (Partition) 机制将数据写入和读取任务分摊到多个 Broker 节点上, 能够实现水平扩展。在农业项目中, 即使数以万计的传感器同时传输数据, Kafka Cluster 依然可以保证数据的快速流转, 避免延迟积压。这对实时监控和响应极为重要。

其次, 是数据持久性和可靠性。Kafka 通过副本机制将数据复制到多个 Broker 节点, 即使某个节点宕机, 数据也不会丢失。这对于农业场景尤为关键, 因为传感器采集的数据往往与作物健康、灌溉控制等直接相关, 一旦数据丢失可能导致生产决策失误。

最后, 是可扩展性。随着农场规模的扩大, 传感器数量会逐渐增加, 单机架构无法应对数据量的增长。而 Kafka Cluster 可以通过增加 Broker 节点来轻松扩展处理能力, 实现系统的动态伸缩。这种设计保证了系统具备良好的扩展性和灵活性。

问题 (3)

请简要描述该项目使用 Netty 的理由。

思路解析

Netty 是一个基于 Java NIO 的高性能网络应用框架，广泛应用于分布式系统和高并发网络服务中。在农业物联网项目中，传感器数量庞大，数据上报频繁，这对网络通信层提出了极高的要求。选择 Netty 的理由主要有以下几个方面。

首先，异步事件驱动模型是 Netty 的最大特点。传统的阻塞式 I/O 处理在面对大规模并发连接时，往往需要为每个连接分配线程，容易导致线程资源耗尽。而 Netty 通过事件驱动机制和 Reactor 模式，能够在少量线程的支持下同时管理成千上万个连接，避免了线程切换和阻塞开销。这一点在农业物联网场景中至关重要，因为成百上千个传感器可能同时上传数据，Netty 能够保证连接的稳定性和高效性。

其次，数据传输效率的提升。Netty 引入了零拷贝（Zero-copy）技术，减少了数据在内核与用户空间之间的拷贝次数，从而降低 CPU 消耗并提升传输速度。对于农业物联网中持续不断的数据流，这种优化能够显著提高整体系统的响应效率。

再次，可扩展性和灵活性。Netty 提供了高度可定制的编程接口，开发者可以根据农业物联网的协议和业务需求，快速构建适配的数据处理管道。例如，可以在数据流中插入自定义的解码器、过滤器或日志模块，以满足复杂业务需求。这种灵活性使得系统不仅能支持现有传感器，还能轻松接入新型设备和协议。

最后，可靠性和成熟度也是选择 Netty 的原因之一。作为广泛使用的开源框架，Netty 已经在大型分布式系统中经过充分验证，具备高稳定性和活跃的社区支持。在农业物联网项目中，系统需要长期稳定运行，Netty 的成熟特性能够降低项目风险。

答案-问题（1）

时序数据库的特点：

（1）时间序列性。以时间戳作为数据的重要维度，数据按照时间顺序进行组织和存储，方便对具有时间先后顺序的数据进行管理和查询。

（2）高写入性能。能够快速处理大量的实时数据写入操作。在物联网场景中，众多传感器会持续不断地产生数据，时序数据库针对这种高频写入进行了优化，可高效存储新数据。

（3）数据压缩率高。对于按时间顺序记录的数据，往往存在一定规律和冗余，时序数据库利用特殊的压缩算法，能有效减少数据存储所需空间，降低存储成本。

（4）丰富的时间窗口函数。支持各种基于时间窗口的计算和分析函数，如在某段时间内求平均值、最大值、最小值等，便于对历史数据进行统计分析。

（5）数据降采样。可以对长时间跨度的大量数据进行降采样处理，在不丢失关键信息的前提下，减少数据量，提高查询效率，适用于对历史数据

在本项目中使用时序数据库的原因：

（1）农业传感器产生的温度、湿度、光照等数据都具有时间序列特征，适合存储在时序数据库中。

（2）高写入性能能够支持多种协议传感器产生的大量实时数据，保证写入的效率与稳定性。

（3）时间窗口函数和降采样功能便于分析不同时间段的环境变化对作物生长的影响，为农业生产提供科学依据。

答案-问题（2）

（1）高吞吐量：农业传感器会持续产生大量数据，Kafka Cluster 模式能处理高并发的数据写入和读取，满足项目中数据传输量大的需求，保证数据快速流转。

(2) 数据持久性和可靠性：集群模式下数据会复制到多个节点存储，即使部分节点故障，也不会导致数据丢失，确保农业数据的安全存储和可靠传输。

(3) 可扩展性：随着项目发展，传感器数量增加或数据量增大时，可通过添加节点轻松扩展集群，适应业务增长。

答案-问题（3）

Netty 采用异步事件驱动的方式处理 I/O 操作，所有 I/O 调用立即返回，通过 ChannelFuture 通知操作结果。在农业物联网大量传感器并发传输数据的情况下，能有效避免线程阻塞，确保系统稳定运行，提升整体性能和响应能力。同时可以减少数据在内存中的拷贝次数，降低 CPU 和内存资源消耗，提高数据传输效率，适合农业物联网中大量传感器数据的快速传输。

(2025年5月系统分析师真题-第4题)

某电商平台设计了如下两个表：

订单表（orders）：包含字段为：订单号（order-id）、用户编号（user-id）、商品编号（product-id）、商品名称（product-name）、优惠券编号（coupon-id）、订单时间（order-time）、订单金额（order-amount）、订单状态（order-status）

用户表（users）：包含字段为：用户编号（user-id）、用户名（user-name）、累计金额（total-amount）、用户VIP级别（user-vip-level）

现要求对数据库进行范式优化，并设计一个触发器用于在订单插入后自动更新用户的VIP等级。

问题（1）

对订单表进行分解，使其满足 BCNF 范式

思路解析

原始 orders 中除主码 order-id 外还存放 product-name，存在如下函数依赖：order-id → user-id, product-id, product-name, coupon-id, order-time, order-amount, order-status 与 product-id → product-name。由于 product-id 不是候选码 却能决定 product-name，违反 BCNF（Boyce-Codd 正规形）“每个决定因素都必须是候选码”的要求。

为消除该 非键决定因素 带来的冗余与更新异常（如修改商品名需批量更新订单、删除最后一笔订单导致商品名丢失等），采用模式分解：将与商品维度相关的依赖剥离到独立的 products 表（主码 product-id），订单事实留在 orders-core（主码 order-id）。分解后，orders-core 内仅存在由候选码 order-id 决定的依赖，products 内由候选码 product-id 决定 product-name，两表各自 满足 BCNF，同时降低存储冗余并消除插入/删除/更新异常。

问题（2）

对用户表进行范式分析，判断属于第几范式，并列出其函数依赖。

思路解析

第一范式（1NF）：表中所有字段为原子值，无重复组或多值字段，满足 1NF。

第二范式（2NF）：主键为单一字段 user-id，所有非主属性完全依赖主键，无部分依赖，满足 2NF。

第三范式 (3NF) : : user-vip-level 依赖于 total-amount, 不满足 3NF。从问题 4 中也可以知道这个依赖关系。

问题 (3)

原始订单表模式分解后是否保持函数依赖; 模式分解后是否为无损分解。

思路解析

一般来说无损连接用公式法 ($(R1 \cap R2) \rightarrow R1 - R2$ 或者 $(R1 \cap R2) \rightarrow R2 - R1$), 函数依赖用定义法即可求解 (分解之后成两个表之后, 订单表包含第一个依赖, 商品表包含第二个依赖, 依赖没有丢失)。

问题 (4)

请补全触发器语句, 并指出该触发器可能存在的性能问题, 提出至少两种优化方案。

```
CREATE -- update-vip-level AFTER INSERT ON orders FOR EACH ROW BEGIN UPDATE users SET
user-vip-level = CASE WHEN total-amount >= 10000 THEN '钻石VIP' WHEN total-amount >=
5000 THEN '黄金VIP' WHEN total-amount >= 1000 THEN '普通VIP' ELSE '普通用户' END WHERE
user-id = --; END;
```

思路解析

在实际生产环境中, 上述 AFTER INSERT 行级触发器虽然语义直观, 但每写入一笔订单就立即执行两次对 users 表的更新: 一次累计消费、一次刷新 VIP 等级, 这意味着高并发场景下同一用户的记录会被频繁加行级排他锁, 不仅放大事务日志和 redo/undo 开销, 还可能造成锁等待、死锁甚至阻塞主业务;

若 user-id 仅有主键索引而 total-amount 和 user-vip-level 列未做合并更新, InnoDB 需要两次 B+Tree 查找并触发页分裂或行迁移, IO 抖动更加明显;

与此同时, 哪怕用户等级并未发生变化依然会执行 UPDATE, 导致无效写放大与 binlog 冗余。

更优实践是: ①在触发器里先判断等级是否真的提升, 只有变动时才更新; ②把实时更新改为异步队列 + 定时批处理, 利用幂等消费和分片并发将热用户拆分到多台 Worker; ③或在 users(user-id, total-amount, user-vip-level) 上建覆盖索引并将两步更新合并为一条 UPDATE ... SET total-amount = total-amount + ?, user-vip-level = CASE ... END, 同时限定事务只锁定目标行; ④若业务允许最终一致性, 可用物化视图或 Spark/Flink 流式聚合离线刷新, 既节省在线库写压力, 又能利用列存/分区表做大规模汇总, 从而全面提升系统吞吐与稳定性。

答案-问题 (1)

订单主表: 订单号, 用户编号, 商品编号, 优惠券编号, 订单时间, 订单金额, 订单状态

商品表: 商品编号, 商品名称

答案-问题 (2)

用户编号是主码, 不存在部分依赖, 但是累计金额决定用户VIP等级, 存在传递依赖, 满足第二范式 (2NF)。

函数依赖关系: 用户编号 $\rightarrow$ 用户名, 累计金额, 用户VIP等级 累计金额 $\rightarrow$ 用户VIP等级

答案-问题（3）

原始订单表存在如下函数依赖：

订单号 → 用户编号，商品编号，商品名称，优惠券编号，订单时间，订单金额，订单状态

商品编号 → 商品名称

分解之后成两个表之后，订单表包含第一个依赖，商品表包含第二个依赖，依赖没有丢失。

若把 R 分解为 R₁ R₂（两表情形），只要

$(R_1 \cap R_2) \rightarrow R_1 - R_2$ 或者 $(R_1 \cap R_2) \rightarrow R_2 - R_1$

可以知道也就是判断 商品编号 → 商品名称是否成立，显然成立，所以无损分解。

答案-问题（4）

请补全触发器语句，并指出该触发器可能存在的性能问题，提出至少两种优化方案。

```
CREATE TRIGGER update-vip-level AFTER INSERT ON orders FOR EACH ROW BEGIN UPDATE users  
SET user-vip-level = CASE WHEN total-amount >= 10000 THEN '钻石VIP' WHEN total-amount  
>= 5000 THEN '黄金VIP' WHEN total-amount >= 1000 THEN '普通VIP' ELSE '普通用户' END  
WHERE user-id = NEW.user-id; END;
```

性能问题分析：

每插入一条订单，都会触发一次更新操作，可能对并发性能造成影响；

若用户下单频繁，可能对 users 表产生频繁写操作；

若未设置适当索引，更新时 WHERE user-id = NEW.user-id 查找会导致全表扫描。

优化方案：

延迟批量处理法：不在触发器中直接更新，而是通过队列表 + 定时任务批量更新用户VIP等级。

增加判断逻辑：在触发器中增加判断，仅在VIP等级实际变化时才执行UPDATE语句，减少无效更新。

使用物化视图或缓存方案：用物化视图定期汇总用户订单金额并更新用户表。

第3章——企业信息化战略与实施

（2021年5月系统分析师真题-第3题）

阅读以下关于数据管理的叙述，在答题纸上回答问题1至问题3。

某大型企业在长期信息化建设过程中，面向不同应用，开发了各种不同类型的应用软件系统，以满足不同的业务需求。随着用户需求和市场的快速变化，要求企业应能快速地整合企业的各种业务能力，为不同类型的用户提供多种流程的业务服务。但现有各个独立的应用系统难以满足日益增长和快速变化的用户需求。

目前该企业各个应用系统主要存在以下问题：

（1）应用系统是异构的、运行在不同软硬件平台上的信息系统；

（2）应用系统的数据源彼此独立、相互封闭，使得数据难以在系统之间交互、共享和融合，即存在“信息孤岛”；

（3）系统是面向应用的，各个应用系统中的数据模型差异大，即使同一数据实体，其数据类型、长度、值均存在不一致甚至相互矛盾的问题。

为此，该企业专门成立了研发团队，希望能够尽快解决上述问题。

问题（1）

李工建议采用数据集成的方式来实现数据的整合，同时构建新系统来满足新的需求。针对题干中的问题（3），李工提出首先应面向企业核心的业务主题，做好企业战略数据规划，建立企业的主题数据库，然后再进行集成系统的开发。请用200字以内的文字简要说明主题数据库的设计要求和基本特征。

思路解析

主题数据库的设计注重稳定性和长期支持企业信息资源的能力。它从企业的核心业务需求出发，通过合理的数据建模来确保数据的准确性与一致性。通过共享数据库的建设，减少了信息孤岛现象，提升了企业信息系统的协同工作能力。采用规范化的设计确保了数据库的高效运作与可维护性。

问题（2）

张工认为数据集成的方式难以充分利用已有应用系统的业务功能，实现不同业务功能的组合，建议采用基于SOA的应用集成方式，将原有系统的功能包装为多个服务，并给出了基本的集成架构，见图

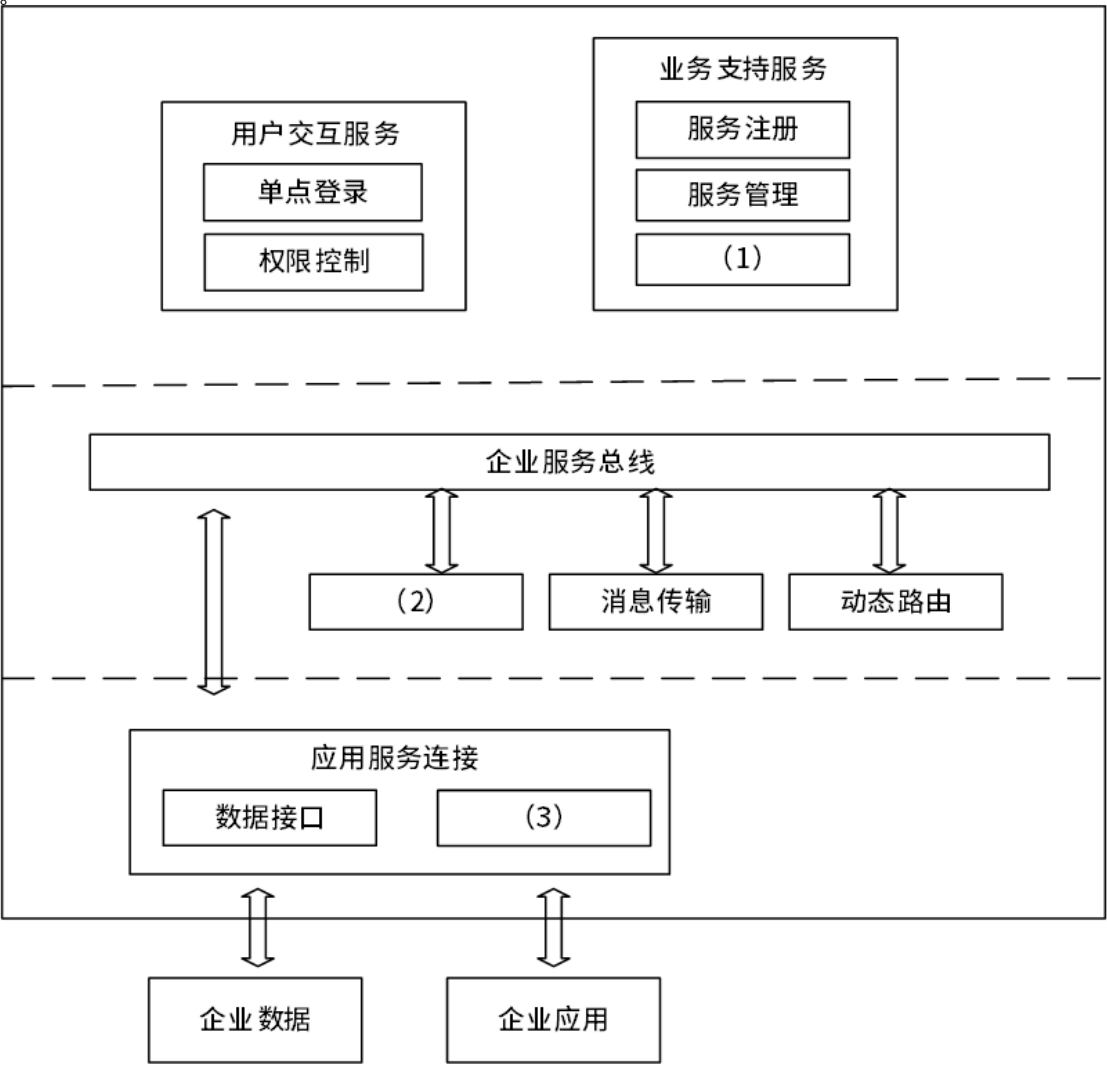


图 4-1 基于 SOA 的集成架构示意图

请补充完善图4-1中（1）—（3）处空白处的内容。

思路解析

SOA（面向服务架构）是一种通过将应用功能拆分为松耦合的服务来实现系统集成的架构。服务编排（Service Orchestration）是指通过定义业务流程，将多个服务组合在一起以实现业务逻辑。数据转换和协议转换是SOA架构中的核心功能，用于确保不同系统和服务之间能够兼容和交换数据。应用程序接口（API）是SOA架构中服务之间通信的核心，提供标准化的接口供不同的服务访问。通过这些组件，SOA能够有效整合企业的不同系统和服务，实现更高效的业务流程和更灵活的系统集成。

问题（3）

研发团队在对张工的方案进行分析后，发现该方案没有发挥SOA的核心理念，即松耦合的服务带来业务的复用，通过服务的编排助力业务的快速响应和创新，未实现"快速整合企业业务能力，为不同类型的用户提供各种不同功能、不同流程的业务服务"的核心目标，目前的方案仅仅是通过SOA实现了系统的集成。请用200字以内的文字分析该方案未满足本项目核心目标的原因。

思路解析

SOA的核心优势在于其松耦合性和业务服务的复用能力。如果服务粒度过大，系统的灵活性会受到限制，难以应对复杂且多变的业务需求。因此，为了实现业务的快速整合和创新，服务的设计应更细粒度，支持灵活的编排与组合，从而能够快速响应用户需求并提高系统的业务创新能力。

答案-问题（1）

设计要求：为了加速应用系统的开发，主题数据库的逻辑结构应独立于当前的计算机硬件和软件的实现过程，应设计得尽可能稳定。

- （1）面向业务主题：主题数据库是面向业务主题来组织的数据存储；
- （2）信息共享：主题数据库是不同应用系统共建共用的共享数据库；
- （3）一次一处输入系统：数据就地采集，就地处理、使用和存储，以及必要的传输、汇总和集中存储；
- （4）由基本表组成：主题数据库由多个达到基本规范化要求的数据实体构成。

答案-问题（2）

- （1）服务编排
- （2）数据转换（协议转换皆可）
- （3）应用程序接口/应用接口

答案-问题（3）

- （1）服务粒度的问题：服务是对原有系统功能的包装，通常是粗粒度的，很难实现真正意义上的细粒度、松耦合的服务。
- （2）服务编排：由于粗粒度的服务，难以进行真正意义上灵活的服务编排。

（2015年5月系统分析师真题-第4题）

阅读以下关于软件系统运行与维护的叙述，在答题纸上回答问题1至问题3。

随着信息化的发展，某银行的中心账务系统，从城市中心、省中心模式已经升级到全国中心模式。

但是处理各种代收代付业务的银行中间业务系统，目前仍然采用省中心模式，由各省自行负责，使得全国中间业务管理非常困难。因此总行计划将银行中间业务系统全部升级到全国中心模式，对各省中间业务进行统一管理。

各省行采用的银行中间业务系统，均为各省自建，或者自行开发，或者自行采购，系统的硬件平台、软件系统、数据模式等均有非常大的差异。同时，对一些全国性的代收代付业务的处理方式，各省行也存在很大的差异。为统一管理，总行决定重新开发一套全国中心模式的银行中间业务系统，用来替代各省自建的中间业务系统，但要求能够支持目前各省的所有中间业务。

问题（1）

各省已建的银行中间业务系统属于遗留系统，在如何对待遗留系统上，设计组存在两种不同的策略：淘汰策略和继承策略。请简要解释这两种策略，并说明新开发的银行中间业务系统适合采用哪种策略及其原因。

思路解析

淘汰策略指对业务价值低、技术债重且难以维护的遗留系统整体停用并以全新系统替代。其优点是可在新架构上“一步到位”清除历史包袱，但在关键行业往往伴随高切换风险、数据清洗成本大、人员培训与流程重塑周期长等问题。继承策略强调在重构/重建新系统的同时，对既有功能模型、数据模型与外部接口协议进行兼容与映射，通过适配层、网关与数据同步机制实现平滑过渡，并允许新旧系统并行期内逐步替换。

本题场景下，各省系统异构严重、承载持续运营中的代收代付等关键业务，且明确“必须支持各省所有现有业务”。因此应采用继承策略：在全国中心侧建立统一标准模型与省级适配器，保留存量规则与数据口径，通过接口适配、数据映射、规则统一与灰度替换实现最小化中断与风险控制，满足监管与连续运营的刚性要求。

问题（2）

遗留系统和新系统之间的转换策略常见的有直接转换、并行转换和分段转换。请简要说明这三种转换策略的含义；并结合银行中间业务的特点，说明该银行新开发的中间业务系统上线时适合采用哪种策略？为什么？

思路解析

直接转换成本与组织复杂度最低，但对数据准确性、流程一致性与新系统成熟度要求极高；在全国级、强监管、资金敏感的银行场景中，一步切换的故障影响面过大，不可取。并行转换通过新旧系统同步跑批与在线核对来降低风险，便于口径校准与人员培训，但在全国范围长期并行会导致成本与维护压力显著上升。分段转换将切换拆解为可控的小批量批次（如先选若干试点省份或先切某些非高峰业务），每段可短期并行+校验后再收口，逐步推广至全国；同时可按业务重要度和依赖关系排定优先级（如先低风险代收，再高频代付），并配置回滚预案、结算对账双轨验证、主数据与代码表统一治理。

在本案例中，省际差异、历史数据复杂度与合规要求决定了分段转换最能兼顾风险、成本与可操作性；必要时在每段内短期采用并行核验，最终实现全国范围的平滑切换。

问题（3）

银行中间业务系统中，最为核心的是业务数据。因此在新旧系统切换时存在一项重要的工作：数据

迁移。考虑到各省中间业务系统的巨大差异，因此需要做好数据迁移前的准备工作。请简要说明数据迁移准备工作的内容。

思路解析

数据迁移的成功依赖充分的前期准备。首先要完成数据盘点：明确各省行来源系统、库表/文件与接口、数据规模与保留周期，并标识敏感字段与合规要求（脱敏/加密）。其次建立统一数据字典，对主数据、交易数据、代码表进行口径统一与差异比对，输出字段映射与转换规则（含单位、精度、编码、时区、字符集）。对不可映射/缺失/脏数据制定修复或兜底策略。技术上需选定并部署 ETL/数据同步工具，设计幂等、可重试、断点续传机制与性能参数。

同时制定迁移验证方案：含多轮演练（全量/增量）、账务对账/笔数与金额校验、抽样核对与性能回归；明确变更窗口与业务冻结范围，落实全量备份、校验签名、回滚脚本与应急联系人矩阵。

（可以写上更加完整）最后完善安全与审计：访问控制、操作留痕与审计报告，确保迁移过程可追踪、可复现、可回退。

答案-问题（1）

继承策略，原因：新开发的系统要求"能够支持目前各省的所有中间业务"，淘汰策略无法达到此要求。

答案-问题（2）

直接转换就是在原有系统停止运行的某一时刻，新系统立即投入运行，中间没有过渡阶段。

并行转换就是新系统和现有系统并行工作一段时间，经过这段时间的试运行后，再用新系统正式替换下现有系统。

分段转换策略也称为逐步转换策略，这种转换方式是直接转换方式和并行转换方式的结合，采取分期分批逐步转换。

以目前该银行的情况来看，适合采用分段转换策略，因为各省行采用的银行中间业务系统，均为各省自建，或者自行开发，或者自行采购，系统的硬件平台、软件系统、数据模式等均有非常大的差异。对这些子系统的支持可分段一个一个处理，降低风险的同时不让成本开支过大。

答案-问题（3）

数据迁移准备工作包括：

- （1）待迁移数据源的详细说明，包括数据的存放方式、数据量和数据的时间跨度。
- （2）建立新旧系统数据库的数据字典，对现有系统的历史数据进行质量分析，以及新旧系统数据结构的差异分析。
- （3）新旧系统代码数据的差异分析。
- （4）建立新旧系统数据库表的映射关系，对无法映射字段的处理方法。
- （5）开发或购买、部署 ETL 工具。
- （6）编写数据转换的测试计划和校验程序。
- （7）制定数据转换的应急措施。

第4章——Web开发

(2022年5月系统分析师真题-第4题)

阅读以下关于微服务架构中的数据管理的叙述，在答题纸上回答问题1至问题3。

某大型电商平台构建了一个在线B2B商店系统。该系统采用微服务架构，将系统功能分解为多个松散耦合且可独立部署的较小组件或服务。最终设计的系统包括了电商系统中常见的服务：客户服务、订单服务、支付服务等，其中：

- 1、客户服务负责对客户相关的信息进行管理和维护；
- 2、订单服务负责对订单信息的管理和维护；
- 3、支付服务负责对在线支付功能和信息的管理和维护等。

为了确保微服务之间的松耦合，每个服务都有自己的数据，其中，订单服务使用了 NoSQL 数据库，客户服务和支付服务使用了关系数据库。

李工认为由于不同服务使用了各自的不同数据库，使得跨服务操作可能存在数据不一致。比如订单与支付的数据一致性问题，系统通过订单服务在本地 NoSQL 数据库中创建订单记录，同时在支付服务的关系数据库中创建支付记录，且必须保证订单记录和支付记录的一致性，该问题在系统构建时需要考虑。

问题（1）

李工建议采用两阶段提交协议（2PC）来解决服务数据的一致性问题。请用200字以内的文字简要说明2PC，说明2PC是否能解决该问题，并简要解释原因。

思路解析

两阶段提交协议（Two Phase Commitment Protocol, 2PC）是一种分布式事务处理协议，用于确保分布式系统中的所有组件都能够一致地处理事务。它将整个事务处理过程分为两个阶段，即准备阶段和提交阶段。

在准备阶段，事务协调器会询问所有的参与者，事务是否可以提交。每个参与者都将执行事务操作，并返回操作结果给事务协调器。如果所有参与者都返回成功，则进入提交阶段；否则，事务会被回滚到事务开始之前的状态。

在提交阶段，如果所有参与者都返回成功，则事务协调器将通知所有的参与者提交事务；否则，将通知所有的参与者回滚事务。

通过这种方式，两阶段提交协议可以确保分布式系统中的所有组件都能够一致地处理事务，从而保证数据的一致性和完整性。

但是此题的前提是不同数据库，涉及到 NoSQL。NoSQL 一般不支持事务，所以不适合引入 2PC。

问题（2）

王工建议采用分布式数据管理方案，用事件驱动架构来解决服务数据的一致性问题，在订单服务和支付服务之间通过可靠的消息队列实现事件的传递，其基本操作步骤如下，请填写其中的空白处。

- （1）订单服务接收订购请求，创建一个订单，该记录状态为（a），发布一个"创建订单"事件；
- （2）（b）接收"创建订单"事件，记录（c），发布一个"支付完成"事件；
- （3）订单服务接收"支付完成"事件，修改订单记录状态为（d）。

思路解析

在事件驱动架构中，各个微服务通过发布和订阅事件来通信，确保系统的松耦合性。在此方案中：

订单服务在接收到订单请求时，会首先创建订单，并将其状态设置为“未支付”。然后，订单服

务发布一个“创建订单”事件，通知支付服务进行支付处理。

支付服务在接收到“创建订单”事件后，记录支付信息，并更新支付状态。一旦支付成功，支付服务发布一个“支付完成”事件，告知订单服务支付已完成。

订单服务接收到“支付完成”事件后，更新订单的状态为“已支付”，表示订单已支付完成。

问题（3）

李工提出王工的方案会有数据库更新和发布事件的原子性问题，例如订单服务创建订单记录和发布“创建订单”事件需要原子性保障，否则会出现数据不一致状态。

王工认为可以使用本地事务发布事件的方法来解决该问题。请给出使用本地事务发布事件的基本方法，并说明该方法的缺点。

思路解析

本地事务发布事件方法的核心思想是利用数据库的事务机制保证业务数据的更新和事件的发布保持一致性。通过在数据库中创建EVENT表，模拟消息队列的功能，将事件记录插入数据库，并使用事务保证这两者的原子性。事务提交后，另一个线程将事件发布到消息队列中。这样可以保证即使在事件发送过程中出现问题，数据更新和事件发布的过程仍然是一致的。

然而，这种方案也有其缺点：

需要开发者手动确保在每次业务更新后都能正确发布事件，容易产生遗漏或逻辑错误。

使用NoSQL数据库时，事务支持较弱，可能无法保证强一致性，尤其在分布式系统中，这种方法会面临更复杂的事务管理问题，因此在使用NoSQL时实现起来较为麻烦。

答案-问题（1）

两阶段提交协议2PC经常用来管理分布式事务。

（1）2PC包含协调者和参与者两类站点，只有协调者才拥有提交或撤销事务的决定权，而其他参与者各自负责在其本地数据库中执行写操作，并向协调者提出撤销或提交事务的意向。

（2）2PC分为两个阶段：表决阶段和执行阶段。

①表决阶段，目的是形成一个共同的决定。协调者给所有参与者发送“准备提交”消息，并进入等待状态，所有参与者给与回复“建议提交”或“建议撤销”。只要有一个结点选择撤销，则整体事务撤销，否则，执行该事务。

②执行阶段，目的是实现这个协调者的决定。根据协调者的指令，参与者或者提交事务，或者撤销事务，并给协调者发送确认消息。

两阶段提交协议 2PC 不能解决当前问题。

（1）分布式数据库遵循的是 CAP 原则，会在一定程度上牺牲一致性。

（2）大多数 NoSQL 数据库并不支持 2PC。

（3）分布式两阶段提交协议2PC一般针对的对象在逻辑上是一个整体，对某一个整体事务需要在多个物理节点上执行时，进行表决和执行，对多个数据库的不同服务并不是很合适。

答案-问题（2）

(a) 未支付 (b) 支付服务 (c) 支付信息 (d) 已支付

答案-问题（3）

本地事务发布事件的方法：

在数据库中创建一张EVENT表，用来模拟消息队列，存储业务数据的状态。

通过一个本地事务，首先更新业务数据记录（如订单服务创建订单），然后将事件（如“创建订单”）插入到EVENT表中。

提交本地事务，确保数据库的更新和事件记录是原子操作。

使用一个单独的线程或服务轮询EVENT表，将事件消息从数据库取出，发送到消息队列（MQ）中，并将事件状态更新为“已发布”。

缺点：

易出错：开发者必须手动确保在数据更新后发布事件，容易出现忘记发布事件或发布失败的情况，特别是在高并发或复杂事务中，可能导致数据和事件的不同步。

NoSQL的局限性：在使用NoSQL数据库时，由于其事务和查询能力有限，实现此方案较为复杂。

NoSQL数据库通常不具备强一致性的事务支持，可能导致数据和事件的不一致。

(2022年5月系统分析师真题-第5题)

阅读以下关于 Web 系统设计的叙述，在答题纸上回答问题1至问题3。

某公司拟开发一个食品供应链溯源系统，该系统需要提供从原材料供应商、加工商、物流、分销商、零售商、消费者的食品供应链全流程溯源。该公司组建了项目组，并召开了项目开发讨论会。会上，张工提出通过二维码扫描获取食品信息，采用中心化数据库作为数据存储媒介；李工提出使用中心化数据库容易产生数据信任、溯源追责困难等问题，建议建立区块链和数据库的映射存储，提供存储和查询操作功能，并提出采用数据接入层、数据核心层、应用表示层三层体系架构实现该食品溯源系统。

问题（1）

去中心化和开放性是区块链的重要特征，请用200字以内的文字简要说明什么是区块链的去中心化和开放性。

思路解析

区块链的去中心化和开放性是其安全性和透明度的关键特性。去中心化使得所有参与者在没有中央机构的情况下进行信任交换，而开放性则确保了任何人都可以监督和验证链上的交易数据，从而增强了系统的可信任性。

问题（2）

分布式交易账本、哈希散列函数、公私钥签名、时间戳就是区块链的核心技术，请从上述技术中选择两种最适合解决数据信任问题的技术，并用300字以内的文字说明原因。

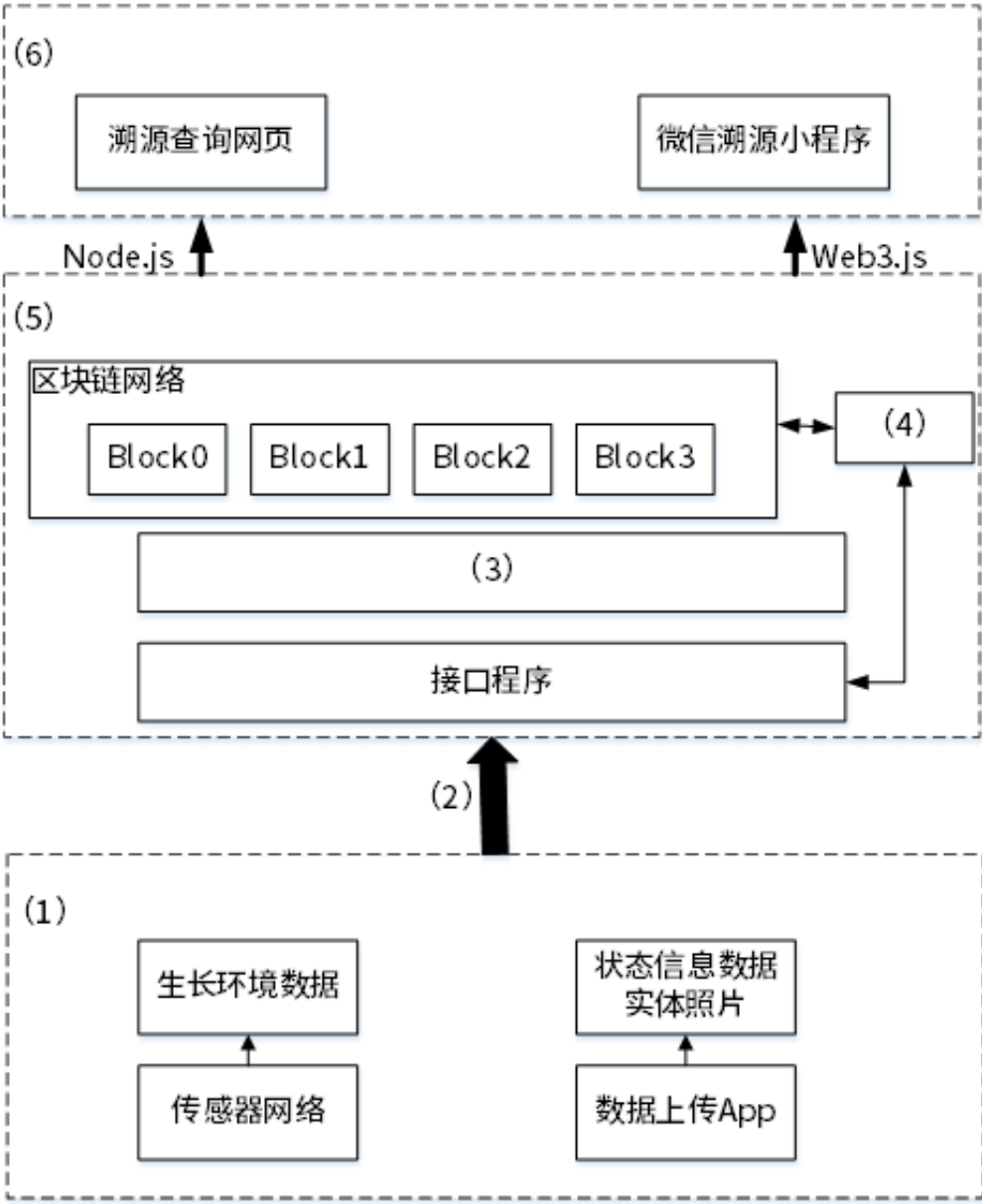
思路解析

分布式交易账本的去中心化特性避免了单点故障和篡改风险，每个节点保存相同的数据副本，确保数据的可追溯性和一致性。而公私钥签名则通过加密验证确保数据来源的合法性，避免了伪造交易，确保了区块链数据的完整性和可信度。结合这两种技术，可以有效解决数据信任问题。

问题（3）

根据李工的建议，该系统将采用三层架构。请从下面给出的（a）~（m）候选项中进行选择，补充

完善图5-1中（1）～（6）处空白的内容，完成该系统的架构设计方案。



- (a) 数据接入层
- (b) 智能合约
- (c) Socket
- (d) Spring
- (e) 应用表示层
- (f) 数据库
- (g) MVC
- (h) 数据核心层
- (i) 传感器网络
- (j) 区块链网络
- (k) 4G/Wifi
- (l) JDBC
- (m) 业务逻辑层

思路解析

三层架构设计通常包括以下几个层次：

数据接入层（Layer 1）：负责数据的输入、采集和传输。在这个层面，数据从外部传入系统，这里可以使用4G/Wifi等技术进行数据接入。

数据核心层（Layer 2）：这一层负责核心的处理和存储，包含数据处理逻辑和数据库等。区块链网络的智能合约通常在此层实现，用于执行和验证智能合约中的逻辑。

应用表示层（Layer 3）：这是用户交互的界面层，通常包括Web界面、移动应用等，负责展示和交互功能。系统的UI和用户交互设计通常放在这一层。

4G/Wifi是数据接入的常见技术，智能合约作为区块链核心功能，负责执行数据验证和业务逻辑，而数据库则存储系统中的静态数据和与区块链关联的数据。

答案-问题（1）

去中心化：区块链采用分布式计算和存储架构，不依赖于单一中心化的硬件或管理机构，系统中的所有节点平等参与，无中心控制。每个节点都有权记录和验证交易，确保数据的透明和公正。由于没有单点控制，区块链避免了中心化结构中可能出现的数据篡改和系统故障风险。

开放性：区块链是一个开放系统，任何人都可以参与到区块链网络中（只要他们符合必要的技术要求）。除了参与方的私密数据外，区块链中的所有交易记录都是公开透明的，任何人都可以查阅。这种开放性确保了信息的透明性和可审计性，同时增加了系统的可信度。

答案-问题（2）

分布式交易账本：区块链的核心之一是将交易账本分散存储在多个节点上，每个节点都有一份完整的账本副本。这种分布式存储方式确保了即使某些节点发生故障，其他节点的数据仍然可以确保一致性。更重要的是，任何想要篡改数据的人都必须修改所有节点的账本副本，这在现实中几乎不可能做到。分布式账本通过去中心化的特性，避免了单点故障和单一管理者的控制，极大增强了数据信任的保障。

公私钥签名：区块链使用公私钥加密技术来验证数据的真实性和防止数据篡改。每个参与者拥有一对公私钥，私钥用于对交易进行签名，而公钥则用于验证签名的合法性。这种机制确保了数据来源的可靠性，并防止了数据在传输过程中被伪造或篡改。由于签名是基于非对称加密算法，它能够有效地防止伪造和篡改行为，从而确保了数据信任。

答案-问题（3）

- （1）（a）数据接入层
- （2）（k）4G/Wifi
- （3）（b）智能合约
- （4）（f）数据库
- （5）（h）数据核心层
- （6）（e）应用表示层

（2020年5月系统分析师真题-第4题）

阅读以下关于 Web 应用系统的叙述，在答题纸上回答问题1至问题3。

某公司拟开发个基于O2O（Online To Offline）外卖配送模式的外卖平台。该外卖平台采用自行建立的配送体系承接餐饮商家配送订单，收取费用，提供配送服务。餐饮商家在该O2O外卖平台发布配送订单后，根据餐饮商家、订餐用户、外卖配送员位置等信息，以骑手抢单、平台派单等多种方式为

订单找到匹配的外卖配送员，完成配送环节，形成线上线下的O2O闭环。

基于项目需求，该公司多次召开项目研发讨论会。会议上，张工分析了O2O外卖平台配送服务的业务流程，提出应采用事件系统架构风格实现订单配送，并建议采用基于消息队列的点对点模式的事件派遣机制。

问题（1）

基于对O2O外卖平台配送服务的业务流程分析，在图5-1的空（1） - （5）处完善O2O外卖平台配送的服务流程。

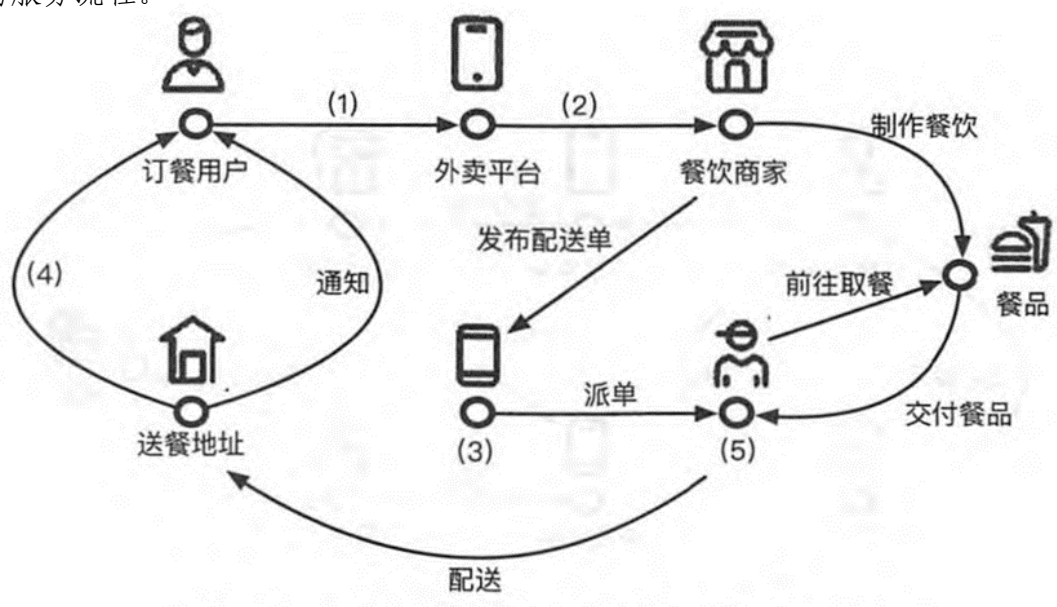


图 5-1 O2O 外卖平台配送的服务流程

思路解析

在O2O外卖平台的配送服务流程中，餐饮商家首先在平台上提交配送订单，开始整个配送流程。订单提交后，平台会将其发布给合适的配送员，基于骑手抢单或平台派单的方式来匹配配送员。

- （1）提交订单：订餐用户通过平台提交订单，这是订单流程的第一步。
- （2）发布订单：餐饮商家发布的订单会通过平台进行处理和发布，确保外卖配送员可以接单。
- （3）外卖平台：外卖平台在系统中充当中枢，管理整个订单的流转和匹配配送员的任务。
- （4）交付餐品：配送员将餐品送达后，通知用户并完成交付，这是整个配送流程的最后一环。
- （5）配送员：配送员是订单流程中的关键角色，负责从平台接单并将餐品送到用户手中。

问题（2）

根据张工的建议，该系统采用事件系统架构风格实现订单配送服务。请基于对事件系统架构风格的了解，补充图5-2的空（1） - （3）处，完成事件系统的工作原理图。

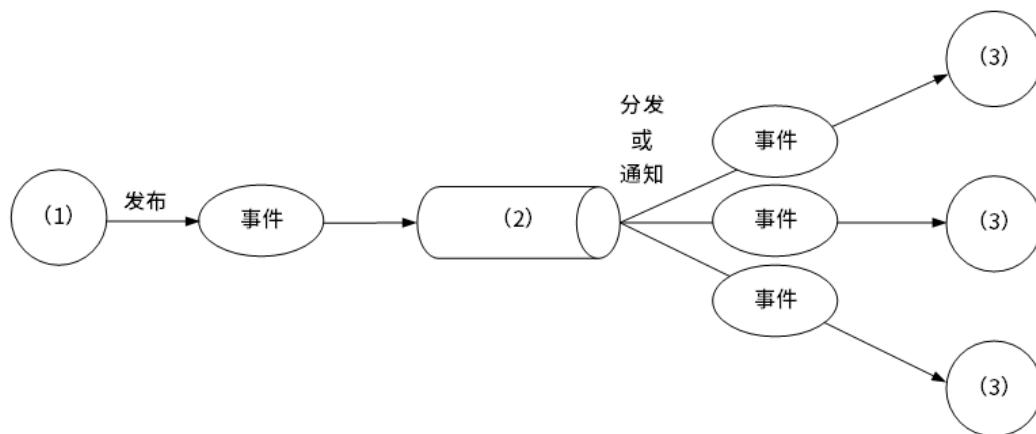


图 5-2 事件系统的工作原理图

思路解析

事件系统架构风格是一种基于事件驱动的架构，通常包含事件源、事件管理器和事件处理器三个主要组件。

事件源（1）：事件源是指发起事件的实体。在O2O外卖配送平台中，事件源可能是用户提交订单、餐饮商家发布订单等。这些动作会触发事件的发生。

事件管理器（2）：事件管理器的角色是接收事件并决定将事件分发给哪个事件处理器进行处理。它根据事件的类型、处理器的注册情况和系统状态来调度事件处理器。

事件处理器（3）：事件处理器是执行与事件相关的操作的组件，可能会是外卖配送员接受任务、处理配送状态等。

这种基于事件驱动的架构可以实现高效的解耦与异步处理，使系统能够响应不同的事件并做出适当的反应。事件系统架构特别适用于处理异步、并发任务，在外卖配送的订单处理中尤为有效。

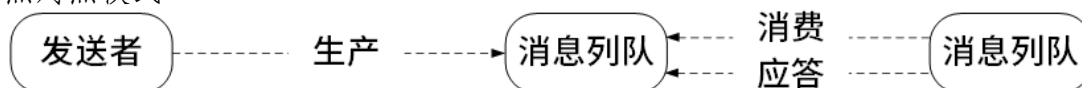
问题（3）

请用200字以内的文字说明基于消息队列的点对点模式的定义，并简要分析张工建议该系统采用基于消息队列的点对点模式的事件派遣机制的原因。

思路解析

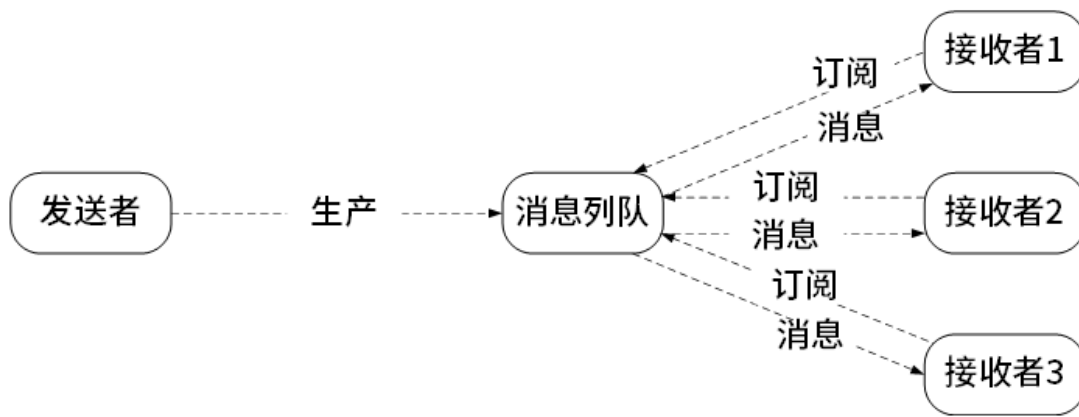
消息队列包括两种模式，点对点模式（point to point, queue）和发布/订阅模式（publish/subscribe, topic）

点对点模式



消息发送者生产消息发送到 queue 中，然后消息接收者从 queue 中取出并且消费消息。消息被消费以后，queue 中不再有存储，所以消息接收者不可能消费到已经被消费的消息。

发布/订阅模式



发布者将消息发送到 Topic, 系统将这些消息传递给多个订阅者。

消息队列的点对点模式通过“队列”和“消费者”的机制, 确保了消息的唯一性和顺序性。每个订单(消息)在队列中仅能被一个配送员(消费者)接单, 这避免了多重消费的问题。在外卖配送场景中, 这样的机制非常重要, 因为每个配送任务需要精确地匹配给一个配送员, 保证了任务的顺利完成。因此, 张工建议使用这种模式来解决任务分配的准确性问题。

答案-问题(1)

- (1) 提交订单
- (2) 发布订单
- (3) 外卖平台
- (4) 交付餐品
- (5) 配送员

答案-问题(2)

- (1) 事件源
- (2) 事件管理器
- (3) 事件处理器

答案-问题(3)

在基于消息队列的点对点模式中, 消息生产者将消息发送到消息队列中, 消息消费者从队列中取出并消费这些消息。每个消息只能被一个消费者消费, 消息被消费后, 队列中不再保存该消息。该模式确保每个消息只会被一个消费者处理, 避免了重复处理的问题。

例如, 在O2O外卖平台中, 外卖订单作为消息生产者将消息发送到消息队列中, 配送员作为消费者从队列中获取订单消息并进行配送。

张工建议采用基于消息队列的点对点模式, 是因为外卖配送订单需要保证每个订单只有一个配送员处理。通过点对点模式, 订单消息被放入队列, 配送员从队列中抢单, 这样可以确保每个订单只能被一个配送员接单, 避免了多个配送员同时处理同一订单的冲突情况。

(2017年5月系统分析师真题-第1题)

阅读以下关于基于微服务的系统开发的叙述, 在答题纸上回答问题1至问题3。

某公司拟开发一个网络约车调度服务平台, 实现基于互联网的出租车预约与管理。公司的系统分析师王工首先进行了需求分析, 得到的系统需求列举如下:

系统的参与者包括乘客、出租车司机和平台管理员三类；

系统能够实现对乘客和出租车司机的信息注册与身份认证等功能，并对乘客的信用信息进行管理，对出租车司机的违章情况进行审核；

系统需要与后端的银行支付系统对接，完成支付信息审核、支付信息更新与在线支付等功能；

针对乘客发起的每一笔订单，系统需要实现订单发起、提交、跟踪、撤销、支付、完成等业务过程的处理；

系统需要以短信、微信和电子邮件多种方式分别为系统中的用户进行事件提醒。

在系统分析与设计阶段，公司经过内部讨论，一致认为该系统的需求定义明确，建议尝试采用新的微服务架构进行开发，并任命王工为项目技术负责人，负责项目开发过程中的技术指导工作。

问题（1）

请用 100字以内的文字说明一个微服务中应该包含的内容，并用 300 字以内的文字解释基于微服务的系统与传统的单体式系统相比的 2 个优势和带来的 2 个挑战。

思路解析

微服务架构的优势在于其能够将一个庞大的单体系统拆分成多个独立的服务单元，从而提升了系统的灵活性和可维护性。然而，微服务架构也带来了新的挑战，特别是服务之间的复杂通信和数据一致性问题，需要开发团队在设计时考虑到服务间的接口标准、容错机制和数据同步方式等。微服务的引入能够使得大型系统的开发与维护更加高效，但也需要一定的技术积累与协调机制。

问题（2）

识别并设计微服务是系统开发过程中的一个重要步骤，请对题干需求进行分析，对微服务的种类和包含的业务功能进行归类，完成表 1-1 中的（1）~（4）。

微服务名称	包含业务功能（至少填写 3 个功能）
乘客管理	（1）
出租车司机管理	（2）
（3）	支付信息审核、支付信息更新、在线支付
订单管理	（4）
通知中心	短信通知、微信通知、邮件通知

思路解析

在微服务架构下，每个微服务都应独立处理特定的业务逻辑。根据题干中的需求，我们可以将系统功能拆分为多个微服务：

乘客管理服务负责乘客信息的注册、身份认证和信用管理。

出租车司机管理服务负责出租车司机的信息注册、身份认证和违章情况的审核。

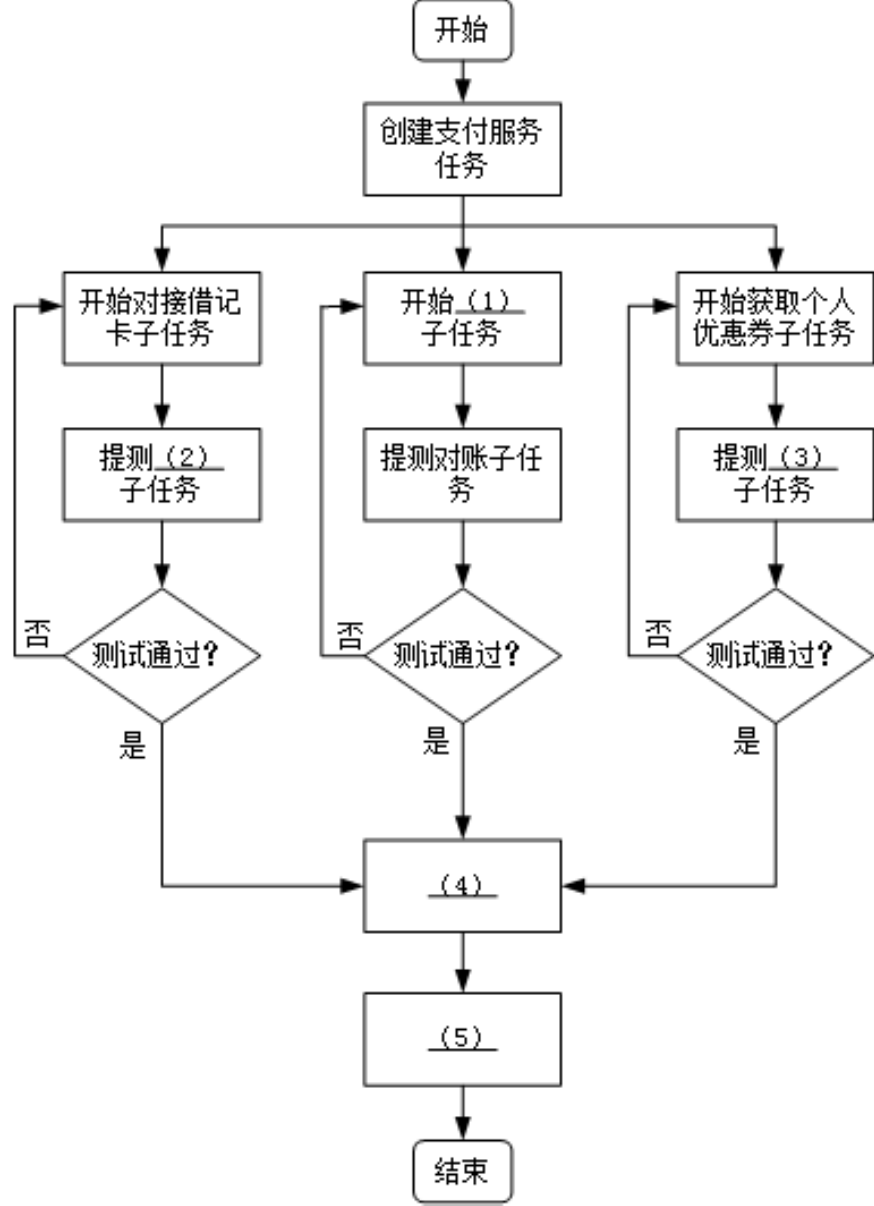
支付管理服务负责与支付系统对接，实现支付信息的审核、更新和在线支付功能。

订单管理服务负责订单的生命周期管理，包括订单的发起、提交、跟踪、撤销等。

每个服务的职责是独立且清晰的，确保微服务架构能够在开发和维护过程中提升效率和灵活性。

问题（3）

为了提高系统开发效率，公司的系统分析师王工设计了一个基于微服务的软件交付流程，其核心思想是将业务功能定义为任务，将完成某个业务功能时涉及的步骤和过程定义为子任务，只有当所有的子任务都测试通过后该业务功能才能上线交付。请基于王工设计的在线支付微服务交付流程，从（a）~（f）中分别选出合适的内容填入图1-1中的（1）~（5）处。



- (a) 提交测试 (b) 全量上线 (c) 对接借记卡
(d) 获取个人优惠券 (e) 试部署 (f) 对账

思路解析

此题考察了微服务交付流程中的任务拆解。在线支付微服务交付流程应当从最基础的对账开始，确保支付系统的正确性；接着进行支付通道的对接（如借记卡接口）；然后用户可以获取个人优惠券；进行试部署，确保新功能能够稳定运行；最后，进行全量上线，正式将该服务推向生产环境。这种分阶段的交付方式有助于确保服务质量，并减少因部署不当导致的故障。通过对每个步骤的合理拆分，可以有效提高开发效率，确保每个阶段的任务都经过严格测试，减少上线后的风险。

答案-问题（1）

一个微服务中应该包含的内容有：资源、对资源的操作、API集合。

微服务的优势：

- （1）解决了复杂性问题。它把庞大的单一模块应用分解为一系列的服务，同时保持总体功能不变。
- （2）让每个服务能够独立开发，开发者能够自由选择可行的技术，让服务来决定API 约定。
- （3）每个微服务都能独立配置，开发者不必协调对于本地服务配置上的变化，这种变化一旦测试完成就被配置了。
- （4）让每个服务都可以独立调整，你可以给每个服务配置正好满足容量和可用性限制的实例数。

微服务架构带来的挑战：

- （1）并非所有的系统都能转成微服务。例如一些数据库层的底层操作是不推荐服务化的。
- （2）部署较以往架构更加复杂：系统由众多微服务搭建，每个微服务需要单独部署，从而增加部署的复杂度，容器技术能够解决这一问题。
- （3）性能问题：由于微服务注重独立性，互相通信时只能通过标准接口，可能产生延迟或调用出错。例如一个服务需要访问另一个服务的数据，只能通过服务间接口来进行数据传输，如果是频繁访问，则可能带来较大的延迟。
- （4）数据一致性问题：作为分布式部署的微服务，在保持数据一致性方面需要比传统架构更加困难。

答案-问题（2）

- （1）乘客信息注册、乘客身份认证、乘客信用信息管理
- （2）出租车司机信息注册、出租车司机身份认证、司机违章情况审核
- （3）支付管理
- （4）订单发起、订单提交、订单跟踪、订单撤销

答案-问题（3）

- （1）（f） （2）（c） （3）（d） （4）（e） （5）（b）

（2018年5月系统分析师真题-第5题）

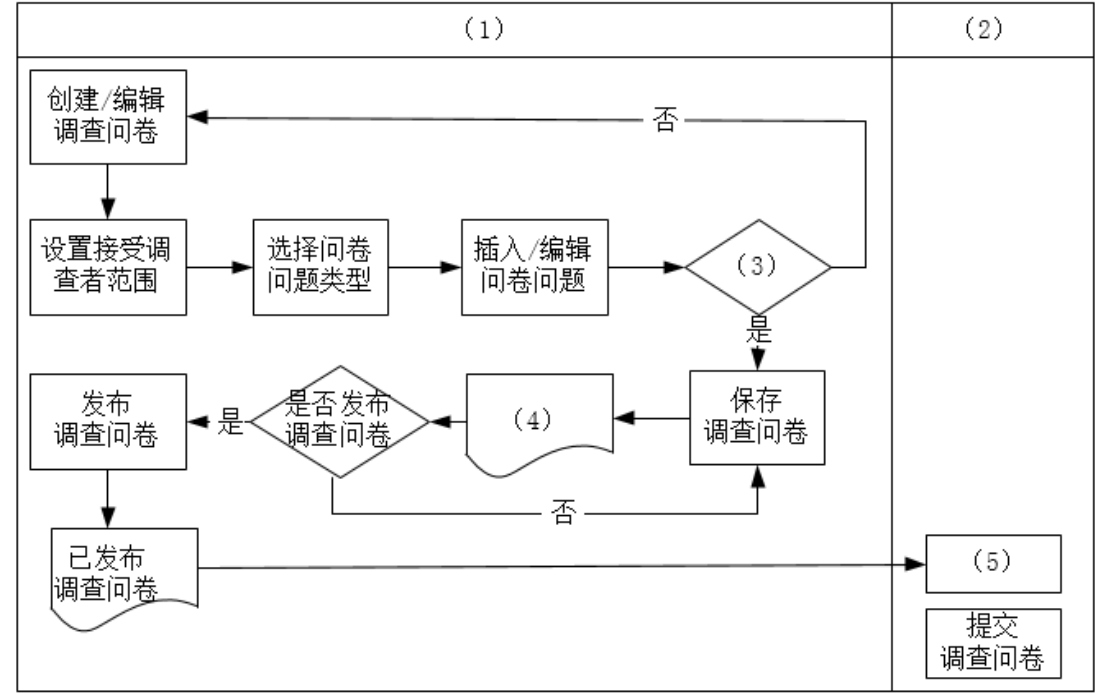
阅读以下关于Web应用设计开发的描述，在答题纸上回答问题1至问题3。

某公司拟开发一个自由、可定制性强、用户界面友好的在线调查系统，以获取员工在课程学习、对公司重大事件的看法、对办公室环境的建议等相关反馈。因需要调查的内容各异，可选择的调查方式多样，故本在线调查系统应满足以下需求：

- 1）支持编辑和视图两种模式，编辑模式只对调查发起者可见，视图模式对接受调查者可见。
 - 2）调查问卷具有可定制性，因调查的内容各异，需要多样的信息采集方式，可设置的调查问题类型包括单选、多选、矩阵类单选、矩阵类多选和开放性问题。
 - 3）操作简单，调查者可以方便地新建和编辑各种问题类型，接受调查者可对每个问题和每个调查问卷给出评论。
 - 4）系统支持显示调查统计结果，以及导出统计结果。
- 针对以上需求，经项目组讨论，拟采用REST架构风格设计实现该在线调查系统。

问题（1）

分析该在线调查系统的业务流程，填写图5-1中（1）～（5）的内容。



思路解析

根据题目描述，该系统的业务流程可以分为几个关键步骤。

调查发起者：负责创建和编辑调查问卷，开始整个流程。

接受调查者：是调查的目标群体，负责填写调查问卷并提供反馈。

是否保存调查问卷：调查问卷在编辑后是否需要保存，这涉及到问卷的保存或发布状态。

已保存的调查问卷/待发布调查问卷：指已保存且未发布的调查问卷或已经准备好的问卷。

显示（查看）调查问卷/填写调查问卷：接受调查者查看并填写调查问卷的过程。

问题（2）

REST 架构风格的核心是资源抽象。在系统设计中，项目组拟将系统中的每一个实体抽象成一种资源。请列举出该系统中的5种资源。

思路解析

REST架构风格强调对系统中的每个实体进行资源抽象，并为每个资源分配一个唯一标识。在该系统中，资源可以包括：

用户：系统的使用者，包括调查发起者和接受调查者。

调查发起者：一个特定角色，负责创建和发布调查问卷。

调查问卷：实际的调查表格，包括调查的具体问题。

调查问题类型：每种调查问题的类型，例如单选题、多选题等。

调查结果：记录所有调查问卷的结果，进行统计分析。

问卷问题的选项：每个问题可能的答案选项。

问卷问题评论：接受调查者对问题的评论。

调查问卷评论：接受调查者对整个问卷的评论。

这些资源都是REST风格的核心部分，每个资源可以通过唯一的URL进行标识和操作。

问题（3）

基于 REST 架构风格对系统进行设计，请简要叙述REST风格的5条关键原则。

思路解析

REST (Representational State Transfer) 是一种面向资源的架构风格，具有以下五个关键原则：

资源抽象：所有可以被客户端操作的事物都被视为资源，比如用户、调查问卷、问题等。

资源唯一标识：每个资源通过唯一的URL来标识，使得资源可访问且唯一。

通用接口：资源的操作通过标准的HTTP方法（GET、POST、PUT、DELETE等）进行，这些操作是通用且规范化的。

资源标识不变：对资源的操作（如创建、删除、修改）不会改变资源的标识，即URL不发生变化。

无状态：每次客户端请求都应包含足够的信息来处理请求，服务器不应依赖于客户端的状态，也就是说，每次请求都是独立的，不依赖于之前的请求。

答案-问题（1）

- （1）调查发起者
- （2）接受调查者
- （3）是否保存调查问卷
- （4）已保存的调查问卷/待发布调查问卷
- （5）显示（查看）调查问卷/填写调查问卷

答案-问题（2）

用户、接受调查者、调查发起者 、调查问卷、调查问题类型 、问卷问题、问卷问题的选项、调查结果、问卷问题评论、调查问卷评论等（回答其中5个即可）。

答案-问题（3）

REST风格的5条关键原则包括：

- （1）网络上的所有事物都被抽象为资源。
- （2）每个资源对应一个唯一的资源标识。
- （3）通过通用的连接件接口对资源进行操作。
- （4）对资源的各种操作不会改变资源标识。
- （5）所有的操作都是无状态的。

(2023年5月系统分析师真题-第4题)

某公司拟开发一个电商平台用户行为分析系统，该系统可支持对电商平台用户行为数据的采集和实时/高效数据处理分析，统计分析用户访问情况。搜集PV/UV商品热度等相关数据。系统应具备以下能力

- （1）对海量用户行为数据的存储和处理能力
- （2）为适应用户行为的随机性，保证系统不会被爆发的日志数据压垮，系统应具备对突发流量的

处理能力

(3) 对用户数据的营私保护能力

针对项目需求，工时召开会议，会上张工提出买那个对大量数据，传统的文件存储于关系型数据库，无法完全解决当前系统问题，应基于大数据技术设计并实现系统，李工提出可就要数据脱敏的方法解决用户隐私的问题。

问题（1）

根据项目要求，该系统需考虑电商平台用户行为的随机性，具有应对突发数据，流量数据的处理能力。请基于大数据相关技术的了解，说明可选那个技术支持流系统应对突发流量暴增情况？并用200字以内说明该技术优势。

思路解析

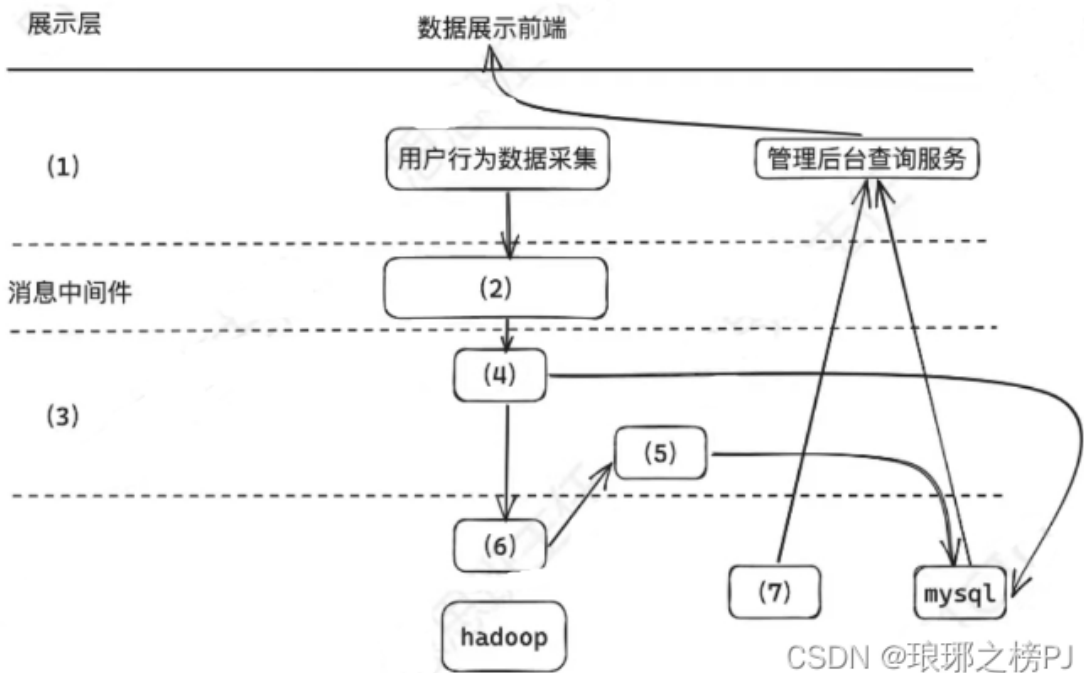
在电商平台中，用户行为数据通常具有高度的随机性，且会受到活动、节假日等因素的影响，可能会导致瞬时的流量暴增。为了应对这种情况，传统的单机架构或数据库系统难以有效处理这种海量的数据流。消息中间件（如Kafka）通过提供分布式的消息传递机制，帮助系统应对突发流量。消息中间件通过异步的方式进行数据传递，避免了直接的同步通信带来的性能瓶颈。在数据流量突增时，消息中间件可以充当缓冲区，暂时存储数据，等到系统负载恢复后再进行处理。这种流量控制与缓冲机制，大大减少了系统受到瞬时高负载冲击的风险，使得整个系统可以平稳运行。此外，消息中间件通过解耦生产者与消费者，降低了系统的复杂度，使得开发和维护更加方便。

问题（2）

根据张工建议，项目组拟采用大数据技术开发该系统。图5给出本项目架构设计方案，请选择

(a) ~ (m) 选项中选出合适的内容，完善（1）~（7）处

(a) 网络层、(b) 业务层、(c) 接入层、(d) 数据处理、(e) 数据采集、(f) Flink实时计算、(g) Flink批处理、(h) Hbase、(i) Redis、(j) NoSQL、(k) kafka、(l) 边缘计算、



(m) 设备层

思路解析

本题涉及到大数据平台架构设计的选择，涉及到多个层次的技术选型。

接入层： 在数据的采集和接入过程中，接入层承担了将外部数据接入系统的责任。通常这一层会使用专门的API、服务网关或负载均衡器来进行数据的采集与转发。这里选择接入层是因为在电商平台中，用户行为数据需要实时、快速地接入平台，并通过合适的方式进行分发。

Kafka： 作为一个高吞吐量的消息中间件，Kafka在大数据系统中被广泛应用于流式数据的传输。它负责在接入层与数据处理层之间传递数据，具有高并发、低延迟和可靠性强等特点。尤其在面对电商平台突发流量时，Kafka可以有效缓解流量暴增的压力。

Flink实时计算与批处理： 在数据处理部分，Flink是一个强大的流式处理框架，能够对实时数据进行高效的计算分析。而Flink批处理则可以对历史数据进行离线处理和分析，配合使用可以大大提高系统的数据处理能力。

Hbase： Hbase是一个分布式的列式存储数据库，适用于存储大量的结构化数据，特别是在需要快速检索时。在本系统中，Hbase负责存储电商平台的行为数据和用户数据，能够支持高并发、高速的读写需求。

Redis： 作为一个内存数据库，Redis可以在系统中作为缓存层使用，大大提高数据访问的速度。在电商平台中，用户行为分析的结果、热门商品数据等可以存储在Redis中，快速响应用户查询请求。

问题（3）

根据李工的建议，项目组拟应用数据脱敏方法对该系统的敏感数据进行脱敏。请用300字以内文字说明常用5种数据脱敏方法。

思路解析

数据脱敏的核心目标是保护用户隐私，同时保证数据在分析中的可用性。在电商平台中，用户的姓名、联系方式、地址、交易记录等数据都属于敏感信息，若泄露会带来严重风险。

五种常见的脱敏方式各有侧重：替换与混排通过调整数据值或顺序，避免敏感字段直接暴露；时空变异与数值变异则通过增加扰动，既保持了数据整体规律，又能有效隐藏真实值；删除是最彻底的方式，适用于非核心分析所需的敏感字段。通过组合使用这些方法，可以在保证分析数据质量的同时，最大限度减少用户隐私泄露的风险，满足电商系统的合规性与安全性要求。

答案-问题（1）

可选消息中间件技术支持流系统应对突发流量暴增情况。消息中间件具有的技术优势：

（1）异步通信，通过消息中间件，不同的服务之间进行异步通信，可以提高系统的吞吐量和并发性能。

（2）解耦合，将消息发送方和接受方进行解耦，从而能独立开发和部署，提高系统的可扩展性和可维护性。

（3）流量控制或者消息缓冲，可以设置流量控制，当消息量过大时，暂停消息的发送，流量小时，继续发送，实现削峰填谷功能。也可以作为消息缓冲功能，作为缓冲区暂存消息，从而减少系统的负载和压力。

答案-问题（2）

（1）c

（2）k

（3）业务层

- (4) f
- (5) g
- (6) h
- (7) i

答案-问题（3）

常用数据脱敏方法为：

- （1）替换。将字符或整数值替换为查找或标准模式中的字符或整数值。例如，可以用列表中的随机值替换名字。
- （2）混排。在一个记录中交换相同类型的数据元素或者在不同行之间交换同一属性的数据元素。例如，在供应商发票中混排供应商名称，以便将发票上的原始供应商替换为其他有效供应商。
- （3）时空变异。把日期前后移动若干天（小到足以保留趋势），足以使它无法识别。
- （4）数值变异。应用一个随机因素（正负一个百分比，小到足以保持趋势），重要到足以使它不可识别。
- （5）取消或删除。删除不应出现在测试系统中的数据。

（2017年5月系统分析师真题-第5题）

阅读以下关于 Web 系统架构设计的叙述，在答题纸上回答问题 1至问题 3。

某公司开发的 B2C 商务平台因业务扩展，导致系统访问量不断增大，现有系统访问速度缓慢，有时甚至出现系统故障瘫痪等现象。面对这一情况，公司召开项目组讨论会议，寻求该商务平台的改进方案。讨论会上，王工提出可以利用镜像站点、CDN内容分发等方式解决并发访问量带来的问题。而李工认为，仅仅依靠上述外网加速技术不能完全解决系统现有问题，如果访问量持续增加，系统仍存在崩溃的可能。李工提出应同时结合Web内网加速技术优化系统改进方案，如综合应用负载均衡、缓存服务器、Web应用服务器、分布式文件系统、分布式数据库等。经过讨论，公司最终决定采用李工的思路，完成改进系统的设计方案。

问题（1）

针对李工提出的改进方案，从 a ~j 中分别选出各技术的相关描述和对应常见支持软件填入表 5-1 中的（1）~（10）处。

技术	相关描述	常见支持软件
负载均衡	（1）	（2）、LVS
缓存服务器	（3）	（4）、Memcached
分布式文件系统	（5）	（6）、（7）、MooseFS
Web 应用服务器	加速对请求进行处理	（8）、（9）、Jetty
分布式数据库	缓存、分割数据、加速数据查找	（10）、Mysql

- （a）保存静态文件，减少网络交换量，加速响应请求
- （b）可采用软件级和硬件级负载均衡实现分流和后台减压
- （c）文件存储系统，快速查找文件
- （d）FastDFS
- （e）HAProxy

- (f) JBoss
- (g) Hadoop Distributed File System (HDFS)
- (h) Apache Tomcat
- (i) Squid
- (j) MongoDB

思路解析

这道题考察了对Web系统架构设计中各项技术的理解及其常见支持软件的配对。我们将李工提出的技术方案中的相关描述和常见支持软件一一对应，确保每项技术有合适的实现工具。

负载均衡(b)：负载均衡可以通过软件和硬件两种方式来实现，目的是将流量分配到不同的服务器上，减轻单个服务器的负担，保证系统高效运作。常见的负载均衡工具有HAProxy，它是一个高效的TCP/HTTP负载均衡器。

静态文件加速(a)：静态文件（如图片、CSS、JS文件等）的存储和访问通常会占用大量带宽。通过保存静态文件并加速响应，可以显著提高系统的访问速度。Squid 是一个广泛使用的缓存代理服务器，可以缓存静态内容，加速网站内容的访问。

文件存储系统(c)：对于大规模的数据存储，FastDFS 和 Hadoop Distributed File System (HDFS) 都是常见的解决方案。它们用于存储和快速查找大量的文件数据。

Web应用服务器(f)(h)：JBoss 和 Apache Tomcat 是两种常见的Web应用服务器。Tomcat 是一个开源的Servlet容器，用于运行Java代码，而 JBoss 是一个企业级应用服务器，支持EJB（企业JavaBeans）等技术。

数据库(j)：MongoDB 是一个流行的NoSQL数据库，适用于存储大规模、灵活的数据，尤其是在需要快速读写操作的环境中表现优秀。

问题(2)

请用100字以内的文字解释分布式数据库的概念，并给出提高分布式数据库系统性能的 3 种常见实现技术。

思路解析

分布式数据库 是将数据分布存储在多个节点上的数据库系统，具有高可用性、可扩展性和容错能力。数据的分布和冗余存储保证了系统即使在某些节点发生故障时，其他节点仍能提供服务。提高分布式数据库性能的常见实现技术包括：

数据分片：这是分布式数据库优化的核心之一，通过将数据分片存储在不同的节点上，可以降低单个节点的压力并提高数据访问效率。常见的分片方式有按范围分片、哈希分片等。

并行查询：多个节点并行处理查询请求，能够显著提高查询的处理速度。分布式数据库通过多个节点同时工作，减少单个节点的查询负担。

缓存技术：使用缓存（如内存缓存）可以显著提高数据库的访问速度。频繁访问的数据被缓存，避免每次都从磁盘读取，减少了磁盘I/O，提高了数据库响应速度。

问题(3)

针对 B2C 商务购物平台的数据浏览操作远远高于数据更新操作的特点，指出该系统应采用的分布式数据库实现方式，并分析原因。

思路解析

一主多从架构是分布式数据库中常用的读写分离机制，适用于数据浏览操作远远高于数据更新操作的场景。主库负责处理数据的写入操作（如插入、更新和删除），而从库则用于处理查询请求。通过这种方式，主库的写入压力得以减轻，从库可以通过多节点并发读取大规模的查询请求，从而显著提高系统的响应速度和吞吐量。

为什么选择一主多从架构：读多写少：B2C平台中的数据浏览操作远远高于数据更新操作，这使得一主多从架构特别适合。主库专注于数据更新，而从库则分担了大量的查询请求，避免了主库的负载过重。读负载分散：从库可以部署在多个节点上，分散查询负载。系统能够在多个从库上进行并行查询，极大提高了系统的查询效率。数据一致性：主库处理写操作，保证了数据的一致性；从库从主库同步数据，不会影响主库的写入性能，同时也能提供高效的查询服务。

答案-问题（1）

- (1) (b)
- (2) (e)
- (3) (a)
- (4) (i)
- (5) (c)
- (6) (7) (d) (g)
- (8) (9) (f) (h)
- (10) (j)

答案-问题（2）

概念：分布式数据库是将数据分布存储在多个物理节点上，逻辑上统一管理的数据库系统。

常见技术：（1）数据分片：将数据按规则划分存储，减少单个节点负载。（2）并行查询：多个节点同时处理查询，提升处理速度。（3）缓存技术：缓存热点数据，减少磁盘 I/O，加快访问。

答案-问题（3）

在本题所涉及到的环境中，由于读取数据的需求非常强烈，涉及到的数据量极大，而更新较少，可以在分布式数据库中采用一主多从的机制来分散读取数据的压力。更新数据时，操作主库，从主库再同步到从库，从库只负责读取数据。

(2016年5月系统分析师真题-第5题)

阅读以下关于Web应用的叙述，在答题纸上回答问题1至问题3。

某软件企业拟开发一套基于Web的云平台配置管理与监控系统，该系统按租户视图、系统管理视图以及业务视图划分为多个相应的Web应用，系统需求中还包含邮件服务、大文件上传下载、安全攻击防护等典型Web系统基础服务需求。

问题（1）

在选择系统所采用的Web开发框架时，项目组对Alibaba开发的WebX框架与轻量级Spring MVC框架进行了对比分析，最终决定采用WebX框架进行开发。请用300字以内文字，从多应用支持、基础服务支持以及可扩展性这三个方面对WebX与Spring MVC框架进行对比。

思路解析

本题主要考查考生对 大型互联网应用框架选择 的理解，要求对比阿里巴巴的 WebX 框架与轻量级 Spring MVC 框架。

首先，从 多应用支持 的角度来看，Spring MVC是一个轻量级的Web MVC框架，适合单一Web应用开发，但在多应用场景下支持不足。而WebX在设计时就考虑了淘宝、天猫等多应用大规模电商平台的需求，提供了 多应用路径生成机制，可自动避免URL冲突，同时支持多个应用在一个工程中共存，层次清晰、扩展方便。

其次，从 基础服务支持 的角度来看，Spring MVC仅提供MVC层面的核心功能，对于邮件服务、文件上传、安全防护等功能，需要依赖第三方库。而WebX是一个 重量级Web开发框架，其内置了大量基础服务，如前端模板、资源加载、持久化组件、表单验证、URL路径映射、大文件上传、邮件服务、安全攻击防护等，开发者无需额外引入过多依赖，能更快搭建完整Web系统。

最后，从 可扩展性 的角度看，Spring MVC作为Spring生态的一部分，依赖Spring本身的扩展能力。而WebX在Spring的基础上引入了 SpringExt扩展机制，允许组件之间的互相扩展与复用，甚至支持开发者通过重写框架逻辑来自定义功能。这使得WebX在复杂、大规模项目中比Spring MVC更具扩展性与灵活性。

综上，Spring MVC适合中小型项目，而WebX更适合大规模、多应用、需要完整基础服务支持的企业级Web系统。

问题（2）

在确定系统采用的持久层技术方案时，项目组梳理了系统的典型持久化需求，对照需求对比分析了 Hibernate和MyBatis两种持久化方案，请分析两种持久化方案对表 5-1中所列项目需求的支持情况，将候选答案序号A或B填入表5-1相应位置。

持久化需求	Hibernate	MyBatis
支持基本对象关系映射，能够生成简单基本的 DAO 层方法	A	(1)
系统业务中可能涉及单次业务超过百万条规模的大批量数据读取需求，因此应方便支持复杂查询操作的 SQL 人工调优	(2)	A
支持复杂的多表关联操作，且应考虑系统部分数据源来自被监控云平台的持久化数据，这部分数据源结构不可更改且可能存在实体关系设计不合理的情况	(3)	(4)
提供良好的数据库移植性支持，支持不同厂商的关系型数据库	(5)	(6)

【候选答案：A 支持 B 不支持或支持差】

思路解析

Hibernate和MyBatis都是常见的 ORM（对象关系映射）框架，均可实现对象与数据库表之间的映射，但二者在设计理念和技术特性上存在差异。

Hibernate属于全自动化ORM框架，强调“对象化操作数据库”，能够自动生成SQL，支持缓存、延迟加载、级联操作，开发效率较高。在复杂关联关系和多表查询上，Hibernate有较好的支持，尤其适用于对数据库操作抽象度要求较高、数据模型稳定的系统。同时，Hibernate具有较好的 数据库移植性，因为其HQL语句是平台无关的，迁移到不同数据库时通常无需修改大量SQL。

相比之下，MyBatis更偏向“半自动化” ORM框架，它需要开发者手工编写SQL语句，因此对复杂SQL的支持和可控性更好。特别是当项目需要 对SQL进行调优 时，MyBatis可以更高效地执行复杂的存储过程或SQL逻辑。但由于SQL需要手工编写，MyBatis的移植性较差，不同数据库厂商的差异需要开发者自行处理。

因此，对比表中不同需求：

- 复杂关联关系映射（1）：Hibernate自动支持，答案A;
- 复杂SQL可控性（2）：MyBatis更好，Hibernate支持较差，答案B;
- SQL调优便利性（3）：Hibernate自动生成SQL，灵活性差，答案B;
- 缓存机制（4）：Hibernate支持一级、二级缓存，答案A;
- 数据库移植性（5）：Hibernate的HQL较好移植，答案A;
- 存储过程支持（6）：Hibernate对存储过程支持较差，答案B。

问题（3）

系统实现相应的配置管理与监控功能时，需要集成云平台侧提供的大量服务以实现配置数据的读取写入与不同视图监测数据的获取。项目组在确定服务集成方案时，对比了REST风格RPC与SOAP RPC两种方案，经过分析讨论，项目组最终决定采用REST风格RPC机制实现服务集成，请判断表5-2中给出的选择REST方案的理由是否合理。

理由	合理：√ 不合理：×
系统后台服务主要提供配置管理数据的读取写入与监测数据的获取,可以较容易映射为典型 CRUD 操作	(1)
REST 风格 RPC 通过 WS-Security 机制支持良好的安全性	(2)
在 REST 风格 RPC 方案中，客户端发出的 HTTP 请求之间支持相互的状态依赖，便于实现多个请求的相互协作处理	(3)
基于 REST 风格 RPC 实现服务集成，客户端请求的处理可以在任何服务器上执行，很容易在服务端实现基于 HTTP 的负载均衡，从而使服务端具备良好的横向可扩展性	(4)

思路解析

本题考查 Web服务集成架构选择。常见的两种方式是 SOAP RPC 和 REST风格RPC。

SOAP属于 重量级Web服务规范，基于WSDL和XML，支持复杂的数据类型、状态管理和事务处理，并且在安全性方面有W3C标准化的 WS-Security 规范。因此SOAP适合对安全性、事务一致性要求较高的企业级应用，例如银行支付系统。

而REST是一种 轻量级服务调用方式，基于HTTP协议，使用GET、POST、PUT、DELETE四种操作对应CRUD操作。REST具有以下优势：

- 无状态性：每个请求独立，服务器无需保存会话，支持横向扩展，容易做负载均衡。
 - 性能高：基于HTTP直接通信，减少了SOAP复杂的XML封装与解析。
 - 开发成本低：调用形式简单，前后端可以轻松集成，适合云平台配置数据的获取与监控场景。
- 因此，表5-2中的理由分析如下：
- （1）REST通过HTTP方法映射CRUD操作，理由合理（√）；
 - （2）REST并不直接提供安全机制，SOAP才具备WS-Security，因此此理由不合理（×）；
 - （3）REST并不支持有状态操作，SOAP才适合有状态事务，因此此理由不合理（×）；
 - （4）REST支持无状态性和负载均衡，理由合理（√）。

答案-问题（1）

- （1）基础服务支持：因为WebX在阿里巴巴和淘宝用了很多年，对于超大访问量的电子商务网站，WebX经受了考验，被证明是成熟可靠的。
- （2）多应用支持：WebX和Spring MVC一样，完全建立在Spring框架之上，可以使用Spring的所有特性。WebX被设计成多个层次，层次间的分界线很清晰，每个层次都足够开放和易于扩展。
- （3）可扩展性：WebX对Spring做了扩展，一个组件可以扩展另一个组件，也可以被其他组件扩展。这种机制使WebX具有较好的扩展性，且比未经扩展的Spring更易使用。

答案-问题（2）

(1) A (2) B (3) B (4) A (5) A (6) B

答案-问题（3）

(1) ✓ (2) × (3) × (4) ✓

(2015年5月系统分析师真题-第5题)

阅读以下关于Web应用系统分析与设计的叙述，在答题纸上回答问题1至问题3。

某汽车配件销售厂商拟开发一套网上销售与交易系统，以扩大产品销量，提升交易效率。项目组经过讨论与分析，初步确定该系统具有首页、商品列表、商品促销、商品库存、商品价格、订单中心、订单结算、支付、用户管理、频道（用于区分不同类别的商品）、搜索、购物车等主要功能。

问题（1）

根据业务逻辑切分系统功能是进行系统功能分解的一项重要原则。项目组目前已经将该系统分解为网站、交易和业务服务三个子系统，请将题干中已经确定的系统功能归入这三个子系统中，填写表5-1中的空白，将解答写在答题纸的对应栏内。

子系统名称	对应功能
网站子系统	
交易子系统	
业务服务子系统	

思路解析

系统功能划分的原则是前端展示交互类 → 网站子系统；交易闭环直接相关 → 交易子系统；后台支撑管理类 → 业务服务子系统。

具体来看：

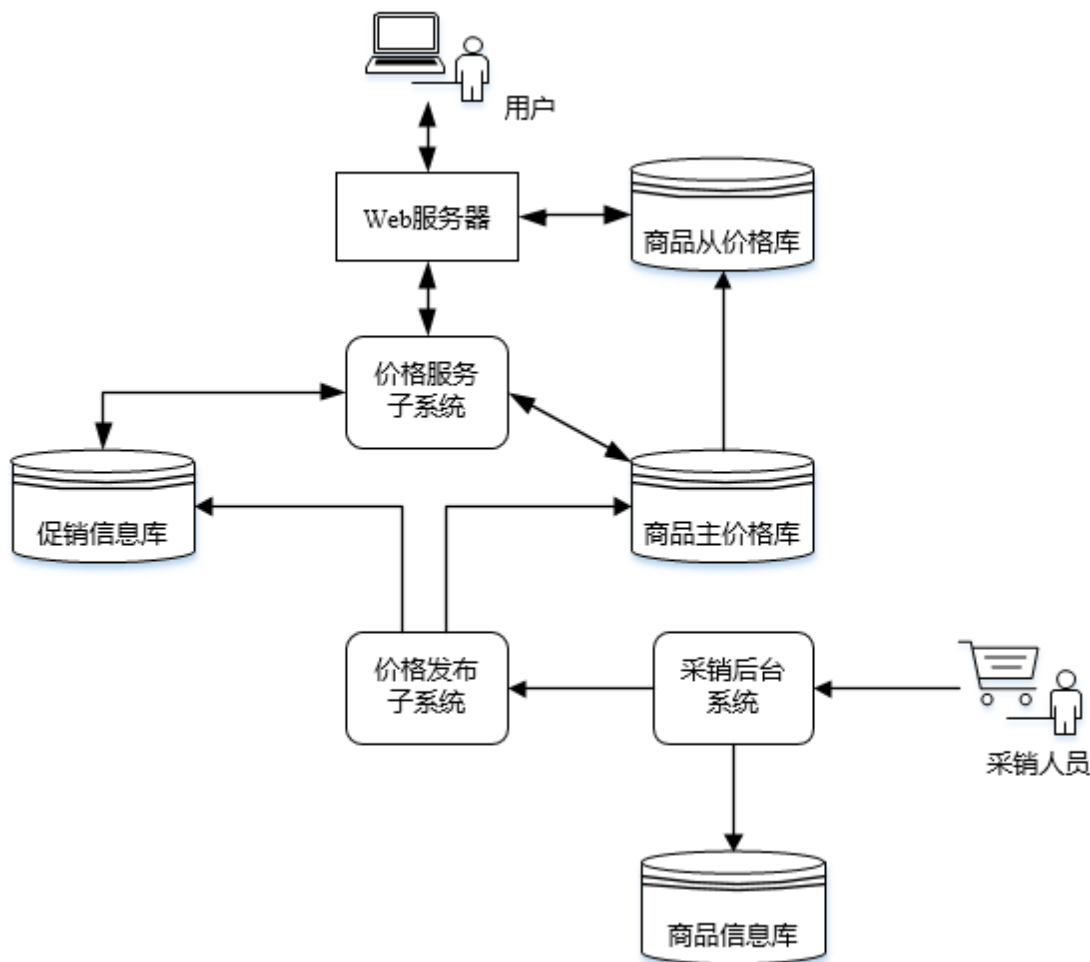
网站子系统：主要负责与用户交互的功能，承担“信息展示”和“访问入口”的作用。例如：首页是门户入口，商品列表提供浏览，频道用于分类导航，搜索提供快速检索。这些功能均与页面交互紧密相关。

交易子系统：核心是“用户下单一结算一支付一完成”的闭环。购物车承载临时订单缓存，订单中心管理订单生命周期，订单结算计算应付金额并确认支付方式，支付负责最终资金流转。它们均是典型的交易相关功能。

业务服务子系统：提供后台数据支撑。例如，商品促销负责活动策略，库存负责数量控制，价格管理提供实时价格，用户管理维护用户资料。这些均是支撑交易正常运转的后台逻辑。

问题（2）

商品实时价格查询是该系统一个重要的业务场景，其完整业务流程如图5-1所示。其中商品实时价格由采销人员在后台设置，包括基础价格与促销信息（例如直降、打折等）；用户在前台商品详情页页面请求实时价格；商品实时价格则由商品的基础价格与促销信息计算得出。



基于上述流程，系统设计人员进一步将业务流程细分为商品价格写逻辑流程、商品价格读逻辑流程和回源写逻辑流程三个部分。根据图5-1所示的业务流程和题干描述，从备选答案中选择正确的选项填写表5-2中的空（a）~（h），将解答写入答题纸的相应栏内。

业务流程	流程描述	备选答案
商品价格写逻辑流程	采销后台系统更新价格，写商品信息库，并通过（a）任务通知（b）更新促销信息库；更新促销信息库，更新商品主价格库的（c）。	商品价格数据 商品价格数据时间戳 同步 异步 商品价格写逻辑流程 商品价格读逻辑流程 回源写逻辑流程 商品信息库 商品主价格库 商品从价格库 促销信息库 价格服务子系统 价格发布子系统 采销后台系统
商品价格读逻辑流程	Web 服务器读取（d）中的价格数据，无过期则直接返回用户；过期或没有命中则执行（e），取最新数据返回用户。	
回源写逻辑流程	价格服务子系统读取（f）和（g），计算价格返回用户，同时异步写商品主价格库；商品主价格库同步数据到（h）。	

思路解析

从业务流程可见，系统设计分为三条逻辑：

商品价格写逻辑流程：采销人员在后台录入或修改商品价格，更新商品信息库，同时通过**（a）异步任务将价格更新推送至（b）价格发布子系统**，并写入**（f）促销信息库与主价格库。主库更新后生成（c）商品价格数据时间戳**，用于判断数据是否为最新。

商品价格读逻辑流程：用户在前台请求价格时，首先访问 Web 服务器，查询**（d）商品从价格库**。若数据有效直接返回；若无效或过期，则触发**（e）回源写逻辑流程**。

回源写逻辑流程：由价格服务子系统读取**（f）促销信息库与（g）商品信息库**，计算出实时价格并返回用户，同时异步写入主价格库。主价格库同步至**（h）商品从价格库**，完成数

据闭环。

问题（3）

在设计实现商品实时价格查询业务流程时，项目组的分析师王工向系统设计师和实现人员提出了一些原则，请分析并判断这些原则的正确性，将应填入表5-3中（a）~（d）处的解答写在答题纸的相应栏内。

原则	正确：√ 不正确：×
垂直扩展--分流原则。前置的 Web 服务器和商品从价格库的架构是无状态的。商品从价格库可以存放全部商品价格信息。这样的设计可以垂直扩展，瓶颈在于商品主价格库的数据复制能力。	(a)
保护系统--降级原则。在系统实现时可以通过开关机制控制回源流程。系统降级时设置为不回源，业务影响为价格不更新，但交易系统的价格为最新价格。	(b)
提高性能--读写分离原则。系统实现时需要将读写逻辑分成几个闭环，互相耦合完成系统功能。	(c)
提高性能--异步化原则。商品价格发布时异步写商品主价格库，回源异步写商品主价格库。	(d)

思路解析

逐条分析：

- （a）垂直扩展-分流原则：题干描述为“前置 Web 服务器与商品从价格库均无状态，可存放所有商品价格信息，瓶颈在于主库复制能力”。然而，这种通过增加从库节点分担读请求的方式属于水平扩展（scale-out），而非垂直扩展（scale-up）。因此该表述错误。
- （b）保护系统-降级原则：系统负载过高时，可通过开关控制回源流程，避免频繁回源导致主库或促销信息库过载。虽然此时价格可能不更新，但保证了交易系统的稳定性。这是典型的降级保护思路，因此正确。
- （c）提高性能-读写分离原则：正确做法应是读走从库，写走主库，避免耦合。而题干描述却称“需要将读写逻辑分成几个闭环，互相耦合完成功能”，这与原则相违背。读写分离的本意是解耦，错误在于误解概念，因此为错误。
- （d）提高性能-异步化原则：系统通过异步写商品主价格库、异步同步从库，从而减少同步等待时间，提高整体性能。这是分布式系统常用优化手段，因此该表述正确。

答案-问题（1）

- 网站子系统： 首页、商品列表、频道、搜索
- 交易子系统： 购物车、订单中心、订单结算、支付
- 业务服务子系统： 商品促销、商品库存、商品价格、用户管理

子系统名称	对应功能
网站子系统	首页、商品列表、频道、搜索
交易子系统	订单中心、订单结算、支付、购物车
业务服务子系统	商品促销、商品库存、商品价格、用户管理

答案-问题（2）

- （a）异步
- （b）价格发布子系统
- （c）商品价格数据时间戳
- （d）商品从价格库

- (e) 回源写逻辑流程
- (f) 促销信息库
- (g) 商品信息库
- (h) 商品从价格库

答案-问题（3）

- (a) ×
- (b) ✓
- (c) ×
- (d) ✓

(2014年5月系统分析师真题-第5题)

某软件公司拟为其客户开发一套基于Web的电子商务系统，该系统向终端用户提供在线购物功能。近期，项目组召开会议对以下两项需求进行了重点讨论：

（1）系统终端用户的界面呈现应提供丰富的多媒体信息，包括文本、图片、动画、视频及语音消息。

（2）系统上线后需应对大量客户端并发请求处理，商家促销活动时，并发用户数可能会达到20万的规模；系统预期用户呈明显地地域集中分布特征。

问题（1）

项目组在讨论实现需求（1）的技术方案时，首先确定了以下技术原则：

- （a）应在开发阶段容易获得良好的协作开发环境支持；
- （b）应考虑客户端浏览器的兼容性；
- （c）应尽可能使系统具有良好的可维护性；
- （d）应考虑公司开发人员的技术学习成本。

项目组就Flex与HTML5两种技术方案进行了论证，综合考虑上述技术原则要求，最终采用了基于Flex的技术方案，请结合需求（1）及上述技术原则，对比Flex与HTML5两种技术方案的优劣，说明采用基于Flex的技术方案的原因。

思路解析

Flex 已经淘汰了，熟悉的人不多，作为了解补充即可。

从四项原则映射比较：（a）协作环境——Flex 时代即有成熟的 IDE、可视化组件库与模块化打包流程，团队协作（UI/逻辑/资源）边界清晰；彼时 HTML5 工具链碎片化（构建、脚手架、调试与跨端适配方案多而杂），落地需要较大工程化投入。（b）兼容性——Flex 依赖统一运行时（Flash Player/AIR），多媒体控件/编解码一致性较好；而 HTML5 在不同浏览器对 Video/Audio 编解码、Canvas 性能、WebGL 支持度等差异显著，需大量降级与 Polyfill。（c）可维护性——Flex 的组件化、事件模型、数据绑定、皮肤机制相对统一，可维护性强；HTML5 需要在 JS 框架/状态管理/模块化/样式工程 等方面自建规范（当时），维护难度上升。（d）学习成本——Flex 面向企业开发提供控件化抽象，UI/媒体能力“开箱即用”；HTML5 要求工程师同时具备 HTML/CSS/JS 与浏览器兼容技能，且多媒体处理链条长、编码协议复杂，学习与踩坑成本更高。

问题（2）

项目组在讨论实现需求（2）的技术方案时，首先确定了以下技术原则：

- （a）系统中商品信息及用户信息按类别划分不同数据库或表存放；
- （b）系统应提供热备份机制以防止服务器意外失效；
- （c）为满足大规模并发处理要求，系统软硬件投入可根据需要追加。

项目组经过集思广益，抽取了HTML静态化、缓存、库表散列、集群与镜像、负载均衡等候选技术手段。请结合需求（2）及相应技术原则，分析上述技术手段在本项目中的可行性，将结果填入表格5-1中。

技术手段	是否可行	原因分析
HTML 静态化		
缓存		
库表散列		
集群与镜像		
负载均衡		

思路解析

本系统在促销峰值面向20 万并发且拥有明显地域集中特征，架构需兼顾吞吐、可用与就近访问。HTML 静态化通过将详情页/活动页预生成，一般配合 ETag/Last-Modified 与增量更新，可将读流量大量转移到 CDN/反向代理，极大减轻动态层与数据库压力。

缓存是核心手段：浏览器+CDN+网关/代理+应用内/分布式（Redis） 的多级缓存，辅以热点隔离、TTL、失效通知、穿透/击穿/雪崩防护，对峰值削峰填谷尤为关键。

库表散列遵循原则（a），按商品/用户/订单等维度分片，结合读写分离、二级索引、跨分片路由，提升水平扩展能力并减少单点热点。

集群与镜像满足原则（b）：数据库主从/半同步/同步复制、对象存储多副本、搜索集群（如分片+副本），应用无状态化后横向扩展，并配置健康检查与自动故障转移。

负载均衡贯穿入口到后端：依据地域将用户引导至就近机房；四/七层负载均衡实现会话保持/权重/熔断/限流，与蓝绿/金丝雀发布结合，实现弹性与稳定。

答案-问题（1）

Flex的优势：大量控件支持、完整的企业化开发流程及工作流、多种框架可供选择。

Flex的劣势：生成的SWF过大、效率问题、较差的图文混排支持。

HTML 5的优势：真正意义上全平台支持、可以胜任后台，比Flex拥有更大、更全面、更活跃的社区。

HTML 5的劣势：缺乏良好的协作开发环境，缺乏完整的开发流程；要编写CSS与JavaScript,学习成本高；浏览器兼容问题、效率问题、保密性及安全性、AJAX跨域通讯等问题。

基于以上比较，在本案例中，由于HTML5缺乏良好的协作开发环境、学习成本高、各种浏览器不兼容，缺乏完整的开发流程，系统可维护性差。因此，选择基于Flex的方案。

答案-问题（2）

技术手段	是否可行	原因分析
HTML 静态化	否	电子商务数据经常发生变化，不适合静态化
缓存	是	对经常访问的数据进行缓存，以提高访问效率
库表散列	是	满足（a）的要求，使输入输出更加均衡
集群与镜像	是	满足（b）的要求，提供热备机制
负载均衡	是	可以按照地域特分布特性，将不同地区的用户分配到不同的服务器

(2012年5月系统分析师真题-第2题)

阅读以下关于企业应用系统开发的叙述，在答题纸上回答问题1至问题3。

某大型咨询服务公司欲对现有的核心咨询业务系统进行升级改造，以适应大量用户的个性化咨询服务要求，提高系统的灵活性。公司主管将核心业务系统的升级改造工作交给了公司的系统分析师王工和李工。

一个月后，王工和李工分别向公司提交了自己的方案。王工主张以公司现有的、采用面向对象技术和Java语言实现的业务系统为基础，针对新的业务需求对系统进行重构、改造与升级。李工则认为现有系统的业务逻辑过于复杂，对系统进行重构的成本太高，可以采用面向服务的思想，提炼可复用的业务功能形成服务，实现系统的灵活性。经过公司相关人员共同开会讨论，最终采用了李工的改造方案。

问题（1）

请用500字以内的文字，从系统业务功能实现和功能集成两个方面对王工和李工的方案进行分析和对比，并结合项目需求说明公司为何会选择李工的方案。

思路解析

从系统业务功能实现角度：王工的方案基于面向对象技术（OO），以对象为核心概念，依靠对象之间的消息传递来实现业务逻辑。此方式结构清晰，封装性好，但业务逻辑往往与代码层紧密绑定，当需求变动时需要修改对象类或方法，并重新部署系统，灵活性不足。

李工的方案基于面向服务的架构（SOA），以服务为核心概念。业务功能封装为服务，并通过标准接口暴露，调用时不依赖具体实现细节。这样能实现粗粒度功能复用，降低了业务变化对系统代码的影响。

从系统功能集成方式角度：面向对象的系统以对象为单元集成功能，通常需要结合工作流引擎编排对象间的调用关系。对象粒度小、集成度有限，系统扩展和跨部门业务流程整合难度较大。

面向服务的系统则以服务为单元进行集成，采用服务组合与编排技术（如BPEL）实现灵活的业务流程重组，可以快速适应业务个性化与多变需求。

选择李工方案的原因：根据题干，公司需要升级现有系统以支持大规模个性化咨询服务，关键目标是降低改造成本并提升灵活性。王工的方案需要重新梳理并改造现有复杂对象模型，开发与维护成本高，且灵活性有限。而李工的方案通过封装现有功能为服务，以低成本方式实现快速改造，并借助服务组合满足多变的业务需求。因此，公司最终选择了李工的方案。

问题（2）

采用服务思想设计系统时，需要考虑服务的耦合性。服务的耦合性可以分为两类，分别是服务契约耦合（Service Contract Coupling）和服务消费者耦合（Service Consumer Coupling）。请对各种耦合关系进行分析，填写（a）~（g），完成表2-1。

耦合关系	耦合类型	是否有害	耦合关系解释
业务逻辑-契约耦合	(a)	(d)	(f)
契约-技术耦合	(b)	(e)	(g)
消费者-实现耦合	(c)	是	服务消费者需要通过服务契约使用服务功能，而不能直接与服务实现相关。
消费者-契约耦合	服务消费者耦合	否	在使用服务功能时，服务消费者需要与服务契约进行绑定。这样能够提高服务消费者和服务之间的独立性。

思路解析

服务耦合关系主要包括契约耦合和消费者耦合两类：

契约耦合：服务的业务逻辑与服务契约绑定，保证调用方只需理解契约即可使用服务。这种耦合符合 SOA 思想，因为它屏蔽了实现细节，使服务具备跨平台、跨厂商复用性。若契约依赖具体厂商技术（如特定中间件协议），会降低可移植性和复用性，因此应采用标准（XML、WSDL、SOAP/REST）。

消费者耦合：服务消费者与服务实现绑定，将导致强依赖，若实现变动则需同时修改消费者，严重影响系统灵活性。正确做法是让消费者仅依赖服务契约，而非具体实现。

SOA 的核心目标是通过松耦合提升复用性与灵活性。因此，良好的服务设计应保证：逻辑与契约耦合，而契约与实现解耦；消费者与契约耦合，而与实现解耦。

问题（3）

在对系统的业务服务进行初步分析后，李工首先提取了客户注册、业务受理和发票开具三个典型的业务服务。但进一步分析后，李工发现这三个服务需要使用数据库中的客户实体、业务实体和发票实体，而发票实体包含客户实体和业务实体的信息，这样会导致发票开具服务内部包含并重复实现客户注册和业务受理的处理过程，降低服务的复用性。请说明这种情况产生的主要原因，并针对这种情况，说明该如何对这三个服务进行重构，使它们具有更好的重用性。

思路解析

题干中的问题实质是服务粒度划分不当。当一个业务服务内部包含了多个实体的处理逻辑时，其他需要这些实体的服务将无法直接复用，只能重复实现逻辑，造成冗余。结果就是发票开具服务中嵌套了客户注册与业务受理的处理过程，破坏了服务的独立性和复用性。

合理的做法是引入实体服务层，将与核心数据实体（客户、业务、发票）相关的操作抽象为独立的服务。这种设计使得高层业务服务（如发票开具）只需组合调用底层实体服务，而不必重复逻辑。这样不仅提升了服务复用性，还减少了冗余，提高了系统的可维护性和扩展性。

答案-问题（1）

从系统业务功能实现来看，采用面向对象技术开发的系统以对象为核心概念，通过对象之间的消息交互完成业务功能；采用面向服务技术开发的系统以服务为核心概念，业务功能需要封装为服务。从系统功能集成方式来看，采用面向对象技术开发的系统以对象为单元进行功能集成，通常采用工作流技术定制业务流程；采用面向服务技术开发的系统以服务为单元进行功能集成，采用服务组合技术实现灵活的业务集成与重组。

根据项目需求，需要对现有业务系统进行升级改造，因此保证升级改造的低成本是一个重要的因素，王工的方案需要现有系统进行功能梳理与重新开发，升级改造成本较高；李工的方案则侧重将现有功能包装为服务，升级改造成本较低。项目还要求提高系统的灵活性，王工的方案以对象技术为

基础，当业务发生变化时，需要在代码层次上进行修改并重新部署，因此系统的灵活性相对较差；李工的方案以大粒度的业务服务复用为基础，当业务发生变化时，可以采用服务组合技术实现系统的运行时功能组合，因此灵活性相对较好。

答案-问题（2）

- (a) 服务契约耦合
- (b) 服务契约耦合
- (c) 服务消费者耦合
- (d) 否
- (e) 是
- (f) 按照SOA的思想，业务逻辑应当与契约耦合，而不应与实现或技术耦合
- (g) 服务契约不应该与具体厂商的实现技术相关，而应该基于标准的XML技术与Web服务标准

答案-问题（3）

针对该问题，应该增加客户实体服务、业务实体服务和发票实体服务，形成实体服务中间层。在这种情况下，客户实体服务和业务实体服务不仅能够为客户注册和业务受理服务提供支持，还可以为发票开具服务提供支持，这样就避免了不可见处理过程在多个服务中存在。

(2012年5月系统分析师真题-第5题)

阅读以下关于Web应用系统开发的问题，在答题纸上回答问题1至问题3。

A公司承担了某企业应用系统的开发任务，用户要求系统最终应发布到Web上供企业员工及企业客户使用。项目组在进行方案论证时，首先肯定了该系统需使用B/S结构，但在系统应采用的底层平台上产生了分歧，一方认为应采用微软.NET平台，一方认为应采用Java企业版平台。经过认真讨论，结合两种平台的特点及项目的实际需求，项目组最终决定采用Java企业版平台作为系统开发运行的基础平台。

问题（1）

请在以下平台特点（1）~（9）中，选择出.NET平台与Java企业版平台各自具备的优势填入表5-1的

（a）~（f）项中，选择出两个平台共有的特点填入表5-1的（g）~（i）项中。

- （1）良好跨平台可移植性支持
- （2）易于部署与配置
- （3）多程序设计语言支持
- （4）良好的Web多层应用开发支持
- （5）丰富的多厂商外部支持
- （6）良好的O/R（对象/关系）映射支持
- （7）针对特定平台的优化支持
- （8）良好的源代码以外的可定制性支持
- （9）良好的Web服务支持

.NET平台	Java企业版平台
(a)	(d)
(b)	(e)
(c)	(f)
(g)	
(h)	
(i)	

思路解析

.NET 平台紧密依赖于微软生态，因此优化性与部署简便性是其典型优势：易于在 Windows 环境下快速安装和配置；提供 CLR（公共语言运行时）支撑多语言开发（C#、VB.NET 等），并且由于与操作系统深度绑定，能获得最佳性能和安全集成。而 Java EE 的核心优势在于跨平台与多厂商支持，基于 JVM 的“一次编译，到处运行”以及庞大的开源生态，使其在复杂企业应用领域得到广泛采用。此外，Java EE 的应用服务器（如 WebLogic、JBoss、Tomcat+Spring）普遍支持丰富的可定制性，适应不同规模与场景。

二者共有的优势主要体现在企业级应用的通用需求：多层架构（Web/Service/DB）、O/R 映射（Hibernate、Entity Framework）、以及 Web Service（SOAP、RESTful）均是现代企业应用平台的标配。整体来看，Java EE 的平台独立性和生态优势更符合题干用户的需求，因此项目组选择 Java 企业版。

问题（2）

MVC（Model-View-Controller）模式是Web应用系统开发中常用的一种软件架构模式。请分别针对基于EJB的重量级框架和基于Struts等的轻量级框架，说明MVC模式中的各组件应采用何种构件实现。项目组在进行需求调研时，发现用户界面部分的变动可能会比较频繁，因此需要降低系统界面与业务逻辑之间的耦合度。MVP（Model--View--Presenter）模式是由MVC模式派生出的一种设计模式，其主要目的是降低MVC模式中模型（Model）与视图（View）的耦合度，请用300字以内文字，从组件耦合度、组件分工及对开发工程化支持等三方面说明MVP模式与MVC模式的主要区别。

思路解析

MVC 是 Web 系统的基础架构模式：模型负责业务逻辑，视图负责用户交互，控制器负责协调。EJB 重量级框架侧重于企业级事务与分布式对象管理，因而业务逻辑多由 EJB 构件实现；视图通常采用 JSP；控制器往往是 Servlet。轻量级框架 Struts 则通过 Action 类承担控制器角色，业务逻辑封装在 POJO+DAO+Service 层，表现层依赖 JSP 与标签库，部署灵活。

MVP 模式是在实践中为解决 MVC 耦合问题而演化而来。MVP 通过 Presenter 在模型与视图之间增加中介层，屏蔽二者直接交互，使系统具有更高的灵活性与可测试性。Presenter 集中业务逻辑，保证视图简单化（仅处理事件和渲染）。在开发工程化方面，MVP 支持 Mock View 与自动化测试

，大大提升敏捷开发与持续集成的可行性。相比之下，MVC 更加轻便直接，但在大型企业应用中往往难以满足复杂性与测试需求。

问题（3）

因为系统中大量业务逻辑涉及企业的核心商业数据，为保证系统数据一致性，完善的事务（Transaction）控制是系统实现时必需考虑重要因素之一。请用200字以内文字说明事务的基本特征，并简单描述EJB规范中提供的两种事务控制的基本方法。

思路解析

事务保证企业应用在并发与异常情况下保持数据一致性。ACID 特性是事务的核心约束：原子性确保部分执行失败不会破坏系统状态；一致性确保数据始终满足约束（如余额不能为负）；隔离性确保并发事务不会互相污染（通过隔离级别控制）；持久性保证提交后的结果不会因系统故障而丢失。

EJB 作为企业级 Java 平台，提供两种事务控制方式：

CMT（容器管理事务）：最常见方式，开发者无需关心事务边界，由容器根据配置自动开启/提交/回滚。其优势是开发效率高，但灵活性有限。

BMT（Bean 管理事务）：开发者通过 JTA 手动编写事务逻辑，可以跨方法或跨资源管理事务，灵活度高但复杂度大。

答案-问题（1）

- （a）~（c）：（2）、（3）、（7），以上三个答案顺序可调换；
- （d）~（f）：（1）、（5）、（8），以上三个答案顺序可调换；
- （g）~（i）：（4）、（6）、（9），以上三个答案顺序可调换；

答案-问题（2）

MVP模式与MVC模式的主要区别为：

（1）在组件耦合度方面：在MVP模式中，视图并不直接使用模型，它们之间的通信通过Presenter进行，从而实现了视图与模型的分离，而在MVC模式中，视图直接与模型交互。

（2）在组件分工方面：在MVP模式中，视图需要处理鼠标及键盘等触发的界面事件，而在MVC模式中这通常是由控制器完成的工作；在MVP模式中，系统核心业务逻辑组织集中在Presenter中，而在MVC模式中，相应的控制器通常只完成事件的分发。

（3）在开发工程化支持方面：MVP模式可更好地支持单元测试，而在MVC模式中，由于模型与视图绑定，因此难以实施相应的单元测试；在MVP模式中，Presenter基于约定接口与视图和模型交互，可更好地支持组件的重用。

答案-问题（3）

事务的基本特征包括：

原子性：一个事务中的所有操作，要么全部完成，要么全部不完成，不会结束在中间某个环节。事务在执行过程中发生错误，会被回滚到事务开始前的状态，就像这个事务从来没有执行过一样。

一致性：在事务开始之前和事务结束以后，数据的完整性限制没有被破坏。

隔离性：两个事务的执行是互不干扰的，两个事务时间不会互相影响。

持久性：在事务完成以后，该事务对数据所作的更改便持久地保存在数据库之中，并且是完全的。

EJB规范支持的两种事务控制方法为：

容器维护的事务（Container Managed Transaction, CMT）：由EJB容器根据部署描述符或EJB构件注释中指定的事务属性自动控制事务的边界，容器维护的事务是方法级的，即默认将一个方法当作一个事务执行，当方法执行的过程中发生系统级异常，容器会自动将事务回滚，从而将方法前面执行的结果恢复。

Bean维护的事务（Bean Managed Transaction, BMT）：由程序员在EJB的源代码中控制事务执行的边界，事务的边界通过Java事务接口（Java Transaction API, JTA）进行控制，Bean维护的事务可以跨越方法的边界。

（2024年5月系统分析师真题-第4题）

随着互联网技术的发展，越来越多的应用需要实现实时双向通信。传统的HTTP协议和WebSocket协议是两种常见的通信协议。在实际应用中，HTTP协议常用于网页浏览和资源请求，而WebSocket协议则在需要实时双向通信的场景中得到广泛应用。

问题（1）

请说明http和websocket连接的异同。

思路解析

此题考察 HTTP 和 WebSocket 的区别。

HTTP。请求-响应模式：客户端发送请求，服务器返回响应，完成一次通信。无状态：每次请求都是独立的，不保留之前的通信状态。常用于网页浏览：用于加载网页内容，如HTML、CSS、JavaScript等。

WebSocket。双向通信：建立连接后，客户端和服务端可以随时互相发送数据，实现实时通信。持久连接：连接建立后保持开放，直到一方主动关闭。适合实时应用：如在线聊天、实时游戏、股票行情推送等。

问题（2）

在现代Web应用中，前端和后端的分离已成为常见的架构设计模式。请分析前端与后端分离的优势与挑战。

思路解析

前后端分离是一种极大提高开发效率和用户体验的架构模式。在传统的单体Web应用中，前端与后端通常是在一个应用内紧密耦合，前端与后端之间的职责和开发周期不易并行化。然而，随着前后端分离的实施，前后端开发团队可以并行开发，从而大幅提高开发效率。前端开发人员专注于UI、交互和响应式设计，后端开发人员专注于处理数据逻辑、数据库操作和业务层功能。这种并行化开发模式不仅节省了开发时间，还可以降低开发过程中的沟通成本和出错率。

此外，前后端分离使得技术栈独立性成为可能，前端和后端可以选择各自最合适的技术栈。前端可以使用现代的JavaScript框架（如Vue.js、React、Angular等），而后端可以使用更合适的后端框架和编程语言（如Spring、Node.js等），并且前后端之间的交互主要通过RESTful API或GraphQL来实现。

另一个重要的优势是用户体验的提升。**单页面应用（SPA）**的实现往往需要前后端分离来支持，它减少了传统Web应用中的页面刷新，从而提高了响应速度，使得用户可以在没有刷新页面的情况

下获得更流畅的交互体验。

尽管前后端分离有诸多优势，但在实际应用中也会面临一些挑战。首先是跨域问题。前后端分离时，前端和后端往往部署在不同的域名、端口或服务器上，这违反了浏览器的同源策略（Same-Origin Policy），导致浏览器默认阻止跨域请求。因此，开发人员需要采取以下解决方案：

在后端使用CORS（Cross-Origin Resource Sharing）来允许跨域请求。

使用代理（如webpack的devServer代理、Nginx反向代理等）将前端的请求代理到后端。

其次，数据同步问题也是前后端分离中的一个常见问题。当后端API接口的数据结构发生变化时，前端需要及时调整，以确保数据结构的一致性。如果前端和后端的接口规范没有得到有效的沟通和合作，可能会导致接口不一致，进而导致页面出现错误。因此，团队间的良好沟通和API接口文档的及时更新显得尤为重要。可以通过使用接口文档自动生成工具（如Swagger）以及前后端联调机制来缓解此类问题。

最后是权限控制问题。前后端分离后，权限控制需要在前端和后端两个层面进行协作。前端通常通过路由守卫来控制用户访问某些页面或功能，但后端仍然需要验证用户的身份和权限，确保数据安全性。因此，前端权限控制需要与后端权限管理相结合，前端可以通过JWT（JSON Web Token）等方式向后端提供用户认证信息，而后端则通过检查用户的权限来决定是否允许访问资源。

问题（3）

思路解析

答案-问题（1）

HTTP协议：基于请求-响应模式，客户端每次发送请求，服务器返回响应，通信完成后连接关闭。

HTTP是无状态的，每次请求都是独立的，不保留之前的通信状态。适用于传统的网页浏览和资源加载，如HTML、CSS、JavaScript等内容的请求。

WebSocket协议：基于双向通信模式，建立连接后，客户端和服务端可以随时互相发送数据，直到一方主动关闭连接。WebSocket提供持久化连接，因此它适用于需要实时交互的应用场景，如在线聊天、实时游戏、股票行情推送等。

答案-问题（2）

前端与后端分离是一种现代Web开发架构，它将前端界面与后端逻辑进行独立分离。这样做的优势在于开发效率、维护性和灵活性等方面。然而，也面临一些挑战，例如跨域请求、数据同步、权限控制等问题。

优势：

开发效率提高：前后端分离后，前端与后端可以并行开发，减少了开发周期。前端开发人员可以专注于UI和用户交互设计，后端开发人员则专注于API设计与数据库操作。

技术栈独立性：前端和后端的独立性使得前端可以使用各种现代的框架（如React、Vue.js等），而后端则可以使用合适的语言和框架（如Node.js、Spring等）。这种独立性为技术栈的选择提供了更多的灵活性。

提升用户体验：前后端分离通常意味着前端采用单页面应用（SPA）架构，减少页面刷新，提供流畅的用户体验。

挑战：

跨域问题：前后端分离时，前端和后端往往部署在不同的域名或端口上，这导致浏览器会因为同源策略而阻止跨域请求。需要使用**CORS（跨域资源共享）**或代理等手段来解决跨域问题。

数据同步问题：前端和后端数据接口分离后，前端需要通过API接口获取数据。如果数据结构发生变化，前端和后端需要协调更新，避免数据同步问题。

权限控制复杂化：前后端分离后，权限控制可能需要在前端和后端两侧同时实现。例如，前端可以通过路由守卫控制用户的访问权限，而后端则需要进一步验证用户的权限并返回相应的资源。

答案-问题（3）

（2025年5月系统分析师真题-第5题）

某智慧城市运维平台采用典型 Web 三层架构。前端展示层用多种浏览器技术实现可视化；业务应用层由多个子系统组成；数据存储层同时引入关系型与非关系型数据库，并按数据来源建立主题库；底层资源通过容器化方式部署在混合云硬件环境上。为保证开发效率，项目组决定采用 MVC 模式，并基于SSM（Spring + Spring MVC + MyBatis）技术栈实现。

问题（1）

解释 MVC 架构，并说明 SSM 框架内部是如何实现 MVC 的。

思路解析

MVC 作为一种经典的架构模式，其核心价值在于分离关注点，通过将应用拆分为 Model、View、Controller 三个独立模块，解决传统开发中代码混杂、耦合度高的问题，从而提升系统的可维护性、可扩展性和可测试性。

具体来看，MVC 的三个模块各有明确边界：

Model不仅包含业务数据（如用户信息、订单详情），还封装了核心业务逻辑（如数据校验、计算规则），是系统的“数据与规则中心”。它不依赖于 View 和 Controller，即便前端展示或请求处理方式变化，Model 层的核心逻辑也可保持稳定。

View专注于数据展示，负责将 Model 中的数据以用户可感知的形式呈现（如网页、移动端界面）。它不处理业务逻辑，仅被动接收 Model 传递的数据，当展示需求变化时（如从 JSP 切换到 Vue），只需修改 View 层而不影响其他模块。

Controller扮演“调度者”角色，负责接收用户请求（如表单提交、API 调用），根据请求内容调用对应的 Model 层逻辑，再将处理结果传递给 View 层展示。它不直接处理业务数据，也不负责展示，仅协调 Model 和 View 的交互。

SSM 框架（Spring + Spring MVC + MyBatis）则是 MVC 模式在 Java Web 开发中的具体落地实现，通过三个组件的分工协作，将 MVC 的思想转化为可执行的技术方案，同时解决了 MVC 模式本身未覆盖的技术细节（如对象管理、数据持久化、横切逻辑处理等）。

Spring MVC：作为 Controller 层的具体实现，它通过DispatcherServlet（前端控制器）统一接收 HTTP 请求，再通过HandlerMapping（处理器映射）找到对应的处理器（Controller 类），由HandlerAdapter执行处理逻辑，最终将结果封装为ModelAndView，交由视图解析器（如InternalResourceViewResolver）解析并渲染 View（JSP、Thymeleaf 等）。其核心是规范了请求处理的流程，让 Controller 层的职责更清晰。

MyBatis：作为 Model 层中“数据持久化”部分的实现，它通过 XML 映射文件或注解，将

Java 对象与数据库表、SQL 语句进行映射，简化了 JDBC 的冗余代码（如连接管理、SQL 执行、结果集转换等）。开发者无需手动编写PreparedStatement，只需定义 Mapper 接口和 SQL，MyBatis 即可自动完成对象与数据的转换，让 Model 层更专注于业务逻辑而非数据操作细节。

Spring: 作为整个框架的“胶水”，它通过IOC（控制反转）容器统一管理所有 Bean（如 Controller、Service、Mapper 等），实现对象的依赖注入（DI），避免了硬编码的依赖关系（如new对象），降低了组件间的耦合；同时通过AOP（面向切面编程）处理日志、事务、权限等横切逻辑（如在 Service 层方法执行前后自动开启 / 提交事务），让业务代码更纯净。

问题（2）

按图示在空格1~6处补全合适的技术或概念，备选项如下。

A. HTML B. 前端展示层 C. 数据采集 D. 数据资源层 E. 数据中台 F. AI算法模型



思路解析

（1）前端展示层：位于系统上层，与“应用层”“系统管理”等并列，属于用户交互相关模块。“前端展示层”用于展示页面内容、接收用户操作，是整个系统对外呈现的部分，正好承接 HTML、CSS 等前端技术。

（2）HTML：该空位于 CSS 和 jQuery 的左边，三者均属于 前端技术栈，是构建网页的基础技术。HTML 是结构层，CSS 负责样式，jQuery 是 JS 库，三者常作为“前端三件套”一同出现。

（3）数据资源层：MySQL 属于 关系型数据库，是典型的数据资源。该层主要是为数据存储与访问提供支撑。“数据资源层”是系统中的底层数据基础，包含数据库、数据湖等组件，MySQL 作为主流数据库正属于此范畴。

（4）数据采集：本行包含“监控数据、故障信息、业务数据”，显然这是 采集系统运行数据 的一层。数据采集技术包括日志采集、监控指标收集等，用于获取运行状态与反馈，支撑上层分析与决策。

（5）数据中台：Docker 是容器平台，属于基础设施；“混合云”和“硬件”也归属于 IaaS/PaaS 层。而“数据中台”通常是基于这些基础设施运行的一个 数据加工与服务共享层，承上启下，为智

能运营和应用系统提供数据能力支撑。

(6) AI算法模型：该空与“智慧指挥”并列，是最上层的智能决策模块。“AI算法模型”用于数据分析、预测、优化等，是实现智能化决策的关键支撑。它将中台数据转化为智能指令，驱动“智慧指挥”等系统。

问题（3）

下表比较公有云、私有云、混合云三种部署模式。请在 1-9 处填入最恰当的词语（高中低），并说明李工最终选择“混合云”的原因。

	公有云	私有云	混合云
所有权	(1)	(4)	(7)
安全性	(2)	(5)	(8)
扩展性	(3)	(6)	(9)

思路解析

公有云由云服务商所有，而私有云由企业自行拥有，混合云则是公有与私有云共享资源，部分由企业拥有。

安全性上，公有云的安全性较低，私有云拥有最高的安全性，混合云在这方面保持适中的控制。
可扩展性上，公有云最具灵活性，私有云则较为有限，而混合云在可扩展性上表现出较高的弹性。

答案-问题（1）

MVC 是一种将应用划分为 Model（模型 / 业务数据）、View（视图 / 页面展示）、Controller（控制器 / 业务流程）三个部分，实现各司其职、松耦合协作的架构模式。
SSM 框架则是对 MVC 模式的具体实现，其中 Spring MVC 充当 Controller，负责请求分发、数据绑定、视图解析，MyBatis 扮演 Model 持久层，完成对象与 SQL 的映射，JSP、Thymeleaf 等前端模板用于生成 View 以展示模型数据，同时 Spring 容器通过统一管理 Bean，提供 AOP、事务等横切能力，让这三者按 MVC 方式实现解耦协作，其整体流程为 Controller 层通过 Spring MVC 接收 HTTP 请求后调用 Service，Service 再通过 MyBatis 操作数据库（Model），最终返回的结果交由 View 进行渲染。

答案-问题（2）

(1) B (2) A (3) D (4) C (5) E (6) F

答案-问题（3）

(1) 低 (4) 高 (7) 中
(2) 低 (5) 高 (8) 中
(3) 高 (6) 低 (9) 中
李工为何最终选择 混合云

安全合规 vs. 弹性成本。监控数据、故障信息等涉及设备状态与隐私，要求本地存储与严格访问控制 → 放在私有云。业务数据分析、高并发 Web 服务需海量算力与带宽 → 利用公有云弹性池。

逐步上云、保护投资。既保留原有数据中心资产，又能在流量高峰“借力”公有云，避免一次性巨额扩容，兼顾旧系统的可用性与新业务的敏捷发布。

灾备与高可用。部署双活或多活架构：私有云出现故障时可快速切换至公有云节点；同理公有云故障可回落本地，提升业务连续性。

技术演进与供应商锁定风险。保留关键组件在私有环境，可减少单一云厂商依赖；同时通过容器-Kubernetes 生态实现“一次构建，多云部署”，灵活迁移。

(2025年11月系统分析师真题-第2题)

某大型电商平台在“双十二”大促期间，访问量激增，订单与商品查询请求达到平时的数十倍。系统架构采用“应用服务器 + Redis 缓存 + MySQL 数据库”的典型三层结构。促销高峰期间，监控发现数据库 CPU 使用率长期超过 95%，查询响应时间显著上升，部分请求超时，严重影响用户下单体验。经排查发现主要存在以下问题：

热点商品数据访问频繁，缓存命中率下降；

大量恶意或异常请求持续查询数据库中不存在的商品 ID，导致缓存中无法命中，每次请求都直接访问数据库，形成缓存穿透问题；

在缓存失效瞬间，高并发请求直接打到数据库，引发数据库压力骤增，出现类似“击穿”现象。

针对上述问题，架构团队提出以下优化方案：

采用 Redis 集群 实现高可用，结合主从复制与哨兵模式；使用 布隆过滤器（Bloom Filter）对商品 ID 做合法性校验，防止无效请求穿透缓存；在缓存与数据库同步策略上，采用 Write-Through（写穿透）模式保证主从数据一致性。

问题（1）

对比 Memcached，说明 Redis 在该场景下具备的两个主要优势。结合本案例，解释为什么布隆过滤器可以有效解决缓存穿透（或击穿）问题。

思路解析

在本案例中，Redis 相比 Memcached 更适合作为核心缓存组件，主要体现在以下两个方面。

首先，Redis 支持丰富的数据结构。Redis 不仅支持简单的 Key-Value 字符串类型，还支持 Hash、List、Set、ZSet、Bitmap、HyperLogLog 等多种数据结构。在电商系统中，商品信息、库存、用户行为统计、排行榜等业务模型复杂，往往需要对数据进行聚合、计数、排序等操作。Redis 可以在缓存层直接完成这些逻辑，减少应用层计算和数据库访问压力。而 Memcached 仅支持简单的字符串结构，复杂业务需要在应用层处理，灵活性明显不足。因此在热点商品、高并发访问场景下，Redis 更有优势。

其次，Redis 支持数据持久化和高可用架构。Redis 提供 RDB 和 AOF 两种持久化机制，可以在缓存节点重启后快速恢复数据，避免缓存雪崩。而 Memcached 数据全部存储在内存中，一旦重启数据即全部丢失。在本案例中，Redis 结合 主从复制 + 哨兵模式，可以实现自动故障转移，保证缓存服务的高可用性，这对于双十二这种高峰业务尤为关键。此外，Redis 还支持集群模式，能够实现水平扩展，进一步分担热点访问压力。因此，从可靠性和可扩展性角度看，Redis 明显优于 Memcached。

在本案例中，大量恶意或异常请求不断查询数据库中不存在的商品 ID，这是导致数据库 CPU 使用率过高的重要原因之一。这类请求即使 Redis 缓存工作正常，也无法命中缓存，每次都会直接访问数据库，形成典型的缓存穿透问题。

布隆过滤器是一种基于位数组和多重哈希函数的概率型数据结构，其核心特点是：可以快速判断一个元素“可能存在”或“一定不存在”。系统可以在缓存层之前，将所有合法商品 ID 预先写入布隆过滤器。当请求到达时，先通过布隆过滤器进行校验，如果判断结果为“一定不存在”，则直接返回结果或拒绝请求，不再访问 Redis 和数据库。这样，大量无效商品 ID 请求会被拦截在系统最外层，显著降低数据库访问压力。

需要注意的是，布隆过滤器存在一定的误判率（可能误判为存在），但不会误判不存在为存在。在电商场景中，即使出现少量误判，也只会导致请求继续访问缓存或数据库，不会影响数据正确性。通过合理设置位数组大小和哈希函数数量，可以将误判率控制在可接受范围内。因此，布隆过滤器在高并发、读多写少的业务场景下，是解决缓存穿透问题的高效手段，也能有效防止缓存失效瞬间引发的数据库击穿风险。

问题（2）

说明在该电商高并发促销场景下，为什么选择使用 Write-Through。并从一致性、写入性能、数据可靠性、系统复杂度等方面，对比 Write-Through 与 Write-Behind 两种策略。

思路解析

在本案例中，系统面临的是促销高峰期的高并发写请求，尤其涉及商品库存、订单状态、价格等核心数据。这类数据一旦出现不一致，可能直接导致超卖、错价、订单异常等严重业务问题。因此，系统在设计缓存写策略时，优先考虑的是一致性和可靠性而非极致的写性能。

Write-Through 的核心特征是：写请求先更新数据库，再同步更新缓存，写操作对业务层而言是同步完成的。这种方式可以确保任何时刻缓存中的数据都是数据库的最新结果，避免出现“缓存有值但数据库未落盘”的情况。在 Redis 集群主从结构下，Write-Through 可以天然配合主从复制和哨兵机制，主节点写成功后再返回结果，极大降低数据不一致风险。

相较之下，Write-Behind 采用异步方式写数据库，虽然写性能更高，但在节点宕机、缓存丢失、队列堆积等异常情况下，存在数据未及时落库甚至丢失的风险。对于双十二这种业务风险远高于性能瓶颈的场景，Write-Through 更符合系统稳定性和业务安全性的设计目标，因此被选用。从一致性角度看，Write-Through 属于强一致性模型。每次写操作必须同时完成数据库与缓存更新后才返回结果，保证缓存与数据库始终保持一致，非常适合库存、订单等核心业务数据。而 Write-Behind 属于最终一致性模型，缓存更新后通过异步方式批量写数据库，在短时间内允许数据不一致，这在金融、电商核心交易场景中风险较高。

从写入性能看，Write-Behind 的优势明显。由于写数据库是异步完成，业务线程只需更新缓存即可返回，吞吐量更高，适合日志、统计、点赞数等对一致性要求不高的场景。而 Write-Through 每次写请求都涉及数据库 IO，在高并发下性能相对较低，但更加稳定可控。

在数据可靠性方面，Write-Through 明显优于 Write-Behind。Write-Behind 如果在异步队列尚未刷盘前发生缓存节点宕机或进程崩溃，可能导致数据永久丢失；而 Write-Through 写入数据库成功即具备持久性保障，结合 Redis 主从和持久化机制，风险更低。

从系统复杂度来看，Write-Behind 需要维护异步队列、失败重试、顺序控制和补偿机制，实现难度和运维成本较高；而 Write-Through 实现简单，逻辑清晰，更符合软考中强调的**“稳定性优先、复杂度可控”**的系统架构设计原则。

问题（3）

针对数据库在热点缓存失效场景下的崩溃问题，请提出两种解决方案，并分别说明其优缺点。

思路解析

在热点商品缓存集中失效的情况下，瞬时高并发请求会绕过缓存直接访问数据库，这是数据库崩溃的直接原因。因此，第一道防线应当放在数据库之前，通过限流或熔断机制对请求进行削峰。

常见做法是在缓存层或网关层引入限流策略，例如基于 QPS、并发数或令牌桶算法，对访问数据库的请求进行控制。当检测到缓存未命中率异常升高或数据库负载接近阈值时，系统可以触发熔断与降级策略，直接返回兜底数据（如“商品信息加载中”）、静态页面或默认库存状态，而不是继续访问数据库。

该方案的核心优势在于：即使缓存彻底失效，也能保护数据库不被打垮，符合软考中强调的“系统稳定性优先”原则。但其缺点也很明显：部分请求被限流或降级，用户体验下降，且并未解决缓存失效的根本原因，因此更适合作为应急保护与兜底方案。

答案-问题（1）

相较于 Memcached，Redis 在该电商高并发促销场景下的两个主要优势是：1. 支持丰富的数据结构，适合复杂业务场景；2. 支持持久化、主从复制与高可用机制，系统可靠性更高

布隆过滤器通过在缓存之前拦截不存在的商品 ID 请求，避免无效请求访问数据库，从而有效解决缓存穿透问题，间接防止击穿现象的发生。

答案-问题（2）

等特点。一旦出现库存不一致、订单丢失或超卖问题，将直接影响交易结果和用户体验，甚至造成经济损失。因此，系统在缓存与数据库协同写入策略的选择上，优先保证数据一致性和可靠性。

Write-Through（写穿）策略在写操作发生时，先将数据同时写入缓存和后端数据库，只有当数据库写入成功后，写操作才算完成。这种方式可以确保缓存与数据库中的数据始终保持一致，避免因缓存与数据库状态不一致而引发库存错误或订单异常问题。因此，在对数据一致性要求极高的电商促销场景下，选择 Write-Through 更符合业务需求。

Write-Through 与 Write-Behind 在一致性、写入性能、数据可靠性和系统复杂度方面的对比如下：

一致性：Write-Through 强一致，Write-Behind 最终一致

写入性能：Write-Behind 更高

数据可靠性：Write-Through 更可靠

系统复杂度：Write-Behind 更复杂

答案-问题（3）

在高并发或异常场景下，系统通常通过限流、熔断和降级来保障整体稳定性。限流通过限制单位时间内的请求数量，防止瞬时流量过大导致系统资源耗尽；熔断在检测到下游服务故障率过高或响应过慢时，暂时中断调用，避免故障进一步扩散；降级则在系统压力过大或部分功能不可用时，主动关闭或简化非核心功能，优先保障核心业务的连续可用。

（2025年11月系统分析师真题-第3题）

某市气象部门计划建设一套天气预测与灾害预警系统，用于对暴雨、高温、大风等极端天气进行预

测和预警。系统需要整合多种数据源，包括：

- 第三方天气服务平台提供的天气 API 数据；
- 分布在城市各区域的温度、湿度、风速等环境传感器数据；
- 历史气象数据存储在后台数据库中。

系统面向不同用户群体，包括普通公众、气象分析人员和应急管理人员，要求支持 Web 端和移动端访问。在系统架构设计和前端开发过程中，项目组讨论了前后端分离架构以及在前端中采用 MVP 或 MVVM 架构模式的问题。

问题（1）

简要说明什么是前后端分离架构，并说明其主要优点。

思路解析

前后端分离架构是指前端只负责页面展示和用户交互，后端只负责业务逻辑处理、数据存储及对外提供 API 接口，二者通过 HTTP/HTTPS + JSON 等方式进行通信。

在本天气预测预警系统中，后端可以统一对接第三方天气 API、传感器数据和数据库，并对外提供标准化的天气数据接口；前端则根据不同用户角色（公众、气象人员）展示不同的界面和功能。这种模式使得前端不依赖后端的具体实现细节，降低了系统耦合度。

前后端分离的一个重要优点是支持多终端统一接入。同一套后端服务可以同时为 Web 端、移动端甚至大屏展示系统提供数据，符合天气系统多场景应用的需求。此外，前后端分离便于团队分工，前端工程师和后端工程师可以并行开发，提高整体开发效率。

从维护和扩展角度看，当天气数据源或预测算法发生变化时，只需调整后端逻辑，不会影响前端展示；而前端样式和交互优化也不会影响后端服务，体现了良好的可维护性和可扩展性。

问题（2）

请把 A~H 给出的说明，填写到下表对应位置。

- A. Presenter 手动更新 View
- B. View 通过数据绑定自动响应 ViewModel 的变化
- C. Model 直接通知 View 刷新
- D. MVVM 支持双向数据绑定
- E. View 实现接口供 Presenter 调用
- F. ViewModel 直接操作 DOM
- H. 数据只能从 View 流向 ViewModel

架构模式	数据更新方式	View与逻辑层如何交互
MVP	(1)	(2)
MVVM	(3)	(4)

思路解析

在 MVP 架构中，View 并不能自动感知数据变化，系统不具备数据绑定能力。当 Model 中的数据发生变化时，并不会直接触发界面刷新，而是需要通过逻辑控制来完成更新流程。通常由 Presenter 协调 Model 与 View 的关系，在获取或更新 Model 数据后，显式地通知 View 进行界面刷新，因此体现为“数据更新依赖显式触发”，对应选项 C（Model 直接通知 View 刷新）。

在 MVP 架构中，为了降低 View 与 Presenter 之间的耦合度，View 通常会定义接口，由

Presenter 持有该接口的引用并进行调用。Presenter 不直接依赖具体的界面实现，而是通过接口方法控制 View 的显示逻辑，从而提升系统的可测试性和可维护性。因此，MVP 中 View 与逻辑层的交互方式体现为 View 实现接口供 Presenter 调用，对应选项 E。

MVVM 架构的核心特征是引入数据绑定机制，尤其是双向数据绑定。View 与 ViewModel 之间通过绑定关系建立联系，当 ViewModel 中的数据发生变化时，界面能够自动刷新；同时，用户在 View 上的操作也可以自动同步到 ViewModel 中，无需显式编写更新代码。这种机制显著降低了界面更新的复杂度，因此对应选项 D（MVVM 支持双向数据绑定）。

在 MVVM 架构中，View 不再通过接口或方法调用与逻辑层交互，而是以声明式的方式绑定 ViewModel 的属性和状态。View 只需关注数据的展示规则，当 ViewModel 状态发生变化时，界面会自动响应并更新，从而实现 View 与逻辑层的解耦。因此，MVVM 中 View 与逻辑层的交互方式表现为 View 通过数据绑定自动响应 ViewModel 的变化，对应选项 B。

答案-问题（1）

前后端分离是一种将用户界面（前端）与业务逻辑及数据处理（后端）通过接口解耦的系统架构模式。

其主要优点包括：1. 提高系统开发效率；2. 提升系统的可维护性和可扩展性；3. 支持多终端访问；4. 有利于系统性能优化和团队协作。

答案-问题（2）

（1）C （2）E （3）D （4）B

第5章——软件工程

（2021年5月系统分析师真题-第1题）

阅读以下关于系统分析与设计的叙述，在答题纸上回答问题1至问题3。

某软件企业拟开发一套基于移动互联网的在线运动器材销售系统，项目组决定采用 FAST 开发方法进行系统分析与设计，在完成了初步的调查研究之后进入了问题分析阶段，分析系统中存在的问题以及改进项，其分析的主要内容包括：

- （1）器材销售订单处理的时间应该减少20%；
- （2）移动端支持IOS和Android两类操作系统；
- （3）器材销售订单处理速度太慢导致很多用户取消订单；
- （4）后台服务器硬件配置比较低；
- （5）用户下单过程中应该减少用户输入的数据量；
- （6）订单处理过程中用户需要输入大量信息；
- （7）利用云计算服务可以降低50%的服务器处理时间；
- （8）公司能投入的技术维护人员数量有限；
- （9）大量的并发访问会导致App页面无法正常显示。

问题（1）

FAST 开发方法在系统分析中包括了初始研究、问题分析、需求分析和决策分析等四个阶段，请简要说明每个阶段的主要任务。

思路解析

FAST (Facilitated Application Specification Technique) 是一种面向需求和快速设计的分析方法，强调在短周期内完成系统的分析与建模。其四个阶段的核心任务如下：

初始研究阶段：重点在于对企业的现状进行快速摸底，包括业务目标、系统边界、资金预算、相关利益相关者等。此阶段往往通过访谈和文档调研完成，产出项目章程和初步需求文档。它的意义在于明确“项目做什么”和“是否值得做”。

问题分析阶段：核心是识别并定义当前系统存在的问题及改进机会。通过因果分析、鱼骨图等方法，分析性能瓶颈、用户体验问题、硬件制约等。例如案例中“订单处理过慢导致用户取消”就是一个直接的问题。此阶段确保项目目标紧密对应实际问题，避免系统建设脱离实际。

需求分析阶段：是最关键的阶段，旨在将“问题”转化为“系统应该怎么做”。需求不仅包含功能需求（订单管理、支付接口等），还包括非功能需求（如并发量、可用性、安全性）。通过用户访谈、原型验证等手段，编制需求规格说明书，为后续设计提供明确依据。

决策分析阶段：重点是从多个可选方案中选出最优方案，常用的标准包括 操作可行性、技术可行性、经济可行性、进度可行性。在本案例中，可能需要对“自建服务器”与“云服务”两种方案做比较，从技术、成本和进度多方面进行权衡。

问题（2）

在问题分析阶段，因果分析方法常用于分析系统中的问题和改进项，请结合题目中所描述各项内容，将题干编号（1）-（9）填入表1-1的（a）-（d）中。

表 1-1 问题、机会、目标和约束矩阵

项目：在线运动器材销售系统		项目经理：Xinyou S.	
创建者：Cindy S.		最后修改人：Cindy S.	
创建日期：2021 年 3 月 12 日		最后修改日期：2021 年 3 月 28 日	
因果分析		系统改进目标	
问题/机会	原因/结果	系统目标	系统约束条件
(a)	(b)	(c)	(d)

思路解析

因果分析的核心思想是将“问题本身”与“原因”“目标”“约束”加以区分，从而为后续改进提供清晰思路：

- (a) 问题/机会
 - (3) “器材销售订单处理速度太慢导致很多用户取消订单” —— 直接反映了系统的痛点，属于亟需解决的核心问题。
- (b) 原因/结果
 - (4) 服务器硬件配置不足，直接导致处理效率低。
 - (6) 订单输入信息过多，造成用户下单流程繁琐。
 - (9) 并发访问过大导致 App 页面无法显示，本质上是性能问题的结果。

- (c) 系统目标
 - (1) 订单处理时间减少 20% —— 典型的量化目标。
 - (5) 下单时减少用户输入 —— 提升用户体验的目标。
 - (7) 云计算降低 50% 处理时间 —— 技术层面的性能改进目标。
- (d) 系统约束
 - (2) 必须兼容 IOS 与 Android —— 技术开发上的限制。
 - (8) 技术维护人员有限 —— 后续运维条件上的限制。

问题（3）

在决策分析阶段，需要对候选方案所述内容按照操作可行性、技术可行性、经济可行性和进度可行性进行分类。请将下列（1）~（5）内容填入表1-2的（a）-（d）中。

- （1）新开发的器材销售系统能够满足用户所需的所有功能；
- （2）系统开发的成本大约需要40万元人民币；
- （3）需要对移动端App开发工程师进行技术培训；
- （4）系统开发周期需要6个月；
- （5）系统每年维护的费用大约5万元人民币。

表 1-2 候选方案指标分类

可行性准则	候选方案描述
操作可行性	(a)
技术可行性	(b)
经济可行性	(c)
进度可行性	(d)

思路解析

在决策分析中，四类可行性常常作为评估标准：

- 操作可行性：判断系统是否能够满足最终用户的业务需求。（1）“系统能够满足用户所有功能”属于操作层面。
- 技术可行性：判断开发团队是否具备实施该系统的技术能力。如果需要培训开发人员（3），说明现阶段技术能力不足，需要补充，因此归入技术可行性。
- 经济可行性：从成本与收益出发进行评估。（2）开发成本 40 万元、（5）每年维护费 5 万元，都是直接经济投入。
- 进度可行性：强调开发周期与项目时间表是否可行。（4）开发周期需 6 个月，属于进度维度。

答案-问题（1）

- 初始研究阶段：主要任务是对业务需求的初步了解和预估，包括对项目的可行性、范围和约束因素等的评估和确定。此阶段的输出是项目策划文档和初步业务需求文档。
- 问题分析阶段：主要任务是对现有业务问题的分析和定义，确定所需要解决的问题，以及分析问题原因和影响。此阶段的输出是问题定义文档和问题分析报告。

需求分析阶段：主要任务是对业务需求进行详细的分析和定义，梳理出功能、性能、安全等方面的需求，并将需求进行优先级和整合，以形成需求规格说明书。此阶段的输出是需求规格说明书和数据模型文档。

决策分析阶段：主要任务是对系统设计和开发的技术方案进行分析和评估，制定系统开发方案和工作计划，并对项目的进度和风险进行管理和控制。此阶段的输出是系统设计方案、工作计划和项目管理计划。

答案-问题（2）

- （a）（3）
- （b）（4）（6）（9）
- （c）（1）（5）（7）
- （d）（2）（8）

答案-问题（3）

- （a）（1）
- （b）（3）
- （c）（2）（5）
- （d）（4）

（2019年5月系统分析师真题-第2题）

阅读以下关于基于MDA（Model Driven Architecture）的软件开发过程的叙述，在答题纸上回答问题1至问题3。

某公司拟开发一套手机通讯录管理软件，实现对手机中联系人的组织与管理。公司系统分析师王工首先进行了需求分析，得到的系统需求列举如下：

用户可通过查询接口查找联系人，软件以列表的方式将查找到的联系人显示在屏幕上。显示信息包括姓名、照片和电话号码。用户点击手机的"后退"按钮则退出此软件。

点击联系人列表进入联系人详细信息界面，包括姓名、照片、电话号码、电子邮箱、地址和公司等信息。为每个电话号码提供发送短信和拨打电话两个按键实现对应的操作。用户点击手机的"后退"按钮则回到联系人列表界面。

在联系人详细信息界面点击电话号码对应的发送短信按键则进入发送短信界面。界面包括发送对象信息显示、短信内容输入和发送按键三个功能。用户点击发送按键则发送短信并返回联系人详细信息界面；点击"后退"按钮则回到联系人详细信息界面。

在联系人详细信息界面内点击电话号码对应的拨打电话按键则进入手机的拨打电话界面。在通话结束或挂断电话后返回联系人详细信息界面。

在系统分析与设计阶段，公司经过内部讨论，一致认为该系统的需求定义明确，建议基于公司现有的软件开发框架，采用新的基于模型驱动架构的软件开发方法，将开发人员从大量的重复工作和技术细节中解放出来，使之将主要精力集中在具体的功能或者可用性的设计上。公司任命王工为项目技术负责人，负责项目的开发工作。

问题（1）

请用300字以内的文字，从可移植性、平台互操作性、文档和代码的一致性等方面说明基于MDA的软件开发方法的优点。

思路解析

MDA（Model Driven Architecture）作为一种以模型为核心的开发方法，具有明显的优势。通过标准化的建模和自动化生成代码的方式，MDA提高了系统的可移植性、跨平台的互操作性，并保证了代码和文档的一致性。这些优势使得开发过程更加高效、灵活和可维护。

问题（2）

王工经过分析，设计出了一个基于MDA的软件开发流程，如图2-1所示。请填写图2-1中（1）~（4）处的空白，完成开发流程。

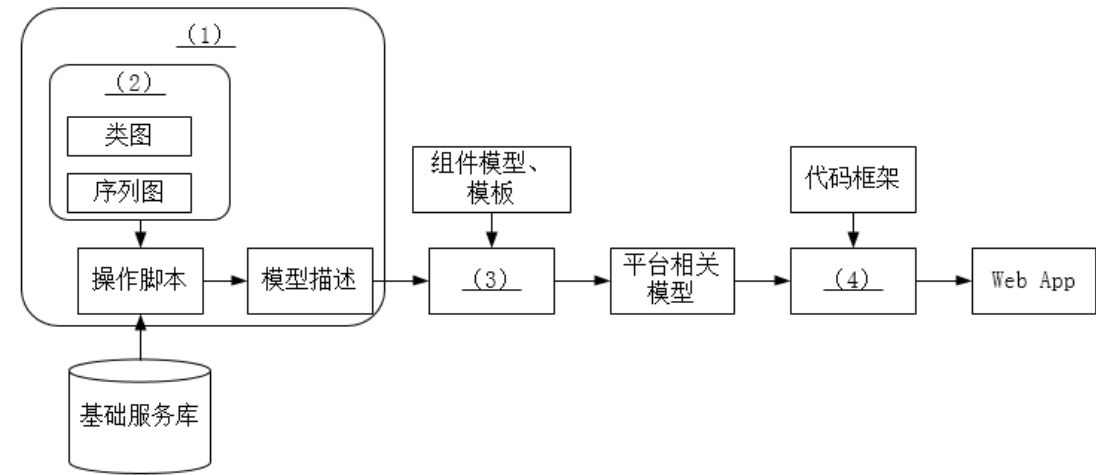


图2-1 基于MDA的软件开发流程

思路解析

在MDA的开发过程中，首先需要创建平台无关模型（PIM），它是描述系统功能和行为的抽象模型，不依赖于任何特定平台。

接下来，使用UML建模来进一步描述系统的结构、组件、行为等。这一建模步骤帮助开发团队理解系统的设计需求。

然后，进行模型变换（映射），即将PIM转换为平台相关模型（PSM），这个过程是将业务模型映射到具体平台的技术细节。

最后，通过模型生成源代码的步骤，将转换后的PSM自动生成实际的源代码，完成从设计到代码的转化。

通过这一流程，MDA能够确保从需求到实现的每个阶段都保持一致，且减少了手工编码的工作量。

问题（3）

王工经过需求分析，首先建立了该手机通信录管理软件的状态机模型，如图2-2所示。请对题干需求进行仔细分析，填写图2-2中的（1）~（5）处空白。

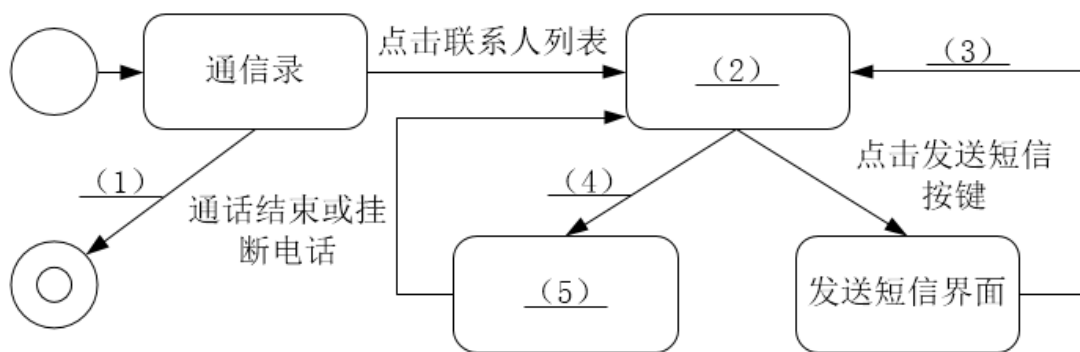


图2-2 手机通信录管理软件状态机模型

思路解析

状态机模型描述了系统在不同状态下如何响应事件。在本问题中，状态转换是基于用户的操作：

(1) 点击后退按钮：在多个界面（如联系人详细信息界面、发送短信界面等），用户点击“后退”按钮后，系统返回前一个状态或界面。

(2) 联系人详细信息界面：用户从联系人列表点击进入该界面，查看联系人的详细信息。

(3) 点击发送按键或点击后退按钮：在联系人详细信息界面，用户可以选择发送短信或返回前一个界面。

(4) 点击拨打电话按键：在联系人详细信息界面，用户点击拨打电话按钮，系统进入拨打电话界面。

(5) 拨打电话界面：用户在该界面进行电话操作，通话结束后系统返回联系人详细信息界面。

答案-问题（1）

可移植性：在MDA方法中，首先开发平台无关模型（PIM），然后通过模型转换将其转换为平台相关模型（PSM）。一个平台无关模型可以转换为多个不同平台的相关模型，简化了软件的移植过程。当需要将软件移植到另一个平台时，只需将PIM转换为目标平台的PSM，而不需要从头开始编写代码，极大提升了可移植性。

平台互操作性：由于MDA是基于模型驱动的开发方法，整个开发过程遵循标准化的建模规范，这使得不同平台之间的系统能够更加容易地互操作。通过标准化的模型，MDA可以帮助开发者构建平台之间互操作的系统，提高了跨平台集成和通信的可能性。

文档和代码的一致性：MDA通过自动化工具将模型直接转换为代码，这意味着生成的代码与模型保持一致性。由于代码直接来源于模型，开发过程中的文档、设计和实际代码之间不容易出现偏差和不一致，从而降低了开发和维护的复杂性和风险。

答案-问题（2）

(1) 平台无关模型（PIM）

(2) UML建模

(3) 模型变换（映射）

(4) 模型生成源代码

答案-问题（3）

(1) 点击后退按钮

- (2) 联系人详细信息界面
- (3) 点击发送按钮或点击后退按钮
- (4) 点击拨打电话按钮
- (5) 拨打电话界面

(2019年5月系统分析师真题-第5题)

阅读以下关于Web应用系统的叙述，在答题纸上回答问题1至问题3。

某公司因业务需要，拟在短时间内同时完成"小型图书与音像制品借阅系统"和"大学图书馆管理系统"两项基于B/S的Web应用系统研发工作。

小型图书与音像制品借阅系统向某所学校的所有学生提供图书与音像制品借阅服务。所有学生无需任何费用即可自动成为会员，每人每次最多可借阅5本图书和3个音像制品。图书需在1个月之内归还，音像制品需在1周之内归还。如未能如期归还，则取消其借阅其他图书和音像制品的权限，但无需罚款。学生可通过网络查询图书和音像制品的状态，但不支持预定。

大学图书馆管理系统向某所大学的师生提供图书借阅服务。有多个图书存储地点，即多个分馆。搜索功能应能查询所有的分馆的信息，但所有的分馆都处于同一个校园内，不支持馆际借阅。本科生和研究生一次可借阅16本书，每本书需在1个月内归还。教师一次可借阅任意数量的书，每本书需在2个月内归还，且支持教师预定图书。如预定图书处于被借出状态，系统自动向借阅者发送邮件提醒。借阅期限到达前3天，向借阅者发送邮件提醒。超出借阅期限1周，借阅者需缴纳罚款2元/天。存在过期未还或罚款待缴纳的借阅者无法再借阅其他图书。图书馆仅向教师和研究生提供杂志借阅服务。

基于上述需求，该公司召开项目研发讨论会。会议上，李工建议开发借阅系统产品线，基于产品线完成这两个Web应用系统的研发工作。张工同意李工观点，并提出采用 MVP (Model View Presenter) 代替MVC的设计模式研发该产品线。

问题（1）

软件产品线是提升软件复用的重要手段，请用 300 字以内的文字分别 简要描述什么是软件复用和软件产品线。

思路解析

软件复用的目标是通过利用现有资源来减少开发和维护成本，而软件产品线则是通过共享核心资产、定制和配置不同的产品，达到规模化开发的目标。软件产品线方法能够提高开发效率并降低错误发生的风险，是提升软件复用的有效策略。

问题（2）

产品约束是软件产品线核心资产开发的重要输入，请从以下已给出的（a）～（k）各项内容，分别选出产品的相似点和不同点填入表5-1中（1）～（8）处的空白，完成该软件产品线的产品约束分析。

- （a）项目当前状态；（b）项目操作；（c）预定策略；（d）会员分类；（e）借阅项目数量；（f）项目的类型和属性；（g）检索功能；（h）与支付相关的用户信息；（i）图书 编号；（j）教师；（k）学生

相似点	用户通用数据，如姓名、电话、住址等	
	(1)	
	项目通用数据：项目存储位置、(2)	
	(3)：预定、借阅、归还	
	(4)	
	(5)	
	借阅策略	允许那些顾客可以借阅
		(6)
	逾期惩罚	在什么情况下借阅权限可被修改
		当借阅的项目没有如期归还时，该采取何种措施
不同点	提醒策略	顾客发出预定请求时，如果项目处于被借阅状态，如何处理
		是否需要向顾客发出一个通知，以提醒其归还该项目
	(7)	哪些顾客可以预定
		预定请求何时过期
	收费方式	成为顾客是否需要付费
		发出预定请求时是否需要付费
		当延期归还时，是否需要付费
	(8)	
	馆际互借	

思路解析

在构建软件产品线时，产品约束分析是非常关键的一步，它帮助识别不同产品之间的相似性和差异性。

相似点： h（与支付相关的用户信息）：两者都需要处理与用户支付相关的功能。 a（项目当前状态）：这两个系统都需要管理项目（如图书或音像制品）的当前状态。 b（项目操作）：借阅、归还、预定等基本操作在两个系统中都是共有的。 g（检索功能）：两个系统都需要提供检索功能来帮助用户查找相关物品。 f（项目的类型和属性）：都需要管理不同类型的借阅项目及其属性。 e（借阅项目数量）：系统需要控制借阅的数量，虽然具体限制不同，但两者都涉及此功能。

h（与支付相关的用户信息）：两者都需要处理与用户支付相关的功能。

a（项目当前状态）：这两个系统都需要管理项目（如图书或音像制品）的当前状态。

b（项目操作）：借阅、归还、预定等基本操作在两个系统中都是共有的。

g（检索功能）：两个系统都需要提供检索功能来帮助用户查找相关物品。

f（项目的类型和属性）：都需要管理不同类型的借阅项目及其属性。

e（借阅项目数量）：系统需要控制借阅的数量，虽然具体限制不同，但两者都涉及此功能。

不同点： c（预定策略）：小型系统不支持预定，而大学图书馆管理系统支持教师预定图书。

d（会员分类）：两者在会员分类上有所不同，小型系统的会员只有学生，大学图书馆管理系统的会员包含本科生、研究生和教师。

c（预定策略）：小型系统不支持预定，而大学图书馆管理系统支持教师预定图书。

d（会员分类）：两者在会员分类上有所不同，小型系统的会员只有学生，大学图书馆管理系统的会员包含本科生、研究生和教师。

问题（3）

MVP 模式是由 MVC 模式派生出的一种设计模式。请说明张工建议借阅系统产品线采用 MVP 模式代替 MVC 模式的原因。

思路解析

MVC（模型-视图-控制器）和MVP（模型-视图-演示者）都是分层架构模式，但MVP比MVC提供了更高的解耦度。在MVC中，View和Model之间的耦合度较高，而在MVP中，所有的交互通过Presenter进行，使得View和Model之间没有直接依赖。这种设计改善了代码的可测试性和可扩展性，尤其对于复杂的UI和交互操作，MVP能够更加清晰地分离不同的职责，提高系统的灵活性和可维护性。因此，张工建议采用MVP模式来更好地应对系统的复杂性和未来的扩展需求。

答案-问题（1）

软件复用是指将已有软件的各种知识、组件、模块等用于新软件的开发，以缩减开发和维护的成本。它通过重用已经开发的代码、设计和架构来加速软件开发过程，提高开发效率并减少重复工作。软件复用能够通过共享模块、工具、接口等减少开发周期，降低风险，并提高代码质量。

软件产品线是指一组共享共同核心资产（如架构、模块、组件等）的软件产品集合，这些产品针对特定领域需求或市场需求进行定制，能够满足不同客户的需求。软件产品线方法通过统一管理和优化核心组件，提升软件开发的可扩展性和维护性，降低开发成本，提高软件的质量和生产效率。

答案-问题（2）

(1) h (2) a (3) b (4) g (5) f (6) e (7) c (8) d

答案-问题（3）

MVP与MVC相比，最显著的差异在于层次之间的耦合度。MVP将模型（Model）和视图（View）完全分离，所有的交互都通过演示者（Presenter）传递。这种分离方式能够提高软件组件及架构的重用性，同时也便于修改和扩展。由于Presenter负责与Model交互并更新View，View的复杂度减少，且它与Model的依赖关系被彻底隔离，使得系统具有更好的可扩展性和可测试性。

（2014年5月系统分析师真题-第4题）

某企业经过多年的信息化建设，存在大量的应用软件系统，为了保证这些系统的运行与维护，专门组建应用系统维护部门。该部门的主要工作是保证系统的正常运行、处理问题以及扩展这些系统的功能，以满足企业业务功能的变化与扩展。

目前该部门存在人员流失、变更频繁，文档丢失或长期失于维护，维护成本愈来愈高等问题，具体表现为：

问题（1）：随着时间和人员的变动，程序被多人修改，往往导致程序难以理解，注释混乱，流程复杂；

问题（2）：随着不断修改程序和增加新的功能，模块之间的耦合关系日益复杂，维护成本不断增加。

这些问题导致新来的维护人员需要直接面对大量流程、结构复杂的源程序，维护困难，往往一次改动需要设计大量的软件模块。

为解决应用系统维护部门面对的问题，企业信息部门组织了专门的专家讨论会。各位专家一致认为，逆向工程与重构工程是目前预防性维护采用的主要技术，应该采用逆向工程的技术方法，重构相关应用系统文档，同时采用软件重构来降低软件代码的复杂性，最终降低维护成本。

问题（1）

软件的逆向工程是分析已有程序，寻求比源代码更高级的抽象表现形式。与之相关的概念包括软件

重构、设计恢复、重构工程等。请说明设计恢复中常见的恢复信息的4种级别。

思路解析

设计恢复是逆向工程的核心活动，其目标是从已有源代码中提取出比代码层次更高的抽象模型，使维护人员能更快理解系统。

实现级：主要恢复算法流程、局部过程逻辑、变量作用域等，帮助维护人员理解程序的基本实现思路。

结构级：上升到程序整体层次，包含模块结构、调用关系、数据结构设计，相当于重建软件体系结构蓝图。

功能级：以用户需求为导向，恢复对象模型、控制流图、数据流图，揭示系统在业务逻辑上的处理过程，是联系代码与需求的桥梁。

领域级：最高层次的恢复，利用 UML 状态图、部署图、业务流程模型等，建立系统与业务领域的对应关系，使维护能从业务抽象层面切入。

问题（2）

重构是对软件内部结构的一种调整，目的是不改变软件功能的前提下，提高其可理解性，降低其修改成本。请说明软件重构的三个类别，并简要说明常见的重构方法。针对题干中的问题（1）和问题（2），宜采用何种重构方法？

思路解析

软件重构强调在不改变功能的前提下优化结构。

代码重构：针对函数级、语句级，如“提取方法”“引入解释变量”等，可显著提高代码清晰度，降低维护人员的理解门槛。

设计重构：面向对象层面，如“提取接口”“用多态替代条件判断”，通过降低类与类之间的耦合，提升模块的可重用性和可扩展性。

架构重构：在系统整体架构层面进行，如单体改造为分层架构、微服务，解决长期演进中的复杂性积累问题。

对应题干：

问题（1）由于多人修改导致流程复杂，应当使用 提取方法（Extract Method）与 提取类，将大而杂的代码块拆分为职责单一的小方法和小类，使逻辑更直观，注释可替代为自解释方法名，提升可理解性。

问题（2）模块间耦合过高，则适合使用 提取接口或 委托替代继承等手段，减少直接依赖关系。通过在模块之间引入接口层，可缓解因功能增加导致的依赖网状化，进而降低维护成本。

问题（3）

软件重构做出的修改可能导致程序运行变慢，但也更容易进行软件的性能优化和调整，请分析原因。

思路解析

重构的首要目标是可维护性与可理解性，而非运行效率。为了实现结构优化，往往会增加额外的函数调用、抽象接口或判断分支，在短期内带来轻微的性能下降。例如，将大方法拆解为多个小方法会增加调用开销；将条件判断替换为多态可能在实例化或调度时引入额外开销；数据库结构调整

（如增加中间表、冗余字段）可能导致查询效率下降。

但长期来看，性能优化必须依赖清晰的架构与可控的复杂性。重构后的系统具有更高的模块化与内聚性，逻辑边界清楚，开发人员能够更快找到性能瓶颈（如热点模块、耗时 SQL、内存泄露点），并在不影响全局的前提下进行局部优化。结构清晰还利于引入缓存、并发处理、负载均衡等性能改进手段。因此，尽管短期性能可能下降，但可维护性与可优化性的大幅提升为后续性能调优创造了条件，整体利大于弊。

答案-问题（1）

- （1）实现级：过程的设计模型。
- （2）结构级：程序和数据结构信息。
- （3）功能级：对象模型、数据和控制流模型。
- （4）领域级：UML状态图和部署图。

答案-问题（2）

软件重构的三个类别：代码重构、设计重构、架构重构。常见的重构方法：

- （1）提取方法（Extract method）
- （2）用委托来代替继承（Replace Inheritance with Delegation）
- （3）用子类代替型别码（Replace Type with Subclasses）
- （4）用多态来代替条件判断（Replace conditional with polymorphism）
- （5）模板函数
- （6）提取类
- （7）提取接口

问题（1）可采用提取方法的重构方法解决，问题（2）可采用提取接口的方法解决。

答案-问题（3）

为了使软件更容易理解，同时又需要考虑到各种兼容性，在重构时，可能需要在代码中增加冗余的判断、冗余的代码或结构；也可能需要修改已有的数据库结构和索引等，导致程序运行变慢。但从长远来看，由于重构以后的软件结构更加清晰，代码复杂性更低，更易于理解，在性能调优时更容易分析瓶颈之所在，然后加以解决，因此，软件重构也更容易进行软件的性能优化和调优。

（2013年5月系统分析师真题-第5题）

阅读以下关于软件维护的叙述，在答题纸上回答问题1至问题4。

某企业两年前自主研发的消防集中控制软件系统在市场上取得了较好的业绩，目前已成功应用到国内外众多企业用户的消防管理控制系统中。该软件系统通过不同型号消防控制器连接各种消防器件，实现集中式消防告警信息显示与控制。日前该软件系统的系统维护主管组织召开会议，对近期收集到的该软件系统的如下维护申请进行了讨论：

- （1）将系统目前的专用报表功能改成通用报表生成功能，以适应未来用户可能提出的报表格式变化需求；
- （2）在界面语言中增加德文支持，以满足新增的两个德国企业用户的使用要求；
- （3）修正系统在与控制器通信时由于硬件网络故障导致的与控制器之间的连接失效，而控制器后

续无法重新连接的问题；

- (4) 完成因某已支持型号消防控制器通信协议改变而导致的相应软件修改；
- (5) 增加与某新型号消防控制器的互连通信功能，以支持在后续推广过程中与该型号消防控制器进行互连；
- (6) 修正用户报告的由于系统从Win7平台迁移到最新发布的Win8平台而导致的部分显示功能失效的问题；
- (7) 修正用户报告的系统自动切换消防平面图形时部分器件位置显示错误的问题；
- (8) 改进系统历史消防告警记录的检索算法，以提高系统在处理大量消防报警记录时的效率；
- (9) 将系统目前针对某一特定数据库管理系统的数据库访问方式改成基于数据源的通用数据库访问方式，以适应用户可能提出的使用其他数据库管理系统的需求。

问题（1）

软件维护的内容通常分为正确性维护、适应性维护、完善性维护以及预防性维护四类，请用300字以内文字说明这4类软件维护内容的特点与区别。

思路解析

软件维护类型的划分主要依据产生维护的原因。若用户报告错误/缺陷，则归为正确性维护，其特点是必须立即处理，以保证现有功能正确执行。若因外部环境变化（如新操作系统、新数据库、新协议）而导致系统不兼容，则属于适应性维护，特点是保证系统继续在新环境中可用。若用户提出新功能、性能优化或界面改进，即超出现有功能范围的扩展，属于完善性维护，特点是以提升系统价值和满足业务发展为目标。最后，预防性维护是一种“前瞻性”维护，不一定解决当下问题，而是重构、补充文档、改进结构等，使系统在未来易于维护。四类维护在触发原因、迫切性与目标方面均有所区别，考试时可按“改错—适应—增强—防患”思路记忆。

问题（2）

请根据不同类型软件维护的特点，将上述（1）—（9）的维护申请填入表5-1相应的类别中。

维护类型	维护申请
(1) 正确性维护	
(2) 适应性维护	
(3) 完善性维护	
(4) 预防性维护	

思路解析

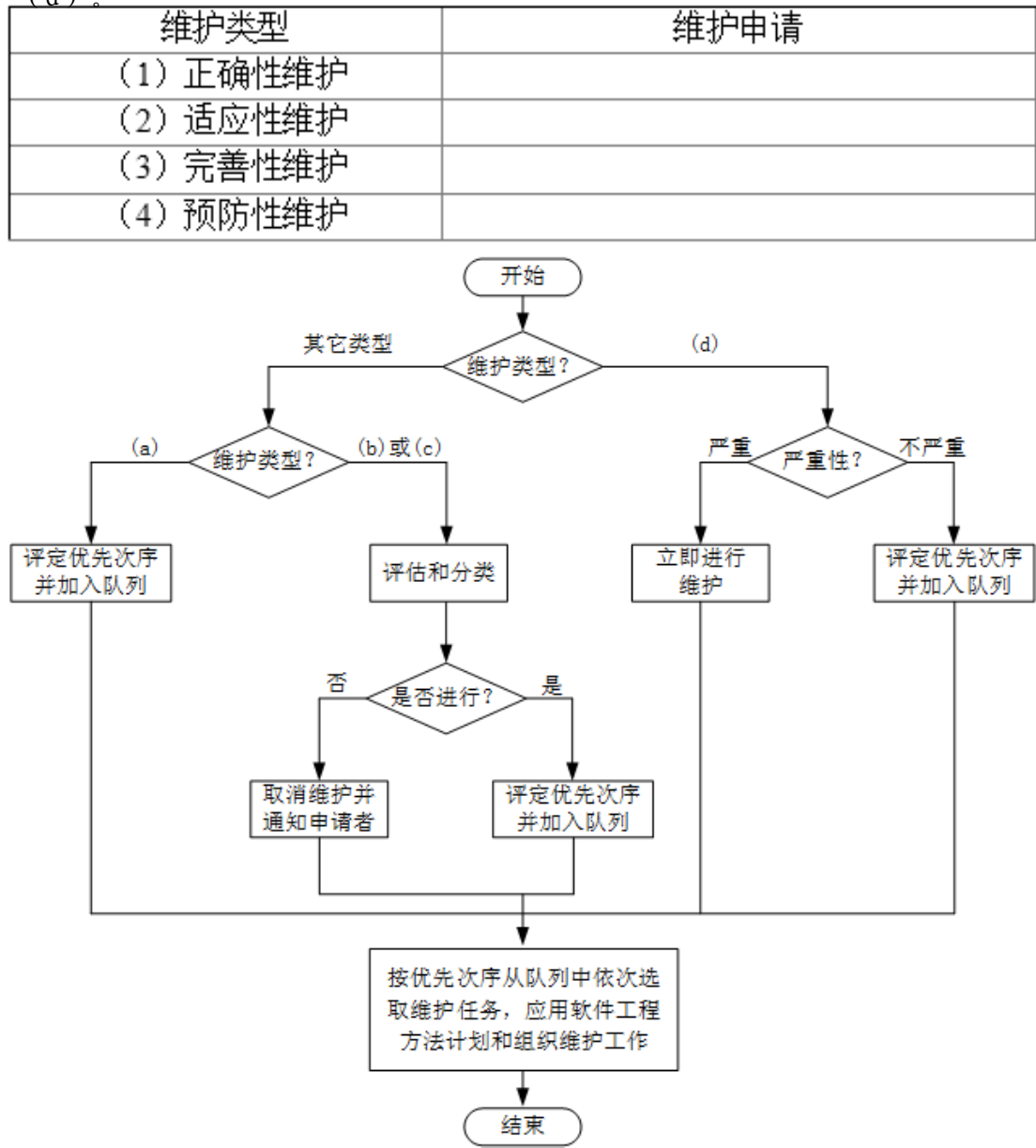
- 分类依据在于每项需求的根本原因。
- (3) (7) 属于正确性维护，分别是修复通信故障导致的无法重连问题，以及平面图器件位置显示错误，均为“错误修正”。
 - (4) (6) 属于适应性维护，前者是消防控制器协议更新导致的不兼容，后者是操作系统从 Win7 升级到 Win8 带来的显示问题，均因外部环境变化引发。
 - (2) (5) (8) 属于完善性维护，分别是新增德语界面支持、增加对新型号控制器的支持、优化检索算法，均为功能扩展或性能改进。
 - (1) (9) 属于预防性维护，前者将报表改为通用生成，后者将数据库访问改为基于数据源的通用

访问，均为未来需求与可扩展性提前重构。

通过对比可以发现，正确性和适应性维护通常是“必须立即实施”，而完善性和预防性维护则偏向“提升与前瞻”。

问题（3）

对于软件维护申请的处理，首先需要确定维护工作的类型，不同类型的维护工作，其处理流程往往不同，图5-1是区分不同维护类型的典型处理流程，请将表5-1中代表维护类型的序号（1）~（4）填入图5-1的（a）~（d）。



思路解析

图中流程的关键在于区分维护工作的必然性与迫切性。

正确性维护（1）：只要发现系统错误，必须修复，因此对应（d）立即进入修正流程。

适应性维护（2）：当环境变化时，维护必然发生，但紧迫性略低于错误修复，因此对应（a）直接进入排队执行。

完善性维护（3）和 预防性维护（4）：这两类维护通常需要先经过可行性与必要性评估，是否实施需取决于成本/收益比，因此对应（b）（c）流程框。

问题（4）

在讨论第（4）项维护工作的方案时，项目组发现与相应的消防控制器通信的子模块重用了企业之前淘汰的某款软件中实现相应功能的源代码，而现有技术人员没有参加过该代码的开发工作，并且重用的这部分代码开发不规范，其数据和代码结构设计都很差，相应的文档极不完整，维护记录也很简略。针对上述情况，项目组一致认为在原有代码基础上进行修改维护难度很大，因此决定采用软件工程的方法对该子模块进行重新设计、编码和测试，请从可行性和该模块后续可维护性两个方面简要分析采取这种方案的原因。

思路解析

在软件维护实践中，“重构/重写 vs 修修补补”是常见决策点。若现有模块存在三大缺陷：文档缺失、结构混乱、人员脱节，则修改代价往往大于重写。重新开发有两个优势：

可行性优势：相比阅读难懂的旧代码，直接重新设计能更快满足控制器新协议的适配需求，避免引入新的隐蔽错误。

可维护性优势：新模块由当前团队主导，必然有完整设计文档与测试用例，符合现代软件工程方法（如模块化设计、接口抽象、单元测试）。这样未来若控制器再次升级，只需在已有规范代码基础上调整即可，维护成本显著下降。

答案-问题（1）

（1）改正性维护。为了识别和纠正软件错误、改正软件性能上的缺陷、排除实施中的误使用，应当进行的诊断和改正错误的过程就称为改正性维护。

（2）适应性维护。在使用过程中，外部环境（新的硬、软件配置）、数据环境（数据库、数据格式、数据输入/输出方式、数据存储介质）可能发生变化。为使软件适应这种变化，而去修改软件的过程就称为适应性维护。

（3）完善性维护。在软件的使用过程中，用户往往会对软件提出新的功能与性能要求。为了满足这些要求，需要修改或再开发软件，以扩充软件功能、增强软件性能、改进加工效率、提高软件的可维护性。这种情况下进行的维护活动称为完善性维护。

（4）预防性维护。这是指预先提高软件的可维护性、可靠性等，为以后进一步改进软件打下良好基础。通常，预防性维护可定义为“把今天的方法学用于昨天的系统以满足明天的需要”。也就是说，采用先进的软件工程方法对需要维护的软件或软件中的某一部分（重新）进行设计、编码和测试。

答案-问题（2）

- （1）正确性维护 （3）（7）
- （2）适应性维护 （4）（6）
- （3）完善性维护 （2）（5）（8）
- （4）预防性维护 （1）（9）

答案-问题（3）

- （a）： （2）
- （b）（c）： （3）（4）
- （d）： （1）

答案-问题（4）

目前的模块无文档，且已属于淘汰模块，重新开发比修改维护成本更低、效果更好，同时有利于后期的维护（一方面是开发人员对新开发的模块有很高的熟悉程度，另一方面新开发会有完善的文档）。

第6章——系统规划

(2019年5月系统分析师真题-第1题)

阅读以下关于软件系统分析的叙述，在答题纸上回答问题1至问题3。

某软件企业为电信公司开发一套网上营业厅系统，以提升服务的质量和效率。项目组经过分析，列出了项目开发过程中的主要任务、持续时间和所依赖的前置任务，如表1-1所示。在此基础上，绘制了项目PERT图。

表 1-1 网上营业厅系统 PERT 图

任务名称	持续时间（周）	前置任务	松弛时间
A.问题分析	2	-	-
B.数据建模	3	A	-
C.业务过程建模	6	B	(a)
D.数据库设计	2	B	(b)
E.接口设计	3	B、C、D	(c)
F.程序设计	4	B、D	(d)
G.单元测试	7	D、E、F	(e)
H.集成测试	2	G	-
I.安装和维护	2	H	-

问题（1）

PERT 图采用网络图来描述一个项目的任务网络，不仅可以表达子任务的计划安排，还可以在任务计划执行过程中估计任务完成的情况。针对表1-2中关于PERT图中关键路径的描述（1）（5），判断对PERT图的特点描述是否正确，并说明原因。

编号	PERT 图特点
(1)	关键路径是 PERT 图中工期最长的路径
(2)	一个 PERT 图仅包含唯一的一条关键路径
(3)	关键路径在项目执行过程中不会变化
(4)	PERT 图中关键路径越多说明项目越复杂
(5)	关键路径上的任务不能延迟

思路解析

关键路径（Critical Path）是指从项目起点到终点 持续时间总和最长 的那条路径。由于其上各活动的 总时差（Total Float）为 0，一旦任一活动延迟都会直接导致 项目总工期顺延，因此（1）“最长路径上的工作不能延误所以称为关键路径”是正确的。与此相对，非关键活动 具有一定机动时间，可在不影响总工期的前提下适度调度。关于（2），“一个 PERT 图只能有一条关键路径”不正确，因为在 存在多条并行且总工期相同的路径 时，可能会出现 多条关键路径；此时项目受控难度更高。对于（3），关键路径并非一成不变，项目执行过程中只要某条非关键路径上的延误累计超过其总时差，就会与原关键路径等长或更长，关键路径便会发生转移，故（3）不正确。关于（4），关键路径越多，意味着并行活动多、可机动空间小，进度统筹与资源冲突风险更大，判断正确。最后（5）指出关键路径上的任务不能延迟，否则会 直接推长总工期，这一点符合关键路径定

义，因此正确。

此外需要澄清一个常见误解：PERT/网络图天然能够表达并行关系。当某两个或多个活动的前置条件已满足时，即可并行从相同事件节点向不同方向发出多条活动箭线，这恰恰是网络计划技术的优势所在。通过 正推/逆推 计算最早/最晚时间，管理者可在执行中 动态监控关键路径变化，并结合赶工（Crashing）与资源平衡（Resource Leveling）等手段进行纠偏，这也是 PERT 在大型信息化项目中被广泛采用的根本原因。

问题（2）

根据表1-1所示任务及其各项任务之间的依赖关系，计算对应PERT图中的关键路径及项目所需工期。

思路解析

求解关键路径的标准步骤为：

- （1）正推（Forward Pass）：自项目起点沿箭线方向依次计算各事件的 最早发生时间（TE-early），对每个事件取其所有进入路径 最晚的最早完成时刻；其上活动的 最早开始/完成（ES/EF）随之确定。
- （2）逆推（Backward Pass）：自项目终点逆向回推各事件的 最晚发生时间（TE-late），对每个事件取其所有离开路径 最早的最晚开始约束；其上活动的 最晚开始/完成（LS/LF）随之确定。
- （3）路径时长对比：统计从起点到终点各可达路径的持续时间总和，取 最长者为关键路径。依据题干给定的数据与图（略），计算结果显示路径 ABCEGHI 的总持续时间为 25 周，且该路径上各活动满足 ES=LS、EF=LF（总时差为 0），因而判定其为 唯一关键路径。

问题（3）

根据表1-1所示任务及其各项任务之间的依赖关系，分别计算对应PERT图中任务C~G的松弛时间（SlackTime），将答案填入（a）（e）中的空白处

表 1-1 网上营业厅系统PERT图

任务名称	持续时间（周）	前置任务	松弛时间
A.问题分析	2	-	-
B.数据建模	3	A	-
C.业务过程建模	6	B	(a)
D.数据库设计	2	B	(b)
E.接口设计	3	B、C、D	(c)
F.程序设计	4	B、D	(d)
G.单元测试	7	D、E、F	(e)
H.集成测试	2	G	-
I.安装和维护	2	H	-

思路解析

松弛时间（Total Float，TF）的计算公式为 $TF = LS - ES = LF - EF$ 。

在具体计算中，先按 正推 求出各活动的 最早开始/完成（ES/EF）：起点事件最早时间记为 0，从前置活动的 EF 最大值 推得后继活动的 ES， $EF=ES+工期$ ；再按 逆推 求出各活动的 最晚开始/完成（LS/LF）：终点事件的最晚时间取为项目总工期（本题为 25 周），从后继活动的 LS 最小值 回推前驱活动的 LF， $LS=LF-工期$ 。将得到的（ES, EF, LS, LF）代入上式即可得各活动的 TF。根据子问题 2 的结果，路径 ABCEGHI 为关键路径，关键活动的总时差必为 0，因此 C、E、G 的 $TF = 0$ ；而偏离关键路径的 D、F 位于另一条次长路径上（与关键路径存在 3 周的时差余量），经正

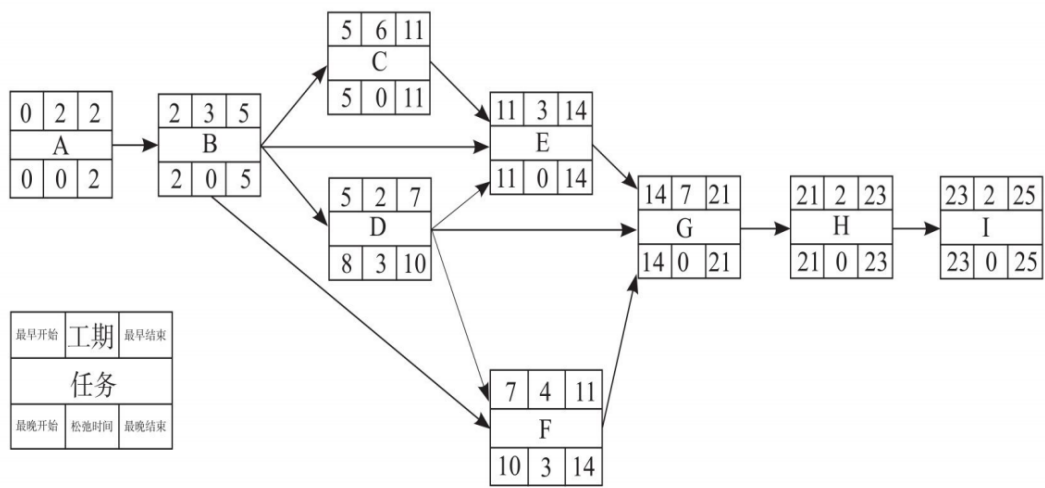
推/逆推可得 D 的 TF=3、F 的 TF=3。

答案-问题（1）

- （1）正确。项目中最长路径上的工作，是不能延误的，所以这条路径是一个项目成败关键，故命名关键路径。
- （2）不正确。一个PERT图可以有多个关键路径。
- （3）不正确。关键路径在项目执行过程中可能发生变化，只要非关键活动延误超过其总时差，关键路径就会发生变化。
- （4）正确。关键路径越多，说明项目活动并行度高，而且大量任务不容延误，所以项目管理的复杂度高，风险也大。
- （5）正确。关键路径上的任务不能延迟，一旦延迟会导致项目工期延长。

答案-问题（2）

根据 PERT 图，可以看出路径 ABCEGHI 为最长路径（图中最早最晚开始时间和结束时间一致），是本题的唯一关键路径，总工期为 25 周。



答案-问题（3）

- (a) 0 (b) 3 (c) 0 (d) 3 (e) 0

(2016年5月系统分析师真题-第1题)

阅读以下关于软件系统可行性分析的叙述，在答题纸上回答问题1至问题3。

某软件开发企业受对外贸易公司委托开发一套跨境电子商务系统，项目组从多个方面对该电子商务系统进行了可行性分析，在项目组给出的可行性分析报告中，对项目的成本、收益情况进行了说明：建设投资总额为300万元，建设期为1年，运营期为4年，该方案现金流量表如表1-1所示。

阶段	0	1	2	3	4	合计
折现系数	1	0.91	0.83	0.75	0.68	
-开发成本	300					300
-运营成本		40	50	60	70	220
总成本	300	340	390	450	520	
折现值	300	336.4	377.9	422.9	470.5	
-运营收益		160	180	200	220	760
总收益		160	340	540	760	
折现值		145.6	295	445	594.6	

问题（1）

（12 分）软件系统可行性分析包括哪几个方面？用200以内文字说明其含义。

思路解析

软件系统可行性分析的内容和基本概念是进行软件系统可行性分析的前提。软件系统的可行性分析包括经济可行性、技术可行性、法律可行性和用户使用可行性，分别从项目建设的经效益、技术方案、制度因素和用户使用等四个方面对系统建设的必要性和可能性进行评估。

问题（2）

（7 分）成本和收益是经济可行性评价的核心要素，成本一般分为开发成本和运营成本， 收益包括有形收益和无形收益，请对照下列7项内容，将其序号分别填入成本和收益 对应的类别。

- a) 系统分析师工资；
- b) 采购数据库服务器；
- c) 系统管理员工资；
- d) 客户满意度增加；
- e) 销售额同比提高；
- f) 软件许可证费用；
- g) 应用服务器数量减少。

类别	选项
开发成本	(1)
运营成本	(2)
有形收益	(3)
无形收益	(4)

思路解析

成本和收益的识别是经济可行性中成本效益分析的基础，成本主要描述系统的投入，收益主要描述系统建成后的产出。软件生存期中的成本按照系统投入的阶段可分为开发阶段的成本和建成后的运营成本两类，软件生存期中的收益按照量化放肆不同可分为有形收益和无形收益。系统分析师工资和采购数据库服务器属于系统开发阶段的投入，所以属于开发成本；系统管理员工资和软件许可证费用属于系统建成后运营阶段的投入，所以属于运营成本；销售额同比提高和应用服务器数量减少都意味着系统收益的增加，可以直接量化，所以属于有形收益；而客户满意度增加同样能够增加系统收益，但是无法直接量化，所以属于无形收益。

问题（3）

（6分）根据表1-1所示现金流量表，分别给出该解决方案的静态投资回收期、动态投资回收期和投资收益率的算术表达式或数值（结果保留2位小数）。

思路解析

投资回收期就是投资收回的期限，也是用系统方案所产生的净现金收入回收初始全部投资所需要的时间。对于投资者来讲，投资回收期越短越好，从而减少投资的风险。计算投资回收期时，根据是否考虑资金的货币时间价值，可分为静态投资回收期和动态投资回收期。投资收益率又成为投资利润率，是指投资收益占投资成本的比率，用来反映投资的收益能力。各指标计算方法：

（1）静态投资回收期=累计净现金流量开始出现正值的年份数-1+|上年累计净现金流量|/当年净现金流量；

（2）动态投资回收期=累计折现值开始出现正值的年份数-1+|上年累计折现值|/当年折现值；

（3）投资收益率=投资收益/投资成本×100%。

答案-问题（1）

（1）经济可行性。主要评估项目的建设成本、运行成本和项目建成后可能的经济收益。

（2）技术可行性。研究的对象是信息系统需要实现的功能和性能，以及技术能力约束。

（3）法律可行性。具有比较广泛的内容，它需要从政策、法律、道德、制度等社会因素来论证信息系统建设的现实性。

（4）用户使用可行性。从信息系统用户的角度来评估系统的可行性，包括企业的行政管理和工作制度、使用人员的素质和培训要求等。

答案-问题（2）

（1）开发成本：（a）系统分析师工资、（b）采购数据库服务器

（2）运营成本：（c）系统管理员工资、（f）软件许可证费用

（3）有形收益：（e）销售额同比提高、（g）应用服务器数量减少

（4）无形收益：（d）客户满意度增加

答案-问题（3）

静态投资回收期= 累计净现金流量开始出现正值的年份数-1+上年累计净现金流量/当年净现金流量
= (3-1) + (390-340) / (200-60) = 2.36年

动态投资回收期 = 累计折现值开始出现正值的年份数-1+上年累计折现值/当年折现值 = (3-1) + (377.9-295) / ((200-60) × 0.75) = 2.79年

投资收益率 = 投资收益/投资成本 × 100% = 594.6/470.5 = 126.38%

(2015年5月系统分析师真题-第1题)

阅读以下关于软件项目进度管理的叙述，在答题纸上回答问题1至问题3。

某软件公司启动了一个中等规模的软件开发项目，其功能需求由5个用例描述。项目采用增量开发模型，每一次迭代完成1个用例；共产生5个连续的软件版本，每个版本都比上一个版本实现的功能多

。 每轮迭代都包含实现、测试、修正与集成4个活动，且前一个活动完成之后才能开始下一个活动。不同迭代之间的活动可以并行。例如，1个已经实现的用例在测试时，软件开发人员可以开始下一个用例的实现。实现和修正活动不能并行。

每个活动所需的工作量估算如下：

- (1) 实现1个用例所需的时间为10人天；
- (2) 测试1个用例所需的时间为2人天；
- (3) 修正1个用例所需的时间为1人天（平均估算）；
- (4) 集成1个用例所需的时间为0.5人天。

项目开发过程中能够使用的资源包括：5名开发人员共同完成实现和修正工作、2名测试人员完成测试工作和1名集成人员完成集成工作。

该项目的Gannt图（部分）如图1-1所示。

(单位：天)	1	2	3	4	5	6	7	
实现 1								
测试 1								
修正 1								
集成 1								
实现 2								
测试 2								
修正 2								
集成 2								

问题（1）

根据题目描述中给出的工作量计算方法，计算1个用例的实现，测试、修正、集成4个活动分别所需的日历时间（单位：天）

思路解析

实现时间：实现1个用例所需的时间为10人天，共有5人负责开发，需要2天实现。
测试时间：测试1个用例所需的时间为2人天，共有2人负责测试，需要1天测试。
修正时间：本题存在不严谨的地方，标准不统一。根据题干修正1个用例所需的时间为1人天（平均估算），5人负责开发和修正，计算得出0.2天，但事实上，项目由于工作面受限，有不可压缩的工期。所以还需要结合甘特图一并分析。根据甘特图可知，修正花费的日历时间为0.5天。
集成时间：集成1个用例所需的时间为0.5人天，共有1人负责集成，需要0.5天集成。

问题（2）

- (1) 根据图1-1给出的项目Gannt图，估算出项目开发时间。
- (2) 计算测试人员和集成人员在该项目中的平均工作时间（占项目总开发时间的百分比）。

思路解析

(1) 总工期 14 天的由来：项目采用流水线节拍推进。每用例在其生命周期中分别占用开发（2 天实现 + 0.5 天修正）、测试（1 天）、集成（0.5 天）资源。由于不同用例可以在不同岗位并行（例如用例1测试的同时，用例2实现），因此整体呈现管线化叠加。根据题干所附甘特图的节拍

：第 1 个用例启动后第 2 天末进入测试，第 3 天末进入修正，第 3.5 天末进入集成；随后新用例以稳定节拍进入流水。沿此节拍累计到第 5 个用例完成集成，总历时为 14 天，这是图示节拍叠加下的最短可行工期。

（2）平均工作时间占比：占比=（该角色在整个历时内处于“忙碌”的时间）÷（项目总工期）。根据甘特图可知，

测试：5 个用例 × 1 天/用例 = 5 个岗位日的有效忙碌量；考虑穿插空隙与节拍，使两名测试在 14 天窗口内的平均忙碌占比约为 35.7%（题干标准答案）。直观理解：测试位点分散在每个用例的实现之后，存在等待与资源互不饱和，从而形成约三分之一的平均占用。

集成：5 个用例 × 0.5 天/用例 = 2.5 天岗位日；同理在 14 天总历时内平均占比 17.9%。

问题（3）

在项目实施过程中，需不断将实际进度与计划进度进行比较分析，进行项目进度计划的修正与调整，以保证项目工期。用300字以内文字，从活动和资源的角度，说明项目进度计划调整所涉及的内容。

思路解析

进度调整的核心在于同时管理工序网络与资源约束。在活动层面，首先识别关键路径/关键链：任何关键活动的拖延都会直接推长总工期，故常用赶工（增加成本换时间）与快速跟进（顺序活动改为部分并行）两类策略；此外，通过细化里程碑与交付物分段，减少大颗粒活动的等待与返工。对非关键活动，充分利用其时差吸收局部扰动，避免对全局产生连锁反应；一旦时差被耗尽，需要重算网络参数，警惕其转为关键路径。需求变更或范围澄清引起的工作包增减，必须回到 WBS/网络计划/成本与资源基线统一重估，保持三重基线一致性与可追踪性。

在资源层面，关注瓶颈岗位（如题中测试、集成）与关键设备。采用资源平衡（调整非关键活动起讫以削峰填谷）、限额资源计划（以资源为约束重排时序）、错峰并行（在不违反前后置约束下提升并发）等方法，提高系统整体吞吐。

答案-问题（1）

实现：2天，测试：1天，修正：0.5天，集成：0.5天。

答案-问题（2）

（1）14天

（2）测试人员：35.7%，集成人员：17.9%

单位:天	1	2	3	4	5	6	7	8	9	10	11	12	13	14
实现 1														
测试 1														
修正 1														
集成 1														
实现 2														
测试 2														
修正 2														
集成 2														
实现 3														
测试 3														
修正 3														
集成 3														
实现 4														
测试 4														
修正 4														
集成 4														
实现 5														
测试 5														
修正 5														
集成 5														

答案-问题（3）

- （1）关键活动的调整，若关键活动延误，一般调整的方法主要是缩短后续关键活动的持续时间。
- （2）非关键活动的调整，当非关键路径上某些工作的持续时间延长，但不超过其时差范围时，则不会影响项目工期，进度计划不必调整。
- （3）增减工作项目，由于编制计划时考虑不周，或因某些原因需要增加或取消某些工作，则需重新调整网络计划，计算网络参数。
- （4）资源调整，若资源供应发生异常时，应进行资源调整，资源供应发生异常是指因供应满足不了需要。资源调整的方法是进行资源优化，提高资源利用率。

(2014年5月系统分析师真题-第2题)

某电子商务公司为了扩大业务规模，提高企业的信息化程度与工作效率，决定由公司的IT部门开发一套ERP系统。在系统建设之初，该公司召开了项目论证会，对于项目的可行性进行了分析。

在论证会上，公司主管领导王总首先介绍了公司目前的运营情况，分析了竞争对手的优势和劣势，认为该项目的成功实施将大大提高公司的竞争力，因此对该项目大力支持，并要求项目必须按期、高质量地完成。公司的中高层管理人员一致认为现有业务大量依靠人工记录，效率低下，急切希望能够依靠该系统的部署提高工作效率，也表示将大力支持项目的开发工作。

公司IT部门的负责人李总对项目开发中的技术问题进行了分析，他认为IT部门多年来一直从事J2EE平台上的应用开发，开发经验丰富。由于近两年来基于B/S结构的Web应用逐渐流行，加之该项目是公司的内部项目，系统功能清楚明确，因此建议可以首先基于B/S架构，采用Web技术进行项目开发，让员工边学边练，即使项目进展不顺利，也可以迅速切换到J2EE平台之上。李总还透露，IT部门最近通过内部关系得到了某竞争对手公司的部分核心业务组件，只要稍加修改就可以立即加入到新系统中，这样就会大大加快项目的进度。

公司业务部门的代表小张表示，ERP系统上线后需要大量的数据采集工作，现有业务人员没有足够的IT技能，短期内难以保证数据录入质量。另外，新系统可能会导致一些现有业务流程的改变，业务

人员一开始将难以适应新的业务处理方式，可能会抵制使用该系统。

最后，公司销售部门和财务部门的代表在会上对ERP系统的建设成本和收益进行了详细的分析和比较，给出了相应的结论。

问题（1）

在信息系统建设项目中，通常从经济可行性、技术可行性、法律可行性和用户使用可行性四个方面来进行可行性分析。请用300字以内的文字简要说明项目可行性分析这四个方面各自的主要内容。

思路解析

经济重在ROI 与现金流，明确预算边界、收益清单与回收期；技术强调“需求-能力匹配”，要校验现有团队与基础设施能否达成性能与安全目标，并识别接口改造、数据治理与迁移复杂度；法律则把关知识产权、数据合规、网络安全、税务票据、行业监管等，明确风险与整改成本；用户使用侧重组织变革，评估数据采集负担、流程重塑冲击、培训成本与激励约束。

问题（2）

请根据题干描述的项目开发总体思路，用600字以内的文字对技术可行性、法律可行性和用户使用可行性进行初步分析，并给出分析结论。

思路解析

技术方面的关键并非“会不会写后端”，而是 端到端工程化能力：前端框架选型、构建链、权限与审计、单点登录、性能与安全、浏览器兼容等都需要体系化沉淀。“边学边做”常见风险包括：需求节奏快于能力沉淀、返工与技术债累积、质量与安全失控；一旦“切回”将引发技术栈迁移成本（比如页面重构、接口协议调整、测试回归）。建议以熟悉栈为主线，设立低风险模块试点引入Web 新技术，形成最佳实践/脚手架/代码规范，再滚动复制。

法律层面，“引入竞争对手核心组件”极易触发知识产权与商业秘密纠纷；即使来源“内部关系”，也难以证明授权合法充分，极有可能造成高额赔偿与声誉损失，属于红线。

用户使用方面，ERP 成功的标志是数据质量与流程落地。应实施 主数据治理（编码、口径、责任人）、分阶段上线（先财务/库存等边界清晰模块）、双轨运行与数据校验、岗位画像+分层培训、KPI 与激励联动。在高层背书下，通过流程梳理与制度配套提升执行可行性。

问题（3）

假设本项目有甲、乙、丙三个解决方案，投资总额均为1000万元，建设期均为2年，运营期均为4年，运营期各年末净现金流入量总和为2000万，年利率为10%，三种方案的现金流量表如表2-1所示。

方案 \ 阶段		建设期			运营期				
		0	1	合计	2	3	4	5	合计
折现系数		1	0.91		0.83	0.75	0.68	0.62	
甲	年初投资额	700	300	1000					
	年末净现金流量				300	400	500	800	2000
乙	年初投资额	600	400	1000					
	年末净现金流量				200	400	600	800	2000
丙	年初投资额	800	200	1000					
	年末净现金流量				400	500	500	600	2000

请根据表2-1中的数据，计算甲、乙、丙三种方案的净现值，给出具体计算过程，并说明那种方案最优。

思路解析

净现值（NPV）衡量项目在考虑资金时间价值后的绝对收益；净现值率（NPVR）=NPV/折现后投资额，衡量单位折现投资所产生的收益效率。

题设已给定贴现系数（收益年各期系数 0.83/0.75/0.68/0.62，建设期第 1 年投资折现系数 0.91），按表 2-1 将各期现金流分别折现后求和，再扣除折现投资额即可得到 NPV，再求 NPVR。在三方案总投资相同的背景下，选择 NPV 最大或 NPVR 最大均可；当两者排序一致时结论更稳健。本题计算显示：方案丙在**绝对收益（NPV=437）与收益效率（NPVR=0.445）**上均领先，因此优选丙。

答案-问题（1）

经济可行性也称为投资收益分析或成本效益分析，主要评估项目的建设成本、运行成本和项目建成后可能的经济收益。

技术可行性也称为技术风险分析，研究的对象是信息系统需要实现的功能和性能，以及技术能力约束。

法律可行性也称为社会可行性，具有比较广泛的内容，它需要从政策、法律、道德、制度等社会因素来论证信息系统建设的现实性。

用户使用可行性也称为执行可行性，是从信息系统用户的角度来评估系统的可行性，包括企业的行政管理和工作制度、使用人员的素质和培训要求等，可以细分为管理可行性和运行可行性。

答案-问题（2）

（1）技术可行性：公司IT部门具有丰富的J2EE平台开发经验，但缺乏Web技术应用经验，如果项目基于B/S架构，则技术上不可行。

（2）法律可行性：直接修改竞争对手的核心业务组件，属于侵权对方知识产权行为，法律上不可行。

（3）用户使用可行性：用户可能会抵制使用新系统，在用户使用可行性上存在问题。但由于公司中高层领导大力支持该项目，这个问题可以通过相关工作来得到解决。

答案-问题（3）

方案甲：

$$(1) NPV = 300 \times 0.83 + 400 \times 0.75 + 500 \times 0.68 + 800 \times 0.62 - (700 + 300 \times 0.91) = 412$$

$$(2) NPVR = 412 / (700 + 300 \times 0.91) = 0.423$$

方案乙：

$$(1) NPV = 200 \times 0.83 + 400 \times 0.75 + 600 \times 0.68 + 800 \times 0.62 - (600 + 400 \times 0.91) = 406$$

$$(2) NPVR = 406 / (600 + 400 \times 0.91) = 0.421$$

方案丙：

$$(1) NPV = 400 \times 0.83 + 500 \times 0.75 + 500 \times 0.68 + 600 \times 0.62 - (800 + 200 \times 0.91) = 437$$

$$(2) NPVR = 437 / (800 + 200 \times 0.91) = 0.445$$

因为方案丙的净现值和净现值率均最大，因此，方案丙最优。

(2012年5月系统分析师真题-第1题)

阅读以下关于软件项目管理的叙述，在答题纸上回答问题1至问题4。

某软件企业为电信公司开发一套网上营业厅系统，以提升服务的质量和效率。项目组经过分析，列出了项目开发过程中的主要任务、持续时间和所依赖的前置任务，如表1-1所示。在此基础上，可分别绘制出管理该系统开发过程的PERT图和Gantt图。

任务名称	持续时间(周)	前置任务	松弛时间
A. 需求获取	2	-	-
B. 业务流程建模	2	A	-
C. 数据建模	3	B	(a)
D. 过程设计	7	B	(b)
E. 数据库设计	5	C	(c)
F. 界面设计	1	C,D	(d)
G. 报表设计	4	D,E	(e)
H. 程序设计	5	F,G	(f)
I. 测试和文档	7	G	-
J. 安装	3	H,I	-

问题（1）

请用300字以内的文字分别解释说明PERT图和Gantt图的具体含义，并说明两种方法所描述开发过程的差异。

思路解析

在项目进度控制中，PERT 图以网络模型呈现活动与依赖，便于识别关键路径与非关键活动的总时差（Slack）。通过计算每个活动的最早开始/完成（ES/EF）与最迟开始/完成（LS/LF），可快速定位进度风险点与工期压缩的边界。关键路径上的松弛为 0，任何延迟都会牵动总工期；非关键活动的正松弛为资源平衡、穿插施工提供了回旋空间。PERT 适用于方案推演、瓶颈识别与“假设情景（What-if）”分析，是制定里程碑和压缩工期策略（如快速跟进、赶工）的重要依据。

Gantt 图则用可读性极强的条形与时间轴联动展示起止时间、持续时间、进度百分比与里程碑，能直观看到多活动并行与重叠情况和资源负荷节奏。在执行期，Gantt 便于周/日度例会对偏差进行“计划—实际”对比，并配合基线与关键路径高亮显示。简言之：PERT 强逻辑、强分析，回答“为什么会这么排、哪里会卡”；Gantt 强可视、强跟踪，回答“何时干到哪、资源怎么排”。两者常结合使用：先用 PERT 定逻辑与关键链，再用 Gantt 编排节拍与资源。

问题（2）

根据表1-1所示活动及其各项活动之间的依赖关系，分别计算对应PERT图中活动C~H的松弛时间（Slack Time）填入（a）~（f）中。

思路解析

松弛时间（Slack/Float）= LSES（或 LFEF），反映活动在不延误总工期前提下可容许的最大小幅延迟。计算步骤为：①按依赖自前向后进行正推，求出各节点/活动的最早开始与最早完成；②自后向前进行反推，在以目标完工时点为约束下得到最迟开始与最迟完成；③两者相减即得松弛。依据本题给定网络，关键路径为 ABCEGIJ，其上活动松弛恒为 0，因此与该链条相连且在链上的 C、E、G、I（对应（a）（c）（e）项中的相关活动）松弛为 0。对不在关键链上的活动，如 F、H，通过比较其最早/最迟时点差值，可分别得到 6 周与 2 周的总时差；而与 E 分支并行的某活动（题中对应（b））仅有1 周可滑动空间。

要点：松弛并非“可浪费时间”，而是用于资源调配、避峰填谷的调度余量。计划优化中，常利用非关键活动松弛对人力进行平衡，避免关键链资源遭到抢占。同时还需关注自由时差（不影响紧后

活动最早开始的那部分时差)与总时差的区别,确保对下游不产生连锁影响。本题给出的(a)~(f)数值体现了上述计算逻辑与关键/非关键活动的差异。

问题(3)

根据表1-1所示活动及其各项活动之间的依赖关系,计算对应PERT图中的关键路径及所需工期。

思路解析

关键路径是一个相关任务序列,该序列上任务工期之和最大。图1-1所示PERT图中,包括了6条路径:

- (1) 路径序列: ABCEGHJ 工期之和: 24周
- (2) 路径序列: ABCEGIJ 工期之和: 26周
- (3) 路径序列: ABCFHJ 工期之和: 16周
- (4) 路径序列: ABDFHJ 工期之和: 20周
- (5) 路径序列: ABDGHJ 工期之和: 23周
- (6) 路径序列: ABDGIJ 工期之和: 25周

各路径之和以 26 周为最大,因此ABCEGIJ为关键路径,总工期为 26 周。其中,关键链上的活动松弛为 0。

问题(4)

如果将关键路径对应的工期作为期望工期;通过资源调配和任务调度,可以使得最优工期比期望工期少2周时间;根据项目组开发经验,解决项目开发过程中所有可能会遇到的问题最多需要8周时间,即最差工期比期望工期多8周时间。请计算项目最可能的开发工期。

思路解析

题设给定:以关键路径工期 26 周作为期望(ED);通过资源调配可将最优(OD)压至 24 周(少 2 周);考虑问题处理与风险预留,最差(PD)可达 34 周(多 8 周)。依据三点估算法的加权平均模型:

$$D = \frac{OD + 4 \times ED + PD}{6}$$

其中 1-4-1 权重体现了期望值的主导与两端不确定性的对称影响。代入数值得 27 周,即最可能工期。

三点估算源于 PERT 概率模型,假设工期服从近似 β 分布,以期望值作为中心,同时吸收乐观/悲观两端的信息。

答案-问题(1)

PERT(项目评估与评审技术)图是一种图形化的网络模型,描述一个项目中任务和任务之间的关系,每个节点表示一个任务,通常包括任务编号、名称、开始和结束时间、持续时间和松弛时间。

Gantt图是一种简单的水平条形图,它以一个日历为基准描述项目任务,横坐标表示时间,纵坐标表示任务,图中的水平线段表示对一个任务的进度安排,线段的起点和终点对应在横坐标上的时间分别表示该任务的开始时间和结束时间,线段的长度表示完成该任务所需的时间。

PERT图主要描述不同任务之间的依赖关系;Gantt图主要描述不同任务之间的重叠关系。

答案-问题（2）

(a) 0 (b) 1 (c) 0 (d) 6 (e) 0 (f) 2

答案-问题（3）

关键路径：A — B — C — E — G — I — J，所需工期 26 周。

答案-问题（4）

最可能的开发工期 = (最优工期 + 期望工期 × 4 + 最差工期) / 6 = (24 + 26 × 4 + 34) / 6 = 27 周

第7章——嵌入式系统设计

(2023年5月系统分析师真题-第2题)

随着嵌入式计算资源快速提升，容器技术（Docker）发挥重要作用，某公司对原有平台升级，公司将平台升级任务交给了张工，张工经过分析、调研，提出在园嵌入式操作系统平台上采用容器技术的升级方案，但该方案引发了争议。

问题（1）

争论焦点是采用容器技术还是虚拟机（VM）技术。李工指出由于容器技术共享主机内核能像虚拟机一样完全隔离，系统存在安全问题；如果采用虚拟机技术除满足需求外，还保证了系统的安全和稳定，会上领导根据系统升级的初衷选择了张工的升级方案，请用300字以内的文字说明容器技术和虚拟技术的含义，并简要论述公司领导采纳容器技术的原因。

思路解析

容器技术和虚拟机技术的差异在于隔离的层级和资源的利用方式。虚拟机提供的是一种更完整的隔离，可以独立运行多个操作系统实例，这意味着它们各自拥有完整的内核和操作系统。这种方式对于需要强隔离的应用场景是理想的，例如某些需要高安全性的应用或操作。

容器技术则更侧重于高效的资源利用，通过共享宿主操作系统的内核，容器之间实现了更轻量级的隔离。虽然容器的隔离级别相对较低，但在许多现代应用中，容器提供的性能和资源利用优势，使得它成为部署和管理大规模分布式系统的首选。通过采用现代安全技术，容器的安全性也能得到有效保障。

问题（2）

下表给出了虚拟技术和容器技术的性能对比表，请根据下面的（a）~（h）的8个性能指标；判断这些指标属于哪类对比项，补充完善3-1的（1）~（8）的空白处。

（a）分钟级、（b）包含 GuestOS，G量级以上、（c）跨操作系统平台迁移、（d）CPU与内存按核、按G分配（e）毫秒级、（f）Cgroups，进程级别、（g）VM伸缩，CPU/内存手动伸缩、（h）实例自动伸缩、CPU内存自动在线伸缩。

对比项	虚拟机技术	容器技术
镜像大小	①	仅包含运行的Bin/Lib,M量级
资源要求	②	CPU与内存按单核、低于G量级分配
启动时间	③	④
可持续性	跨物理机迁移	⑤
弹性伸缩	⑥	⑦
隔离策略	操作系统、系统级别	⑧

思路解析

在对比容器技术与虚拟机技术的性能时，可以从启动时间、资源分配、伸缩性和跨平台迁移等方面进行分析。

虚拟机技术（VM）通常启动时间较长，需要几分钟来启动，因为每个虚拟机都包含独立的操作系统，这使得虚拟机的资源占用和管理更为复杂，通常使用手动配置来调整资源分配，如CPU和内存。

容器技术则更加高效、轻量化，启动速度为毫秒级，并且它通过共享主机操作系统的内核来运行，因此容器的资源分配非常灵活，能够实现按需伸缩。容器还支持自动伸缩，使得在负载增加时可以自动增加容器实例，以满足业务需求。

这些特性使得容器技术在现代分布式系统中更加适用，尤其在需要高效管理大量实例的场景中，如微服务架构等。与虚拟机相比，容器的资源利用更加高效，启动和扩展更加灵活，且能够轻松支持自动化管理和跨平台迁移。

答案-问题（1）

容器技术是一种轻量级的隔离技术，多个容器共享主机内核，每个容器运行一个或一组应用，具备自己的文件系统、运行环境等，实现应用的快速部署和隔离。虚拟机技术通过虚拟化软件在物理主机上模拟出多个独立的虚拟计算机，每个虚拟机有自己的操作系统、硬件资源，实现完全隔离。容器技术相比虚拟机更加轻量化，资源占用少，启动速度快，能更高效利用硬件资源，满足系统升级中对资源利用优化的需求。且在当前技术发展下，安全问题可通过安全策略、安全工具等手段进行防护，在满足业务需求的同时，能以更低成本、更高效率完成部署，契合系统升级初衷。

答案-问题（2）

- ① b
- ② d
- ③ a
- ④ e
- ⑤ c
- ⑥ g
- ⑦ h
- ⑧ f

(2022年5月系统分析师真题-第3题)

阅读以下关于基于模型的系统工程方法的描述，在答题纸上回答问题1至问题3。

某公司长期从事宇航系统的研制任务。随着宇航系统能力的不断提升，软件的规模急剧增加，对应的缺陷密度呈现出几何增长，失效问题越来越严重。为了保证产品的质量，公司领导要求在重点项目中实施基于模型的系统工程（MBSE）。由于辅助动力控制系统关系着飞行器的飞行安全，有必要作为试点工程首先实施MBSE方法。王工作为该系统的主要负责人，承担起了此项工作。

问题（1）

王工以前一直负责用传统开发方法进行系统设计，深知基于文本的系统设计方式存在天然的局限性。王工接到指示后，研究了MBSE的相关理论基础和工程实现方法，提出了用MBSE方法设计辅助动力控制系统的实施方案。请用300字以内的文字说明什么是MBSE方法，并列举出三个基于文本设计方式的局限性。

思路解析

MBSE方法通过模型化设计过程，避免了传统方法中存在的文档歧义和沟通不畅问题。MBSE通过使用统一的模型语言和工具，确保设计阶段的每个细节都能清晰表达和验证，从而提高系统的整体质量和效率。而传统的基于文本的设计方式在大规模、复杂系统中往往难以避免这些局限性。

问题（2）

MBSE的核心技术是模型方法论，王工对目前主流建模方法Harmony-SE（harmony for System Engineering）方法、RUP-SE（Rational Unified Process for System）方法和Vitech MBSE 方法进行了分析、对比，鉴于该系统具备安全攸关系统的特殊性，强调验证的有效性，王工认为Harmony-SE 方法比较适合宇航环境开发。图3-1给出了Harmony-SE 方法的产品生命周期结构。请根据以下三种有关建模方式的描述，判定哪种描述是 Harmony-SE 方法的正确说明？并简要说明王工在本项目中选择 Harmony-SE 方法的理由。

- 描述1：一种面向对象的设计方法，可以将设计工作按照角色、工件和任务类型进行分解，生成新的分解结构，实现并行设计与迭代开发。
- 描述2：系统模型贯穿系统研发的整个生命周期，能够较好处理复杂与整体问题。
- 描述3：一种并行系统工程活动，由四个基本系统工程活动组而成，这四种活动通过公共系统设计库相关联。

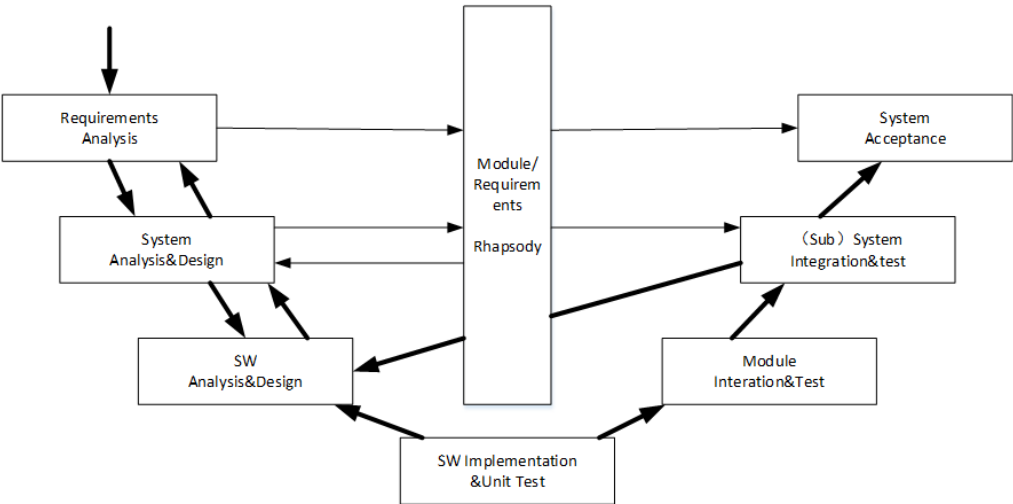


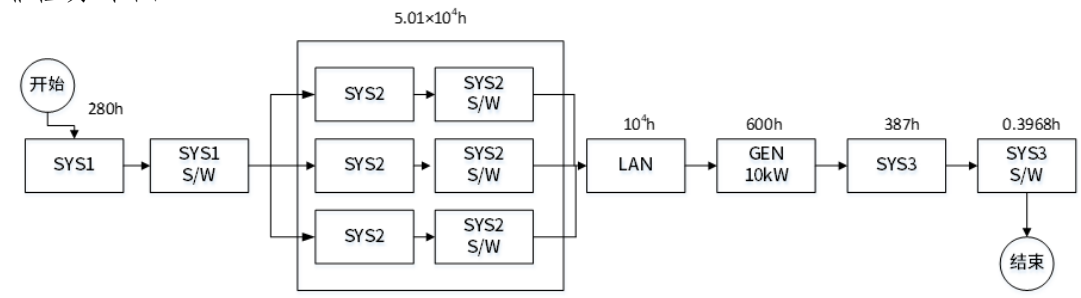
图 3-1 Harmony-SE 建模方法的全产品生命周期

思路解析

Harmony-SE方法专注于通过系统生命周期中的模型来处理复杂系统问题，特别强调模型的统一性和跨阶段的验证。与其他方法相比，Harmony-SE在系统工程领域中为复杂系统提供了更完整的建模和验证支持，这使其成为适用于宇航系统等高安全性需求项目的最佳选择。

问题（3）

MBSE方法可以通过软硬件建模来预先评估系统的可靠性。王工带领项目成员针对其负责的辅助动力控制系统中的进排气门控制器组件开展了基于MBSE系统分析工作。图3-2给出了进排气门控制器的可靠性分布图。



王工指出：系统是由软硬件组成，以往考虑可靠性指标仅仅关注硬件失效问题，而不考虑软件失效对系统可靠性的影响。如果不考虑软件失效对系统可靠性的影响，本系统可靠性的MTTF（Mean Time To Failure，平均失效前时间）应该为125.9h
(参考公式： $\frac{1}{T_{TFS}} = \sum_{i=1}^n \frac{1}{T_{TF_i}} = \frac{1}{280} + \frac{1}{5.01 \times 10^4} + \frac{1}{10^4} + \frac{1}{600} + \frac{1}{387} = 0.007942(1/h)$)。

请对照图3-3给出的考虑软件失效情况下的系统可靠性分布图（这里，假设SYS1软件非常可靠，系统仅仅将圈中的软件可靠性加入系统可靠性的评估过程），说明此时的系统可靠性（MTTF）应大于还是小于不考虑软件失效的MTTF？并计算出此时的系统可靠性（MTTF）是多少？

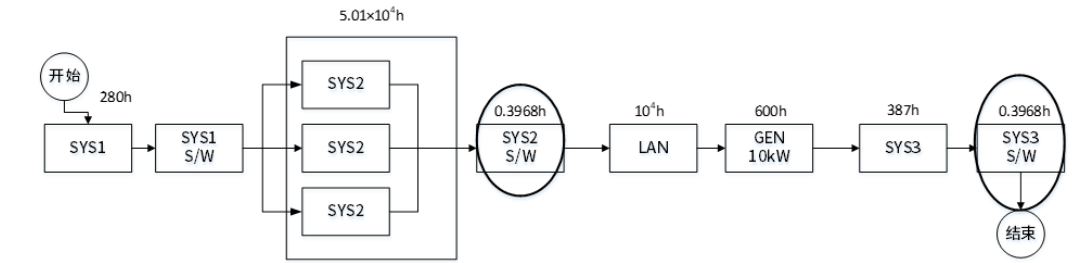


图 3-3 考虑软件失效情况下的系统可靠性分布图

思路解析

考虑软件失效情况下的系统可靠性：

系统的可靠性是硬件和软件失效概率的综合考虑结果。根据图3-3的可靠性分布图，系统的可靠性会受软件失效的影响。因为软件故障率较高，当考虑软件失效时，系统的故障率增加，因此，整体的MTTF会较不考虑软件失效时的MTTF低。通过计算，考虑软件失效后的系统MTTF约为0.1981小时，明显低于不考虑软件失效时的125.9小时。这表明，软件失效对系统的可靠性有显著影响，导致系统可靠性降低。

答案-问题（1）

基于模型的系统工程（MBSE）是一种形式化的方法，用于支持与复杂系统的开发相关的需求，设计，分析，验证和确认。与以文档为中心的工程，MBSE将模型放在系统设计的中心。MBSE是向以模型为中的系列法转变这长期趋势的部分，这些法被应于机械、电和软件等程领域，以期望取代原来系

统程师们所擅长的以档为中的法，并通过完全融系统程过程来影响未来系统程的实践。

基于文档的设计方法的局限性主要有：

(1) 在基于文档的方法中，许多文档是由不同的作者生成的，以从各种利益相关者的观点（例如系统行为，软件，硬件，安全，安全性或其他学科）中捕获系统的设计。不利于利益相关者之间的沟通，容易产生歧义。

(2) 开发复杂系统的能力有限，基于本的设计案法进前期仿真验证。

(3) 自然语言容易引入形容词等模糊描述，很难保证准确性。

答案-问题（2）

正确的描述是【描述2】。【描述1】描述1是对RUP-SE方法的描述。【描述3】描述3是对vitech MBSE方法的描述。根据王工对MBSE的研究，RUP-SE是一种面向对象的设计方法，vitech MBSE方法是一种并行系统，并不满足宇航系统所要求的高安全性和强调验证性的要求。而Harmony-SE建模方法是贯穿系统研发的整个生命周期，用于支持与复杂系统的开发相关的需求、设计、分析、验证和确认。符合宇航系统所描述的安全性和验证性的要求。

答案-问题（3）

(1) 小于不考虑软件失效的MTTF。

(2) 系统可靠性的计算如下：

$$1/280+1/(5.01*104) +1/104+1/600+1/387+(1/0.3968+1/0.3968)$$

$$=0.007942+(1/0.3968+1/0.3968)$$

$$=0.007942+2/0.3968$$

$$=0.007942+5.040322$$

$$=5.048264 (1/h)$$

所以系统的可靠性为：1/5.048264==0.1980879H。

(2021年5月系统分析师真题-第2题)

阅读以下关于嵌入式实时系统设计的相关技术的描述，回答问题1至问题3。

问题（1）

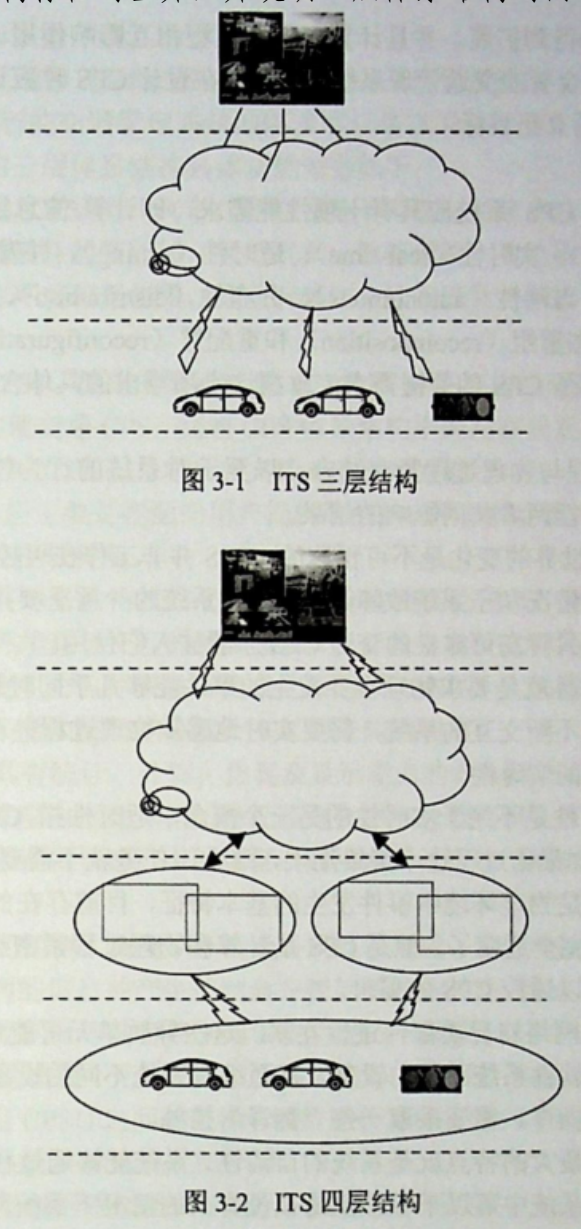
某公司长期从事嵌入式系统研制任务。近期公司承担了一项面向交通领域的智能交通系统（ITS），为了将信息、通信、传感、控制及计算机等技术有效地集成运用于整个地面交通管理，达到智能交通管理的要求，经公司讨论决定，采用信息物理融合系统（Cyber Physical System，CPS）技术来保证ITS达到实时、准确、高效的智能交通管理的目的。公司领导层将此任务交给王工承担论证工作。王工在广泛调研的基础上提交了总体实施方案供讨论，大家在高度肯定总体实施方案的基础上，提出了一些问题，并就这些问题提出了补充意见。

思路解析

这三个需求是CPS设计中的核心特性，它们共同确保了CPS在复杂、动态的环境中能够灵活、高效地运行。异质性保证了不同设备和系统的互操作性，分布性确保了系统的灵活扩展性，而动态重组和重配置则提供了应对突发情况和环境变化的能力。

问题（2）

讨论会上，与会者在CPS的体系结构构建上出现意见分歧，王工提出的智能交通管理系统拟采用传统的CPS三层体系结构（即物理层、网络层和应用层）。张工对此方案提出了异议，认为三层体系结构不能体现智能、融合等特点，由于智能交通管理系统信息量大，计算和控制是关键，应考虑将系统分解为四层体系结构，以充分体现智能、控制和信息融合的特点。经讨论，最后采纳了张工的建议。图3-1和图3-2分别是王工和张工提出的分层体系结构。请用150字以内的文字简要说明王工提出的三层体系结构中各层的功能划分，并详细分析图3-2，指出张工提出的四层体系结构与王工提出的结构存在的差异，并说明四层体系结构的两个显著优点。



思路解析

张工提出的四层架构更好地反映了智能交通管理系统的需求，特别是在高效数据处理和控制方面，使得系统更加智能和高效。而王工的三层架构在处理大规模信息和实时决策方面可能存在局限性。

问题（3）

王工在总体实施方案中强调，智能交通管理系统采用CPS体系结构后，由于本系统安全与否，直接涉

及车辆、驾驶员以及行人的生命安全，因此必须开展智能交通管理系统的安全性分析，寻找出潜在风险。通常风险可分为基本风险和特定风险，而特定风险涵盖了人为因素带来的风险和环境因素带来的风险。请识别出智能交通管理系统存在的5种特定风险。

思路解析

特定风险是指与智能交通管理系统的操作和运行密切相关的风险，涵盖了人为因素和环境因素。人为因素如驾驶员和行人行为不当可能导致直接的交通事故；环境因素如天气恶劣或设备故障则会影响系统的正常运行，造成系统失效。安全性分析是确保交通系统顺利运行的重要环节，有助于提前识别和减轻这些潜在风险。

答案-问题（1）

王工在总体实施方案中指出：CPS是在嵌入式系统、传感器技术和网络技术的基础上发展起来的，嵌入式系统使设备具有智能化能力，传感器网络使设备具有感知能力，这两者的结合就产生了"计算深度嵌入物理过程中"的效果，使得物理系统能力得到扩展，并且计算与物理过程相互影响作用，这正是CPS的系统目标，也是智能交通管理系统的最终目标。基于此需求，对比现有系统，报告分析出CPS系统应具有十项需求，其中：异质性（heterogeneity）、分布性（distribution）、动态重组（recomposition）和重配置（reconfiguration）是CPS的关键需求。请用300字以内的文字解释说明上述三个需求的具体含义。

答案-问题（2）

物理层。交通CPS中，汽车、道路设备不再仅仅是简单的机械设备，而将是嵌入大量传感器、计算、控制部件的智能体。智能汽车、智能道路、智能桥梁等交通智能设备分布在环境中，直接与物理环境相互作用，这些有感知、计算以及控制等功能的交通设备构成了交通CPS的物理层。

网络层。单一的、孤立的智能汽车（CPSU）并不能构成交通CPS，只有各种交通CPSU互联互通才能实现交通CPS，交通CPS体系结构中的网络层正是将大量异构CPSU连接起来，实现交通CPSU的互联互通，并支持CPSU的互操作。

应用层。应用层主要是指面向用户提供服务的应用软件，例如智能汽车的车载软件、交通管理部门的集中监控软件等。

张工的四层体系结构与王工的三层体系结构相比，其显著特点是张工的感知层与王工的物理层功能一致，而差别在于张工在感知层之上增加了计算/控制层，使得功能划分更清晰，层次结构更加明确，可确保信息的计算、控制和融合处理的有效性。

张工提出的系统结构具有以下优势：

（1）增加计算/控制层中的计算能力可以有效获取来自感知层或者其他层的数据，可增加综合统计及分析能力，也可对数据经分析、运算后传到其他层，实现交通管理的统计、计算、仿真及显示能力，同时也可降低感知层的计算负荷。

（2）增加计算/控制层的控制能力可以接收来自计算层的数据或来自应用层的用户控制信息，加工处理后生成控制命令，经网络层传送到感知层和计算层，实现对感知层CPSU的控制。增加控制能力可以将比如导航仪、车速控制器等具有控制与调度功能的子系统能力进行信息融合管理。

答案-问题（3）

驾驶员、行人、乘客、车辆维修人员、操作人员、路面结冰、轮胎爆裂、火灾/水灾、爆炸/自燃、汽油/机油泄漏、信号灯故障、网络故障或丢失

(2020年5月系统分析师真题-第2题)

阅读以下关于嵌入式实时系统设计的相关技术的描述，回答问题1至问题3。

某公司长期从事嵌入式系统研制任务，面对机器人市场的蓬勃发展，公司领导决定自主研制一款通用的工业机器人。王工承担了此工作，他在泛调研的基础上提出：公司要成功地完成工业机器人项目的研制，应采用实时结构化分析和设计（RTSAD）方法，该方法已被广泛应用于机器人顶层分析和设计中。

问题（1）

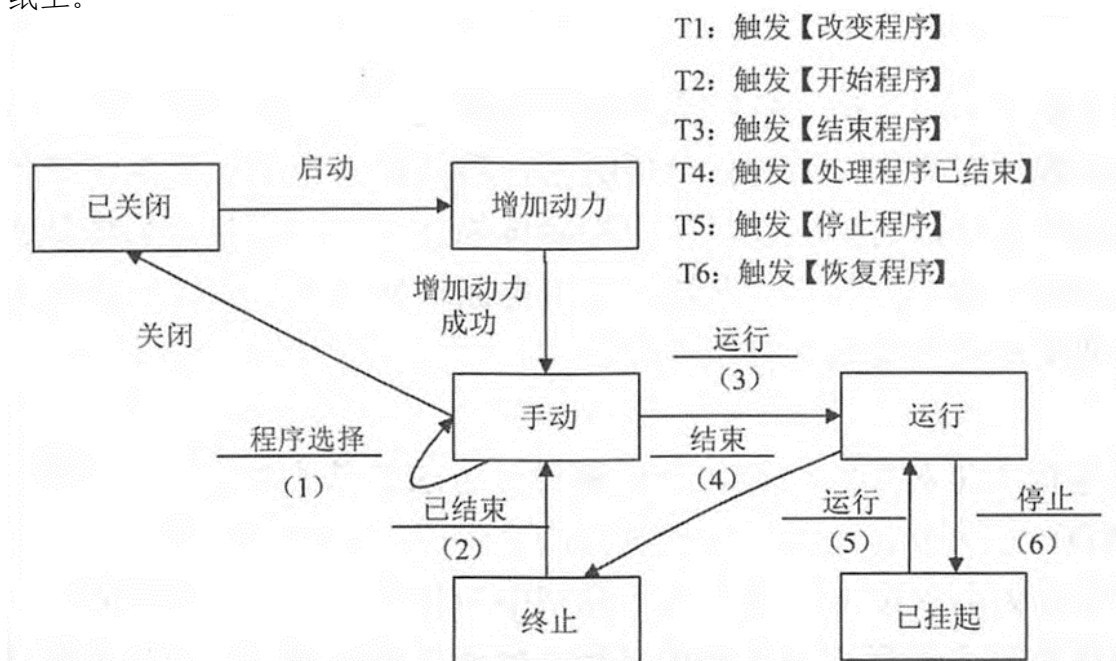
实时结构化分析和设计（RTSAD）方法分为分析和设计两个阶段。分析阶段要开发个基本模型，即需求模型，基本模型中包含一个环境模型和一个行为模型：设计阶段是一种程序设计方法，该方法在转换分析和事务分析策略中结合使用了模块耦合和内聚标准，用于开发从结构化分析规范开始的设计方案。请用 300 字以内文字说明环境模型、行为模型、模块耦合和内聚的含义；并从模块独立性的角度，说明模块设计的基本原则。

思路解析

RTSAD方法通过分析和设计阶段对系统进行分解和建模，其中环境模型帮助理解系统与外部的交互，行为模型定义系统的行为模式。而模块设计中，内聚和耦合是关键指标。高内聚的模块有助于增强模块独立性和系统的稳定性，而低耦合则减少模块间的依赖关系，提升系统的灵活性。通过合理设计模块，可以提高系统的可维护性和可扩展性。

问题（2）

图3-1给出了机器人控制器的状态转换图，其中T1T6表示了状态转换过程中的触发事件，请将T1-T6填到图3-1中的空（1）-（6）处，完善机器人控制器的状态转换图，并将正确答案填写在答题纸上。



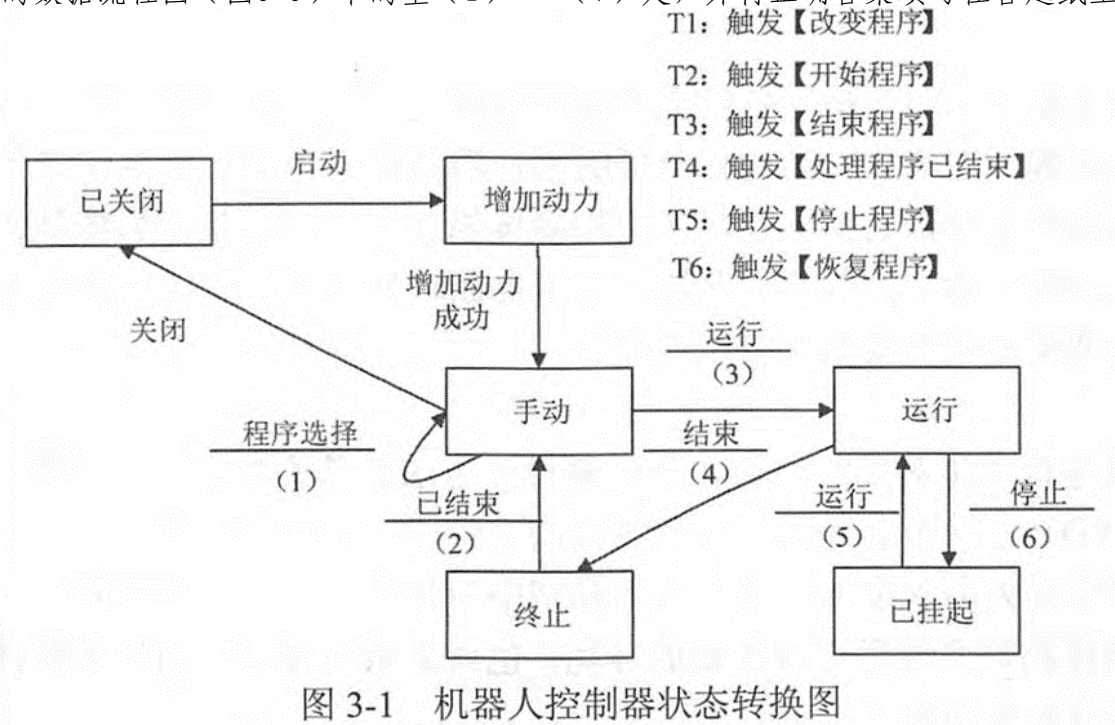
思路解析

状态转换图（STD）描述系统如何响应外部事件，并如何通过状态变化来执行特定行为。

- T1: 触发【改变程序】事件，系统从手动状态切换到程序选择状态。
- T4: 触发【处理程序已结束】事件，系统从手动状态回到增加动力状态。
- T2: 触发【开始程序】事件，系统从增加动力状态进入手动状态。
- T3: 触发【结束程序】事件，系统从运行状态切换到终止状态。
- T6: 触发【恢复程序】事件，系统从已挂起状态返回到运行状态。
- T5: 触发【停止程序】事件，系统从运行状态进入已挂起状态。

问题（3）

参考机器人控制器状态转换图（图3-1）和机器人控制器环境图（图3-2），完善机器人控制器命令的数据流程图（图3-3）中的空（1） - （7）处，并将正确答案填写在答题纸上。



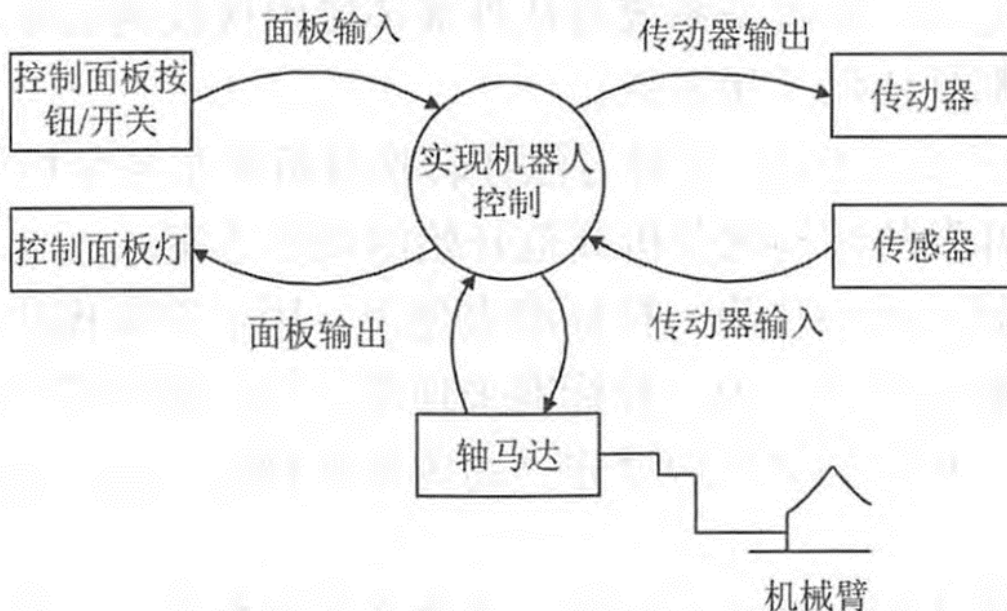


图 3-2 机器人控制器环境图

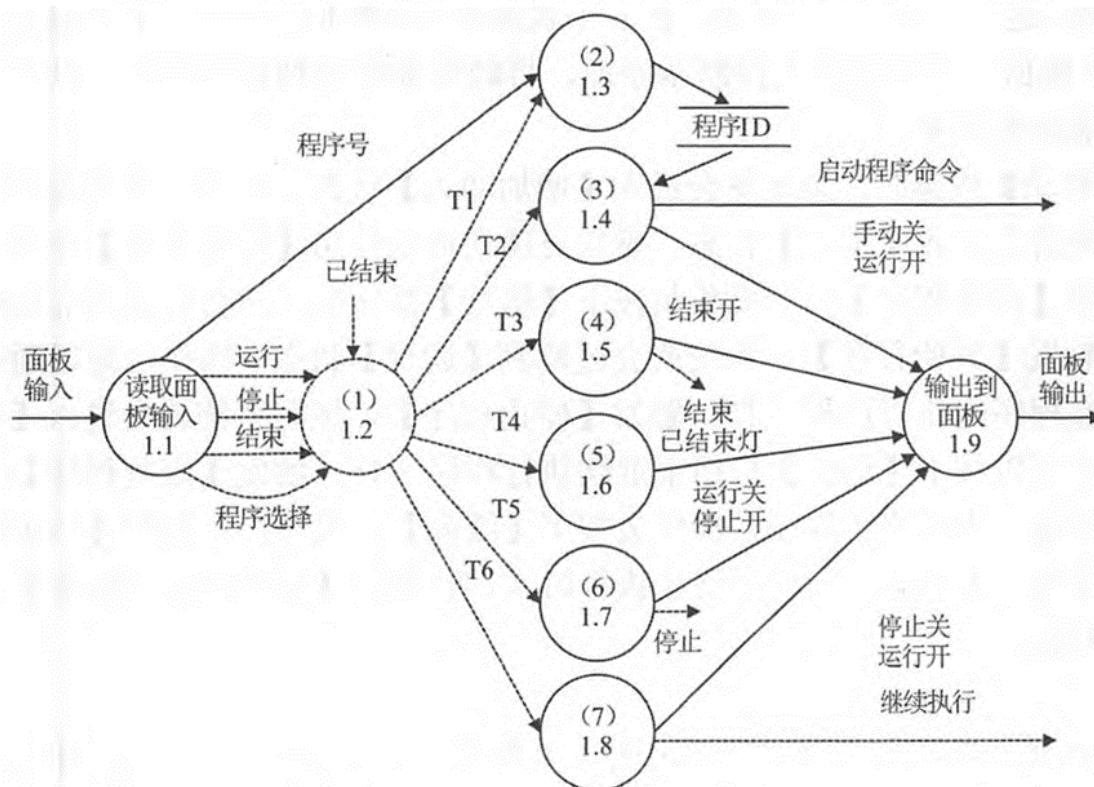


图 3-3 机器人控制器命令的数据流程图

思路解析

数据流程图（DFD）展示了信息在系统中的流动、处理和存储。

- （1）控制机器人：来自控制面板的输入转换为指令发送给机器人，控制机器人的行为。
- （2）修改程序：触发【改变程序】事件后，修改机器人的程序。
- （3）启动程序：触发【开始程序】事件后，启动当前程序。
- （4）结束程序：触发【结束程序】事件后，终止当前程序。
- （5）处理程序结束：当程序完成后，系统进入处理已结束状态。
- （6）停止程序：触发【停止程序】事件后，暂停当前执行的程序。

(7) 继续执行程序：触发【恢复程序】事件后，继续程序的执行。

答案-问题（1）

环境模型描述的是系统运行时所处的环境，也就是系统要连接的外部实体，以及发送给系统的输入和来自系统的输出。行为模型描述的是行为，也就是系统对从外部环境中接收到的输入信息的反应，在实时系统中，这些反应一般都是依赖于状态的。

模块耦合在模块分解过程中作为种标准来使用，用于判断模块间连接性的程度。

模块内聚在模块分解过程中作为 种标准来使用，用于确定模块内部的强度或统一性。

模块设计在模块独立性上的基本原则：高内聚低耦合。

答案-问题（2）

- (1) T1
- (2) T4
- (3) T2
- (4) T3
- (5) T6
- (6) T5

答案-问题（3）

- (1) 控制机器人
- (2) 修改程序
- (3) 启动程序
- (4) 结束程序
- (5) 处理程序结束
- (6) 停止程序
- (7) 继续执行程序

(2019年5月系统分析师真题-第3题)

阅读以下关于安全攸关嵌入式系统相关技术的描述，在答题纸上回答问题1至问题3。

某公司机电管理系列产品被广泛应用于飞行器后，外场事故频繁发生，轻则飞机座舱显示机电设备工作异常，重则系统预警，切入备份运行。这些事故给航空公司带来重大经济损失。

公司领导非常重视航空公司的问题反馈，责令公司王总带队到现场进行故障排查。经过一个多月的排查，故障现象始终未复现，同时，公司实验室内也在反复复现故障，结果未取得显著成效，但发现产品存在偶然丢失协议包的现象。随后，公司领导组织行业专家召开故障分析会。王总在会上对前期故障排查情况进行了说明，指出从外场现象看CCDL协议包丢失是引起系统报警、切换的主要原因。图3-1给出了机电管理产品的工作原理，机电管理系统主要承担了对飞行器的刹车、燃油和环控等子系统进行监视与控制，它对飞行器而言是安全攸关系统，因此，从系统结构上采用了双余度计算机系统。具体工作流程简要说明如下：

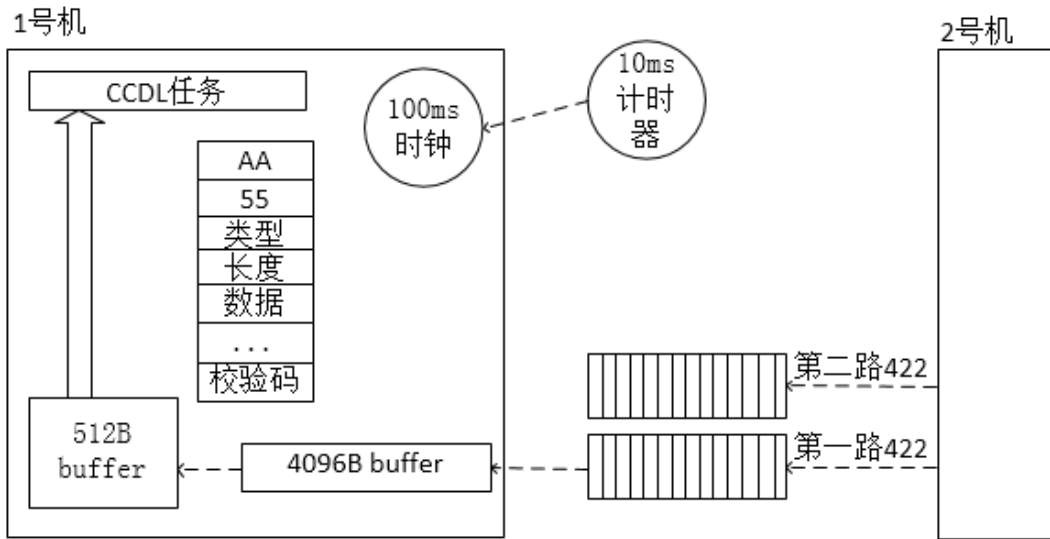


图3-1 机电管理系统工作原理图

1. 机电管理系统由1号计算机和2号计算机组成，双机互为余度备份；
2. 双机中分别驻留了一个100ms周期的CCDL任务，完成双机间的交叉对比和实时监控等工作。10ms定时器作为任务的工作频率；
3. 交叉对比协议包包含一组"AA55"报头、消息长度、数据和校验码；
4. 2号机将协议包通过422总线发送给1号机（422总线接口芯片有8级缓冲）；
5. 1号机通过中断方式将422总线数据接收到大环形缓冲区中（大小为4096B）；
6. 100ms的CCDL定时任务将大环形缓冲区的数据以512B为单位拷贝到小环形缓冲区中（大小512B）；
7. CCDL任务按照协议包格式解析小环形缓冲的数据，如果校验错误，丢弃当前协议包；
8. 在协议包格式正确的情况下，进行数据交叉比对，比对正确则输出；比对不正确，并连续不正确超过门限，则报警。

问题（1）

王总汇报时指出，在设计安全攸关系统软件时，往往不重视安全攸关软件设计方法，不遵守C语言安全编码规范，导致程序质量较差，代码中存在安全隐患。请简要说明表 3-1给出的C语言代码是C语言安全编码标准中（如：MISARC标准）不允许采用的代码结构的原因。

序号	C 语言代码	标准是否允许采用
1	<pre>void static_p(void) {...unsigned int x=2u; if(x=2u){ /*...*/ }else if (x=3u){ /*...*/ } }</pre>	(1)
2	<pre>void static_p(int p_1) {...inti=p_1; switch(i) { } }</pre>	(2)
3	<pre>unsigned int *static_p(unsigned int *p1_ptr) {static unsigned int w=2u; p1_ptr=&w; Return &w; }</pre>	(3)
4	<pre>void foo (unsigned int p_1,unsigned short p_2) void static_p(void) { void(*proc_pointer)(unsigned int, unsigned short)=foo; }</pre>	(4)
5	<pre>unsigned int exp_1(unsigned int *p_1) { unsigned int x=*p_1; (*p_1)=x*x; return(x) } unsigned int exp_2(unsigned int *p_1) { unsigned int x=*p_1; void static_p(void) } unsigned int y=3u,x=0u; x=exp_1 (&y)+exp_2(&y); }</pre>	(5)
6	<pre>void static_p(void) { unsigned short s=0; unsigned int *p1_ptr; p1_ptr=(unsigned int *)s; /*...*/ }</pre>	(6)

思路解析

1) 不允许：违背（建议）规则6.3。MISAR-C标准要求开发人员显式标识数据类型的长度和符号特性，避免直接使用标准数据类型。如果没有使用typedef来标识数据类型的长度和符号特性，在不同平台上可能会导致类型不一致，增加系统不稳定的风险。比如在不同系统上，int类型的大小可能不同，因此建议使用typedef定义类型，如typedef unsigned int uint32_t;，确保数据类型的一致性和可移植性。

2) 不允许：违背（建议）规则6.3、（强制）规则15.3和（强制）规则15.5。在使用switch语句时，MISAR-C标准强制要求switch语句必须包含default子句，并且至少有一个case子句。缺少default子句会使得程序在没有匹配任何case时不做任何处理，这会导致未处理的情况或未定义行为。确保switch语句包含default子句可以增强程序的健壮性。

3) 不允许：违背（建议）规则6.3。使用标准数据类型而没有显式定义数据类型的符号和长度特性，会增加程序出现错误的风险，尤其是在跨平台或者不同编译器的环境下。因此，标准建议使用typedef明确标识数据类型，减少出现不同数据类型大小或符号特性的潜在问题。

4) 不允许：违背（建议）规则6.3 和强制规则8.1。MISAR-C标准要求所有函数必须有原型声明，并且函数原型必须在函数定义和调用范围内都是可见的。没有函数原型声明或者函数原型不可见，会导致编译器在函数调用时无法进行类型检查，容易导致运行时错误或行为不确定。

5) 不允许: 违背(建议)规则6.3和强制规则12.2。MISAR-C标准要求程序中的表达式在标准允许的任何次序下应该产生相同的结果。代码中如果依赖于特定的计算顺序,可能导致在不同编译器或不同平台上行为不一致,从而产生潜在的错误。为了保证一致性和可维护性,应避免在表达式中依赖计算顺序。

6) 不允许: 违背(建议)规则6.3和强制规则10.3。MISAR-C标准中强制要求避免危险的类型转换。代码中的类型转换(unsigned int *)s是不安全的,可能导致内存访问错误或未定义行为,尤其是在不同平台或编译器上,指针类型转换可能导致错误的内存地址访问,增加了程序崩溃或错误的风险。

问题(2)

请根据自己对图3-1所示机电管理系统工作原理的分析,用300字以内的文字说明本实例中可能存在哪三方面数据传输时丢失协议包现象,并简要说明原因。

思路解析

大小缓冲不匹配: : CCDL任务每100ms将大缓冲区(4096B)数据拷贝到小缓冲区(512B)。当大缓冲区中接收到大量数据时,如果CCDL任务的处理速度不足以及及时处理这些数据,可能导致数据未能及时从大缓冲区转移到小缓冲区,进而丢失协议包。设计上需要确保数据拷贝的同步,避免因缓冲区溢出而丢包。

100ms时钟不准: : 系统依赖于10ms计时器来保证每100ms任务周期的准确性。如果10ms计时器由于系统干扰或者中断丢失而产生误差,导致100ms周期不准确,则会影响CCDL任务的定时执行。此时,任务可能提前或延迟执行,导致协议包未能及时处理,从而丢包。

422总线缓冲丢字节: : 422总线只有8字节的输入缓冲区。如果数据输入流量较大,而总线缓冲区没有及时读取数据,就可能导致缓冲区溢出,进而丢失部分字节。如果协议包的校验失败,丢失的数据将导致协议包丢失。为了避免此问题,必须确保缓冲区能及时读取并处理数据。

问题(3)

针对以上分析出的三种丢包原因,请举例给出两种以上的修改丢包bug的可能的方法。

思路解析

合并缓冲区: 通过将512B和4096B两个缓冲区合并成一个更大的缓冲区,避免由于缓冲区拷贝不同步而造成丢包问题。合并后,数据可以更加顺畅地传递,减少了在数据拷贝过程中出现丢包的风险。此外,大缓冲区也能更好地容纳更多数据,防止溢出。

精简时钟中断处理程序: 如果10ms时钟中断处理程序过于复杂,可能会导致中断丢失,从而影响100ms任务的准时启动。通过优化中断处理程序,可以减少中断丢失的概率,确保定时任务能够按照预定的周期执行,避免时钟不准确导致丢包。

充分利用422芯片的中断机制: 422芯片提供了缓冲区满、半满和空的中断机制,可以通过配置这些中断机制,确保在缓冲区快满时及时读取数据,从而防止缓冲区溢出导致数据丢失。及时读取数据有助于避免丢包现象的发生。

采用新型校验码: 使用更强的校验码(如CRC)来替代传统的校验和,能够更有效地检测数据传输中的错误,从而减少因校验失败导致的丢包。新型校验码的引入能够提高数据的完整性和传输的可靠性。

缩短100ms任务周期: 将100ms任务的执行周期缩短,可以更及时地处理到达的数据,从而避免

由于长时间的任务周期导致的数据积压和丢包。缩短任务周期有助于提高系统响应速度，及时处理数据，减少丢包的风险。

答案-问题（1）

- 1) 不允许。违背（建议）规则6.3。
- 2) 不允许。违背（建议）规则6.3、（强制）规则15.3和（强制）规则15.5。
- 3) 不允许。违背（建议）规则6.3。
- 4) 不允许。违背（建议）规则6.3 和强制规则8.1。
- 5) 不允许。违背（建议）规则6.3和强制规则12.2. $x=\exp_1(\&y)+\exp_2(\&)$ 应修改为： $x=\exp_1(\&y)$ ； $x+=\exp_2(\&y)$ 。
- 6) 不允许。违背（建议）规则6.3 和强制规则10.3； $(\text{unsigned int } *)s$ 是危险的类型转换。

答案-问题（2）

图3-1可能存在大小缓冲不匹配、时钟不准和422总线缓冲丢字节等三方面丢数据包现象。

（1）大小缓冲不匹配。由于 CCDL任务在每100ms 将大缓冲的数据拷贝到小缓冲后，进行协议包解析。当两缓冲存在大量接收数据包时，如果CCDL 任务处理机制设计不合理，有可能解析时间过长而引起丢包现象。

（2）100ms 时钟不准。由于大小环形缓冲数据交换采用100ms 时钟周期处理，而100ms 时钟周期依赖于 10ms计时器，如果10ms 计时器受到系统干扰可能丢失中断，则引起100ms 时钟周期增加10ms。导致CCDL 任务定时时间不准，不能按100ms 周期解S加入：析协议包，过早解析协议包，而当前周期的协议包还未到达，可能导致包。

（3）422总线缓冲丢字节。422总线有8字节输入缓冲，如果将422总线设置全满中断时，在数据输入流量大的情况下，8字节 FIFO 字节输入缓冲容易溢出，导致字节丢失，一旦协议包校验和失败，会丢弃此包，这样会导致丢包。

答案-问题（3）

- （1）512B和4096B两个缓冲区合并，以减少缓冲区拷贝不同步而产生的丢包问题。
- （2）精简10ms 时钟中断处理程序，避免10ms 中断的丢失，所引发100ms任务不能准时启动。
- （3）充分利用422芯片的缓冲区的满、半满和空中断机制，避免422缓冲区内字节数据未及时读出而丢失，使100ms 任务找不到包头或校验码错。
- （4）采用新型校验码，降低传输中的数据错误。
- （5）缩短100ms 任务的循环周期，以及时处理CCDL 任务。

（2018年5月系统分析师真题-第3题）

阅读以下关于安全关键系统安全性设计技术的描述，回答问题1至问题3。

某公司长期从事计算机产品的研制工作，公司领导为了响应国家军民融合的发展战略，决定要积极参与我国军用设备领域的研制工作，将本公司的计算机及软件产品通过提升和改造，应用到军用装备的安全关键系统中。公司为了承担军用产品的研发任务，公司领导将论证工作交给王工负责。王工经调研分析，提交了一份完整论证报告。

问题（1）

论证报告指出：我们公司长期从事民用市场的计算机研制工作，在研制流程、管理方法以及环境试验等方面都不能达到军用设备相关技术要求。要承担武器装备生产研制工作，就必须建立公司的武器装备生产研制质量体系，需要拿到军方或政府部门颁发的资格认证。从技术上讲，军用设备产品大部分都属于安全关键系统，其计算机及软件的缺陷会导致武器装备失效，因此，公司技术人员应及早掌握相关安全性基本概念和相关设计知识。

- 1) 企业要承担武器装备产品生产任务，需获得一些资格认证，请列举两种资格认证名称。
- 2) 请说明安全关键系统的定义，并列举出两个安全关键系统的实例设备。
- 3) 请简要说明安全性（safety）的具体含义，并给出产品设计时，安全性分析通常采用哪两种方法？

思路解析

企业认证：对于从事武器装备研发的公司而言，获得相应的认证是必不可少的。这些认证旨在确保公司具备足够的技术能力、管理能力以及保障产品质量和安全性的措施。常见的两种认证包括：

国军标质量管理体系认证（ISO-9001）：这是国际通行的质量管理体系认证标准，确保公司产品质量符合国际标准。该认证针对公司整体质量管理体系进行评估，适用于各类军工产品的研发和生产。

武器装备科研生产许可证认证：这是一种针对军事产品研发的专门认证，表明公司在军工产品的设计、开发、生产等方面符合军方的技术要求与标准，能够承担相应的科研和生产任务。

安全关键系统的定义及实例：安全关键系统（Safety Critical System）指的是那些一旦出现失效，可能导致重大人员伤亡、财产损失、环境破坏等灾难性后果的系统。它们对于生命、财产和环境的安全至关重要，必须严格控制和管理。安全关键系统的实例包括：战斗机的航空电子系统：在战斗机中，航空电子系统负责飞机的导航、通信和武器控制等功能，任何系统失效都可能导致飞行器的丧失或任务失败，甚至危及飞行员生命。火控雷达系统：火控雷达系统在军事装备中负责目标锁定、火力控制等任务，失效可能导致武器无法准确打击目标，影响军事行动的成效甚至导致不可挽回的损失。

战斗机的航空电子系统：在战斗机中，航空电子系统负责飞机的导航、通信和武器控制等功能，任何系统失效都可能导致飞行器的丧失或任务失败，甚至危及飞行员生命。

火控雷达系统：火控雷达系统在军事装备中负责目标锁定、火力控制等任务，失效可能导致武器无法准确打击目标，影响军事行动的成效甚至导致不可挽回的损失。

安全性分析及常用方法：安全性（Safety）是指在系统发生故障时，系统能保持的安全状态，尤其是防止灾难性后果的发生。例如，即使发生故障，系统也能维持在一个不会导致严重后果的安全状态。安全性分析是为了识别系统中的潜在风险，并采取有效措施避免故障对系统造成灾难性后果。常用的安全性分析方法包括：故障树分析（Fault Tree Analysis, FTA）：通过树状图形化地表示系统中可能导致灾难性后果的故障，并进一步分析这些故障的根本原因。FTA能够清晰地展示不同故障事件如何层层相互作用，导致系统最终失效。失效模式与影响分析

（Failure Modes and Effects Analysis, FMEA）：FMEA用于识别和评估系统中每一个组件的潜在故障模式及其对系统功能的影响。通过分析故障模式及其影响，可以采取降低系统失效的风险。

故障树分析（Fault Tree Analysis, FTA）：通过树状图形化地表示系统中可能导致灾难性后果的故障，并进一步分析这些故障的根本原因。FTA能够清晰地展示不同故障事件如何层层相互作用，导致系统最终失效。

失效模式与影响分析（Failure Modes and Effects Analysis, FMEA）：FMEA用于识别和评估系统中每一个组件的潜在故障模式及其对系统功能的影响。通过分析故障模式及其影响，可以采取降低系统失效的风险。

问题（2）

IEC 61508 （《电气/电子/可编程电子安全系统的功能要求》）是国际上对安全关键系统规定的一种较完整的安全性等级划分标准，本标准是由国际电工委员会（International Electronic Commission）正式发布的电气和电子部件行业标准（GB/T 20438等同于此标准）。本标准对设备或系统的安全完整性等级（SIL）划分为4个等级（SIL1、SIL2、SIL3、SIL4），SIL4是最高要求。

表3-1给出了本标准对安全功能等级和失效容忍概率的对应关系。请根据自己所掌握的安全功能等级相关知识，补充完善表3-1给出的（1）~（6）空格，并将答案写在答题纸上。

安全功能等级	每项需求失效的平均容忍概率	每小时失效的平均容忍概率
SIL4	(1)	(2)
SIL3	(3)	$\geq 10^{-8}$ to $< 10^{-7}$
SIL2	(4)	(5)
SIL1	$\geq 10^{-2}$ to $< 10^{-1}$	(6)

思路解析

IEC 61508标准是电气、电子、可编程电子安全系统功能要求的标准，适用于对安全性要求极高的行业，尤其是涉及危险控制和预防重大灾难的系统。该标准为设备或系统的安全完整性等级（SIL）划定了四个级别，分别是SIL1、SIL2、SIL3、SIL4，其中SIL4要求最为严格，适用于对安全性要求极高的系统。

在IEC 61508标准中，每个SIL级别都对应着一个失效容忍概率范围，表示系统在特定条件下能够容忍的最大失效概率。较低的SIL等级意味着系统能容忍的失效概率较高，较高的SIL等级则意味着对系统失效的容忍度较低，要求系统具备更高的可靠性。

对于每个SIL等级的失效容忍概率：

SIL1（最低要求）：系统的失效概率应在 $\geq 10^{-5}$ to $< 10^{-4}$ 之间。表示系统可以容忍每5万次事件中发生一次失效。

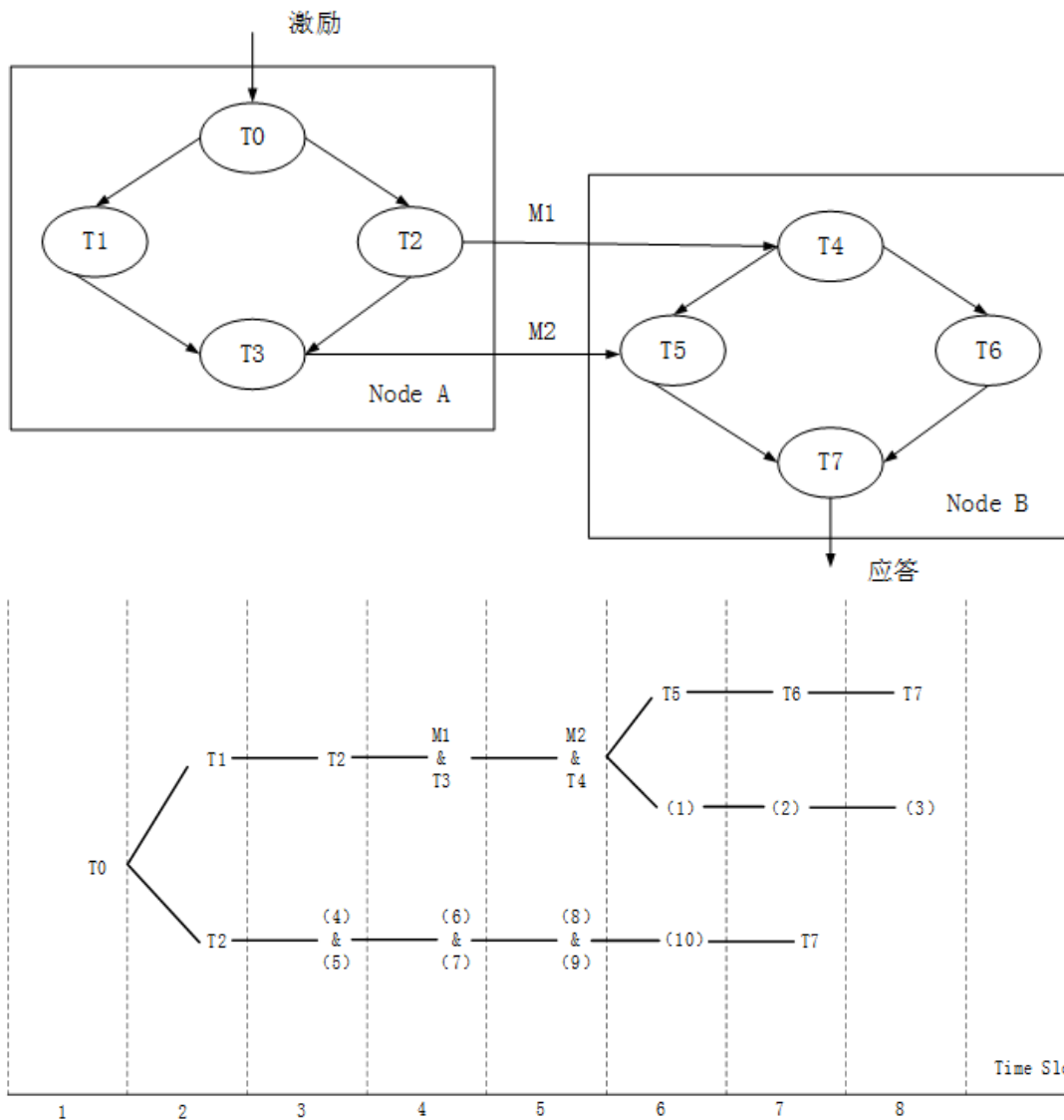
SIL2：要求失效容忍概率在 $\geq 10^{-9}$ to $< 10^{-8}$ 之间，即系统能容忍的失效概率更低，适用于较高安全要求的系统。

SIL3：该级别的失效容忍概率在 $\geq 10^{-4}$ to $< 10^{-3}$ 之间，适用于极为重要的控制系统。

SIL4（最高要求）：失效容忍概率应在 $\geq 10^{-3}$ to $< 10^{-2}$ 之间，是最高安全级别，适用于那些可能导致灾难性后果的系统，如核电厂的安全控制系统。

问题（3）

实时调度是安全关键系统的关键技术。实时调度一般分为动态和静态两种。其中，静态调度是指在离线情况下计算出的任务的可调度性，静态调度必须保证所有任务的时限、资源、优先级和同步的需求。图3-1给出了一组分布式任务执行的优先级关系，请根据图3-1给出任务间的优先级关系实例，按静态调度算法的基本原理，补充完善图3-2给出的任务静态调度搜索树的（1）~（10）空白，并给出最佳调度路径。



思路解析

资源需求等因素进行预先计算。静态调度通常用于系统的硬实时任务，它能够确保所有任务在规定的时间内完成，避免延迟或错过时限，适用于那些需要精确控制的任务。

静态调度的原则：

任务优先级：任务按照其优先级顺序执行，优先级高的任务在低优先级任务之前完成。

任务时限：静态调度要求每个任务的执行必须保证在其规定的时间窗口内完成，否则会导致系统失效。

任务同步与资源分配：在多任务的情况下，静态调度还需要考虑任务间的同步和共享资源的合理分配，防止死锁和资源冲突。

图3-1中给出的任务执行优先级关系表明，任务T6的优先级最高，其次是T5，T7等。根据静态调度算法，系统将在满足优先级和资源分配的情况下，合理安排任务的执行顺序。静态调度的关键是通过计算任务的资源需求和优先级，确定任务执行的先后顺序。

根据静态调度的搜索树，任务的调度路径可以得出：

最佳调度路径：T0，T2，（M1，T1），（T3，T4），（M2，T6），T5，T7

这条路径确保了高优先级的任务优先执行，且系统资源得到了合理的分配，确保所有任务都能在规定时间内完成，避免系统失效。

答案-问题（1）

1) 从事军工科研生产需先获得以下五种认证:

- 1、国军标质量管理体系认证，简称国军标认证（ISO-9001）；
- 2、武器装备科研生产许可证认证，简称许可证认证；
- 3、武器装备科研生产单位保密资质认证，简称保密认证；
- 4、装备承制单位资格审查（装备承制单位资格名录认证和武器装备质量管理体系认证），检测名录认证；
- 5、军用软件研制能力成熟度模型资格认证，简称软件认证（GJB5000）。

2) 安全关键系统是指系统功能一旦失效将引起生命、财产等重大损失以及环境可能遭到严重破坏的系统（如果计算机系统的失效可能引起灾难性的后果，如丧失生命、大量财产损失或环境遭到灾难性损失，则这个计算机系统能被称为"安全关键系统"。）安全关键系统包括如战斗机的航空电子系统，火控雷达系统、火车控制系统、核反应堆系统等。

3) 安全性是指系统在发生关键失效状态下，系统可保持不会导致灾难性后果的时间间隔。安全性分析通常采用：故障树（Fault Tree）分析法；失效模式和影响域（FMEA）分析法。

答案-问题（2）

- (1) $\geq 10^{-5}$ to $< 10^{-4}$
- (2) $\geq 10^{-9}$ to $< 10^{-8}$
- (3) $\geq 10^{-4}$ to $< 10^{-3}$
- (4) $\geq 10^{-3}$ to $< 10^{-2}$
- (5) $\geq 10^{-7}$ to $< 10^{-6}$
- (6) $\geq 10^{-6}$ to $< 10^{-5}$

答案-问题（3）

- (1) T6 (2) T5 (3) T7
- (4) M1 (5) T1 (6) T3
- (7) T4 (8) M2 (9) T6
- (10) T5

最佳调度路径：T0, T2, (M1, T1), (T3, T4), (M2, T6), T5, T7

(2017年5月系统分析师真题-第3题)

阅读以下关于嵌入式多核程序设计技术的描述，回答问题 1至问题 3。

近年来，多核技术已被广泛应用于众多安全关键领域（如：航空航天等）的电子设备中，面向多核技术的并程序序设计方法已成为软件人员急需掌握的主要技能之一。某宇航公司长期从事宇航电子设备的研制工作，随着宇航装备能力需求的提升，急需采用多核技术以增强设备的运算能力、降低功耗与体积，快速实现设备的升级与换代。针对面向多核开发，王工认为多核技术是对用户程序透明的，开发应把重点放在多核硬件架构和硬件模块设计上面，而软件方面，仅仅需要选择一款支持多核处理器的操作系统即可。而李工认为，多核架构能够使现有的软件更高效地运行，构建一个完善的软件架构是非常必要的。提高多核的利用率不能仅靠操作系统，还要求软件开发人员在程序设计中考虑多进程或者多线程并行处理的编程问题。

问题（1）

请用300字以内文字说明什么是多核技术和多线程技术，并回答李工的意见是否正确，为什么？

思路解析

多核技术：

多核技术指的是将多个独立的处理器核心（CPU核心）集成到同一颗芯片上。这些核心可以同时处理不同的任务，从而提高计算性能和系统的整体吞吐量。每个核心都可以独立执行任务，并且它们共享同一颗芯片上的资源（如缓存和内存控制器），这使得多核处理器在处理多个任务时比单核处理器更高效。

多核技术的一个关键优势是其能在硬件层面上实现并行计算，这意味着多个任务可以在不同的核心上并行执行，从而显著提高计算速度。例如，在运行一个并行计算任务时，操作系统会将任务划分成多个子任务并分配到不同的核心上，让这些任务同时执行，从而大幅度缩短处理时间。

多线程技术：

多线程技术指的是在程序内部创建多个执行路径（线程），每个线程负责执行任务的一部分。线程是程序中的最小执行单元，多个线程可以共享程序的资源（如内存空间），但每个线程有独立的执行上下文（如程序计数器、堆栈等）。多线程技术让单个程序能够同时执行多个任务，通常用于提高程序的响应性和效率。

多线程可以通过时间片轮转的方式在单核处理器上模拟并行执行，在多核处理器上，则可以在不同的核心上并行执行不同的线程，从而进一步提高性能。

李工的意见：

李工的观点是正确的。多核技术提供了物理上同时执行多个任务的能力，但若程序没有考虑并行化和多线程设计，那么即使在多核系统上，任务也可能无法充分利用多个核心。程序员需要通过使用多线程或多进程来实现任务的并行处理，从而让多个核心协同工作，提升程序的执行效率。因此，李工强调的通过并行设计来提高多核利用率，是非常重要的。

问题（2）

在多核环境下，线程的活动有并行和并发两种方式，请用300字以内的文字说明这两种方式的含义及差别。

思路解析

并发（Concurrency）：

并发指的是多个任务在同一时间段内进行，任务之间并不一定同时执行，但它们在同一时间窗口内存在。并发处理不要求任务在同一时刻都在执行，而是通过任务调度在多个任务之间切换，从而让每个任务都能在一段时间内得到执行。即使是单核处理器，操作系统也能够通过切换任务的执行顺序（时间片轮转），实现看似同时执行多个任务的效果。

例如，在一个单核处理器上，操作系统会在多个线程间切换，快速轮流给每个线程一定的CPU时间，从而实现并发执行。

并行（Parallelism）：

并行指的是多个任务在同一时刻执行，通常需要多核或多处理器系统的支持。并行处理要求有多个计算单元（如多个核心）能够同时运行多个线程或进程，任务被分解成多个子任务并且这些子任务同时执行，从而大幅度缩短整体执行时间。

并行和并发的主要区别在于：并发强调任务的管理与调度，允许多个任务在同一时间段内存在，但

不要求它们在同一时刻执行；而并行则要求任务真正的在同一时刻同时执行，并且通常依赖于多核或多处理器的硬件支持。

问题（3）

请根据自己所掌握的多核、多线程的知识，判别表 3-1 给出的说法是否正确，并将答案写在答题纸上对应空白处（填写正确或错误）。

序号	说 明	是否正确
1	在面向多核体系结构开发应用程序时，只有有效地采用多线程技术并仔细分配各线程的工作负载才能够达到最高的性能	(1)
2	在面向多核平台设计多线程应用程序时，开发人员应当采用与面向单核平台时不同的设计思想	(2)
3	在多核平台上。多线程一般被当作是一种能够实现延迟隐藏的有效编程手段	(3)
4	多核平台为开发人员提供了一种优化应用程序的渠道，就是通过仔细分配加载到各线程（或各处理器核）上的工作负载就能够得到性能上的提升	(4)
5	在单核平台上，为了简化多线程应用程序的编写和调试，开发人员可能会做一些假设，这些假设也会适应于多核平台	(5)
6	在多核平台上，存储缓冲的 Cache 一致性问题是多核程序设计应当重点考虑的问题，但是，多核 Cache 的伪共享（False Sharing）问题在单核平台上也存在	(6)
7	在单核平台上，开发人员通常遵循优先级较高的线程不会受到优先级较低线程的干扰的思想对代码进行优化，这样的代码在多核平台上运行就会非常不稳定	(7)

思路解析

（1）✓

多核处理器确实能够在物理上同时执行多个任务。每个核心可以独立地执行不同的任务，这意味着系统可以通过多个核心并行处理多个任务，从而提高计算效率。

（2）✓

多核处理器能够在提高计算性能的同时，降低功耗和体积。通过将多个核心集成到一颗芯片上，可以减少物理空间的占用，同时减少功耗，特别是在高性能计算需求的场合，利用多核可以实现更高的能效比。

（3）×

这是错误的。尽管操作系统可以管理多核资源，但仅仅依靠操作系统并不能自动提高性能。为了充分利用多核硬件，程序需要进行多核编程（如多线程、并行计算等）。如果程序未能设计成并行执行，即使硬件具备多核，也不能有效利用每个核心的计算能力。

（4）✓

程序员确实需要在多核环境下考虑任务并行化。如果程序没有经过并行设计，它就无法有效利用多核处理器的所有核心，因此，软件开发人员需要在设计阶段就考虑如何分配任务以实现并行计算，确保多核系统的高效运作。

（5）×

这是错误的。仅通过硬件（如多核处理器）并不能自动获得性能提升，必须通过合适的软件设计来让多核系统发挥最大效能。没有并行化的程序即使运行在多核系统上，也不能提升性能。

（6）×

这是错误的。并发和并行的定义是不同的。并发是指任务之间的交替执行，而并行是指任务同时执行。并行处理要求硬件支持多个核心同时运行任务，而并发处理可以通过时间片轮转在单核处理器上模拟多任务执行。

(7) ✓

多核处理器能够在并行计算中提高性能。当任务能够被有效分解为多个子任务时，多个核心可以同时执行这些任务，从而大大加快整体计算过程。

答案-问题（1）

多核是多微处理器核的简称，是将两个或更多的独立处理器封装在一起，集成在一个电路中。多核处理器是单枚芯片（也称为硅核），能够直接插入单一的处理器的插槽中，但操作系统会利用所有相关的资源，将它的每个执行内核作为分立的逻辑处理器。通过在多个执行内核之间划分任务，多核处理器可在特定的时钟周期内执行更多任务。

多线程是指从软件或者硬件上实现多个线程并发执行的技术。具有多线程能力的计算机因有硬件支持而能够在同一时间执行多于一个线程，进而提升整体处理性能。

李工的意见是正确的。多核处理器虽然带来了强大的计算能力，但如果无法实现程序的并行，那么，大量计算资源将被闲置，造成巨大的浪费。所以在开发中，要注重多核编程技术，从并行计算、共享资源分布式计算、任务分解与调度、Lock-Free编程等方面充分考虑相关问题，以便将多核处理器的性能充分发挥出来。

答案-问题（2）

如果某个系统支持两个或者多个动作（Action）同时存在，那么这个系统就是一个并发系统，即系统中同时有多个线程，可以认为是并发的情况。

如果某个系统支持两个或者多个动作同时执行，那么这个系统就是一个并行系统。当系统中有多个线程，并在同一时刻有两个，或两个以上的线程在运行状态，可认为是并行系统。

答案-问题（3）

(1) ✓ (2) ✓ (3) × (4) ✓ (5) × (6) × (7) ✓

(2016年5月系统分析师真题-第3题)

阅读以下关于嵌入式系统能耗管理设计的描述，在答题纸上回答问题1至问题3。

随着嵌入式系统智能化技术的持续发展，系统的能耗问题已成为亟待解决的难题。某宇航公司研发的一款手持式野外辅助装备，需要较长的巡航时间，由于设计时对功耗问题考虑不周，使得产品出现了批次责任事故。公司领导决定抽调王工等5人组建专门的攻关小组，要求在三个月内完成故障归零。王工接到任务后，经反复论证与实验，给公司提交了一份基于软件动态节能的改进方案，得到了领导及专家的好评。

问题（1）

在嵌入式系统中节能技术一般分为静态节能和动态节能两种，请用300字以内文字分别说明什么是静态节能技术和动态节能技术，并列举三种动态节能技术。

思路解析

本题考查考生对嵌入式系统节能技术谱系的把握与分类表述能力。按照实现阶段与对运行时负载的敏感性，节能手段通常分为静态与动态两大类。静态节能是在编译/链接/部署阶段完成的功耗优化，其核心思想是在不改变硬件电源/时钟配置的前提下，通过算法复杂度优化、循环变换、无效代码消除、常量折叠、函数内联、数据对齐与结构体压缩、内存/缓存亲和布局、减少总线切换次数等手段，尽量减少运算与访存，从而降低动态功耗贡献；这类优化与运行时负载关系弱，但可建立稳态的低功耗“底座”。

此外，在系统架构层面，静态方法还包括模块拆分与冷热路径隔离、中断/轮询机制选择、DMA替代CPU搬运等，目标是减少CPU活跃度和不必要唤醒。与之对应，动态节能强调“按需供给（on-demand）”：系统运行时依据任务到达特性、实时性（deadline/jitter）、温度、剩余电量、设备使用相位等信号，动态改变电压/频率/门控/电源域与设备状态机。

典型技术包括：① DVFS：利用CMOS功耗与V、f关系（ $P \propto CVf$ ）在不违背实时约束的条件下降低V/f；② DPM：面向设备状态机的空闲检测与睡眠→唤醒管理，配合有效时刻策略减少“短空闲不睡、长空闲必睡”的失配与频繁抖动；③ 时钟/电源门控：对未使用外设/总线树进行门控或对电源域断电，实现深睡/部分休眠；④ 体偏压/自适应衬底偏置（ABB）：改变阈值电压兼顾漏电与性能；⑤ 动态阈值电压、功耗感知任务映射/迁移等。

问题（2）

王工的方案得到认可后，他带领团队研究了调度方式和系统能耗之间的关系，请用150字以内的文字说明为了节约系统能耗而采用的设备调度策略的核心思路，并说明设备调度策略与任务实时性的关系。

思路解析

本题聚焦“设备有效时刻调度”思想：系统能量主要消耗在设备工作态（活跃功率）与频繁的状态切换开销上；若盲目“有活就唤醒、无活就睡”，会在短空闲/短任务交替时产生抖动，反使切换能量与延迟额外放大。

核心思路因此是：在满足任务实时约束（deadline/最大抖动/响应时间）的前提下，把设备的唤醒时刻尽可能推迟到“必须开始工作”的最晚安全点，把进入睡眠的时刻尽可能提前到“确定不再被访问”的最早安全点；同时将多次零散访问对齐/合并为批处理（batching），把若干短活跃期合并为较长但更少次的活跃期，以减少状态切换次数与转换能量。该策略配套两项关键判断：① 空闲阈值判定（Idle Threshold）：当预计空闲时间大于“睡眠收益>转换成本”的临界值才关闭设备；② 唤醒提前量（Warm-up Margin）：设备从睡眠到可用需转换时间和转换功率，需预留“提前量”不影响任务开始服务。工程实现通常结合任务到达预测、队列长度、服务时间分布以及**实时调度器（如Rate Monotonic/EDF）的可调度性分析，按deadline或松弛度（slack）**推导设备状态序列。与实时性的关系：设备调度是实时性能的直接变量。一方面，唤醒过晚将造成deadline miss；另一方面，频繁切换会引起响应时间抖动、中断延迟增大，影响上层控制/信号处理稳定性。因此，策略制定必须与实时调度器协同：在EDF下可按最晚可行开始时刻安排唤醒；在固定优先级（如RM）下需确保高优先级任务的抢占与设备占用互不干扰，必要时对设备访问使用优先级继承/天花板协议避免优先级反转。同时，设备级策略应与DVFS协作：当计算核可降频时，将释放出的时间松弛让渡给设备管理，进一步延长睡眠窗口。

问题（3）

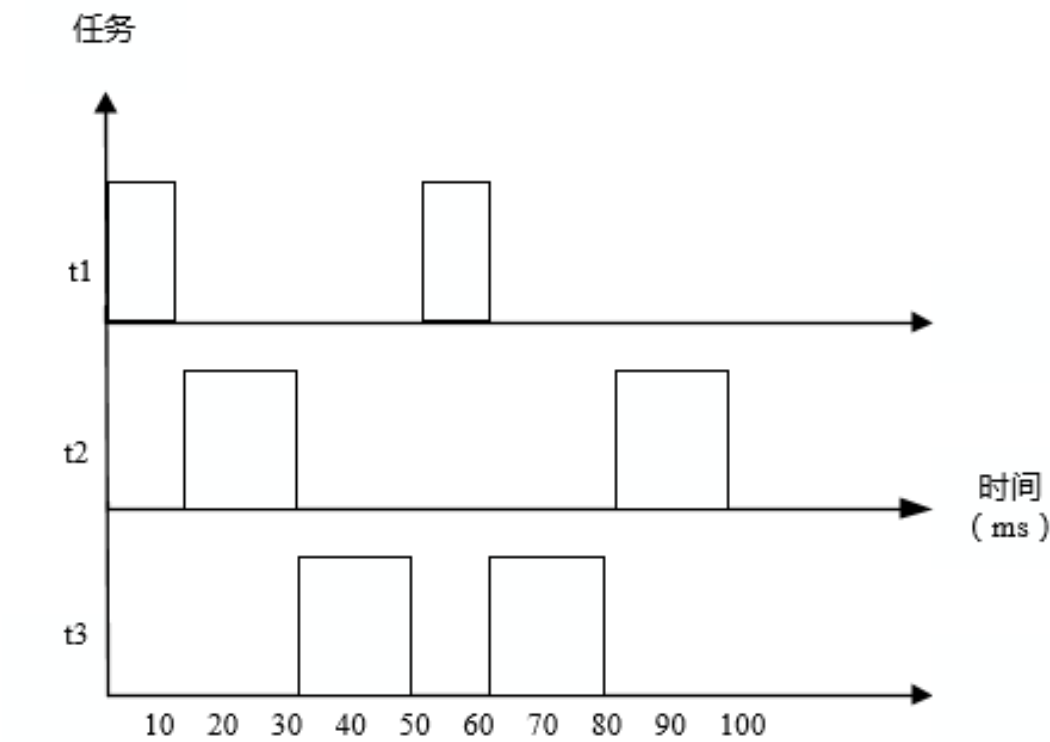
根据上述设备有效调度时刻的策略，分析以下实例，并将答案写在答题纸上。

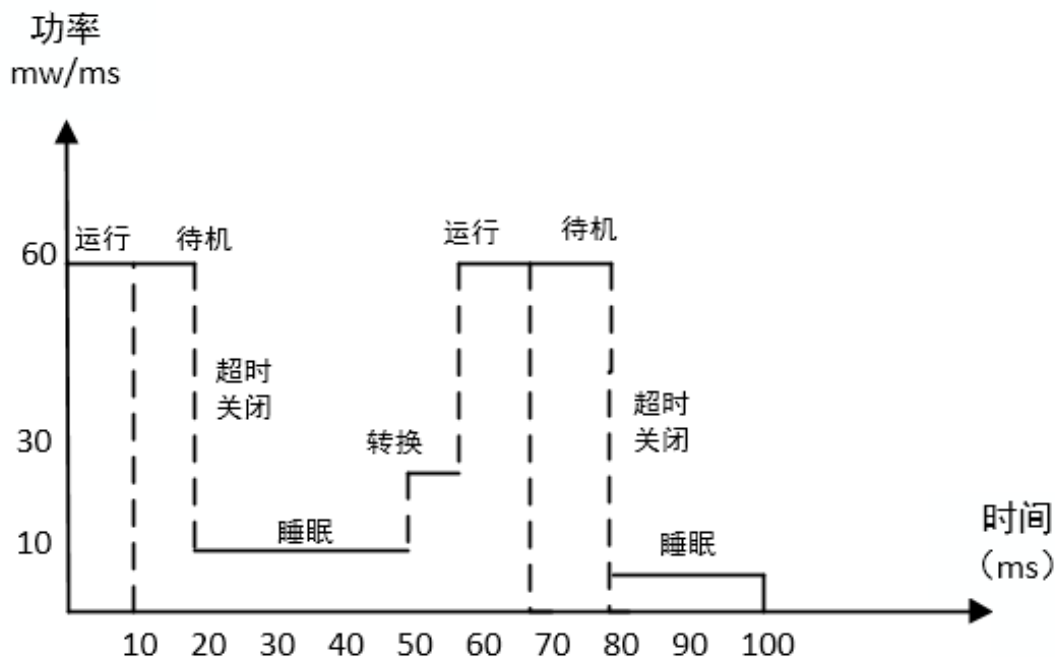
手持式野外辅助装备由三个实时任务和三个独立设备组成（分别表示为：t1、t2 和t3； s1、s2和 s3），表3-1给出了手持式设备实时任务的时间参数表，这些任务使用独立的设备并且所有设备的特征一致，各设备待机超时可直接进入睡眠状态，处于睡眠状态的设备要经过转换状态后才能进入运行状态。假设初始状态（t=0）设备处于工作状态，且所有设备的工作状态功率为60mw/ms，转换功率为30mw/ms，睡眠功率 为10mv/ms。同时假设设备被唤醒的转换时间为10ms。设备待机后被关闭的超时值为10ms。

图3-1给出了未考虑节能需求时任务调度算法，在采用了设备有效时刻调度策略之后，三个设备100ms周期内将按一定顺序被依次关闭和唤醒。图3-2给出了采用设备有效时刻调度策略后s1的功率随时间的变化规律。请计算此时三个设备在100ms 周期内的功耗，并计算100ms周期内，每个设备的节能比。

节能比 = （未节能情况下的系统能耗 - 节能后系统能耗） ÷ 未节能情况下的系统能耗

任务	执行时间 (ms)	周期 (截止期) (ms)	设备使用列表
t1	10	50	s1
t2	20	80	s2
t3	40	100	s3





思路解析

解题关键在于把设备的功率—状态机和时间片段映射为“能量片段求和”：能量=功率×持续时间，在100 ms周期内按工作/转换/睡眠分别累计。题设给出三类功率与两个关键时序参数（待机超时、唤醒转换时间），且图3-2已明确了s1的开—关—唤醒—节奏。通常，未节能调度意味着设备始终保持工作态，因此每台设备的基线能耗是 $60 \times 100 = 6000$ （单位以 $\text{mW} \cdot \text{ms}$ 计）。

采用“有效时刻调度”后，每台设备的状态序列将变为：若在被上层任务访问结束后持续空闲 ≥ 10 ms，则在超时点切到睡眠态；当下一次访问将至，需提前10 ms启动唤醒转换（转换态功率30），并在转换完成时正好切入工作态服务。由图3-2可直接读取s1在100 ms内各状态的时间片段，从而得到 $s1=3400$ 的能量累计（两段各10 ms工作、两段各20 ms睡眠、两段各10 ms转换以及其间的服务片段，逐项乘以相应功率求和即得）。对s2、s3同理，根据其访问相位关系与“空闲 ≥ 10 ms才睡、唤醒需10 ms”的规则，可得到题设给出的分段： $s2=4200$ 、 $s3=4700$ 。将三者与基线比较，计算节能比分别为0.43、0.30、0.22。

答案-问题（1）

静态节能技术是在编译时刻对代码扫描，找出可以针对目标平台特性优化的部分进行代码重构。

动态节能技术是在设备运行时根据任务负载、功率与输入电压平方的线性关系等进行运行状态以达到节能的技术，主流的动态节能技术有动态电压调节、动态频率调节、动态电源管理、动态电压缩放、体偏压调节、自适应衬底偏置等。

答案-问题（2）

在不违背时限，保证延迟的前提下，合理调度任何到各设备上，求得最佳调度以降低系统能耗。设备调度策略是直接影响任务实时性能的因素。

答案-问题（3）

s1、s2和s3在不考虑节能调度算法的情况下，100ms周期内的能耗都是 $60 \times 100 = 6000 \text{mw}$ 。

根据图3-2，在考虑了节能调度算法的情况下：

s1的能耗： $60 \times 10 + 60 \times 10 + 10 \times 20 + 30 \times 10 + 60 \times 10 + 60 \times 10 + 10 \times 20 + 30 \times 10 = 3400\text{mw}$

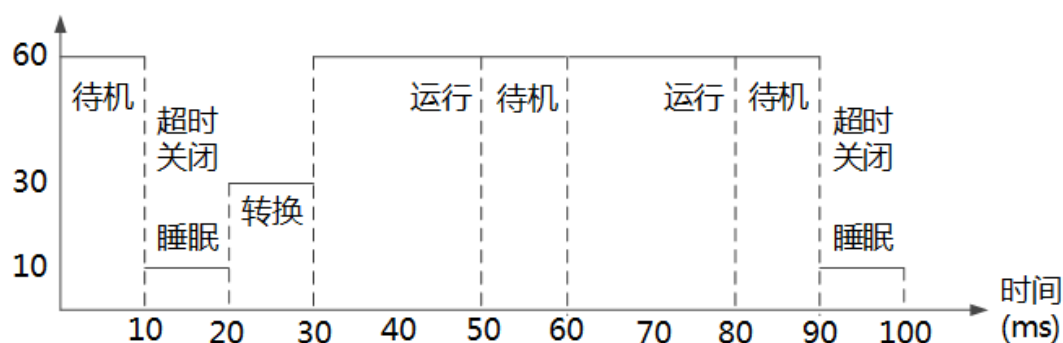
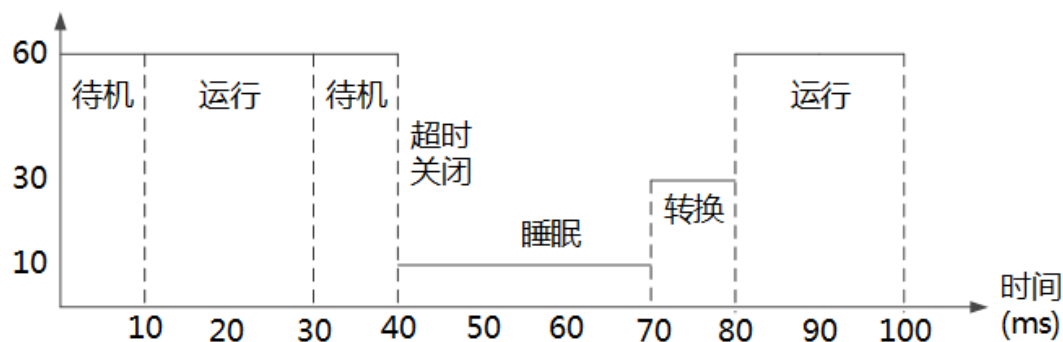
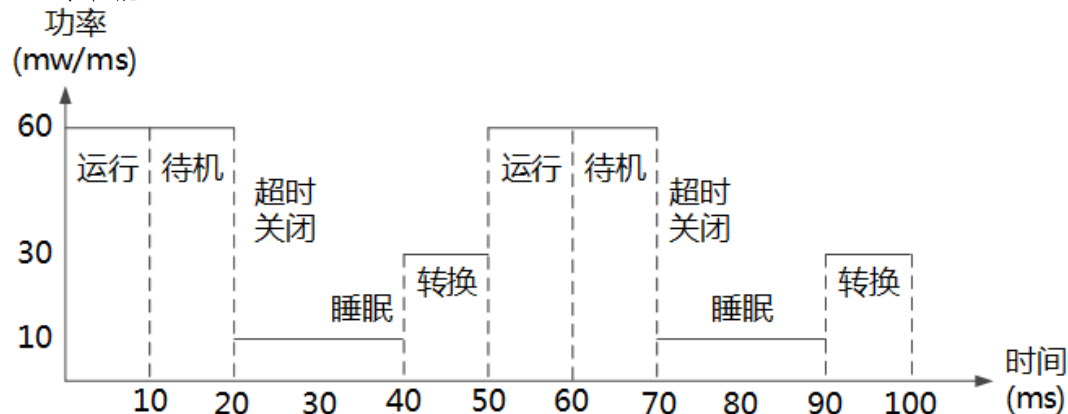
s1的节能比： $(6000 - 3400) / 6000 = 0.43$ 。

s2的能耗： $60 \times 10 + 60 \times 20 + 60 \times 10 + 10 \times 30 + 30 \times 10 + 60 \times 20 = 4200\text{mw}$

s2的节能比为： $(6000 - 4200) / 6000 = 0.3$ 。

s3的能耗： $60 \times 10 + 10 \times 10 + 30 \times 10 + 60 \times 30 + 60 \times 10 + 60 \times 10 + 60 \times 10 + 10 \times 10 = 4700\text{mw}$

s3的节能比： $(6000 - 4700) / 6000 = 0.22$ 。



(2015年5月系统分析师真题-第3题)

阅读以下关于嵌入式系统软件设计的叙述，在答题纸上回答问题1至问题3。

某软件公司长期从事移动智能终端设备等嵌入式软件研制工作，积累了丰富的嵌入式软件开发经验。某日，该公司经理派在嵌入式软件开发方面已具有很强经验的王工程师到某宇航设备研制单位洽谈一项软件合作项目，但是，在与对方洽谈需求时王工程师感觉沟通并不顺畅，许多概念较难达成一致。主要原因是王工程师长期从事移动智能终端的软件开发，开发平台主要是Android操作系统，开发语言是Java，而这次洽谈是王工程师第一次接触宇航系统软件，对于其特殊需求和要求缺少相关知识积累，不了解强实时、高安全和高可靠嵌入式软件设计等方面应用。

问题（1）

王工程师与某宇航设备研制单位洽谈的业务，涉及到了嵌入式系统的知识。根据你的理解请用100字以内文字说明嵌入式系统的主要特点。

思路解析

嵌入式系统的核心特征可从以下几方面总结：

- 专用性：不同于通用计算机，嵌入式系统针对特定应用场景（如飞控、医疗仪器、汽车电子）进行定制，功能目标明确。
- 实时性：需在限定时间内响应外部事件，例如飞控系统若延迟超过阈值可能导致飞行事故；因此实时调度和硬件协同至关重要。
- 软硬件耦合：软件往往需结合特定硬件特性优化，系统设计必须协同考虑 CPU 架构、内存布局、I/O 模块，避免性能瓶颈。
- 资源受限：受制于功耗、体积和成本，嵌入式设备常配备有限的存储与处理能力，要求开发者精简代码、优化调度。
- 可靠性与安全性：在宇航、医疗等关键领域，嵌入式系统需具备容错、冗余、校验机制，确保任务连续性。

问题（2）

王工程师到某宇航设备研制单位洽谈软件项目时，宇航设备研制单位技术人员提出了以下需求：此宇航设备主要面向无人飞行器，用于飞行姿态控制，实现飞行器的自主起飞和着陆，要求应用软件按最高安全等级（即A级软件要求）开发。为了提高飞行器系统的安全性和可靠性，便于应用软件的可重用性，需要选择一款满足宇航设备要求的嵌入式操作系统，并在操作系统之上开发飞行器控制软件。请根据你对嵌入式操作系统选择方法的理解，判断表3-1给出的有关选择嵌入式操作系统的观点是否正确，将解答写在答题纸的对应栏内。

序号	选择嵌入式操作系统的观点	正确：√ 不正确：×
1	根据项目（系统）需要的嵌入式操作系统功能来选择操作系统产品，要考虑系统需求是否覆盖了操作系统的全部功能或部分功能，是否支持文件系统和人机界面，是实时系统还是分时系统。	（1）
2	在微电子技术高速发展的今天，硬件资源受限已不再是嵌入式系统设计的难点，通常的嵌入式操作系统都能满足各类嵌入式系统的需要。	（2）
3	有些 RTOS 只支持该系统供应商的开发工具。也就是说，还必须向操作系统供应商获取编译器和调试器等；有些嵌入式操作系统使用广泛，且有第三方工具可用，因此，选择的余地比较大。	（3）
4	嵌入式操作系统到硬件的移植是一个重要的问题，是整个系统能否按期完工的关键因素，因此，要选择那些可移植性程度高的产品，从而避免嵌入式操作系统难以向硬件移植而带来的种种困难，加速系统的开发进度。	（4）
5	均衡考虑是否需要额外 RAM 或 EEPROM 来满足操作系统对内存的较大要求。有些嵌入式操作系统对内存的要求是目的相关的，如 VxWorks 等，开发人员能够按照应用需求分配所需资源，而不是为嵌入式操作系统分配资源。	（5）
6	嵌入式操作系统是否包含所需的软件部件，例如网络协议栈、文件系统和各种常用外设的驱动等，也是选择嵌入式操作系统重点关注的	（6）
7	在宇航系统中，一般选择嵌入式操作系统时，首先要考虑的是产品的先进性，希望采用更多新技术、新方法，其次才是开放性、成熟度。	（7）
8	有些嵌入式操作系统只能提供弱实时性能，对于需要达到硬实时性能要求的系统就不适用。	（8）

思路解析

选择嵌入式操作系统时，应综合考虑任务调度、实时性、安全性、可靠性与可扩展性。
（1）正确：系统应满足实时性与可靠性需求，这是宇航飞控的核心要求。

- (2) 错误：仅靠移植性和开发工具丰富不足以作为首要标准，关键是能否满足安全和实时要求。
- (3) 正确：操作系统需支持安全认证和故障隔离，避免单点失效。
- (4) 正确：资源占用低、内核精简是宇航嵌入式操作系统的重要特性。
- (5) 正确：应支持分区与模块化，利于功能隔离和安全验证。
- (6) 正确：应支持多任务调度与优先级控制，保证关键任务实时响应。
- (7) 错误：仅关注界面友好性是移动终端关注点，宇航系统首要是安全与实时。
- (8) 正确：系统应具备可裁剪性，以适应不同飞控功能需求。

问题（3）

请用300字以内文字，说明宇航设备的嵌入式软件与移动智能终端嵌入式软件在安全性（safety）、实时性、交互性设计和编码方面的要求有何差异，请填写表3-2中的空格（1）~（8），将解答写在答题纸的对应栏内。

序号	分类	宇航设备的嵌入式软件	移动智能终端的嵌入式软件
1	安全性	(1)	(2)
2	实时性	(3)	(4)
3	交互性	(5)	(6)
4	编码	(7)	(8)

思路解析

安全性：宇航嵌入式软件根据失效影响被划分为 AE 五个等级，A级软件失效可能导致灾难性后果，因此开发需遵循D0-178C 等标准，强调安全验证、冗余设计和严格的过程管控。而移动终端软件以用户娱乐和生活辅助为主，失效风险仅限于用户体验，故无须安全等级分级。

实时性：宇航软件必须保证强实时，即每个控制环节在限定时间内完成，例如飞控姿态调整不能有毫秒级延迟；因此需在设计上紧密结合硬件特性，确保任务分解到各个模块都满足实时性。智能终端软件则为弱实时系统，如播放视频或响应触摸，用户可容忍一定延迟。

交互性：宇航软件大多数为自动控制或后台运行，无需复杂的人机交互，关键是稳定性与容错。而智能终端软件则高度依赖用户交互，界面美观度与操作便捷性是首要。

编码：宇航领域编码严格，需遵循 MISRA-C 等安全子集标准，对函数规模、复杂度、扇入扇出有明确约束，以减少潜在缺陷；移动终端软件虽需遵守基本规范，但对安全性与复杂度要求宽松，更关注功能快速迭代。

答案-问题（1）

嵌入式系统具有以下特点：

- (1) 系统专用性强。嵌入式系统是针对具体应用的专门系统。
- (2) 系统实时性强。很多嵌入式系统对外来事件要求在限定的时间内及时做出响应，具有实时性。
- (3) 软硬件依赖性强。嵌入式系统的专用性决定了其软硬件的互相依赖性很强，两者必须协同设计，以达到共同实现预定功能的目的，并满足性能、成本和可靠性等方面的严格要求。
- (4) 处理器专用。嵌入式系统的处理器一般是为某一特定目的和应用而专门设计的。
- (5) 多种技术紧密结合。嵌入式系统通常是计算机技术、半导体技术、电力电子技术、机械技术与各行业的 具体应用相结合的产物。
- (6) 系统透明性。嵌入式系统在形态上与通用计算机系统差异甚大。
- (7) 系统资源受限。嵌入式系统为了达到结构紧凑、高可靠性和低成本的目的，其存储容量、

I/O设备的数量和处理器处理能力都比较有限。

答案-问题（2）

(1) ✓ (2) × (3) ✓ (4) ✓ (5) ✓ (6) ✓ (7) × (8) ✓

答案-问题（3）

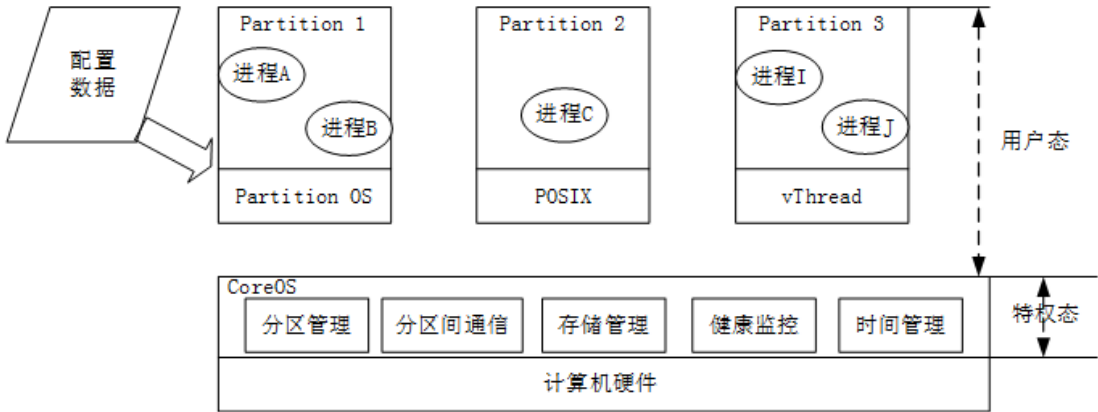
序号	分类	宇航设备的嵌入式软件	移动智能终端的嵌入式软件
1	安全性	(1) 高	(2) 较高
2	实时性	(3) 高	(4) 低
3	交互性	(5) 低	(6) 高
4	编码	(7) 高	(8) 低

(2014年5月系统分析师真题-第3题)

近年来，分区化（Partitioning）技术已被广泛应用于大型嵌入式系统，此项技术重在解决嵌入式系统中多类应用软件共享计算机资源的安全性问题，同时，也可降低软件开发成本，提高软件的可重用能力。某公司承担了一项宇航计算机系统研制任务，要求将以前采用的多处理机系统精简为由单个处理器系统完成，而整体功能应在原有功能不变的基础上，可灵活扩展，并要求原应用软件可被快速移植到新系统。公司将任务交给王工程师组织并承担总体设计工作。在采用何种嵌入式实时操作系统的选型问题上，王工认为宇航系统强调安全性，原系统应用软件功能分布在各自的处理机上，在保持功能不变的情况下，应适当地维持各个软件的相对独立性，因此采用具有分区能力的操作系统（如：VxWorks653）比较合适。

问题（1）

图3-1给出了具有分区能力的操作系统架构，该架构支持分区和进程两类调度，并提供了分区的时间、空间隔离保障，保证了应用软件可安全共享计算机资源。请用300字以内文字说明该类操作系统中分区化技术的主要特点。



思路解析

分区化实时操作系统（如 ARINC 653/VxWorks 653）通过空间+时间的强隔离使多类任务在同一处理器上安全共享。

空间上，内核为每个分区建立独立页表与 MPU/MMU 策略，栈/堆/代码段相互不可见，典型错误（空指针、越界）被捕获并限域处理。

时间上，采用静态主时间框架（Major Frame）+若干时间窗口（Window），窗口精确到 tick（本

题为 1ms)，确定性强，便于形式化验证与认证

。调度方面，先由分区调度决定哪个分区在当前窗口获得 CPU，再由分区内调度器按优先级/周期等策略调度该分区的进程，实现“宏观确定性+微观实时”。此外，内核提供健康监测、重启策略与事件日志，配合 I/O 抽象与消息通道实现故障隔离与可恢复。对于由多处理机整并为单处理器的场景，分区能“保持原子系统相对独立”，降低耦合，支持增量扩展与独立验证，显著提升复用与可维护性。

问题（2）

（16 分）完成该项目总体设计后，王工将软件设计工作交给李工，要求李工在原有软件基础上，完成新环境下的软件设计。经分析，李工统计出了原系统的任务基本情况（见表3-1）。原系统包含三个子系统，为了保证各子系统软件间的相对独立性，将三个子系统移植到新系统的三个分区工作，假设将"显示子系统"、"雷达子系统"和"任务子系统"分别用P1、P2和P3表示，系统的最小时间计时（tick）设为1ms，调度表的主时间框架将是40ms。

根据表3-1给出的原系统各任务周期和最坏执行时间数据，新系统中的所有任务可被调度，请对此进行分析说明，完善表3-2所示的分区调度表的设计，在空（1）~（10）填写正确内容并将解答填入答题纸的对应栏内。（注：WCET, Worst-Case Execution Time）。

序号	子系统	原系统软件分布情况		
1	显示子系统	任务（TASK）	周期（Period）ms	最坏执行时间（WCET）ms
		T1	10	1
		T2	10	2
		T3	40	5
2	雷达子系统			
		T1	10	1
		T2	20	3
3	任务子系统			
		T1	5	1
		T2	20	1

窗口	W01	W02	W03	W04	W05	W06	W07	W08	W09	W10	W11	W12
运行分区	P3	P1	P2	P1	P3	P3	(4)	(5)	P1	P2	P1	P3
子任务	T1	T1	T1	T2	T1	T2		T1	T1	T1	T2	T1
启动时间	(1)	(2)	2	3	5	6	7	10	11	12	13	15
持续时间	1	1	1	(3)	1	1	3	1	1	1	2	1

窗口	W13	W14	W15	W16	W17	W18	W19	W20	W21	W22	W23	W24
运行分区	P1	P3	P1	P2	P1	P3	P3	P2	P3	P1	P2	(9)
子任务	T3	T1	T1	T1	T2	T1	T2		T1	T1	T1	T2
启动时间	16	(7)	21	22	23	25	26	27	30	31	32	33
持续时间	(6)	1	1	1	2	1	1	(8)	1	1	1	(10)

思路解析

本题涉及到两方面的知识，其一是实时系统的周期任务的可调度分析方法；其二是正对问题1中提到

的分区的时间表调度算法，完成对表3-1的实例进行可调度分配，使操作系统按照设置好的调度序列调度分区，一定会保证表3-1中所有任务在最坏情况下按其周期要求执行完成。在可调度分析时，表3-1给出的实例中，分区调度的主时间框架应是实例中所用任务周期的最大公约数，即40ms，这样可调度分析首先应分析在40ms时间范围内是否可保证表3-1中所有任务的最坏执行时间都被至少执行一次。在此前提下，依次类推，完成对20ms周期、10ms周期和5ms周期的所用任务的最坏执行时间都被至少执行一次，如果四分钟分析都可调度，则表3-1给出的实例是可调度的。具体公式如下：

$$1(P3T1) \leq 5ms;$$

$$1(P1T1) + 1(P2T1) \leq 10ms;$$

$$2(P1T2) + 1(P3T2) \leq 20ms$$

$$1(P3T1) \times 8 + 1(P1T1) \times 4 + 1(P2T1) \times 4 + 2(P1T2) \times 2 + 1(P3T2) \times 2 + 5(P1T3) \times 1 + 3(P2T2) \leq 40ms$$

如果上述分析是可调度的，那么完成表3-2的分区调度表就可很容易实现。调度表是分区操作系统调度分区的依据，调度表将时间框架分成若干个窗口，每一个窗口可定义分配给哪个分区运行，启动时间是指本窗口针对主时间框架的第一个窗体"0"的偏移量，持续时间是指分配给此窗口的实际可运行时间。

在填写时必须清楚以下4点：

(1) 分区时间表调度中某窗口的启动时间不一定必须等于上一窗口的启动时间加持续时间。也就是说，窗口间的启动时间可以不连续；(2) 分区时间表调度中的第一个窗口的启动时间一般都为0；(3) 操作系统调度程序在遇见主时间框架最后一个窗口运行完后，应在主时间框架结束时刻重新加载主时间框架，新周期开始从第一个窗口运行；(4) 空窗口(null)是指本时间段操作系统不做任何分区调度，系统运行一种称之为空分区的任务。

答案-问题(1)

分区是一个相对独立实体，对于每个分区来讲，它们在空间和时间上完全隔离；分区由一个以上的进程组成，在时间上，每个分区按预先设定的时间片运行，分区没有优先级，系统中的所有分区按线性方式顺序工作；在空间上，每个分区有自己独立的存储空间，其空间大小是预先分配好的，任何分区内的进程发生故障不会影响其他分区的进程运行。分区技术使应用之间既能够相互独立的工作而不相互影响，又能共享系统资源；各个应用之间故障隔离，防止错误蔓延，可靠性和安全性高。开发人员能够快速地将新的软件模块插入系统，从而实现了高级别的系统集成和更新。

答案-问题(2)

(1) 0 (2) 1 (3) 2 (4) P2 (5) P3

(6) 4 (7) 20 (8) 3 (9) null (10) 7

(2013年5月系统分析师真题-第2题)

某高校计算机学院实验中心有5个机房，200个机位，主要承担学生计算机软件相关课程的上机实践任务。实验中心每年工作时间约为200天，每天平均工作时间为8小时。2013年初，实验中心决定对机房进行升级改造，其中UPS电源等设备不需要重新购置，改造后每个机位机器的基本配置要求如表2-1。实验中心现公开征集硬件改造方案。

	主机				显示器	交换机
技术指标	CPU	内存	硬盘	功率	尺寸 功率	传输速率 功率
规格	1 颗双核	4GB	500GB	250W 以内	21.5 英寸 30W 以内	千兆 40W 以内

一个月后，实验中心的主管从众多的改造方案中选择了A公司和B公司提出的方案进行最终评审。
A公司主张采用PC机更新换代的方式进行改造，其主要的设备和报价如表2-2所示。

	主机（200 台）				显示器（200 台）	交换机（10 台）
技术指标	CPU	内存	硬盘	功率	尺寸 功率	传输速率 功率
规格	1 颗双核	4GB	500GB	230W	21.5 英寸 25W	24 口千兆 30W
单价	3100 元				900 元	1000 元

B公司主张以桌面云的方式对实验中心进行全面改造，其主要的设备和报价情况如表2-3所示。

	服务器（14 台）				瘦客户机（200 台）	显示器（200 台）	交换机（11 台）	机架（2 台）
指标	CPU	内存	硬盘	功率	功率	尺寸 功率	传输速率 功率	尺寸
规格	4 颗 6 核	96GB	10TB	800W	23W	21.5 英寸 25W	24 口千兆 30W	42U
单价	30000 元				500 元	900 元	1000 元	10000 元

实验中心组织多名专家召开论证会，会上专家对这两种方案进行了评审，最终采纳了B公司的改造方案。

问题（1）

桌面云方案的核心是虚拟化技术，请用200字以内的文字说明什么是虚拟化技术，并从应用角度列举3种虚拟化技术。

思路解析

从概念本质看，虚拟化的核心是**“抽象 + 隔离 + 复用”：把底层物理资源抽象为统一的逻辑资源池（抽象），让不同工作负载在独立的运行空间中互不干扰（隔离），并以更高的资源复用度**支撑多任务并行（复用）。

在桌面云场景，服务器虚拟化负责在数据中心将计算节点切分为多个虚拟桌面/会话；存储虚拟化把分散的磁盘/阵列汇聚成统一的卷池，便于精简置备、快照、回滚，提升数据安全性与运维效率；桌面虚拟化/应用虚拟化则把操作系统或应用的执行迁移到中心侧，通过协议（RDP/PCoIP 等）把显示与交互传送到前端的瘦客户机。

与传统“每台 PC 独立部署/维护”相比，虚拟化带来三方面直接收益：一是集中管理（镜像统一、补丁一次性发布、策略统一下发），二是弹性伸缩（按需分配 vCPU、内存，负载高峰可临时扩容），三是安全可控（数据集中在机房、瘦端无本地盘，便于访问控制与取证）。因此，在高校机房这类席位多、负载起伏、镜像统一的场景中，虚拟化/桌面云天然契合需求。

问题（2）

请以A公司方案中的机器性能为基准，从应用模式、投入成本、运维成本、安全性、能源消耗等5个方面对两种方案进行详细比较（其中投入成本与能源消耗要求量化比较），说明为什么专家采纳了B公司的改造方案。

思路解析

本题考察总拥有成本（TCO）与架构演进的综合分析能力，必须同时覆盖应用模式、投入与运维成本、安全与能耗五维，并对投入成本与能耗进行明确量化。

量化计算的关键在于：①基于题干与表格逐项罗列设备清单与单价/功耗；②按机位数（200）、交换机数量、服务器/存储数量分别乘算；③能耗按功率（W）→日耗（Wh）→年耗（kWh）逐步计算。

以上计算结果为：A=810,000 元，B=731,000 元；能耗A=82,080 kWh/年，B=33,808 kWh/年。

在定性维度上，应理解桌面云的三大利好：集中管理（镜像/补丁统一）、弹性扩展（加节点即扩容）、端侧轻量（瘦端低故障率/低能耗）。安全方面，数据集中、最小化前端攻击面与细粒度权限/审计可显著降低风险；运维方面，服务端一次性配置可显著缩短停机与维护窗口。结合财务可行性（CAPEX+OPEX）与教学业务连续性要求，B 方案在 35 年维度呈现出明显的经济性与可运维性优势，这正是专家组选择 B 的决定性原因。

答案-问题（1）

虚拟化是将计算与资源（CPU、内存、存储、网络）从物理形态抽象为多个相互隔离的逻辑实例，使一台或一组物理设备可并行承载多个操作系统/应用环境，从而提高资源利用率、简化运维与弹性扩展。

常见应用包括：①服务器虚拟化（如基于 KVM/VMware 的多虚机并发）；②存储虚拟化（将多盘/阵列抽象为统一卷池提供快照/精简置备）；③桌面虚拟化/应用虚拟化（VDI/应用发布，在数据中心集中运行桌面或应用，通过瘦客户机访问）。

答案-问题（2）

应用模式：A公司方案是一种分散式的计算模式，这种模式每个节点都相对独立，每个节点都要求一定的运算能力，当节点运算能力不足时，便需要整体升级。而B公司方案是一种虚拟化的方案，他将服务器的运算能力按需求分配给各个节点使用，当运算能力不足时，可通过增加服务器的方式，方便的升级。

投入成本：

A公司方案的投入成本为： $(3100+900) \times 200 + 1000 \times 10 = 810000$ 元。

B公司方案的投入成本为： $30000 \times 14 + (500+900) \times 200 + 1000 \times 11 + 10000 \times 2 = 731000$ 元。

所以B公司的方案投入成本较小。

运维成本：B公司方案维护成本比A公司方案低，因为A公司方案中，需要维护的计算机有200台之多，而B公司方案只需要维护14台服务器。

安全性：B公司方案安全性比A公司方案高，因为A公司方案中，200台主机都带存储，都是独立的系统，其中任何一台机器出现安全问题都可以影响到其他机器。而B公司方案中，只要维护好14台服务器即可，此时出安全问题的可能性会低很多。

能源消耗：

A公司方案的能源消耗为： $((230W+25W) \times 200 + 30W \times 10) \times 8 \times 200 = 82080000$ 瓦时=82080千瓦时。

B公司方案的能源消耗为： $(800W \times 14 + (23W+25W) \times 200 + 30W \times 11) \times 8 \times 200 = 33808000$ 瓦时=33808千瓦时。

B公司的方案能源消耗远远低于A公司方案。

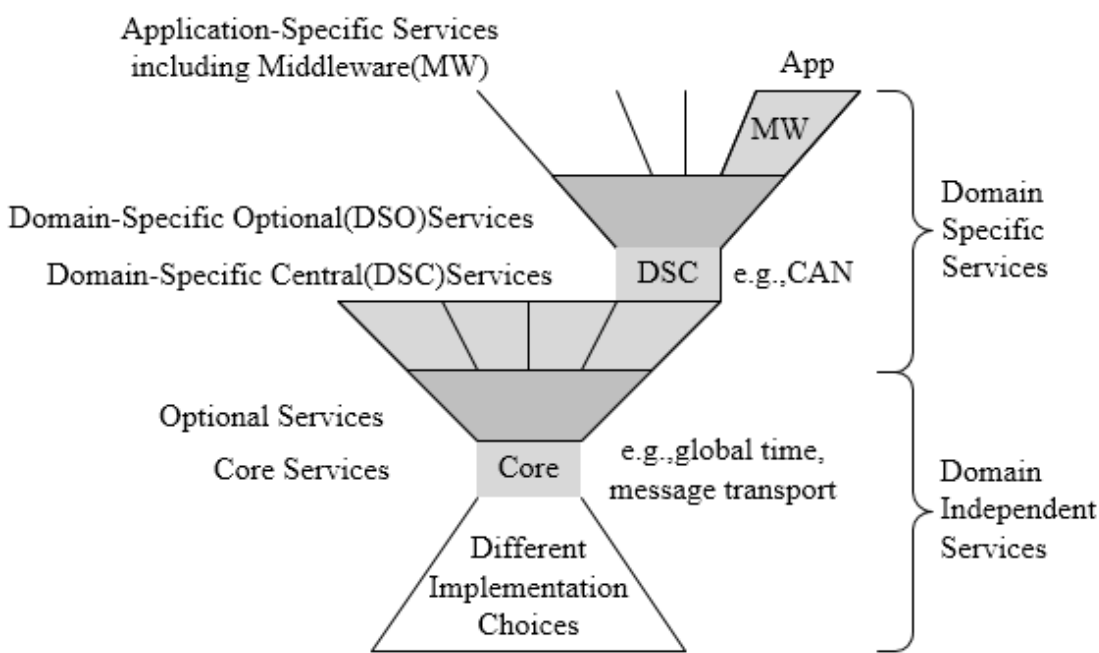
(2013年5月系统分析师真题-第3题)

阅读有关嵌入式系统新架构风格方面的描述，在答题纸上回答问题1至问题3。

在传统的嵌入式系统中，由于应用背景的不同，架构风格存在着差异。某公司长期从事各类嵌入式电子产品的研发工作，为了扩展业务，拟承担安全关键嵌入式产品的研制，为了统一本公司嵌入式产品的架构，以兼顾安全关键系统和非安全关键系统，公司领导层提出了采用国外近年新的跨领域嵌入式系统架构，新架构主要有6个特点：

- (1) 面向构件化
- (2) 开放式
- (3) 支持多种安全级别
- (4) 服务的层次化
- (5) 确定性核心
- (6) 内部组合的标准化

图3-1给出了跨领域嵌入式系统架构，图中CS表示领域无关的核心服务、OS表示领域无关的选择性服务、DSC表示领域相关的中心服务、DSO表示领域相关的选择性服务。该架构通常称为"腰"型架构，将嵌入式系统分为核心服务层、域服务层和应用服务层三个层次，实现了应用间相对无关性设计，架构的腰部为与领域无关的核心服务。



问题（1）

请用200字以内文字说明图3-1"腰"型架构的特点。

思路解析

“腰”型架构之所以被提出，是为了克服传统嵌入式系统架构因应用领域不同而割裂的问题。该架构通过一个稳定的核心服务层形成“腰部”，上层是面向领域与应用的服务，下层是硬件平台。其优势首先在于降低耦合度：应用层不再直接依赖硬件或领域特殊接口，而是通过标准化的核心服务交互，从而提高系统的可移植性与维护性；其次在于复用度高：核心服务与部分域服务可在不同产品中复用，大幅减少重复开发。

该架构还体现了面向构件化和开放式的特点，允许厂商将不同功能模块以标准接口组合，实现灵活裁剪与升级。此外，它能够支持多安全级别的应用，从消费电子到航空航天皆可适用；确定性核心保证了对安全关键应用的实时性与可靠性支持；而标准化的内部组合进一步减少集成风险，提升开发效率。

问题（2）

针对图3-1的"腰"部的核心层，表3-1中给出了10种服务，请判断这些服务中哪些属于核心服务（Core Services），哪些属于选择性服务（Optional Service），将结果填到表3-1中。

序号	服务类型	服务归属 Core Services 填 CS Optional Service 填 OS
1	内部连接服务	
2	外部存储器管理服务	
3	基本通信服务	
4	资源管理服务	
5	高级通信服务	
6	高级定时器服务	
7	基本计时器服务	
8	构件执行服务	
9	可靠性服务	
10	基本配置服务	

思路解析

要求分析给出的服务归属于哪种服务类型，这需要一定的经验，但同时也有技巧性。如，在题目给出的服务中，即有"基本***服务"也有"高级**服务"，这种情况，"基本***服务"必然是属于核心服务，而"高级**服务"属于选项服务。因为核心服务就有不可少的，最基本的服务的理念，而选择服务是可有可无的。依据该原则，已可解决半数问题。另外的服务类型则需要根据你的经验进行判断。

问题（3）

该公司遵从图3-1定义的嵌入式架构风格，梳理了现有产品序列，定义了公司产品中涉及的软件主要包括以下4类：

- （1）基于分区化的嵌入式操作系统（如：VxWorks653）
- （2）存储管理部件（如：文件系统）
- （3）支持产品互联的多种网络协议部件（如：TCP/IP、ppp、CAN、1394等）
- （4）应用软件

请根据你所掌握上述软件的相关知识，说明这些软件应归属图3-1中核心服务层、域服务层和应用服务层中的哪一层。并举例说明上述软件提供的服务中哪些归属于架构的OS、CS、DSC和DSO服务，填写表3-2。

序号	服务类型	服务举例
1	CS	
2	OS	
3	DSC	
4	DSO	

思路解析

首先，应根据软件的通用性与领域依赖性进行层次划分。嵌入式操作系统是所有应用的基础，属于核心服务层，其时间管理、调度、通信、中断管理等功能属于CS；如果操作系统还提供文件系统等部件，这些功能并非每个应用必需，因此划为OS。

其次，网络协议栈显然与特定产品领域相关（如消费电子侧重 TCP/IP，车载系统侧重 CAN，总线通信依赖 1394），因此属于域服务层。

其中一些协议（如 TCP/IP）在现代嵌入式系统中已是事实上的标准，属于DSC，而其他如 PPP、1394 等仅在特定场景使用，属于DSO。最后，应用软件直接实现产品功能，如车载控制软件、医疗监测软件，位于应用服务层，其需求与具体业务相关，不再归入 CS/OS/DSC/DSO 范畴。

答案-问题（1）

耦合程度低、复用程度高、面向构件化、开放式、支持多种安全级别、服务的层次化、确定性核心、内部组合的标准化等特点。

答案-问题（2）

判断依据主要有两点：

“基本”类服务必然是核心服务（CS）。因为核心服务层的目标是为所有嵌入式系统提供最基础的运行支撑，缺失这些服务，系统无法运行。例如，时间管理、任务调度、内存管理、通信与中断管理是嵌入式操作系统的五大支柱功能，必须属于核心服务。

“高级”类服务归入选择性服务（OS）。这些功能往往在某些特定应用中才需要，并非所有嵌入式系统都依赖。例如高级任务调度（支持多级调度、复杂优先级策略）、高级通信服务（如 QoS 保证）、高级中断管理（支持嵌套中断控制与异常处理增强）等，属于增强功能。

序号	服务类型	服务归属 Core Services 填 CS Optional Services 填 OS
1	内部连接服务	OS
2	外部存储器管理服务	OS
3	基本通信服务	CS
4	资源管理服务	OS
5	高级通信服务	OS
6	高级定时器服务	OS
7	基本计时器服务	CS
8	构件执行服务	CS
9	可靠性服务	OS
10	基本配置服务	CS

答案-问题（3）

核心服务层（1）（2） 域服务层（3） 应用服务层（4）

序号	服务类型	服务举例
1	CS	任务管理服务、任务间通信服务、内存管理服务、中断服务和时间管理服务等
2	OS	文件的创建、打开、删除等服务
3	DSC	网络初始化服务、数据发送/接收服务、网络配置服务等
4	DSO	应用或上层中间件软件需要扩展的服务，如 RTC 时钟等

（2012年5月系统分析师真题-第3题）

阅读以下关于嵌入式软件虚拟化方面的叙述，在答题纸上回答问题1至问题2。

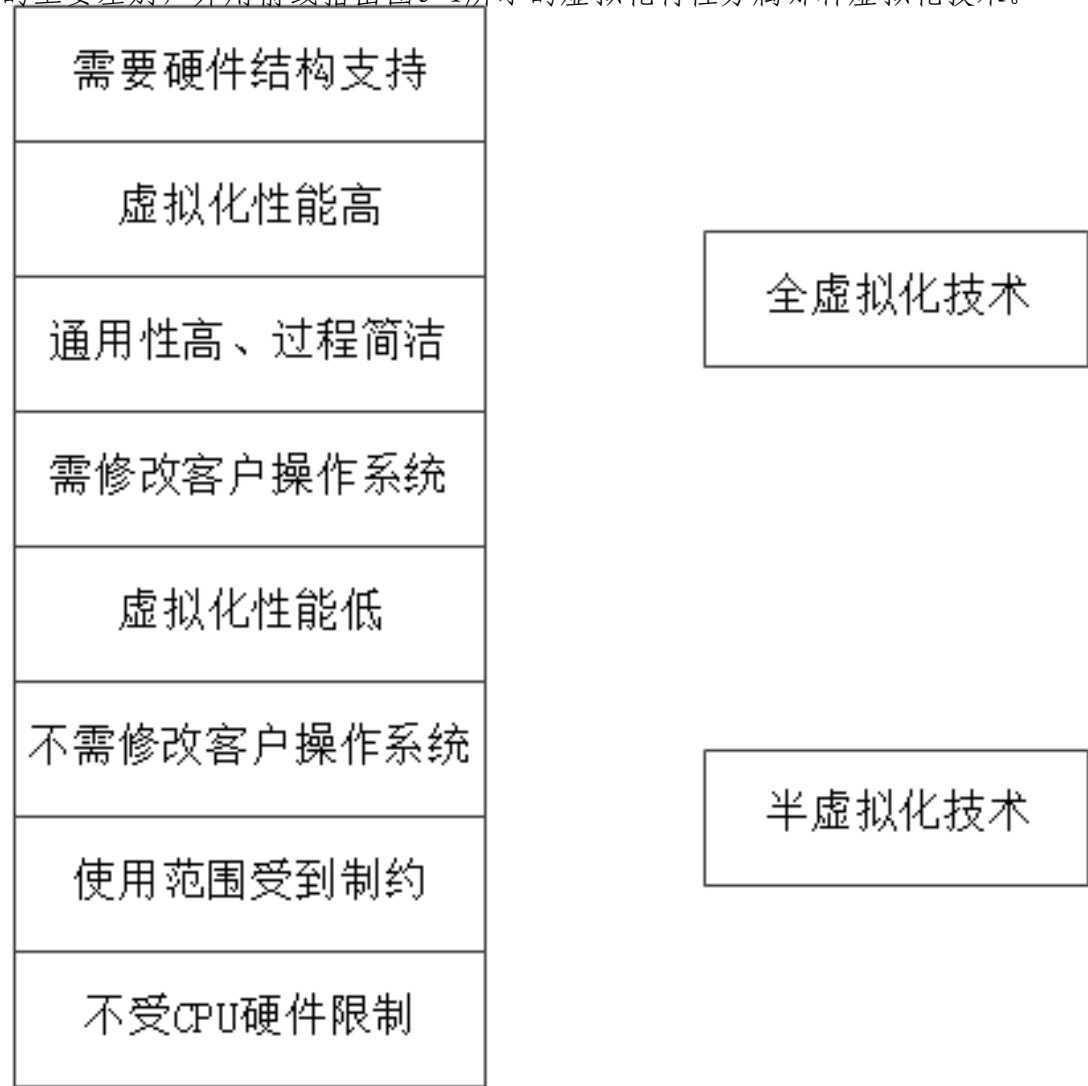
计算机虚拟化是一种型的计算模式，能够动态组织多种计算资源，隔离硬件体系结构和软件系统之间的依赖关系，实现透明化、可伸缩的计算系统构架，提高计算资源的使用效率和遗产软件的重用。

某公司是一家长期从事嵌入式软件研制的单位。随着虚拟化技术发展，公司决策层决定在公司原有

嵌入式实时操作系统基础上，研制具备虚拟化能力的新一代嵌入式操作系统产品。公司将该项目的总体设计任务交给了王工，要求他充分调研用户需求，尽快拿出项目的计划书，并给出项目的技术实施途径。一个月后，王工向公司决策层提交了项目计划书和技术实施途径报告，在公司讨论会上引起技术争议，其焦点主要集中在两个方面：虚拟化体系架构问题和虚拟化中的安全因素问题。

问题（1）

当前虚拟化的实现技术主要有全虚拟化和半虚拟化两种，请用200字以内的文字说明两种虚拟化技术的主要差别，并用箭线指出图3-1所示的虚拟化特性分属哪种虚拟化技术。



思路解析

全虚拟化和半虚拟化的最大区别在于客户操作系统是否需要修改。全虚拟化由于直接模拟硬件接口，对客户 OS 完全透明，因而兼容性极好，但上下文切换与指令陷入带来明显开销，性能偏低。半虚拟化通过在客户内核中增加 Hypercall 接口，绕开指令陷入，性能开销小，适合性能敏感场景，但局限在于需要获得操作系统源码并进行修改，且不易推广到闭源商业系统（如 Windows）。在嵌入式场景中，半虚拟化因内核可控性强、性能优越而被广泛应用，而全虚拟化则更适合支持多操作系统混合运行的通用计算环境。

问题（2）

具有虚拟化能力的嵌入式实时操作系统目前仅支持单核处理器，而不支持多核，请用100字以内的文字说明其主要原因。表3-1给出如果采用虚拟化技术支持多核时存在的四种共享资源和四种潜在安全

隐患，请用箭线指出这些潜在安全隐患分别会发生在哪种共享资源的环境下。

序号	潜在的安全隐患	共享资源
1	潜在的旁路通道遭数据泄漏攻击	共享引导逻辑
2	潜在的故障注入攻击	共享 CACHE
3	潜在的隐蔽通道攻击和服务拒绝攻击	共享 I/O 逻辑
4	执行影像的安全性受到潜在威胁	结构的复杂性

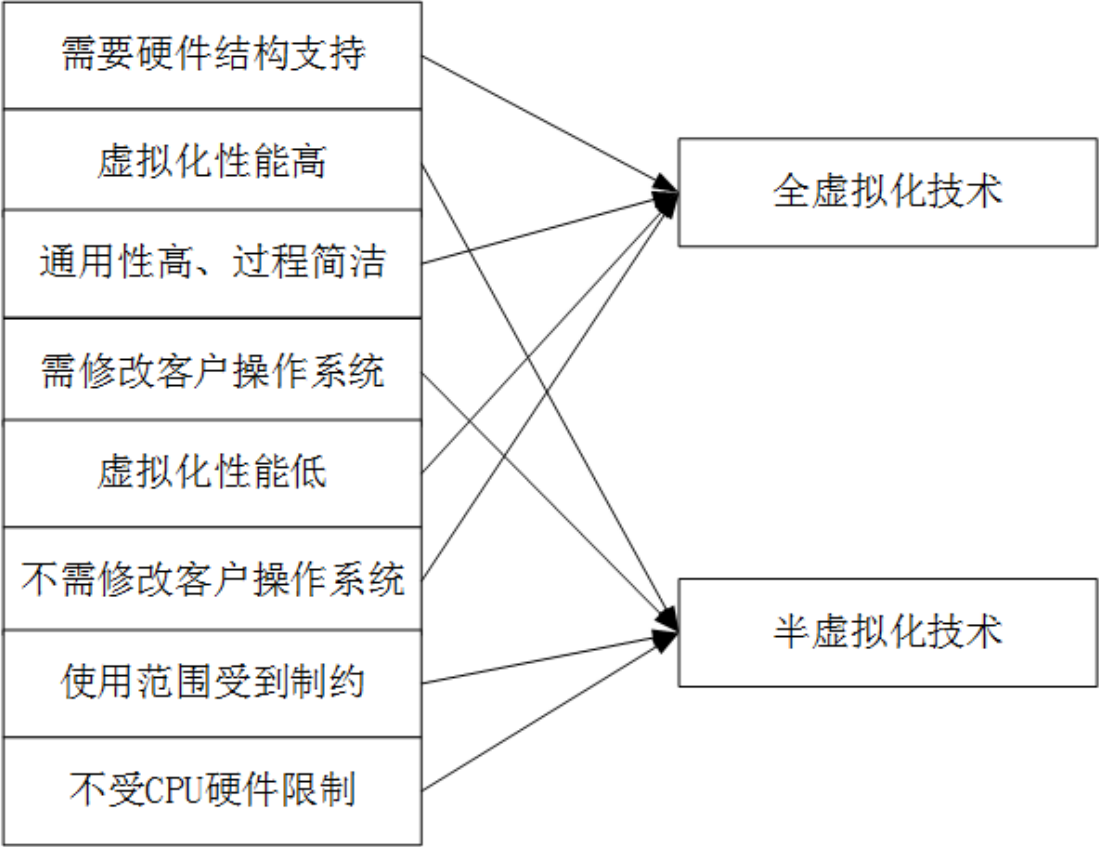
思路解析

嵌入式实时操作系统强调分区隔离（空间与时间），而单核环境下调度器可以严格保证时间片分配、缓存独占与 I/O 排他。但在多核环境中，多个分区可能同时运行，导致共享缓存、共享总线、共享 I/O 控制器等资源竞争。一旦虚拟化加入 Hypervisor 层，管理难度加大，分区间可能通过共享资源建立旁路通道（Side Channel）或隐蔽通道（Covert Channel），严重威胁安全。例如共享缓存可通过缓存命中率泄漏敏感信息，共享 I/O 可被滥用造成拒绝服务，复杂架构则容易被故障注入攻击利用。此外，共享引导逻辑若被篡改，还会导致启动阶段执行镜像遭到攻击。

因此，虽然单核虚拟化已可用于空间隔离与任务整合，但向多核迁移仍需解决资源隔离与安全强化问题（如缓存分区、I/O 虚拟化隔离、可信引导）。这也是目前虚拟化嵌入式实时 OS 难以支持多核的核心原因。

答案-问题（1）

全虚拟化技术能够在硬件辅助虚拟化的支持下，运行任何不需要修改的客户操作系统；半虚拟化技术不需要硬件辅助虚拟化的支持，通过精心修改客户操作系统内核，在只改动有限量内核代码、替换为相应Hypercall的基础上实现虚拟化的。



答案-问题（2）

虚拟化操作系统目前仅仅支持单核处理器，不支持多核的主要原因是：由于多核存在共享资源访问

虚拟化后会影响分区化的安全性。

序号	潜在的安全隐患	共享资源
1	潜在的旁路通道遭数据泄漏攻击	共享引导逻辑
2	潜在的故障注入攻击	共享 CACHE
3	潜在的隐蔽通道攻击和服务拒绝攻击	共享 I/O 逻辑
4	执行影像的安全性受到潜在威胁	结构的复杂性

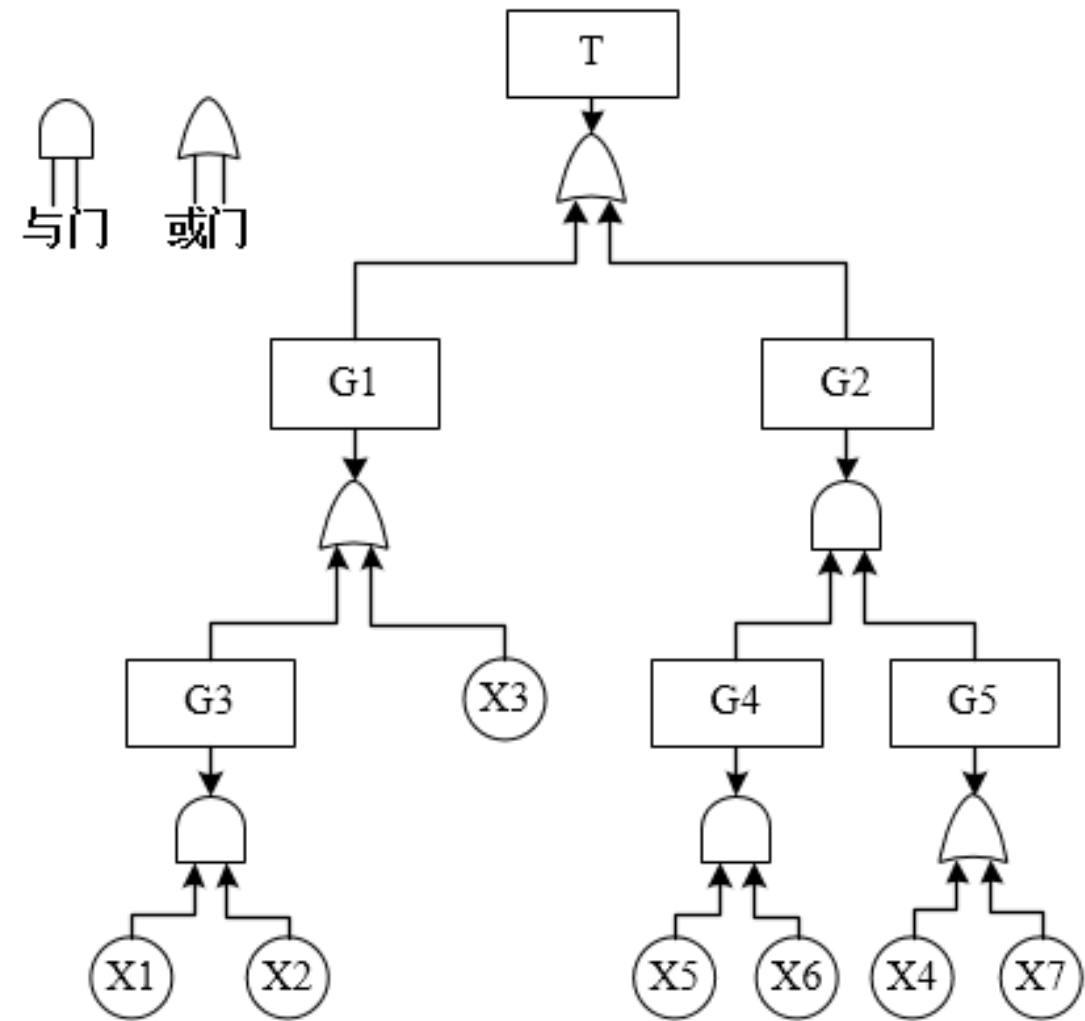
(2012年5月系统分析师真题-第4题)

阅读以下关于软件可靠性的叙述，在答题纸上回答问题1至问题3。

某企业欲研制某宇航控制系统。该系统对软件的可靠性要求很高，分配给软件的可靠性指标为 $R \geq 0.99$ 。根据软件结构设计方案，该软件由7个模块X1, X2, ..., X7组成，分别完成不同的控制功能。

为了保证该控制系统能够满足环境可靠性指标，李工认为应该首先采用故障树方法分析，预测出可靠性指标。该企业主管采用了李工的建议，建立了该软件的故障树模型（如图4-1所示），评估出了每个模块的可靠性指标，同时在每个模块的设计与实现过程中，采用了流程优化、结构优化、降低设计复杂度等方法来提高模块的可靠性指标。

软件开发完成后，项目组对该软件进行了相应的可靠性测试，得到了各模块的失效概率为： $FX1=FX2=0.05$ ， $FX3=0.008$ ， $FX4=0.07$ ， $FX5=FX6=0.05$ ， $FX7=0.08$ ，通过计算割集的失效概率来近似计算整个软件的可靠性指标，计算结果表明该软件的可靠性未达到分配的指标要求。



问题（1）

请给出该故障树的所有最小割集。

思路解析

由图4-1看，顶事件由四条互斥路径触发：一条为单模块X3失效直接导致顶事件（对应一阶割集）；另一条为X1与X2同时失效（与门）；其余两条路径分别要求 $X4 \wedge X5 \wedge X6$ 与 $X5 \wedge X6 \wedge X7$ 同时失效。

根据割集定义：能单独导致顶事件发生的、且不含冗余元素的失效集合即为最小割集。因此得到四个互不包含的最小割集： $\{X3\}$ 、 $\{X1, X2\}$ 、 $\{X4, X5, X6\}$ 、 $\{X5, X6, X7\}$ 。其中K1是一阶割集，意味着单点故障即可引发系统失效，风险最高；K2为二阶，K3、K4为三阶，阶数越高，路径需要的联合失效越多、发生概率越小，对系统可靠性的威胁相对更低。该分解为后续定量评估与改进指明了关键脆弱点（X3）。

问题（2）

根据题中给出的、经过可靠性测试后得到的每个模块的失效概率，计算每个割集失效概率，并近似计算出整个软件的失效概率，说明该软件的可靠性测试指标确未达要求。

思路解析

在最小割集互不相交且各事件失效率较小、相互独立的“稀有事件”假设下，可用一次近似（割集概率求和）估计顶事件概率。代入各模块测试得到的失效概率，四个割集的贡献从大到小依次为： $K1(0.008) > K2(0.0025) > K4(0.0002) > K3(0.000175)$ 。总失效概率为0.010875，据此可靠度 $R \approx 0.989125$ ，低于分配指标0.99，说明系统仍存在显著单点薄弱环节。进一步可算出K1占总失效的比例 $\approx 0.008/0.010875 \approx 73.6\%$ ，K2贡献 $\approx 22.97\%$ ，其余两条路径占比较小。

问题（3）

请进一步分析导致该软件可靠性测试未能满足分配指标要求的原因，并给出后续的改进策略。

思路解析

根据“最小割集越小、可靠性要求越高”的原则，一阶割集K1决定了系统的脆弱上限。当前K1的0.008几乎“吃掉”指标余量（R目标0.99对应允许失效 ≈ 0.01 ），导致总失效超标。首要措施应在X3上同时采用设计改良+测试增强+冗余容错三管齐下：在设计面降低圈复杂度、去耦关键路径、应用防御式编程与形式化规格；在验证面通过边界/异常/压力与组合路径的高标准覆盖；在结构面以1oo2D/2oo3投票或冷/热备份等方式避免单点。其次针对K2，可通过降低X1/X2失效率或削弱其相关性（差异化实现、隔离资源，避免共因失效）来进一步压低总失效。配合可靠性增长模型（如JM/Musa）设定阶段目标与试验计划，按割集贡献迭代收敛，可较高性价比地将R提升至 ≥ 0.99 。

答案-问题（1）

通过对故障树进行分析，可以得出其所有的最小割集为

： $K1=\{X3\}$ ， $K2=\{X1, X2\}$ ， $K3=\{X4, X5, X6\}$ ， $K4=\{X5, X6, X7\}$ 。

答案-问题（2）

通过对故障树进行分析，可以得出其所有的最小割集为

: K1={X3} , K2={X1, X2} , K3={X4, X5, X6} , K4={X5, X6, X7} 。

则对应割集的失效概率分别为:

$F(K1)=0.008$, $F(K2)=0.05 \times 0.05=0.0025$, $F(K3)=0.07 \times 0.05 \times 0.05=0.000175$, $F(K4)=0.05 \times 0.05 \times 0.08=0.0002$

则整个软件的失效概率近似为: $P(T)=0.008+0.0025+0.000175+0.0002=0.010875$, $R=1-0.010875=0.989125<0.99$, 因此, 软件的可靠性没有达到分配的指标要求。

答案-问题（3）

根据最小割集的定义，最小割集中所含模块数越少，此最小割集中模块的可靠性要求就越高。所以 X3模块的可靠性要求最高，为关键功能模块。由于X3模块对应的割集失效概率过高，导致了软件未能达到分配的指标要求。

改进方法（答出一种即可）：（1）采用多轮重复测试的方法，使得模块X3的失效概率收敛，满足可靠性指标要求；（2）重新设计模块X3，提高其可靠性，降低其失效概率。（3）割集K1为一阶割集，容易出现单点故障。可以在X3模块所在层次或其上级增加"与门"，降低K1割集的失效概率。

(2025年5月系统分析师真题-第3题)

随着嵌入式系统向智能化、联网化发展，传统的单一固化式部署已无法满足灵活性和敏捷性需求。容器技术特别是Docker的引入，为嵌入式系统部署提供了新的解决方案。

问题（1）

某嵌入式开发团队在实际部署Docker容器过程中，依据业务和技术特性，总结出提升容器系统质量的10项关键措施。为方便管理与优化，团队将这些措施归类为以下7个方面：镜像的规范性、运行时的规范性、性能与资源效率、兼容性、可靠性、易用性、安全性。以下是部分对应关系，请将表格补充完整（共10项措施，填空部分用序号表示）：

措施编号	措施名称	所属方面
1	基于多阶段构建优化镜像	(1)
2	制定基础镜像命名规范	镜像的规范性
3	控制容器权限与挂载方式	安全性
4	使用健康检查机制	(2)
5	采用异步日志机制	性能与资源效率
6	提供标准化入口脚本	易用性
7	优化镜像层级结构	(3)
8	控制定时任务及启动项	(4)
9	使用跨平台构建工具	(5)
10	引入统一配置中心	(6)

思路解析

（1）镜像的规范性。多阶段构建可以将构建依赖和运行环境分离，使最终镜像更精简、结构清晰，有利于规范化镜像内容，降低体积与安全风险。

（2）可靠性。通过 HEALTHCHECK 指令或外部监控手段，可以及时发现容器异常并触发自恢复机制，从而保障系统运行的持续性与可靠性。

- (3) 镜像的规范性。镜像层级优化可减少构建时间、提升缓存复用率，同时降低层之间的依赖耦合，增强镜像构建的可维护性和规范性。这个凯恩认为当作性能与资源效率也可以。
- (4) 运行时的规范性。控制启动项和定时任务能避免容器启动后自动运行不必要服务，防止资源浪费或行为不可控，确保容器运行过程更清晰规范。
- (5) 兼容性。构建工具如 Docker Buildx 支持生成多架构镜像（arm64、amd64 等），解决不同硬件平台部署问题，提高平台兼容性。
- (6) 易用性。统一配置中心可集中管理环境变量、配置文件等，提升配置一致性与部署便捷性，降低人为配置错误风险，提高系统易用性。

问题（2）

接下来，团队重点对运行容器的“切换时间”进行了性能优化，尤其关注无状态连接和数据库连接两类典型场景。以下为优化前后的切换耗时对比数据：

场景类型	优化前切换时间(ms)	优化后切换时间(ms)
无状态连接	230	120
数据库连接	310	190

请列举三种可行的优化方法，并简要说明其原理或适用条件。

思路解析

通过引入预热机制、连接池技术和启动项精简等策略，可以有效降低了无状态和数据库连接类容器的切换时间，升系统响应速度和资源利用效率。

问题（3）

从无状态连接与数据库连接两类场景的优化效果来看，你认为哪类场景更容易优化？为什么？请结合容器特点进行分析。

思路解析

无状态连接类容器因其结构简单、切换自由度高、对状态依赖低，更容易通过容器级优化（如预热、副本复用、镜像精简）获得显著的切换性能提升，优化成本低、收益明显。相比之下，数据库连接类容器由于状态复杂性和一致性要求，优化受限，提升空间相对较小。

答案-问题（1）

(1) 镜像的规范性 (2) 可靠性 (3) 镜像的规范性 (4) 运行时的规范性 (5) 兼容性 (6) 易用性

答案-问题（2）

- 方法一：容器预热机制（预启动或Warm Pool）：预先启动备用容器实例，在主容器切换前就绪，避免启动时加载延迟。：无状态连接尤为适合，可快速启用备用实例，应对高并发或快速扩缩容。
- 方法二：连接复用与连接池优化：对于数据库连接，避免每次切换重建连接，可通过连接池实现连接复用。数据库连接类容器，可大幅减少连接建立与认证时间，提升效率。
- 方法三：精简容器启动项与镜像体积：优化容器镜像结构、移除不必要启动项（如定时任务、冗余

服务），加快初始化速度。所有容器通用，尤其适用于嵌入式资源有限的环境，可减轻系统负担。

答案-问题（3）

从无状态连接与数据库连接两类场景的优化效果来看，无状态连接场景更容易优化，原因如下：
无状态连接无需保存上下文信息。无状态服务在切换时不涉及会话状态或持久连接恢复，因此容器重启或替换后不需要做额外的状态同步或恢复操作，优化空间更大，响应更快。
容器启动更轻量、初始化更快。无状态容器通常只需加载基础依赖和服务进程，镜像精简，启动流程简单。通过减少镜像层、移除启动项等手段，更容易实现显著加速。
更适合使用预热、并发副本等优化机制。无状态容器可以使用预热容器池、滚动替换等方式实现“无感切换”，这些方式在数据库连接类场景中往往受限于连接状态和一致性要求。
数据库连接类容器通常涉及连接建立、身份认证、连接池初始化等过程，优化难度较高；
即便使用连接池等手段，也需考虑连接状态同步、事务一致性与并发控制，增加优化复杂度；
数据库服务对可靠性与顺序性要求更高，很多轻量优化手段（如快速销毁重建）不适用。

第8章——软件需求工程

（2014年5月系统分析师真题-第1题）

某企业委托软件公司开发一套运动器材综合销售平台，以改进已有的销售管理系统，拓展现有的实体店销售模式，综合管理线上线下的器材销售业务。该软件公司组建项目组开发该系统，现正处于需求获取阶段。经过项目组讨论，由于目标系统业务功能比较复杂，所以在需求获取中针对不同类型的业务需求，采用不同的需求获取方法。项目组列出可选的需求获取方法包括：用户访谈、联合需求计划（JRP）、问卷调查、文档分析和实地观察等。

需求获取的要求如下：

- （1）获取已有销售管理系统中所实现的实体店销售模式和过程；
- （2）获取系统的改进需求和期望增加的业务功能；
- （3）获取当前业务过程中的详细数据并深入了解这些数据产生的原因；
- （4）从企业管理人员、销售人员、各种文档资源等尽可能多的来源获取需求；
- （5）消除需求中出现的冲突，尽可能获取全面、一致的需求；
- （6）尽可能多地让用户参与需求获取过程。

问题（1）

联合需求计划（JRP）是一种流行的需求获取方法。请说明什么是JRP，JRP与其他需求获取方法相比有什么优势？

思路解析

JRP 的本质是一种结构化的群体沟通与决策方法。与传统的单个用户访谈或书面调查不同，它强调群体讨论，由主持人控制会议节奏，保证各方观点被充分表达，并在会议中形成可执行的共识。在复杂业务场景下，需求往往涉及多个部门，容易出现需求冲突与歧义，若仅依靠访谈和调查，难以及时发现并解决。通过 JRP，可以快速聚合意见，并在必要时结合原型工具，让用户“所见即所得”，从而提升需求质量。其缺点是组织成本高、参会人员要求高，但对于 ERP、销售平台等跨部门复杂系统，JRP 的优势尤为突出。

问题（2）

针对题目中所描述的需求获取要求（1）~（6），选择最适合的需求获取方法填入表1-1中的（a）~（f）处。

需求获取要求	需求获取方法
（1）	（a）
（2）	（b）
（3）	（c）
（4）	（d）
（5）	（e）
（6）	（f）

思路解析

- （1）已有模式与过程：实体店的操作流程可通过实地观察直接获取，也可通过文档分析查阅制度与操作手册。
- （2）改进需求与新增功能：需要与用户进行深度交流，最适合用户访谈或JRP，以捕捉真实痛点与改进方向。
- （3）详细数据及成因：单靠文档难以理解，需要用户访谈结合JRP，深入挖掘数据背后的业务逻辑与原因。
- （4）多渠道获取需求：可通过问卷调查覆盖范围广的员工群体，同时辅以文档分析，保证不同来源的数据输入。
- （5）消除冲突、获取一致需求：这是 JRP 的典型优势，依靠群体讨论与主持人调和来达成共识。
- （6）提高用户参与度：JRP 强调用户深度参与全过程，因此是最合适的方法。

问题（3）

由于该企业销售规模较大，所积累的企业业务文档数量庞大，所以只能通过抽样实现不同类型的文档分析。如果对于每种类型的文档要求90%的可信度（可信度因子为1.645），那么不同类型的文档分别需要抽样多少份就能达到该要求？

思路解析

在统计抽样中，样本容量公式为：

$$n = \frac{Z^2 \cdot p(1-p)}{E^2}$$

代入

$$\begin{aligned} n &= \frac{1.645^2 \times 0.25}{0.1^2} \\ &= 0.25 \times \left(\frac{1.645}{0.1}\right)^2 \\ &= 67.65 \end{aligned}$$

结果取整为 68。

Z = 可信度因子（置信区间对应的正态分布值，本题 1.645）；

p = 成功概率，取最大变异值 0.5；

E = 容许误差（即 1可信度 = 0.10）

此计算说明，为在大样本文档集中达到 90% 置信水平，至少需要抽样 68 份文档。通过抽样分析，可显著降低工作量，同时保持统计结果的可靠性。

答案-问题（1）

联合需求计划是一个通过高度组织的群体会议来分析企业内的问题并获取需求的过程，它是联合应用开发的一部分。JRP是一种相对来说成本较高的需求获取方法，但也是十分有效的一种。它通过联合各个关键用户代表、系统分析师、开发团队代表一起，通过有组织的会议来讨论需求。JRP将会起到群策群力的效果，对于一些问题最有歧义的时候、对需求最不清晰的领域都是十分有用的一种方法。

它的优势是（1）发挥用户和管理人员参与系统开发过程的积极性，提高系统开发效率；（2）降低系统需求获取的时间成本，加速系统开发周期；（3）采用原型确认系统需求并获取设计审批，具有原型化开发方法的优点。

答案-问题（2）

- （a）实地考察或文档分析
- （b）用户访谈或联合需求计划
- （c）用户访谈或联合需求计划
- （d）问卷调查或文档分析
- （e）联合需求计划
- （f）联合需求计划

答案-问题（3）

样本大小 = $0.25 \times (1.645 \div (10.90)) = 67.65063 \approx 68$ 份

（2024年11月系统分析师真题-第1题）

阅读以下关于软件维护的叙述，在答题纸上回答问题1至问题3。

【说明】

某企业自主研发了超市自助结算系统具备以下主要功能：

自动计价：顾客在结算时，通过自助设备扫描商品条码，系统能够快速识别商品，在屏幕上清晰显示商品的名称、规格、单价等详细信息，同时系统自动计算商品总价，并将结果返回显示在屏幕上，供顾客确认。

付款管理：当顾客完成商品选购并确认总价后，可选择多种付款方式（如现金、银行卡、移动支付等）进行付款。付款成功后，财务清算系统会及时接收到支付完成的通知，并进行相应的账务处理。

库存管理：一旦交易成功，系统会依据订单中的商品信息，自动对库存商品数量进行相应减少操作，并将变动情况准确记入库存表中，以确保库存数据的实时性和准确性，为后续的采购和销售决策提供可靠依据。

交易管理：系统会全面、准确地记录每一笔交易的详细信息，包括交易时间、顾客信息、商品清单、付款金额、付款方式等。这些记录将被妥善保存，以便在后续需要时（如顾客查询、财务审计、数据分析等）能够快速、便捷地进行查询和调用。

问题（1）

请用300字分别介绍流程图和数据流图并比较它们的不同之处。

思路解析

流程图（Flowchart）：

流程图是用图形符号表示操作步骤、判断逻辑和控制流的工具，广泛应用于程序设计、算法描述、业务流程等方面。流程图的基本目的是通过图形化的方式，展示从开始到结束的各个步骤及其之间的控制关系。常用的流程图符号包括：

起止框（椭圆）：表示流程的开始或结束。通常使用椭圆形状来标识流程的起始点和终结点。

处理框（矩形）：表示流程中的处理步骤或操作，如“计算总价”、“生成报告”等。矩形框内通常包含操作的简要描述。

判断框（菱形）：用于表示条件判断或决策节点，例如“是否登录成功？”根据条件结果，流程会根据“是”或“否”分支，进入不同的操作路径。

输入/输出框（平行四边形）：表示流程中的数据输入或输出，如“输入用户名”、“导出文件”等。平行四边形用于表示用户与系统之间的数据交换。

箭头：表示流程步骤的顺序或控制流的方向，指示流程的执行路径。

流程图的主要特点：

关注控制流和步骤执行的顺序。

用于描述具体操作和决策的逻辑，适用于算法设计和业务流程。

强调过程中的每一步骤，清晰展示步骤间的顺序关系。

数据流图（Data Flow Diagram, DFD）：

数据流图则用于描述系统中数据的流动、处理和存储，它侧重于展示数据如何在系统中流转、如何通过各个模块处理、存储和传递。数据流图的目的是帮助分析系统需求，明确数据流转的路径和系统功能。数据流图的主要组成元素包括：

外部实体（矩形）：表示系统外部的参与者，如“用户”、“外部系统”等。这些外部实体与系统有数据交互。

数据流（箭头）：表示数据在系统中的传递方向，箭头指示数据从一个元素流向另一个元素，如“订单信息→库存系统”。

加工（圆角矩形或椭圆）：表示数据处理过程，即系统内进行数据加工、转换或运算的模块，如“处理订单”、“验证数据”。

数据存储（开口矩形或双横线）：表示数据的存储位置，如“数据库”、“文件系统”等。数据存储用来保持系统中的信息，供后续操作使用。

数据流图的主要特点：

侧重于数据的流动和存储，聚焦于数据的处理和传输。

强调数据如何在系统中流转，而不是具体的控制流程。

用于需求分析和系统建模，帮助开发团队理解系统的数据交互方式。

流程图与数据流图的区别：

关注点不同：流程图关注的是操作的步骤和控制流，侧重描述“如何做”和“做什么”的过程，强调步骤的执行顺序和判断条件。数据流图则专注于数据的流动和处理，它关注的是“数据是如何流转、存储和加工的”，以数据驱动，描绘了系统内各个部分的数据交互和转换过程。

应用阶段不同：流程图常用于程序设计阶段，帮助开发人员理解和设计具体的操作流程，如算法的步骤、业务逻辑的执行路径等。数据流图更多应用于需求分析阶段，用于描述系统的功能需求，帮助开发团队理解系统中的数据流和信息交互。

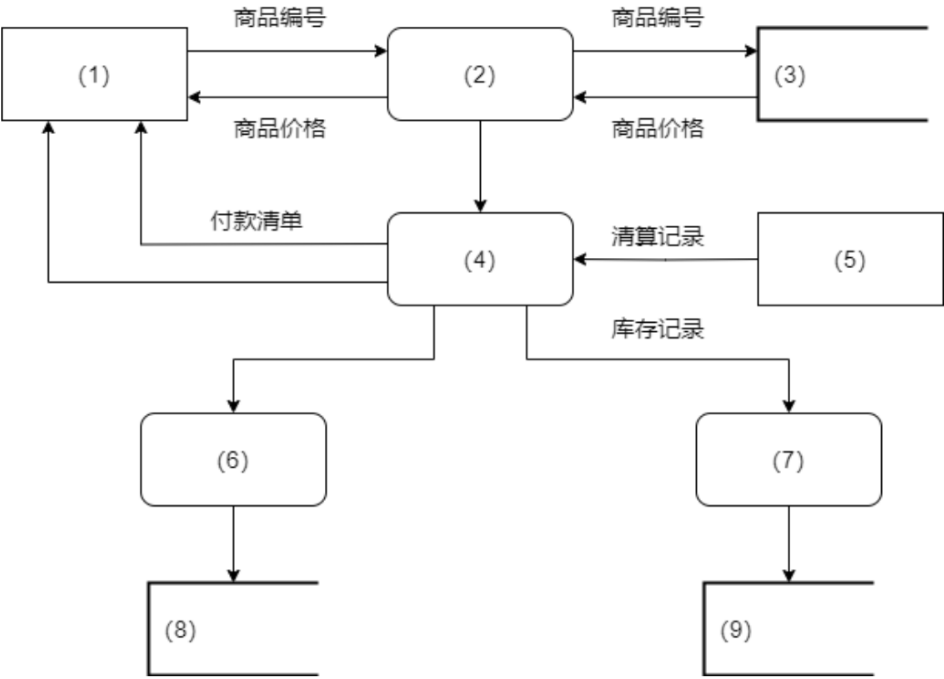
构建方法不同：流程图侧重于具体步骤的执行，通过图形符号展示操作的顺序和逻辑，通常是描述一个过程中的“如何做”。数据流图则注重数据的流向和变换，通过数据流、加工和存储的元

素，描绘数据如何在系统中传递和处理，通常是描述系统中的“做什么”。

问题（2）

请根据题意，将下列备选内容的序号，填入下图数据流图中（1）至（9）的位置。

（a）顾客、（b）计算总价、（c）商品、（d）付款结算、（e）财务清算系统、（f）库存管理、（g）交易管理、（h）库存信息、（i）交易信息。



思路解析

（1）a 顾客：顾客在超市自助结算系统中是外部实体，触发了整个流程的开始，负责提供商品信息、确认总价并完成付款。

（2）b 计算总价：计算总价是系统处理商品信息的步骤，顾客通过扫描商品条码，系统会计算出商品的总价。

（3）c 商品：商品数据是系统中的基本输入，顾客通过扫描条码获取商品的名称、规格、单价等信息，用于计算总价。

（4）d 付款结算：付款结算是系统中的重要环节，顾客确认总价后，选择付款方式并完成支付，结算过程记录并处理付款信息。

（5）e 财务清算系统：财务清算系统接收到付款完成的通知后，会处理账务信息，更新财务数据。

（6）g 交易管理：交易管理负责记录每笔交易的详细信息，确保交易的完整性、可查询性，并为后续审计或分析提供数据。

（7）f 库存管理：库存管理模块根据交易信息实时更新库存数量，以确保库存数据的准确性和及时性。

（8）i 交易信息：交易信息用于记录每一笔交易的详情，包括商品清单、付款方式等，以便后续查询和分析。

（9）h 库存信息：库存信息用于存储商品的数量和其他相关数据，确保库存管理模块能准确掌握库存状态。

答案-问题（1）

流程图是用图形符号表示流程中的操作步骤、判断逻辑及控制流，展现从开始到结束的完整过程。常被用于描述算法、业务流程、程序逻辑或操作步骤。它包括起止框（椭圆）：表示流程开始或结束。处理框（矩形）：如"计算成绩""生成报告"。判断框（菱形）：分支条件（如"是否登录成功？"）。输入/输出框（平行四边形）：如"输入用户名""导出文件"。箭头：指示步骤顺序。

数据流图用于展示系统中数据的流动、存储和处理，强调数据如何被加工、传递和存储。常被用于系统分析阶段，明确功能需求及数据交互关系。它的核心元素有外部实体（矩形）：系统外部参与者（如"用户""银行"）。加工（圆角矩形/椭圆）：如"处理订单""验证数据"。数据存储（开口矩形/双横线）：如"数据库""文件"。数据流（箭头）：数据传递方向（如"订单信息→库存系统"）。

它们两者的区别在于，流程图的核心目标是描述步骤顺序和控制流，关注点在于操作的执行，以流程驱动，用于程序设计等阶段；数据流图旨在描述数据流动和处理逻辑，聚焦数据流转，以数据驱动，常用于系统需求分析等阶段。

答案-问题（2）

(1) a (2) b (3) c (4) d (5) e (6) g (7) f (8) i (9) h

(2025年11月系统分析师真题-第1题)

某大型连锁企业主营产品销售业务，在全国范围内设有多家分支机构和销售网点。随着企业规模不断扩大，原有的信息系统已无法满足企业精细化管理和业务发展的需要，项目组列出可选的需求获取方法包括：用户访谈、联合需求计划（JRP）、问卷调查、文档分析和实地观察等。

需求获取的要求如下：

- (1) 获取已有销售管理系统中所实现的实体店销售模式和过程；
- (2) 获取系统的改进需求和期望增加的业务功能；
- (3) 获取当前业务过程中的详细数据并深入了解这些数据产生的原因；
- (4) 从企业管理人员、销售人员、各种文档资源等尽可能多的来源获取需求；
- (5) 消除需求中出现的冲突，尽可能获取全面、一致的需求；
- (6) 尽可能多地让用户参与需求获取过程。

问题（1）

请用 300 字简要说明系统需求、用户需求、业务需求的涵义。

思路解析

在信息系统建设过程中，需求通常分为三个层次，彼此递进、相互支撑。

业务需求是最高层需求，反映企业的战略目标或管理诉求，例如提高销售效率、加强精细化管理等。它往往由企业高层或业务负责人提出，具有宏观性和方向性，是项目立项的重要依据。

用户需求来源于具体使用系统的人员，如销售人员、管理员等，关注日常业务操作层面的问题，例如希望系统能快速录入订单、自动生成报表等。用户需求具有较强的角色差异性，是连接业务需求与系统实现的桥梁。

系统需求是在分析和整理业务需求、用户需求后形成的可实施需求，通常以功能需求和非功能需求形式表达，如性能、安全、可靠性等，是系统设计和开发的直接依据。三者层层分解，缺一不可。

问题（2）

对比分析 JRP、问卷调查、用户访谈、实地观察、文档分析，填写下方表格（高，中等，低）

方法	信息广度	信息深度	用户参与度	信息集成度
JRP	中等	高	(3)	高
问卷调查	高	(2)	低	低
用户访谈	(1)	高	高	中等
实地观察	低	高	中等	(5)
文档分析	中等	中等	(4)	低

思路解析

在需求获取过程中，不同方法在信息覆盖范围、挖掘深度、用户参与程度以及信息整合能力等方面存在明显差异。

用户访谈通常通过一对一或小范围交流的方式进行，能够较深入地了解用户的真实需求和使用痛点，但由于参与用户数量有限，其覆盖范围不够全面，因此信息广度为中等。

问卷调查适合在较大范围内快速收集用户意见，信息覆盖面广，但问题形式多为封闭式，缺乏深入追问的机制，难以挖掘隐性和深层次需求，因此信息深度较低。

****JRP（联合需求计划）****通过组织用户、管理人员、系统分析师和开发人员共同参与需求讨论，使用户在需求获取过程中直接参与分析和决策，参与程度高，是用户参与度最高的一种需求获取方法。

文档分析主要依赖已有的业务制度、流程文档和历史资料进行需求提取，整个过程不需要用户直接参与，属于非交互式需求获取方式，因此用户参与度较低。

实地观察通过对用户实际工作环境和操作流程的观察，可以获得较真实、深入的需求信息，但主要依赖分析人员的理解和归纳，缺乏多角色协同讨论与统一决策机制，其信息集成度处于中等水平。

问题（3）

针对题目中所描述的需求获取要求（1）~（6），选择最适合的需求获取方法填入表1-1中的（a）~（f）处。

需求获取要求	需求获取方法
(1)	
(2)	
(3)	
(4)	
(5)	
(6)	

思路解析

（1）已有模式与过程：实体店的操作流程可通过实地观察直接获取，也可通过文档分析查阅制度与操作手册。

（2）改进需求与新增功能：需要与用户进行深度交流，最适合用户访谈或JRP，以捕捉真实痛点与

改进方向。

(3) 详细数据及成因：单靠文档难以理解，需要用户访谈结合JRP，深入挖掘数据背后的业务逻辑与原因。

(4) 多渠道获取需求：可通过问卷调查覆盖范围广的员工群体，同时辅以文档分析，保证不同来源的数据输入。

(5) 消除冲突、获取一致需求：这是 JRP 的典型优势，依靠群体讨论与主持人调和来达成共识。

(6) 提高用户参与度：JRP 强调用户深度参与全过程，因此是最合适的方法。

答案-问题（1）

业务需求：描述组织或企业在战略层面希望通过信息系统解决的业务问题或实现的业务目标，关注“为什么要建系统”。

用户需求：描述不同角色用户在实际工作中希望系统提供的功能和支持，关注“用户要做什么”。

系统需求：在业务需求和用户需求基础上，进一步细化为系统必须具备的功能需求和非功能需求，关注“系统如何实现”。

答案-问题（2）

(1) 中等 (2) 低 (3) 高 (4) 低 (5) 中等

方法	信息广度	信息深度	用户参与度	信息集成度
JRP	中等	高	高	高
问卷调查	高	低	低	低
用户访谈	中等	高	高	中等
实地观察	低	高	中等	中等
文档分析	中等	中等	低	低

答案-问题（3）

- (a) 实地考察或文档分析
- (b) 用户访谈或联合需求计划
- (c) 用户访谈或联合需求计划
- (d) 问卷调查或文档分析
- (e) 联合需求计划
- (f) 联合需求计划